

Nullexit: Unified Project Document

Omar Alaaeldein

July 10, 2026

Contents

1	Project Directory Tree	3
2	README.md	4
3	agent.md	9
4	devref.md	14
5	diagrams.md	53
6	toggle.sh	58
7	recover.sh	75
8	setup.sh	82
9	scripts/toggle-linux.sh	83
10	scripts/recover-linux.sh	94
11	scripts/setup-linux.sh	97
12	docker-compose.yml	98
13	scripts/common.sh	101
14	scripts/watcher.sh	107
15	scripts/crypto.sh	111
16	.aiignore	112
17	.gitignore	112
18	.host_ips	113
19	Recover-Gateway.applescript	113
20	Toggle-Gateway.applescript	113
21	benchmarks/benchmark.sh	113
22	benchmarks/test_load.py	114

23	black_list.txt	115
24	cloud-init.yaml	116
25	launchd/com.nullexit.wake-recovery.plist	117
26	post-rules.txt	118
27	scripts/Dockerfile.routing-fix	118
28	scripts/Dockerfile.rule-compiler	119
29	scripts/Toggle-Gateway.bat	119
30	scripts/diagnose-host-leak.sh	121
31	scripts/dns-proxy.py	130
32	scripts/fix-docker-bridge-collision.sh	132
33	scripts/fix-ssh-delay.sh	137
34	scripts/generate_tex.py	137
35	scripts/logger/go.mod	140
36	scripts/logger/main.go	140
37	scripts/pf.conf	143
38	scripts/reinstall-host-tailscale.sh	144
39	scripts/routing-fix.sh	153
40	scripts/rule-compiler/go.mod	157
41	scripts/rule-compiler/main.go	157
42	scripts/setup-common.sh	168
43	scripts/unlock-files.sh	174
44	white_list.txt	174

1 Project Directory Tree

```
nullexit/
|-- .aiignore
|-- .gitignore
|-- .host_ips
|-- LICENSE
|-- README.md
|-- Recover-Gateway.applescript
|-- Toggle-Gateway.applescript
|-- agent.md
|-- benchmarks
|   |-- benchmark.sh
|   `-- test_load.py
|-- black_list.txt
|-- cloud-init.yaml
|-- devref.md
|-- diagrams.md
|-- docker-compose.yml
|-- launchd
|   `-- com.nullexit.wake-recovery.plist
|-- post-rules.txt
|-- recover.sh
|-- scripts
|   |-- Dockerfile.routing-fix
|   |-- Dockerfile.rule-compiler
|   |-- Toggle-Gateway.bat
|   |-- common.sh
|   |-- crypto.sh
|   |-- diagnose-host-leak.sh
|   |-- dns-proxy.py
|   |-- fix-docker-bridge-collision.sh
|   |-- fix-ssh-delay.sh
|   |-- generate_tex.py
|   |-- logger
|   |   |-- go.mod
|   |   `-- main.go
|   |-- pf.conf
|   |-- recover-linux.sh
|   |-- reinstall-host-tailscale.sh
|   |-- routing-fix.sh
|   |-- rule-compiler
|   |   |-- go.mod
|   |   `-- main.go
|   |-- setup-common.sh
|   |-- setup-linux.sh
|   |-- toggle-linux.sh
|   |-- unlock-files.sh
|   `-- watcher.sh
|-- setup.sh
|-- toggle.sh
`-- white_list.txt
```

2 README.md

```
1 # nullexit: Tailscale + Cloudflare WARP Docker Gateway
2
3 > **Last updated:** July 10, 2026 [Architecture & Flow Diagrams ](./diagrams.md) [Full Dev Reference ](./
  devref.md)
4
5 **nullexit** is a chained network gateway that routes all Tailscale exit-node traffic through a Cloudflare WARP
  VPN tunnel double-encrypting every packet, hiding your ISP metadata, and providing network-wide DNS ad-
  blocking (AdGuard Home) and kernel-level IP threat blocking (`ipset`/`iptables`) for every device on your
  mesh.
6
7 ---
8
9 ## 1. Prerequisites
10 - Docker and Docker Compose installed.
11 - A Tailscale account.
12 - **macOS:** Install the standalone Tailscale CLI via Homebrew (`brew install tailscale`) and register `
  tailscaled` as a system service (`brew services start tailscale`). No `.app` GUI install required.
13
14 ## 2. Installation & Setup
15
16 Run the interactive setup script for your OS. It downloads `wgcf`, generates fresh Cloudflare WARP WireGuard
  keys, prompts for a Tailscale Auth Key, and writes your `.env`.
17
18 ```bash
19 ./setup.sh # macOS
20 ./scripts/setup-linux.sh # Linux
21 ```
22
23 ### Reference `.env`
24 ```env
25 WIREGUARD_PRIVATE_KEY=<Generated>
26 WIREGUARD_PUBLIC_KEY=<Generated>
27 WIREGUARD_ADDRESSES=<Generated>
28 TS_AUTHKEY=tskey-auth-...
29 GATEWAY_RULE_PROFILE=medium # light | medium | heavy
30 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds and
  have a healthy path.
31 GATEWAY_MSS=1120
32 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
  internet is NOT hijacked).
33 GATEWAY_HIJACK_HOST=true
34 GATEWAY_USE_EXIT_NODE=true
35 WARP_FAIL_THRESHOLD=6 # consecutive warp=off polls before auto-shutdown (default 6 = 30s)
36 KILL_SWITCH=false # enforce strict PF lock that breaks SSH if VPN fails
37 # On campus/enterprise networks (WPA2-Enterprise / 802.1x), AP Client Isolation blocks direct
38 # LAN traffic between devices. Tailscale attempts a local P2P upgrade anyway, which causes
39 # macOS gvproxy to SNAT-mangle the reply's source port poisoning the phone's WireGuard
40 # endpoint and blackholing 100% of its traffic. Setting this to false drops those local
41 # hole-punch packets so the phone safely falls back to Tailscale DERP relays.
42 # watcher.sh auto-detects 802.1x and AP isolation on every Wi-Fi change and overrides
43 # this setting automatically you only need to set this manually if auto-detection fails.
44 TAILSCALE_ALLOW_LAN_P2P=false # auto-managed; true only on trusted hotspots/home networks
45 ADGUARD_PASSWORD=nullexit
46 BLOCKED_COUNTRIES="kp il" # dynamically block country IP ranges via ipdeny.com
47 ```
48
49 ## 3. Deploy the Gateway
50
51 **macOS (Recommended):**
52 ```bash
53 ./toggle.sh
54 ```
55 Running it again stops the gateway and restores your network. Handles the full lifecycle: Colima VM, containers
  , DNS hijacking, Tailscale exit-node routing, rule compilation, and sleep prevention.
56
57 **Linux / Manual:**
58 ```bash
59 docker compose up -d
60 ```
61
62 ## 4. Post-Deployment
63
64 ### Approve the Exit Node
65 1. Go to [Tailscale Admin Machines](https://login.tailscale.com/admin/machines).
66 2. Find the `tailscale` device `...` **Edit route settings** enable as Exit Node.
67
68 ### Verify Traffic Flow
```

```

69 On any Tailscale client device, select this gateway as the exit node, then visit [api.ipify.org](https://api.
ipify.org) or run:
70 ```bash
71 curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
72 # expect: warp=on
73 ```
74
75 ### AdGuard DNS Setup
76
77 To ensure client devices correctly route DNS through the AdGuard container (and do not bypass it via `
tailscaled`'s internal resolver), you must configure your Tailscale Admin Console:
78 1. Go to the DNS tab in the [Tailscale Admin Console](https://login.tailscale.com/admin/dns).
79 2. Enable MagicDNS.
80 3. Under Nameservers, add a Custom nameserver and enter the gateway's Tailscale IPv4 address (run `
tailscale ip -4` on the gateway machine to find it, e.g., `100.x.x.x`).
81 4. Click the three dots (...) next to the nameserver you just added and enable "Use with exit node".
82 5. Delete any other Global Nameservers (like Google or Cloudflare).
83 6. Toggle ON Override local DNS.
84
85 Traffic will now be correctly forced into the AdGuard sinkhole, and mobile devices will be blocked from
bypassing it via encrypted DNS (DoH/DoT).
86
87 > [!WARNING]
88 > Disable "Private DNS" on mobile devices. Android's Private DNS (DoT/DoH) bypasses port 53 interception
entirely. Go to `Settings Connections More connection settings Private DNS Off`.
89
90 > [!WARNING]
91 > Bandwidth doubling: Streaming 1GB of video on a remote device consumes 1GB download + 1GB upload on the
host's network. Be mindful on metered connections.
92
93 ### AdGuard Dashboard
94 Navigate to `http://<gateway-tailscale-ip>:3000` credentials: admin / nullexit.
95
96 ### Access Any Mesh Device From Anywhere (SFTP/SSH)
97
98 Once the gateway exit node is online, any device on your Tailscale mesh can reach any other mesh device,
anywhere in the world as long as both devices are on the mesh and the target device has an SSH server
running. No port forwarding, no public IPs, no firewall holes. You can SSH into any device using its
MagicDNS hostname (e.g. `ssh username@my-macbook`) or its assigned Tailscale IP (`100.x.x.x`).
99
100 ##### Example: Browse your Android phone's files from your Mac
101
102 1. On your Android phone, install [Termux](https://termux.dev/) and [Termux:Boot](https://f-droid.org/
packages/com.termux.boot/) from F-Droid.
103 2. In Termux, install and start the SSH server:
104 ```bash
105 pkg install openssh
106 sshd
107 ```
108 Termux defaults to port 8022 (ports below 1024 require root on Android).
109 3. Find your phone's Tailscale IP:
110 ```bash
111 tailscale ip -4 # e.g. 100.87.42.87
112 ```
113 4. Stop Android from killing Termux do ALL of these (Samsung One UI is especially aggressive):
114 - Settings Apps Termux Battery Unrestricted
115 - Device Care Battery Background usage limits Sleeping apps / Deep sleeping apps remove Termux if
listed
116 - In Termux, run `termux-wake-lock` to hold a persistent wakelock
117 5. On your Mac/PC, open [Cyberduck](https://cyberduck.io/) (or any SFTP client: WinSCP, FileZilla, FE File
Explorer on iOS) and connect to:
118 - Server: `100.87.42.87` (your phone's Tailscale IP)
119 - Port: `8022`
120 - Protocol: SFTP (SSH File Transfer Protocol)
121 - Username: (your Termux username, usually `u0_a123` run `whoami` in Termux to check)
122 - Password: (your Termux password set with `passwd` in Termux if you haven't already)
123
124 That's it you can now browse, download, and upload files to your phone from any device on the mesh, no matter
where either device is physically located. This works identically for any mesh device: Mac Mac, Windows
Linux, phone NAS, etc.
125
126 > Troubleshooting: If the connection hangs for ~30 seconds before failing, your phone's SSH server isn't
running. Open Termux and run `sshd` again. If Android keeps killing it despite Unrestricted battery, the `
termux-wake-lock` command is your fix.
127
128 ---
129
130 ## 5. Multi-Layer Firewall & Kill-Switch
131
132 ### Layer 1 DNS Sinkhole (AdGuard Home)
133 Blocks ads and tracking domains before they are downloaded. In automated Lighthouse tests on ad-heavy sites:

```

```

134 - **Time to Interactive:** ~22% faster (51.0s 41.8s)
135 - **Speed Index:** ~20% faster (15.3s 12.8s)
136
137 Customize blocking via `black_list.txt` and `white_list.txt`. Rule profiles (`light` / `medium` / `heavy`) are
    set in `.env`.
138
139 ### Layer 2 Kernel IP Firewall (`ipset`/`iptables`)
140 On every startup, the `rule-compiler` fetches threat-intelligence feeds (Spamhaus DROP, Feodo Tracker, Emerging
    Threats, CINS) and compiles **~16,700 unique malicious IPs/CIDRs** into the kernel `FORWARD` chain
    blocking both outbound C2 connections and inbound attack traffic. Zero configuration required.
141
142 ### Layer 3 Geo-IP Blocking
143 Dynamically blocks all traffic to and from specific countries using live IP ranges from `ipdeny.com`.
144 To add countries to your blocklist, open your `.env` file and set the `BLOCKED_COUNTRIES` variable with 2-
    letter ISO country codes (e.g. `BLOCKED_COUNTRIES="kp il cn ru"`), then restart the gateway.
145
146 ### Layer 4 macOS PF Kill-Switch
147 A native macOS Packet Filter (`pf`) kill-switch that enforces a strict default-deny policy on your physical Wi-
    Fi interface (`en0`). When `KILL_SWITCH=true` is set in your `.env`, it drops all outgoing traffic except
    the Cloudflare WARP endpoints and Tailscale DERP relays.
148
149 - **Failsafe Design:** If the VPN tunnel crashes or Docker dies, your Mac completely loses internet access
    rather than leaking your real IP.
150 - **Remote-Access Safe:** Carefully designed to whitelist Tailscale's control plane (`192.200.0.0/24`), `utun*`
    tunnel traffic, and local LAN subnets. This guarantees that your remote SSH sessions and local AirDrop
    will survive even when the kill switch engages.
151 - **Setup Requirement:** The toggle scripts need permission to manipulate the network and firewall in the
    background without prompting you for a password. You must configure your passwordless sudo config by
    running exactly:
152 ```bash
153 echo "$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/bin/killall,
    /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/brew, /usr/local/
    bin/brew, /usr/bin/python3 $PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 $PWD/scripts/dns-proxy.py,
    /usr/local/bin/python3 $PWD/scripts/dns-proxy.py" | sudo tee /etc/sudoers.d/nullexit
154 ```
155
156 ---
157
158 ## 6. Toggle Scripts
159
160 ### macOS `toggle.sh`
161 Fully automated lifecycle script. Also supports a native `.app` launcher:
162 ```bash
163 osacompile -o "Toggle Gateway.app" Toggle-Gateway.applescript
164 ```
165
166 Optional `.env` settings:
167 - `GATEWAY_BYPASS_PING=true` proceed even if pre-flight connectivity checks fail.
168 - `GATEWAY_USE_EXIT_NODE=false` DNS-only mode, skip exit-node routing.
169 - `GATEWAY_HIJACK_HOST=false` skip DNS hijacking on the host (provides VPN/adbblocking only to external
    Tailscale peers).
170 - `GATEWAY_MSS=1360` override TCP Maximum Segment Size for the tunnel (default is calculated automatically).
171 - `HOST_LEAK_PROBE=false` disable background host-egress leak probe (default: true).
172 - `HOST_MTU=1200` override the host Tailscale interface MTU. Defaults to 1200 to fit inside the 1280-byte WARP
    tunnel.
173 - `KILL_SWITCH=true` enforce strict network lock that breaks SSH if the VPN fails.
174 - `STOP_COLIMA_ON_EXIT=true` fully shut down the Colima VM on toggle-off (saves battery on dedicated hosts).
175 - `TAILSCALE_ALLOW_LAN_P2P=true` allow direct Tailscale LAN P2P connections between devices on the same
    network.
176 - **Default: `false`** On campus or enterprise Wi-Fi (WPA2-Enterprise / 802.1x), AP Client Isolation blocks
    local device-to-device traffic. When Tailscale still attempts a local P2P upgrade, macOS's `gvproxy` layer
    SNAT-mangles the reply's source port. WireGuard's endpoint roaming treats this as a legitimate address
    change and updates the phone's outbound path to the newly-mangled port which has no listener. This
    blackholes 100% of the phone's traffic while the DERP relay path is abandoned. Setting this to `false`
    preemptively drops all RFC1918-destined Tailscale UDP from the gateway so the phone receives a clean
    failure and safely falls back to DERP.
177 - **Auto-managed:** `watcher.sh` automatically detects WPA2-Enterprise via `system_profiler SPAirPortDataType`
    (the `airport` binary was removed in macOS 14.4) and probes for AP isolation by pinging the default
    gateway (`route -n get 1.1.1.1`) on every network change. It writes the detected value to `
    lan_p2p_detected` in the repo root, which `routing-fix.sh` re-reads every 30 seconds **no restart required
    ** Only set this manually if auto-detection fails. See `devref.md 36` for the full SNAT endpoint poisoning
    analysis and `devref.md 40` for the `airport` removal background. *(Note: Direct P2P between the gateway
    container and the host Mac itself is always permitted via the Docker bridge to bypass DERP relays,
    regardless of this setting).*
178 - Set to `true` on trusted home networks or Windows/mobile hotspots (no AP isolation) for a faster, lower-
    latency direct path.
179 - `WARP_FAIL_THRESHOLD=3` number of consecutive failed checks before forcing a gateway teardown (default: 6
    checks = 30s).
180 - `WARP_ENDPOINT_1` / `WARP_ENDPOINT_2` override the Cloudflare WARP WireGuard endpoints.
181
182 ### Linux `scripts/toggle-linux.sh`

```

```

183 ```bash
184 ./scripts/toggle-linux.sh # toggle ON/OFF
185 ./scripts/recover-linux.sh # recovery tool
186 ```
187
188 ### Sleep Prevention (macOS)
189 When the gateway is ON, `caffeinate -i` runs in the background to prevent idle system sleep, keeping Docker and
    all services alive even on battery. The display can still sleep normally.
190
191 ### Auto-Recovery (macOS post-wake / post-roam)
192 Install the launchd LaunchAgent so the gateway self-heals after sleep/wake and Wi-Fi roams:
193 ```bash
194 cp ./launchd/com.nulllexit.wake-recovery.plist ~/Library/LaunchAgents/
195 sed -i '' "s|__NULLEXIT_HOME_|$(pwd)|" ~/Library/LaunchAgents/com.nulllexit.wake-recovery.plist
196 launchctl load -w ~/Library/LaunchAgents/com.nulllexit.wake-recovery.plist
197 ```
198 See `devref.md 11.16` for the full deep dive.
199
200 ---
201
202 ## 7. Networking Notes
203
204 - **Port conflicts:** None. Tailscale uses dynamic UDP ports; WARP uses outbound UDP:2408. The gateway binds
    `0.0.0.0:41642/udp` on the host for Tailscale's WireGuard listener this is intentional and required to
    receive inbound hole-punch packets from peers on the internet. All other internal ports (AdGuard :5335,
    SOCKS5 :1080) are bound to `127.0.0.1` only and are not reachable from the LAN.
205 - **Tailscale P2P vs DERP:** On the same Wi-Fi, Tailscale establishes a fast P2P connection. On cellular (CGNAT
    ), it always falls back to a DERP relay (~80-200ms). See `devref.md 5.1`.
206 - **AirDrop / AirPlay:** Unaffected the gateway only touches standard Wi-Fi/Ethernet interfaces, not `awdl0`.
207 - **VPN coexistence:** Do **not** run a local VPN client (WARP, Mullvad, NordVPN) simultaneously with the exit
    node. Both fight for the default route and will blackhole your connection.
208 - **Banking & financial sites:** May intermittently block logins through WARP. Banks use anti-fraud databases
    that flag datacenter IP ranges (like Cloudflare's `104.28.x.x`) as VPN/proxy traffic. Success fluctuates
    depending on which WARP IP you land on. If a site blocks you, either disable the exit node temporarily for
    that session or use the bank's mobile app (which often uses different fraud detection).
209 - **MSS clamping (Bufferbloat):** Double-tunneling adds ~120-160 bytes of overhead per packet. This is now
    automatically handled natively via the macOS `pf` firewall, which universally applies TCP MSS Clamping to
    all outbound packets to strictly prevent MTU fragmentation and bufferbloat. No manual `.env` configuration
    is required. Additionally, the host's Tailscale interface MTU is explicitly lowered to 1200 (configurable
    via `HOST_MTU` in `.env`) to guarantee packets fit inside the container's WARP tunnel without fragmenting.
210
211 ---
212
213 ## 8. Privacy & Security Hardening
214
215 The stack shifts trust across four layers: WARP hides traffic from your ISP, AdGuard blocks tracking at DNS,
    Tailscale encrypts device-to-device, and `ipset` blocks known malicious IPs at the kernel.
216
217 For higher security, the gateway supports these drop-in upgrades:
218
219 | Layer | Default | Hardened |
220 |---|---|---|
221 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, no-logs, anonymous payment) |
222 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH |
223 | Coordination | Tailscale (US) | Headscale (self-hosted) |
224 | Auth | Persistent OAuth keys | Ephemeral auth keys |
225 | Content | WireGuard asymmetric | WireGuard + Pre-Shared Keys (PSK) |
226
227 See `devref.md 8` for the full threat model, Snowden programme analysis, and trust assumptions.
228
229 ---
230
231 ## 9. Debugging
232
233 - **`output.log`** All stderr from `toggle.sh`, `recover.sh`, `setup.sh`, and the rule-compiler is written
    here. The WARP Watcher (`WARP_FAIL_THRESHOLD`) and the Host Leak Probe (`HOST_LEAK_PROBE`) also write all
    their events here `LEAK`, `ROTATE`, `HOST-PROBE failed`, and WARP shutdown notices. Check this first.
234 - **`bash scripts/diagnose-host-leak.sh`** One-shot host-routing diagnostic. Runs 8 checks and classifies your
    state into one of four known scenarios (System Extension conflict, SOCKS5 fallback, IPv6 leak, route-table
    freeze) or OK, and prints the exact fix command. Check 5 uses `route -n get 1.1.1.1` to ask the kernel
    which interface real traffic actually resolves through (more accurate than `netstat -rn | grep default` on
    macOS). Pass `--fix` to apply the remediation automatically. Pass `--watch` (or `--watch 30`) to run a full
    baseline diagnostic then continuously monitor warp/IPv6/default-route for leaks, alerting on any state
    change.
235 - **`bash scripts/host-leak-probe.sh`** Sub-second host-egress prober (enabled automatically via `
    HOST_LEAK_PROBE=true` in `.env`). Polls `cdn-cgi/trace` every 300ms directly from the host NIC not via `
    docker exec` so it catches flash-leaks invisible to the in-container WARP Watcher. All state changes, curl
    errors, and probe timeouts land in `output.log`. Grep with `grep LEAK output.log`.
236 - **`bash scripts/fix-docker-bridge-collision.sh`** One-shot fix for Docker bridge IP conflicts. If your local
    Wi-Fi router assigns you an IP in the `172.17.0.0/16` range, it will violently collide with Docker's
    default internal bridge, permanently killing your internet. This script detects the collision and auto-

```

```

patches your Colima VM to use a safe subnet (e.g. `10.200.0.1/24`).
237 - devref.md Complete architecture reference, routing deep-dives, and full resolved-issues log. Read this
      before modifying any routing or firewall logic.
238
239 > [!CAUTION]
240 > Two infinite routing loops are possible. (1) If a hotspot device providing internet to the gateway host
      is also using the gateway as its exit node, packets bounce forever between the two. (2) When the host's
      default route is redirected to utun*, WARP endpoint packets can loop back into the tunnel. Both are
      documented in devref.md 11.10 and 11.18 and are automatically handled by toggle.sh.
241
242 ---
243
244 ## 10. Acknowledgements
245 - SyameimaruKoa(https://github.com/SyameimaruKoa): Dual-stack TCP MSS clamping, SIGHUP state-tracking
      in the routing sidecar, and TS_AUTH_ONCE integration.
246
247 ## 11. License
248 GNU Affero General Public License v3. See [LICENSE](./LICENSE).
249
250 ## 12. Changelog
251
252 - July 10, 2026 Bug fixes:
253 1. Resolved a race condition where Wi-Fi roaming caused the host IP to leak as the raw ISP IP (132.x) after
      every network switch. The WARP Watcher was firing nuclear shutdown while recover.sh --post-wake was
      still intentionally recreating the warp container. Fixed by adding a /tmp/nullexit-warp-inhibit.marker
      that pauses the watcher's failure counter during post-wake recovery. See devref.md 25.
254 2. Resolved a startup race condition where the pre-flight connectivity check [1/3] Gateway reachable via
      Tailscale would fail because the host's tailscaled had not yet propagated the netmap immediately after tailscale up --reset.
      Upgraded the check to a 5-attempt retry loop with a 1-second delay. See devref.md 26.
255 3. Resolved a concurrency collision where the network changes from a nuclear recover.sh teardown triggered
      watcher.sh to run recover.sh --post-wake in parallel, corrupting container states. Implemented a shared
      lifecycle lock (/tmp/nullexit-toggle.lock) across both scripts to safely skip post-wake logic during
      teardowns. See devref.md 27.
256 4. Resolved an infinite auto-recovery loop after a tunnel failure shutdown. When the WARP Watcher triggered
      nuclear recover.sh to teardown the gateway, the active-state marker file /tmp/nullexit-gateway-active.
      marker was leaked on disk. The resulting network configuration changes from the teardown then triggered 
      watcher.sh, which saw the marker and immediately restarted the gateway, looping indefinitely. Fixed by
      explicitly deleting the marker file at the start of nuclear recover.sh. See devref.md 28.
257 5. Resolved a pre-flight check false negative during cold boots where the [1/3] Gateway reachable via
      Tailscale check would fail. Because gluetun blocks UDP STUN traffic to the public internet, Tailscale is
      forced to use a slower DERP-assisted handshake which takes ~30-35 seconds on cold boots. Increased the
      retry loop limit to 45 attempts (45s) to give the handshake sufficient time to complete. See devref.md
      29.
258 6. Resolved a startup race condition during restarts where docker compose build would fail with DNS
      resolution errors because physical Wi-Fi was power-cycled during teardown and Colima booted before the host
      obtained a DHCP lease. Implemented a unified wait_for_dhcp_settle helper (configured with a 30s timeout
      to tolerate macOS DHCP lease latency) in scripts/common.sh and added it to the start of toggle.sh. See
      devref.md 30.
259 7. Resolved a bug where enabling the PF Kill-Switch during SOCKS5 fallback mode blocked all host traffic (
      including tailscaled daemon connections to the control plane) because SOCKS5 does not route non-TCP/
      system traffic. Removed kill-switch activation from the SOCKS5 fallback path in toggle.sh. See devref.md
      31.
260 8. Resolved a bug where host-side tailscale up calls passed the --reset flag, which aggressively cleared
      macOS Tailscale credentials and logged the user out. Removed the --reset flag from all host-side
      invocations to preserve authenticated sessions during exit-node state toggling. See devref.md 32.
261 9. Resolved a bug where container connectivity check failures in toggle.sh and recover.sh were discarded
      to /dev/null. Redirected stderr of Docker exec checks to output.log to prevent diagnostic blind spots
      during tunnel outages. See devref.md 33.
262 - July 6, 2026 Edge-case security and stability hardening: DNS Watcher interface independence, robust lock
      -file PID verification, fail-closed crypto integrity check, strict iptables pinning for tailscale0tun0,
      Docker port binding to 127.0.0.1 to prevent LAN exposure, routing verification via netstat -nr. See 
      devref.md 12.
263 - July 4, 2026 Colima bridged networking, SOCKS5 proxy migrated natively to Tailscale, headless prompt
      crashes fixed, proxy bypass domains added to fix LAN P2P. Python dependencies completely eliminated: 
      logger and rule-compiler ported to blazing-fast Go multi-stage Docker builds. Added crypto.sh to
      enforce cryptographic signature validation on all core bash scripts. Massively refactored toggle.sh and 
      recover.sh to eliminate ~200 lines of duplicated code by migrating network and process logic to scripts/
      common.sh. Added a concurrency mutex to toggle.sh, resolved unbound TS_AUTHKEY check crashes on setup,
      and integrated Go validation to the CI pipeline.
264 - July 3, 2026 Critical Security Updates: Addressed the overnight silent IP leak and improved geo-
      blocking. Introduced scripts/host-leak-probe.sh (a sub-second host-egress leak detector) and a
      statistical threshold auto-shutdown watcher. Added unlock-files.sh to safely reset file permissions.
      Resolved numerous startup regressions and system bugs.
265 - July 2, 2026 Two infinite routing loops resolved (devref.md 11.18): host-VM tunnel loop (WARP bypass
      routes now via gateway IP + manual utun* redirection) and Docker Compose subnet takeover (10.200.1.0/24
      lock). --accept-routes=true now explicit on all tailscale up --reset calls.
266 - July 2, 2026 Auto-recovery daemon documented in 6 above (scripts/watcher.sh + launchd plist). See 
      devref.md 11.16.
267 - July 2, 2026 Linux scripts (scripts/toggle-linux.sh, recover-linux.sh, setup-linux.sh) documented
      in 6.

```



```

268 - **July 1, 2026** `recover.sh --post-wake` ships; wired to watcher daemon. Triggered on every sleep/wake or
    network-state change.
269 - **June 28, 2026** 8 internal scripts moved to `scripts/`. User-facing files (`toggle.sh`, `recover.sh`, `
    setup.sh`, `docker-compose.yml`) stay at repo root.
270 - **June 28, 2026** All hardcoded install paths removed. `SCRIPT_DIR`-relative resolution and `
    __NULLEXIT_HOME__` placeholder used throughout.
271
272 ### Cryptographic Integrity Verification
273
274 nullexit uses a built-in cryptographic integrity checker designed to provide tamper-*evidence* against
    accidental edits, logic bugs, or non-privileged malware. During setup, a 256-bit `NULLEXIT_SEED` is
    injected into your `.env` file. The core endpoint scripts (`toggle.sh`, `recover.sh`, etc.) are hashed
    using HMAC-SHA256, and their signatures are stored in `.signatures`.
275
276 *(Note: This is not a strict boundary against an attacker who already has write access to the repo, as they
    could simply read the seed and re-sign the scripts. It is designed as a strict footgun-catcher).*
277
278 If any script is modified without authorization, the gateway will loudly fail to boot. If you make intentional
    edits to the bash scripts, you must re-sign them by running:
279 ```bash
280 ./scripts/crypto.sh --sign
281 ```

```

3 agent.md

```

1 # nullexit Detailed Agent Analysis
2
3 > Complete per-file breakdown of every function, constant, duplication, and bug found.
4 > **Note (July 2026):** `sync-rules.py` and `logger.py` have been fully ported to Go (`scripts/rule-compiler/
    main.go` and `scripts/logger/main.go`) to dramatically improve performance and reduce container footprint
    via multi-stage Alpine builds. The analysis below reflects their previous Python states.
5
6 ## Important Agent Instruction
7 - **NEVER use `cat << EOF` or terminal redirections to write or modify files.** You must always use your native
    file-editing tools (`replace_file_content` or `multi_replace_file_content`). Using `cat EOF` leads to
    duplicated text, broken markdown headers, and structural corruption (which previously destroyed the section
    numbering in `devref.md`).
8 - **Always run `bash scripts/crypto.sh --sign` after making any modifications to the core bash scripts.** This
    is required to update the cryptographic HMAC-SHA256 signatures; otherwise, the startup integrity checks
    will fail and block execution.
9 - **Always run `git diff` and print the changes to the user *before* requesting or executing any git commit
    command.** The user must be shown the diff so they can review and understand the changes. Based on this
    diff, formulate a highly precise, detailed, and descriptive commit message (explaining exactly what was
    changed and why) rather than a generic summary, and present it to the user alongside the diff.
10 - **Always use the `--restart` flag when asked to restart the toggle (e.g., run `./toggle.sh --restart`).**
11 - **NEVER execute any `sudo` commands (especially `sudo route`, `sudo dscacheutil`, or any network-modifying
    commands) without explicitly explaining to the user what the command does and why it is needed FIRST. Do
    not assume implicit permission. Wait for the user's approval before running it.**
12 - **When fixing an error and using logs to debug, always show the user the reasoning and the specific logs that
    support this theory.**
13 - **When the user tells you to "push", this means read git diffs and prepare commit messages that correspond to
    these changes (do not ignore any changes). If the changes can be atomized into multiple commits for easier
    understanding, do so.**
14 - **When the user says "latex this", it means you should run `python3 scripts/generate_tex.py` to regenerate
    the unified LaTeX document.**
15
16 ## How to Verify Gateway is Working
17 Whenever modifications are made to the gateway scripts, routing, or containers, execute these verification
    checks in order:
18 1. **Container Health:** Run `docker compose ps` to ensure all containers (`warp`, `adguardhome`, `routing-fix
    `, `tailscale`) are `running` and `healthy`.
19 2. **DNS Hijacking Status:** Run `networksetup -getdnsservers "Wi-Fi"` (or appropriate network service) and
    ensure it is set exclusively to the gateway's Tailscale static IP (typically `100.100.21.8`).
20 3. **DNS Query Interception:** Run `dig @100.100.21.8 google.com` (using the gateway static IP from `ADGUARD_IP
    .txt`) to ensure AdGuard Home is active, intercepting, and resolving queries.
21 4. **Double-Tunnel Egress:** Run `curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp` to verify the
    egress is routed through Cloudflare WARP (`warp=on`).
22 5. **Host Leak Monitoring:** Run `tail -n 30 output.log` and verify the background watchers and host leak
    probe are reporting healthy status without `LEAK` warnings.
23
24 ---
25
26 ## Part 1: macOS Main Scripts
27
28 ### toggle.sh (1281 lines, 56KB)
29

```

```

30 **Functions (27 total):**
31
32 | # | Function | Lines | Purpose |
33 |---|-----|-----|-----|
34 | 1 | `run_with_timeout()` | L3064 | Run a command with a timeout watchdog; kills after N seconds |
35 | 2 | `run_gui_cmd()` | L6880 | Run a GUI command as the logged-in console user |
36 | 3 | `write_gateway_active_marker()` | L8890 | Write timestamp to `/tmp/nullexit-gateway-active.marker` |
37 | 4 | `clear_gateway_active_marker()` | L9295 | Remove gateway active marker + TUNNEL_FAILED_CLOSED.marker |
38 | 5 | `start_sleep_prevention()` | L108154 | Start `caffeinate` in background with shutdown trap to revert DNS
39 | 6 | `stop_sleep_prevention()` | L157167 | Stop caffeinate process via PID file |
40 | 7 | `start_dns_watcher()` | L173191 | Background loop to re-hijack DNS every 30s for Wi-Fi roaming |
41 | 8 | `stop_dns_watcher()` | L193203 | Stop DNS watcher via PID file |
42 | 9 | `start_warp_watcher()` | L220279 | Background WARP liveness monitor; triggers recover.sh after N
43 | 10 | `stop_warp_watcher()` | L281291 | Stop WARP watcher via PID file |
44 | 11 | `start_host_leak_probe()` | L297310 | Start host-egress leak prober (300ms polling) |
45 | 12 | `stop_host_leak_probe()` | L312322 | Stop host leak probe via PID file |
46 | 13 | `cleanup_handler()` | L325382 | Error/signal handler: resets DNS, stops all watchers, tears down
47 | 14 | `restart_tailscaled_daemon()` | L449463 | Restart tailscaled via brew services (user then system
48 | 15 | `disconnect_tailscale_host()` | L465477 | Disconnect host Tailscale with retry on failure |
49 | 16 | `get_active_service()` | L480508 | Detect active macOS network service name (e.g., "Wi-Fi") |
50 | 17 | `add_warp_bypass_routes()` | L524546 | Add static routes for WARP endpoints (162.159.192.1,
51 | 18 | `remove_warp_bypass_routes()` | L548552 | Remove WARP bypass routes |
52 | 19 | `setup_exit_node_routing()` | L558577 | Override default route to point through Tailscale utun interface
53 | 20 | `reset_dns()` | L588595 | Reset DNS to 1.1.1.1 on ACTIVE_SERVICE + ENO_SERVICE |
54 | 21 | `cleanup_network_state()` | L600645 | Nuclear cleanup: disable proxies, flush DNS cache, flush routes,
55 | 22 | `enable_socks_proxy()` | L663682 | Enable system-wide SOCKS5 proxy on macOS |
56 | 23 | `disable_socks_proxy()` | L684689 | Disable SOCKS5 proxy |
57 | 24 | `stop_local_dns_proxy()` | L702710 | Kill local Python DNS proxy |
58 | 25 | `start_local_dns_proxy()` | L713768 | Start Python DNS proxy (UDP:53 TCP:5354) |
59 | 26 | `force_dns_to_gateway()` | L780820 | Force DNS to a specific IP with 3-retry verification loop |
60 | 27 | `is_gateway_active()` | L833854 | Check if gateway is running (containers, DNS, SOCKS proxy) |
61
62 **Constants:**
63
64 | Constant | Value | Line |
65 |-----|-----|-----|
66 | `LOG_FILE` | `$PWD/output.log` | L8 |
67 | `PID_FILE` | `/tmp/nullexit-caffeinate.pid` | L105 |
68 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L169 |
69 | `WARP_WATCHER_PID_FILE` | `/tmp/nullexit-warp-watcher.pid` | L170 |
70 | `HOST_LEAK_PROBE_PID_FILE` | `/tmp/nullexit-host-leak-probe.pid` | L171 |
71 | `SOCKS_PROXY_PORT` | `1080` | L661 |
72 | `PATH` | export | `/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH` | L396 |
73 | Gateway active marker | `/tmp/nullexit-gateway-active.marker` | L89 |
74 | TUNNEL_FAILED_CLOSED marker | `${SCRIPT_DIR}/TUNNEL_FAILED_CLOSED.marker` | L94 |
75 | WARP endpoints | `162.159.192.1`, `162.159.193.1` | L535-544 |
76 | DNS fallback | `1.1.1.1` | L592 |
77 | DNS search domain | `ts.net` | L794 |
78 | `LOCK_FILE` | `/tmp/nullexit-toggle.lock` | L62 |
79
80 ---
81
82 ### recover.sh (566 lines, 23KB)
83
84 **Functions (7 total):**
85
86 | # | Function | Lines | Purpose |
87 |---|-----|-----|-----|
88 | 1 | `step()` | L40 | Print bold step header |
89 | 2 | `ok()` | L41 | Print green success message |
90 | 3 | `warn()` | L42 | Print yellow warning message |
91 | 4 | `fail()` | L43 | Print red failure message |
92 | 5 | `die()` | L44 | Print red error and exit |
93 | 6 | `run_with_timeout()` | L5674 | Run command with timeout (bash-native, no GNU coreutils) |
94 | 7 | `restart_tailscaled_daemon()` | L118129 | Restart tailscaled daemon via brew services |
95
96 **Constants:**
97
98 | Constant | Value | Line |
99 |-----|-----|-----|
100 | Color codes | `RED`, `GREEN`, `YELLOW`, `BOLD`, `NC` | L37-38 |
101 | `SCRIPT_DIR` | `${dirname "${BASH_SOURCE[0]}"}` | L50 |
102 | `PATH` | export | `/opt/homebrew/bin:/usr/local/bin:$PATH` | L77 |
103 | `PID_FILE` | `/tmp/nullexit-caffeinate.pid` | L387 |

```

```

104 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L407 |
105 | `HOST_LEAK_PROBE_PID_FILE` | `/tmp/nullexit-host-leak-probe.pid` | L423 |
106 | WARP endpoints | `162.159.192.1`, `162.159.193.1` | L329-336 |
107 | TUNNEL_FAILED_CLOSED marker | `${SCRIPT_DIR}/TUNNEL_FAILED_CLOSED.marker` | L51 |
108
109 **Duplicated logic from toggle.sh:**
110 *(Note: As of July 2026, **ALL** of the below duplicated logic has been successfully extracted to `scripts/
    common.sh`!)*
111 - `run_with_timeout()` simpler subshell version vs toggle's PID-tracking version
112 - `restart_tailscaled_daemon()` near-identical (uses warn()/ok() vs echo)
113 - Active network service detection inline at L217-233 (toggle has `get_active_service()` function)
114 - ENO service resolution inline at L230-233 (same as toggle L515-516)
115 - WARP bypass routes inline at L329-337 (toggle has `add_warp_bypass_routes()` )
116 - Exit node routing setup inline at L340-345 (toggle has `setup_exit_node_routing()` )
117 - DNS cache flushing L296-302 (same as toggle L611-616)
118 - Wi-Fi power-cycling L442-461 (similar to toggle L626-637)
119 - Proxy disabling L282-288 (same as toggle L604-608)
120 - Sharing services reset L586 (same as toggle L642)
121 - PID file stop pattern L387-399, L407-419, L423-435 (same as toggle L157-167, L193-203, L312-322)
122 - KILL_SWITCH check L194 (same as toggle L141, L469)
123 - ADGUARD_IP.txt reading L138-142, L209-213 (same pattern as toggle L1080)
124
125 ---
126
127 ### setup.sh (410 lines, 18KB)
128
129 **Functions (4 total):**
130
131 | # | Function | Lines | Purpose |
132 | ---|-----|-----|-----|
133 | 1 | `step()` | L10 | Print bold blue step header |
134 | 2 | `ok()` | L11 | Print green success message |
135 | 3 | `warn()` | L12 | Print yellow warning message |
136 | 4 | `die()` | L13 | Print red error and exit |
137
138 **Constants:**
139
140 | Constant | Value | Line |
141 | ---|-----|-----|
142 | Color codes | `RED`, `GREEN`, `YELLOW`, `BLUE`, `BOLD`, `NC` | L7-8 |
143 | `RECOMMENDED_TS_VERSION` | `1.98.5` | L71 |
144 | `ADGUARD_PASSWORD` | `nullexit` | L215 |
145 | Default .env values | `GATEWAY_RULE_PROFILE=medium`, `GATEWAY_MSS=1120`, etc. | L240-248 |
146
147 ---
148
149 ## Part 2: Linux Scripts
150
151 ### scripts/toggle-linux.sh (931 lines, 39KB)
152
153 **Functions (17 total):**
154
155 | # | Function | Line | Purpose |
156 | ---|-----|-----|-----|
157 | 1 | `run_with_timeout()` | L19 | Runs command with timeout watchdog, kills on expiry |
158 | 2 | `run_gui_cmd()` | L57 | macOS code uses `stat -f '%Su' /dev/console`, DEAD CODE on Linux |
159 | 3 | `start_sleep_prevention()` | L74 | Uses `systemd-inhibit` (Linux) to block sleep |
160 | 4 | `stop_sleep_prevention()` | L116 | Kills sleep prevention process via PID file |
161 | 5 | `start_dns_watcher()` | L130 | Background daemon polling DNS every 30s, re-hijacks if changed |
162 | 6 | `stop_dns_watcher()` | L149 | Kills DNS watcher process via PID file |
163 | 7 | `cleanup_handler()` | L162 | Trap handler for ERR/INT/TERM/HUP restores DNS, kills bg processes |
164 | 8 | `disconnect_tailscale_host()` | L273 | Disconnects host Tailscale from mesh |
165 | 9 | `get_active_service()` | L283 | Gets active network interface via `ip route` (Linux) |
166 | 10 | `reset_dns()` | L304 | Restores DNS via `resolvectl revert` (Linux) |
167 | 11 | `cleanup_network_state()` | L314 | Flushes DNS cache, routes, power-cycles network interface |
168 | 12 | `enable_socks_proxy()` | L361 | Stub prints message that Linux global SOCKS is DE-specific |
169 | 13 | `disable_socks_proxy()` | L367 | Stub prints skip message |
170 | 14 | `stop_local_dns_proxy()` | L383 | Kills Python DNS proxy processes |
171 | 15 | `start_local_dns_proxy()` | L394 | Starts Python DNS proxy (UDP:53 TCP:5354) + hijacks DNS |
172 | 16 | `force_dns_to_gateway()` | L461 | 3-attempt loop to set + verify DNS via `resolvectl` |
173 | 17 | `is_gateway_active()` | L495 | Checks if containers running or DNS was hijacked |
174
175 **Shared constants with macOS toggle.sh:**
176
177 | Constant | Value | Both |
178 | ---|-----|-----|
179 | `SOCKS_PROXY_PORT` | `1080` | toggle-linux L359, toggle.sh L661 |
180 | `PID_FILE` | `/tmp/nullexit-cafeinate.pid` | toggle-linux L71, toggle.sh L105 |
181 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | toggle-linux L128, toggle.sh L169 |
182 | ARP threshold | `15` devices | toggle-linux L518, toggle.sh L863 |
183 | Colima memory | `0.6` (600MB) | toggle-linux L578, toggle.sh L928 |

```

```

184 | Colima swap | `400MB` | toggle-linux L587, toggle.sh L937 |
185 | Tailscale wait loop | `60 iterations 1s` | toggle-linux L613, toggle.sh L1010 |
186 | NoState stuck threshold | `40 consecutive` | toggle-linux L628, toggle.sh L1025 |
187
188 ---
189
190 ### scripts/recover-linux.sh (257 lines, 10KB)
191
192 **Functions (6 total):**
193
194 | # | Function | Line | Purpose |
195 |---|-----|-----|-----|
196 | 1 | `step()` | L24 | Print formatted step header |
197 | 2 | `ok()` | L25 | Print green success message |
198 | 3 | `warn()` | L26 | Print yellow warning message |
199 | 4 | `fail()` | L27 | Print red failure message |
200 | 5 | `die()` | L28 | Print red error + exit 1 |
201 | 6 | `run_with_timeout()` | L33 | Simplified timeout (immediate SIGKILL, no PID tracking) |
202
203 ---
204
205 ### scripts/setup-linux.sh (416 lines, 18KB)
206
207 **Functions (4 total):**
208
209 | # | Function | Line | Purpose |
210 |---|-----|-----|-----|
211 | 1 | `step()` | L10 | Print formatted step header |
212 | 2 | `ok()` | L11 | Print green success message |
213 | 3 | `warn()` | L12 | Print yellow warning message |
214 | 4 | `die()` | L13 | Print red error + exit 1 |
215
216 **~95% identical to macOS setup.sh.** Only differences:
217 1. Desktop shortcuts (L366-391 Linux `.desktop` files vs macOS `.app` via `osacompile`)
218 2. Final instructions text
219 3. .env template: macOS adds `GATEWAY_USE_EXIT_NODE`, `WARP_FAIL_THRESHOLD`, `HOST_LEAK_PROBE`, `KILL_SWITCH`
   Linux omits these
220
221 ---
222
223 ## Part 3: Utility Scripts
224
225 ### scripts/diagnose-host-leak.sh (722 lines, 38KB)
226
227 **Purpose:** Comprehensive host-routing diagnostic. 8 checks classify host IP leaks. Supports `--fix` and `--watch` modes.
228
229 **Duplicated logic:**
230 - `resolve_active_service()` identical pattern as toggle.sh, recover.sh, fix-docker-bridge-collision.sh
231 - `run_with_timeout()` reimplemented timeout
232 - Color codes RED/GREEN/YELLOW/BOLD/NC with tty detection
233 - WARP check via `cloudflare.com/cdn-cgi/trace`
234 - `export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"`
235 - ADGUARD_IP.txt reading pattern
236
237 ### scripts/fix-docker-bridge-collision.sh (402 lines, 18KB)
238
239 **Purpose:** Fixes Docker bridge subnet collisions with `10.200.1.0/24`.
240
241 **Duplicated logic:**
242 - `resolve_active_iface()` same interface detection as diagnose-host-leak.sh
243 - Color codes, step()/ok()/warn()/fail()/die() formatting helpers
244
245 ### scripts/fix-ssh-delay.sh (63 lines, 2.7KB)
246
247 **Purpose:** One-shot fix for ~20s SSH delay. Standalone, minimal duplication (uses hardcoded / instead of color functions).
248
249 ### scripts/host-leak-probe.sh (110 lines, 5.5KB)
250
251 **Purpose:** Sub-second host-egress leak prober at 300ms intervals.
252
253 **Duplicated logic:**
254 - WARP check via cdn-cgi/trace (same curl + awk as toggle.sh WARP Watcher)
255 - Color codes (hardcoded; status updates and warnings are conditioned on TTY detection to prevent log pollution)
256 - LOG_FILE = `$PROJECT_ROOT/output.log`
257
258 ### scripts/reinstall-host-tailscale.sh (740 lines, 31KB)
259
260 **Purpose:** Complete uninstall + reinstall of host Tailscale.

```

```

261
262 **Duplicated logic:**
263 - `with_timeout()` yet another bash timeout implementation (polling-based, different from diagnose and toggle
    versions)
264 - Color codes + logging functions
265 - System Extension detection logic (similar to diagnose-host-leak.sh)
266
267 ### scripts/routing-fix.sh (266 lines, 10.5KB)
268
269 **Purpose:** Runs INSIDE warp container. Maintains routing table 200, FORWARD rules, country blocking, IP
    blocklists, tunnel health checks.
270
271 **Duplicated logic:**
272 - (Fixed) WARP_ENDPOINT defaults are now centralized in common.sh
273 - WARP health check via cdn-cgi/trace (another instance)
274 - iptables dual-backend pattern (for-loop over `iptables iptables-legacy`)
275
276 ### scripts/watcher.sh (269 lines, 13KB)
277
278 **Purpose:** Long-running launchd daemon for post-wake/post-roam recovery.
279
280 **Key function `detect_lan_p2p_mode()`:**
281 Added July 10, 2026. Called on startup and on every network state change (Listener 2 NET: events). Determines
    whether the current Wi-Fi network safely allows direct Tailscale LAN P2P connections:
282 1. **WPA2-Enterprise check:** `system_profiler SPAirPortDataType` reads the `Security:` field for the current
    association. Enterprise = 802.1x = always AP-isolated sets `allow=false`.
283 2. **AP isolation probe:** `route -n get 1.1.1.1` finds default gateway IP, then `ping -c 3 -t 1 -q "$gw"` if
    0 responses on a non-empty network, AP isolation is active sets `allow=false`.
284 3. **Output:** Writes `true` or `false` to `.lan_p2p_detected` in the repo root.
285 4. **Consumer:** `routing-fix.sh` reads this file every 30 seconds and enforces/relaxes the RFC1918 DROP rule
    accordingly no restart required.
286
287 **Duplicated logic:**
288 - PATH export (similar to toggle.sh/recover.sh)
289 - Marker file pattern (`/tmp/nullexit-gateway-active.marker`)
290
291 ### scripts/dns-proxy.py (92 lines, 2.9KB) Clean, standalone
292
293 ### scripts/logger.py (144 lines, 5.9KB) Clean, standalone
294
295 ### scripts/sync-rules.py (666 lines, 29KB)
296
297 **Internal duplication:**
298 - `fetch_remote_domains()` and `fetch_remote_ips()` are structurally identical (fetch cache sanity check
    stale fallback). Should be a single `fetch_with_cache(url, parser_fn, cache_dir)`.
299 - `.env` parsing custom hand-rolled reader duplicates bash scripts' `grep/sed` approach
300 - AdGuard credentials `admin:nullexit` hardcoded (also in AdGuardHome.yaml as bcrypt hash)
301
302 ---
303
304 ## Part 4: Cross-Cutting Issues
305
306 ### Functions Identical Across macOS Linux
307
308 | Function | macOS toggle | macOS recover | macOS setup | Linux toggle | Linux recover | Linux setup |
309 |-----|-----|-----|-----|-----|-----|-----|
310 | `run_with_timeout()` | | | identical | identical | |
311 | `stop_local_dns_proxy()` | | | identical | | |
312 | `start_local_dns_proxy()` | | | ~95% similar | | |
313 | `is_gateway_active()` | | | ~80% similar | | |
314 | `step/ok/warn/die` | | | identical | identical |
315
316 ### Platform-Adapted Functions (same logic, different OS commands)
317
318 | Function | Linux Command | macOS Command |
319 |-----|-----|-----|
320 | `get_active_service()` | `ip route get 1.1.1.1 \| awk '{print $5}'` | `route get default` + `networksetup -
    listnetworkserviceorder` |
321 | `reset_dns()` | `resolvectl revert` | `networksetup -setdnsservers` |
322 | `cleanup_network_state()` | `ip link set dev down/up` | `networksetup -setairportpower off/on` + proxy
    disable |
323 | `force_dns_to_gateway()` | `resolvectl dns` + `resolvectl domain -ts.net` | `networksetup -setdnsservers` +
    `--setsearchdomains ts.net` |
324 | `start_sleep_prevention()` | `systemd-inhibit --what=sleep` | `caffeinate -i` |
325 | `enable/disable_socks_proxy()` | Stub (no-op on Linux) | `networksetup -setsocksfirewallproxy` |
326 | DNS cache flush | `resolvectl flush-caches` | `dscacheutil -flushcache` + `killall -HUP mDNSResponder` |
327
328 ### Features Missing From Linux Scripts
329
330 These exist in macOS toggle.sh but NOT in toggle-linux.sh:
331

```

```

332 1. `write_gateway_active_marker()` / `clear_gateway_active_marker()`
333 2. `restart_tailscaled_daemon()`
334 3. `start_warp_watcher()` / `stop_warp_watcher()` WARP liveness monitor
335 4. `start_host_leak_probe()` / `stop_host_leak_probe()` host egress leak prober
336 5. `add_warp_bypass_routes()` / `remove_warp_bypass_routes()` / `setup_exit_node_routing()`
337 6. `LOG_FILE` structured logging (timestamps to output.log)
338 7. MSS clamping injection (step 4c)
339 8. Rule count reporting
340 9. `--post-wake` mode in recover
341 10. Container teardown on failure in cleanup_handler
342
343 ---
344
345
346
347 ## Part 6: Constant Duplication Map
348
349 | Value | Files |
350 |-----|-----|
351 | `162.159.192.1` (WARP endpoint) | docker-compose.yml, routing-fix.sh (now dynamically loaded via .env/common.sh!) |
352 | `5335` (AdGuard DNS port) | docker-compose.yml, AdGuardHome.yaml, post-rules.txt |
353 | `5354` (mapped DNS port) | docker-compose.yml, dns-proxy.py |
354 | `41642` (Tailscale WireGuard) | docker-compose.yml, routing-fix.sh, post-rules.txt |
355 | `1080` (SOCKS5 port) | docker-compose.yml, toggle.sh |
356 | `1120` (MSS clamp) | post-rules.txt, .env |
357 | `admin:nullexit` (AdGuard creds) | AdGuardHome.yaml (bcrypt), sync-rules.py (plaintext) |
358 | `10.200.1.0/24` (Docker subnet) | (Fixed) no longer duplicated |
359 | `America/New_York` (timezone) | docker-compose.yml (4 services) |
360 | `/tmp/nullexit-caffeinate.pid` | (Fixed) centralized in common.sh |
361 | `/tmp/nullexit-dns-watcher.pid` | (Fixed) centralized in common.sh |
362 | `/tmp/nullexit-host-leak-probe.pid` | toggle.sh, recover.sh |
363 | `/opt/homebrew/bin:...` (PATH) | (Fixed) centralized in common.sh |
364
365 ---
366
367 ## Part 7: Refactoring Outcome (Implemented July 2026)
368
369 **Decision: "Scripts-only" architecture**
370 Instead of introducing a new `lib/` directory, all shared code was moved into the existing `scripts/` directory to keep the project hierarchy flat and simple.
371
372 ### 1. `scripts/common.sh` (288 lines, 10KB)
373 Extracts the fundamental primitives and networking logic used across orchestrator scripts:
374 - **Formatters:** `step()`, `ok()`, `warn()`, `fail()`, `die()`
375 - **Timeout Wrapper:** `run_with_timeout()` (Canonical version with PID tracking, graceful SIGTERM, and SIGKILL fallback)
376 - **Daemon Lifecycle:** `restart_tailscaled_daemon()`, `stop_pidfile_daemon()` (collapsed caffeinate, DNS watcher, WARP watcher, and host leak prober teardown logic)
377 - **Network Interface/Routing:** `get_active_service()`, `get_en0_service()`, `bounce_wifi_interfaces()`, `add_warp_bypass_routes()`, `remove_warp_bypass_routes()`
378 - **Proxy/DNS Cleanup:** `disable_all_proxies()`, `flush_dns_cache()`, `reset_sharing_services()`
379 - **Configuration Helpers:** `read_adguard_ip()`, `is_kill_switch_enabled()`, `get_warp_endpoint_1()`, `get_warp_endpoint_2()` (which centralizes the hardcoded 162.159.192.1 / .193.1 into .env logic)
380
381 ### 2. `scripts/setup-common.sh` (~350 lines)
382 Extracts the heavy, duplicated installation logic shared between `setup.sh` (macOS) and `scripts/setup-linux.sh`:
383 - Docker and Colima prerequisite checks
384 - `wgcf` binary download and WARP key generation
385 - `.env` configuration file generation
386 - Interactive Tailscale Auth Key and AdGuard Rule Profile prompts
387
388 ### Sourcing Pattern
389 - macOS root scripts (`toggle.sh`, `recover.sh`, `setup.sh`) source via:
390   `source "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/common.sh"`
391 - Linux sibling scripts (`scripts/toggle-linux.sh`, etc.) source via:
392   `source "$(dirname "${BASH_SOURCE[0]}")/common.sh"`

```

4 devref.md

```

1 # nullexit Development Reference & Resolved Issues
2
3 > **Last updated:** July 10, 2026
4 > **Purpose:** Provide any LLM or developer with complete project understanding, debugging history, and resolved issues so they can make informed changes without re-reading every file.

```

```

5 > **Diagrams:** See [diagrams.md](./diagrams.md) for system architecture, toggle.sh flowcharts, monitoring
   layer, traffic sequence, recover.sh decision tree, and the full failureself-healing map.
6
7 ---
8
9 ## 1. What Is nullexit?
10
11 nullexit is a **Tunnel-in-Tunnel VPN gateway** that chains **Tailscale** (mesh VPN) through **Cloudflare WARP**
   (exit VPN). It runs on a macOS host inside Docker containers managed by **Colima** (lightweight Linux VM).
   The goal: any device on the user's Tailscale mesh can use this gateway as an exit node, and all traffic
   exits through Cloudflare WARP achieving double encryption, ISP invisibility, and network-wide ad-blocking
   via **AdGuard Home**.
12
13 ### Traffic Flow
14 ```
15 Phone/Laptop  Tailscale mesh (WireGuard)  Mac host  Docker container
16   Tailscale decrypts  Gluetun re-encrypts  WARP tun0  Cloudflare  Internet
17   ```
18
19 ### SOCKS5 Failover Path (if exit node is unavailable)
20 ```
21 DNS:  Browser  host 127.0.0.1:53  Python proxy  TCP:5354  AdGuard  WARP DNS
22 TCP:  Apps  macOS SOCKS5 proxy (127.0.0.1:1080)  container  tun0  WARP  Internet
23   ```
24
25 ---
26
27 ## 2. Architecture
28
29 ### Containers (all share `warp`'s network namespace via `network_mode: service:warp`)
30 | Container | Image | Role |
31 |-----|-----|-----|
32 | `warp` | `qmcgaw/gluetun:v3.41.1` | Gluetun WireGuard client  Cloudflare WARP. Owns the network namespace.
   Strict firewall. |
33 | `tailscale` | `tailscale/tailscale:v1.98.4` | Advertises as exit node on the mesh. Also provides the built-in
   SOCKS5 proxy fallback via `TS_SOCKS5_SERVER=1080`. |
34 | `routing-fix` | `alpine:3.20` | Sidecar that maintains routing tables + iptables rules every 30 seconds. |
35 | `rule-compiler` | `golang:1.22-alpine` / `alpine:3.20` | One-shot startup container that compiles 500k+ DNS
   block rules in <2s and immediately exits. |
36 | `adguardhome` | `adguard/adguardhome:v0.107.77` | DNS sinkhole for ads/trackers. Listens on port 5335.
   Upstream DNS: `127.0.0.1:53` (through WARP). |
37
38 ### Port Mappings (on host via Colima)
39 | Host Port | Container Port | Protocol | Purpose |
40 |-----|-----|-----|-----|
41 | 5354 | 5335 | TCP+UDP | AdGuard DNS |
42 | (Tailscale IP):3000 | 3000 | TCP | AdGuard web UI (Internal to Mesh) |
43 | 41641 | 41641 | UDP | Tailscale WireGuard direct |
44 | 1080 | 1080 | TCP | SOCKS5 proxy |
45
46 ### Host Components
47 - **Colima VM** Linux VM running Docker (600MB RAM + 400MB swap)
48 - **tailscaled** Standalone Tailscale daemon via `brew install tailscale` + `brew services start tailscale` (
   NOT the GUI `.app`)
49 - **macOS network settings** DNS, SOCKS proxy, routing all manipulated by `toggle.sh`
50
51 ---
52
53 ## 3. Key Files
54
55 | File | Lines | Purpose |
56 |-----|-----|-----|
57 | `toggle.sh` | ~1265 | **Main script.** Detects state, toggles gateway ON/OFF. Handles DNS hijacking,
   Tailscale exit node, SOCKS proxy, sleep prevention, timeouts, cleanup. |
58 | `setup.sh` | 406 | **One-time setup.** Installs deps (Docker, Tailscale, wgcf), generates WARP keys, writes
   `.env`, configures AdGuard via API, starts containers. |
59 | `docker-compose.yml` | 194 | Service definitions for all 5 containers. |
60 | `scripts/routing-fix.sh` | 201 | Maintains routing tables (table 200 for SOCKS5, table 52 for CGNAT) and
   FORWARD iptables rules in both nftables and legacy backends. Loads and enforces the IP blocklist via `ipset`
   . Runs in a 30-second loop. |
61 | `post-rules.txt` | 18 | Gluetun iptables rules loaded at container start. FORWARD accept for tailscale0, NAT
   MASQUERADE on tun0, DNS redirect to port 5335, IPv6 drop, TCP MSS clamping. |
62
63 | `scripts/dns-proxy.py` | 91 | Local DNS proxy. UDP:53 TCP:5354 with proper 2-byte length prefix. Fallback
   when exit node is unavailable. |
64 | `scripts/rule-compiler/` | ~800 | Unified threat compiler (ported to Go from Python). **DNS pipeline:**
   fetches remote blocklists, deduplicates against AdGuard native filters, optimizes subdomains (~50%
   reduction), compiles to AdGuard syntax. **IP pipeline:** fetches threat-intel feeds (Spamhaus, Feodo, ET,
   CINS), strips private/Tailscale ranges, outputs atomic `ipset` restore file. Built dynamically in Docker
   multi-stage build. |

```



```

65 | **adguard/work/** | | **Bind-mounted container data folder. Off-limits from outside Docker READ-ONLY-EXCEPT-VIA-`sync-rules`. See README 6.** |
66 | `recover.sh` | -557 | Nuclear recovery script (gain: use `--post-wake` for non-destructive refresh after sleep/wake or Wi-Fi roam keeps the gateway live while still re-hijacking DNS + refreshing Tailscale exit-node + force-recreating warp if unhealthy). Resets DNS on ALL services, disables proxies, flushes routes, stops containers, kills sleep prevention, power-cycles Wi-Fi. Cryptographically verified. |
67 | `scripts/diagnose-host-leak.sh` | 580+ | **One-shot host-routing diagnostic.** Runs 7 checks (Tailscale, SOCKS5, CDN-cgi/trace, IPv6 leak, default route, output.log hints, gateway IP), classifies into ONE of three known leak scenarios (A=SOCKS5 fallback, B=IPv6 leak, C=route freeze) or OK, writes a timestamped report to `host-leak-diagnostic-<UTC>.txt`, and prints a ready-to-run fix on stdout. Pass `--fix` to apply the matched remediation and re-verify egress. Pass `--watch` (or `--watch 30`) to run a full baseline then continuously monitor warp/IPv6/default-route every N seconds, alerting on any state change. See 10.30. |
68 | `scripts/host-leak-probe.sh` | -110 | **Continuous sub-second host-egress prober.** Launched automatically by `toggle.sh` when `HOST_LEAK_PROBE=true` in `.env` (default). Polls `cdn-cgi/trace` every 300ms directly from the host NIC via `curl` not via `docker compose exec` so it detects flash-leaks invisible to the in-container WARP Watcher. Logs only on state change (`LEAK`, `ROTATE`, `HOST-PROBE failed/timeout`) to `output.log`. `curl errors` (`2>>output.log`) also land there. Stopped gracefully (`SIGTERM`) by `toggle.sh` `STOP` path, `cleanup_handler`, and `recover.sh`. See 10.30.1. |
69 | `scripts/fix-docker-bridge-collision.sh` | -290 | **One-shot fix for 9.13.** Detects Docker's `172.17.0.0/16` bridge colliding with the host Wi-Fi subnet (the symptom that produces `default 172.17.0.1 UGScg en0` in `netstat -rn`), picks a non-colliding `docker.bip` from a small candidate set, idempotently edits `~/colima/default/colima.yaml` with a timestamped backup, restarts Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer Docker's bridge, and re-runs `toggle.sh` + `scripts/diagnose-host-leak.sh`. Supports `--dry` and `--skip-toggle`. |
70 | `scripts/watcher.sh` | 194 | **Long-running post-wake + post-roam daemon.** Launched by launchd LaunchAgent; subscribes to `com.apple.powermanagement` events via `log stream` and to network-state changes via `scutil n.watch`. Both fire `recover.sh --post-wake`. Single-instance lock + `SCRIPT_DIR`-relative resolve of `RECOVER`. See 10.29. |
71 | `scripts/common.sh` | -40 | **Shared script primitives.** Centralizes formatted echo statements (`step`, `ok`, `warn`, `die`) and the safe background `run_with_timeout` execution wrapper. Sourced by all orchestrator scripts across macOS and Linux. |
72 | `scripts/setup-common.sh` | -350 | **Shared installation logic.** Centralizes logic for verifying Docker, installing Tailscale/wgcf, generating `.env`, and prompting for auth keys. Sourced by `setup.sh` and `setup-linux.sh`. |
73 | `scripts/toggle-linux.sh` | -900 | **Linux sibling of `toggle.sh`. Sources `common.sh`. Native Linux equivalents for every macOS-specific primitive `nmcli radio wifi off/on` instead of `networksetup -setairportpower`, `ip link set ... down/up` fallback, `systemd-resolved` wiring. Paired with `.desktop` shortcuts. |
74 | `scripts/recover-linux.sh` | -250 | **Linux sibling of `recover.sh`. Sources `common.sh`. Same nuclear semantics (flush, restart, verify) but using `nmcli` / `ip link` instead of macOS `networksetup`. |
75 | `scripts/setup-linux.sh` | -80 | **Linux sibling of `setup.sh`. Sources `setup-common.sh`. Distro detection (`apt/dnf/pacman`), installs Linux-specific dependencies, creates `.desktop` shortcuts. |
76 | `scripts/Toggle-Gateway.bat` | -300 | **Windows Toggle helper.** Moved from root to `scripts/`. Helps invoke the framework on Windows if applicable. |
77 | `scripts/reinstall-host-tailscale.sh` | -740 | **Nuke-and-reinstall tool for host Tailscale.** Completely purges Tailscale, its settings, macOS Background Items (`.btm` database), and stale System Extensions. Wipes ALL Login Items via `sftool reset-login-items` (requires sudo). Useful for crisis recovery but destructive to other login items. |
78 | `scripts/fix-ssh-delay.sh` | -100 | **Fixes SSH connection delays.** One-shot script to address SSH delays caused by DNS or routing issues, useful in crisis situations. |
79 | `scripts/logger/` | -100 | **Internal logger piped from `scripts/routing-fix.sh` inside the `warp` container. Ported to Go from Python. Built via Docker multi-stage build. |
80 | `black_list.txt` | 71 | Custom domains to block (ads, trackers, telemetry). Supports `$important` modifier. |
81 | `white_list.txt` | 222 | Domains to force-allow (YouTube, Apple services, etc.). Always wins over blocks. |
82 | `.env` | -14 | WARP WireGuard keys, Tailscale auth key, rule profile. **Contains secrets & NULLEXIT_SEED.** |
83 | `ADGUARD_IP.txt` | 1 | Static gateway Tailscale IP (fallback for dynamic resolution). |
84 | `scripts/unlock-files.sh` | -20 | **One-shot stale-permission fix.** Uses atomic rename (`cp` + `mv`) to replace locked inodes (mode `000`/`0444` from old `chmod` decisions) with fresh writable ones no `chmod` called. |
85 | `scripts/crypto.sh` | -50 | **Cryptographic integrity enforcement.** Uses `NULLEXIT_SEED` from `.env` to sign core bash scripts (HMAC-SHA256) and strictly verify them on start to prevent manipulation. Pass `--sign` or `--verify`. |
86 | `scripts/pf.conf` | 35 | **macOS native firewall ruleset.** Enforces the default-deny kill-switch, allows local LAN / Tailscale traffic, and applies TCP MSS Clamping (`max-mss 1160`) to prevent WireGuard MTU fragmentation. |
87 | ---
88 |
89 |
90 | ## 4. toggle.sh Flow
91 |
92 | ### State Detection
93 | `is_gateway_active`() returns true if EITHER containers are running OR host DNS is not `1.1.1.1`. This dual check prevents the script from starting when it should stop (e.g., containers crashed but DNS is still hijacked).
94 |
95 | ### START Path (gateway OFF ON)
96 | 1. **Reset DNS to 1.1.1.1** Prevents deadlocks during startup
97 | 2. **Disconnect host Tailscale** `tailscale down` (prevents exit-node routing during container boot)
98 | 3. **Compile DNS rules** `docker compose run --rm rule-compiler` (Go binary inside multi-stage Docker build)
99 | 4. **Boot Colima VM** `colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged` if not running

```



```

100 - **Why Bridged Networking?** The `--network-mode bridged` and `--network-address` flags are critical. By
    default, Colima creates an isolated network. Bridged mode forces the VM to receive its own IP address
    directly from your local router's DHCP server, placing it on the same physical subnet as your host machine.
    This bypasses macOS network isolation and is absolutely required for peer-to-peer (P2P) connections, mDNS,
    and local LAN service discovery to function properly across the container boundary.
101 5. **Configure VM swap** 400MB swap file inside the VM to prevent OOM
102 6. **Clean corrupted AdGuard config** Remove empty `AdGuardHome.yaml`
103 7. **Start containers** `docker compose up -d`
104 8. **Wait for gateway Tailscale** Poll `tailscale status` for "offers exit node" (up to 60s, abort at 40
    consecutive NoState)
105 9. **Resolve gateway IP** From `ADGUARD_IP.txt` or `docker compose exec tailscale tailscale ip -4`
106 10. **Connect host to mesh** Verify `tailscaled` is running (auto-start if needed), then `tailscale up --reset
    --ssh=true --accept-dns=false --accept-routes=true --exit-node=` ( `--accept-routes=true` must be explicit
    `--reset` reverts it to `false` otherwise, silently preventing the default route from switching to `utun
    *`)
107 11. **Pre-flight checks** [1/3] `tailscale ping` gateway, [2/3] `dig +tcp` AdGuard DNS, [3/3] WARP container
    internet
108 12. **If all pass** Set exit node + hijack DNS to gateway IP (single server, no fallback)
109 13. **If any fail** Enable SOCKS5 proxy + local DNS proxy as fallback
110 14. **Final DNS re-force** `force_dns_to_gateway` called again after exit-node transition (tailscaled clobbers
    DNS briefly)
111 15. **Enable sleep prevention** `caffeinate -i` in background to prevent idle sleep
112
113 ### STOP Path (gateway ON OFF)
114 1. Disconnect host Tailscale (`tailscale down`)
115 2. `docker compose down -t 5`
116 3. Reset DNS to 1.1.1.1
117 4. Stop local DNS proxy
118 5. **Stop sleep prevention** Kill caffeinate process
119 6. Full network state cleanup (proxies off, DNS cache flush, route flush, Wi-Fi power cycle)
120
121 ### Safety Mechanisms
122 - `run_with_timeout()` Pure-bash watchdog for every system command (15s/120s)
123 - `cleanup_handler()` Trap on ERR/INT/TERM/HUP restores DNS to 1.1.1.1, kills caffeinate, and tears down
    Tailscale
124 - `force_dns_to_gateway()` Sets DNS and VERIFIES via `networksetup -getdnsservers` (3 attempts with backoff)
125 - `SUCCESS_RUN` flag Cleanup only runs if script didn't complete successfully
126 - `start_sleep_prevention()` / `stop_sleep_prevention()` Manages `caffeinate -i` via PID file at `/tmp/
    nullexit-caffeinate.pid`
127
128 ---
129
130 ## 5. Routing & Firewall Architecture (Inside Container)
131
132 ### IP Rules (`ip rule show`)
133 | Priority | Match | Table | Purpose |
134 |-----|-----|-----|-----|
135 | 99 | `to 100.64.0.0/10` | 52 | CGNAT range Tailscale routes (injected by routing-fix) |
136 | 100 | `from 172.18.0.2` | 200 | Container's own traffic (SOCKS5 proxy) tun0 |
137 | 101 | all | 51820 | Gluetun's WireGuard fwmark tun0 |
138
139 ### Table 200 (SOCKS5 proxy traffic)
140 - `162.159.192.1 via $DOCKER_GW dev eth0` WARP endpoint exception (prevents tunnel loop)
141 - `default dev tun0` Everything else through WARP
142
143 ### iptables (TWO separate stacks!)
144 - **nftables backend** (`iptables`) Used by Gluetun's `post-rules.txt`. FORWARD policy DROP.
145 - **legacy backend** (`iptables-legacy`) Used by Tailscale's `ts-forward`. FORWARD policy ACCEPT.
146 - Both stacks apply to every packet. FORWARD RELATED,ESTABLISHED must exist in BOTH.
147
148 ### post-rules.txt (loaded by Gluetun)
149 ```
150 FORWARD: ACCEPT for tailscale0 in/out
151 INPUT: ACCEPT for tailscale0
152 NAT POSTROUTING: MASQUERADE on tun0
153 MANGLE: TCP MSS clamp to 1180
154 NAT PREROUTING: Redirect DNS (53) 5335 on tailscale0 and eth0
155 iptables: DROP FORWARD on tailscale0
156 ```
157
158 ---
159
160 ### 5.1 Tailscale Local Network Discovery (DERP Bypass)
161 By default, Gluetun's strict NAT swallows all of Tailscale's UDP hole-punching packets, forcing Tailscale to
    fall back to incredibly slow DERP relay servers (often adding 500ms+ latency).
162
163 To fix this, we use `FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8` in `docker-compose.yml`
    (on the `warp` container). This creates `ip rule` bypasses (Priority 99, Table 199) that route all packets
    destined for private IP ranges *outside* the WARP tunnel directly to the host's `eth0`.
164 - **172.16.0.0/12** is especially critical because many modern Wi-Fi routers (and Docker itself) assign IPs in
    this block (e.g., `172.17.x.x`).

```

```

165 - This bypass allows Tailscale's UDP packets to reach devices on the same Wi-Fi directly, establishing a fast
    peer-to-peer connection while the actual web traffic inside the tunnel remains fully routed through WARP.
166
167 ---
168
169 ### 5.2 Firewalling & Per-Device Access Control
170 Because every mesh device's traffic passes through the `warp` container's network namespace, this namespace
    serves as the ultimate choke point for network-wide firewall rules.
171
172 **Where to Put Rules**
173 The right place to add firewall rules is `scripts/routing-fix.sh`. It runs a 30-second loop inside the
    namespace and already handles re-injecting rules that Gluetun resets on reconnect. **Do not use `post-rules
    .txt` for dynamic rules**, as Gluetun flushes and resets it upon VPN reconnects.
174
175 **The Dual iptables Backend Constraint (CRITICAL)**
176 The container runs two simultaneous iptables backends: `iptables` (for the nftables backend used by Gluetun)
    and `iptables-legacy` (for Tailscale). Every rule **must** be added to both stacks, or it will silently
    fail for certain traffic.
177
178 **Avoiding Duplication**
179 Because `scripts/routing-fix.sh` runs in a loop, you must always check for a rule's existence with `-C` before
    appending with `-A`. Otherwise, your rules will duplicate every 30 seconds.
180
181 **Per-Device Identification & Filtering**
182 Each mesh device has a stable `100.x.x.x` Tailscale IP, which is visible as the `src` IP in the `FORWARD` chain
    . This is how you identify devices for filtering.
183 * **Domain-level:** AGuard Home (port 3000, credentials `admin/nullexit`) provides a REST API that supports
    per-client blocking rules based on their Tailscale IP.
184 * **IP-level:** Use `iptables` in `scripts/routing-fix.sh` (e.g., `iptables -I FORWARD -s 100.x.x.x -d <
    blocked_ip> -j DROP`).
185 * **Country-level:** Add `ipset` to the `routing-fix` apk install step in `docker-compose.yml`, and load CIDR
    ranges from `ipdeny.com` into an ipset, then block that set in the `FORWARD` chain.
186
187 ## 6. macOS Quirks to Remember
188
189 1. **Colima SSH tunnel is TCP-only** UDP ports (like 5354/udp) are NOT accessible from host. Use `dig +tcp` or
    the Python DNS proxy (UDPTCP converter).
190 2. **`docker compose exec -T` injects `\r`** Always `tr -d '\r'` when capturing output for comparisons or `
    networksetup` commands.
191 3. **`brew services` can get stuck** Fix with `sudo brew services restart tailscale`. Common state: `brew
    services list` shows `started` but `tailscale status` shows "Tailscale is stopped."
192 4. **No `timeout` command** macOS lacks GNU `timeout`. Use the `run_with_timeout()` bash function.
193 5. **`sudo -v` prompts even with NOPASSWD** Use `sudo -n` (non-interactive) everywhere.
194 6. **DNS is scoped to en0** macOS scutil DNS resolver is scoped to en0 (Wi-Fi). DNS changes on other
    interfaces (like USB Ethernet) are ignored unless en0's service is also updated. The script always sets DNS
    on BOTH `$ACTIVE_SERVICE` and `$ENO_SERVICE`.
195 7. **tailscaled clobbers DNS during exit-node transition** `force_dns_to_gateway` is called a second time at
    the end of the ENABLE branch to counteract this.
196 8. **`tailscale up --reset` resets all unspecified flags to defaults Every `--reset` call MUST also pass `--
    accept-dns=false` explicitly or tailscaled re-enables DNS management.
197 9. **`docker compose ps` CWD pitfall** `docker compose` commands require a `docker-compose.yml` in the current
    working directory. Use `docker ps` for raw container listing from any CWD; `docker compose ps` requires
    the project directory.
198
199 ---
200
201 ## 7. Environment Variables
202
203 ### `.env` File
204 | Variable | Purpose |
205 |-----|-----|
206 | `WIREGUARD_PRIVATE_KEY` | WARP WireGuard private key (from `wgcf generate`) |
207 | `WIREGUARD_PUBLIC_KEY` | WARP WireGuard public key |
208 | `WIREGUARD_ADDRESSES` | WARP WireGuard address (e.g., `172.16.0.2/32`) |
209 | `TS_AUTHKEY` | Tailscale auth key for the container |
210 | `NULLEXIT_SEED` | 256-bit seed for HMAC-SHA256 script integrity signing (generated by `setup.sh`) |
211 | `GATEWAY_RULE_PROFILE` | Rule compilation tier: `light`, `medium`, `heavy` |
212 | `GATEWAY_BYPASS_PING` | (Optional) `true` to proceed even if pre-flight checks fail |
213 | `GATEWAY_USE_EXIT_NODE` | (Optional) `false` to skip exit node, DNS-only mode |
214 | `GATEWAY_HIJACK_HOST` | (Optional) `false` to skip DNS hijacking on the host (VPN/adblocking for Tailscale
    peers only) |
215 | `GATEWAY_MSS` | (Optional) TCP MSS clamp value (default 1120); 1180 for speed on healthy paths |
216 | `WARP_FAIL_THRESHOLD` | (Optional) Consecutive `warp-off` polls before auto-shutdown (default 6 = 30s; 3 = 15
    s) |
217 | `WARP_ENDPOINT_1` | (Optional) Override Cloudflare WARP WireGuard endpoint IP 1 (default `162.159.192.1`) |
218 | `WARP_ENDPOINT_2` | (Optional) Override Cloudflare WARP WireGuard endpoint IP 2 (default `162.159.193.1`) |
219 | `KILL_SWITCH` | (Optional) `true` to enforce strict PF lock drops all host traffic if VPN fails. Breaks SSH
    if VPN dies. |
220 | `HOST_LEAK_PROBE` | (Optional) `false` to disable the 300ms host-egress leak prober (default: `true`) |
221 | `STOP_COLIMA_ON_EXIT` | (Optional) `true` to fully shut down the Colima VM on toggle-off (saves battery on
    dedicated hosts) |

```

```

222 | `ADGUARD_PASSWORD` | AdGuard Home web UI password (default: `nullexit`; username always `admin`) |
223 | `BLOCKED_COUNTRIES` | Space-separated 2-letter ISO codes to block via ipdeny.com CIDR ranges (e.g. `"kp il cn
    ru"`) |
224
225 ---
226
227 ## 8. Diagnostic Commands
228
229 ### Container Health
230 ```bash
231 docker ps --format '{{.Names}} {{.Status}}'
232 docker compose exec -T tailscale tailscale status
233 docker compose exec -T warp wget -q0- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace
234 ```
235
236 ### SOCKS5 Proxy
237 ```bash
238 curl --socks5-hostname 127.0.0.1:1080 --max-time 5 -s https://www.cloudflare.com/cdn-cgi/trace
239 ```
240
241 ### AdGuard DNS
242 ```bash
243 dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5
244 ```
245
246 ### Firewall & Routing
247 ```bash
248 # nftables FORWARD chain
249 docker compose exec -T warp iptables -L FORWARD -v --line-numbers
250 # Legacy FORWARD chain
251 docker compose exec -T warp iptables-legacy -L FORWARD -v --line-numbers
252 # IP rules + routing tables
253 docker compose exec -T warp ip rule show
254 docker compose exec -T warp ip route show table 199
255 docker compose exec -T warp ip route show table 200
256 ```
257
258 ### Host macOS
259 ```bash
260 tailscale status
261 brew services list | grep tailscale
262 networksetup -getdnsservers Wi-Fi
263 networksetup -getsocksfirewallproxy Wi-Fi
264 sysctl net.inet.ip.forwarding
265 ```
266
267 ---
268
269 ## 9. Development Issue Tracker
270
271 This section documents issues encountered during development, their status, and resolutions.
272
273 ### 9.1 Exit Node Return Path (`rx 0`) RESOLVED
274 **Symptom:** Gateway showed `tx 936 rx 0` transmitted but never received return traffic.
275
276 **Root Cause:** `docker-compose.yml` included `FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10`. Gluetun created a
    strict routing rule (`ip rule add to 100.64.0.0/10 lookup 199`, priority 99) that forced all Tailscale
    CGNAT traffic to bypass the VPN via the Docker bridge. Return packets were sent to the macOS host instead
    of back through `tailscale0`.
277
278 **Fix:** Removed `FIREWALL_OUTBOUND_SUBNETS` entirely. `scripts/routing-fix.sh` now injects `lookup 52`, and
    return packets correctly flow back into `tailscale0`. The SOCKS5 proxy was repurposed as a bulletproof
    failover.
279
280 ### 9.2 Gluetun Resets nftables FORWARD Rules
281 **Symptom:** FORWARD RELATED,ESTABLISHED rule disappears after Gluetun health check.
282
283 **Fix:** `scripts/routing-fix.sh` re-adds to both iptables backends every 30 seconds. Legacy backend (policy
    ACCEPT) acts as safety net.
284
285 ### 9.3 docker compose exec `\r` Injection
286 **Fix:** All `docker compose exec` output piped through `tr -d '\r'`.
287
288 ### 9.4 Colima VM Memory / Swap Thrashing Latency
289 **Symptom:** After running nullexit flawlessly for over 24 hours straight to test stability, the 0.5GB VM RAM
    configuration began experiencing severe network latency on the Tailscale mesh later in the day.
290
291 **Root Cause:** Services (like AdGuard, Tailscale, Docker logs) slowly accumulate cache and state over extended
    periods. Under the tight 512MB physical RAM limit, this slow creep forced the Linux kernel to aggressively
    swap memory to the 512MB SSD swap file. The constant disk I/O thrashing blocked process execution, causing
    DNS resolutions and packet forwarding to delay significantly (making the internet feel "very slow").

```

```

292
293 **Proof (Empirical Observation):** While in this OOM-thrashing state, direct DNS queries through the Tailscale
    mesh (`dig @100.100.21.8 google.com`) would intermittently hang or take several seconds to resolve. After
    restarting with the 600MB configuration, the exact same query immediately dropped to a lightning-fast **41
    ms** response time. (Note: Querying the mapped host port via `127.0.0.1:5354` will always time out due to
    Gluetun's strict leak-prevention firewall blocking non-VPN ingress traffic).
294
295 **Fix:** Increased VM base physical RAM to 600MB (`--memory 0.6`) and reduced the swap file to 400MB. This
    provides enough native RAM headroom for long-running state caches without forcing heavy I/O swapping.
    Subdomain deduplication still reduces blocklists by ~60%. Memory profiles (`light`/`medium`/`heavy`) let
    users trade off coverage for memory.
296
297 ### 9.5 tailscaled Daemon Broken State
298 **Symptom:** `brew services list` shows `started` but `tailscale status` shows "stopped".
299
300 **Fix:** `sudo brew services restart tailscale`. Toggle script now detects and auto-restarts.
301
302 ### 9.6 Stderr Invisibly Swallowed
303 **Symptom:** Failures were silent due to stderr redirected to `output.log`.
304
305 **Fix:** Tailscale wait loop now shows output. Critical commands show errors inline.
306
307 ### 9.7 Missing iptables in routing-fix Container
308 **Root Cause:** `alpine:3.20` does not have iptables installed. Commands failed silently with `2>/dev/null ||
    true`.
309
310 **Fix:** Updated `docker-compose.yml` to `apk add --no-cache iptables iproute2` before `scripts/routing-fix.sh`
    `.
311
312 ### 9.8 DOCKER_GW Variable Parsing Bug
313 **Root Cause:** `ip route show default` printed two lines; `awk '{print $3}'` picked up both, causing route
    commands to fail.
314
315 **Fix:** Added `head -1` to the parsing chain.
316
317 ### 9.9 Native SSH & SFTP Security (Zero Local Attack Surface)
318 **Context:** macOS "Remote Login" (`sshd`) and "File Sharing" (`smbd`) open ports on the local network (e.g.
    `172.x.x.x`), exposing the host to brute-force attacks on public Wi-Fi.
319
320 **Implementation:** The script strictly enforces `tailscale up --ssh=true` whenever the mesh state is reset.
    This empowers the user to completely disable native macOS Remote Login and File Sharing. This provides an
    incredible zero-trust security advantage: even if an attacker steals your Mac username and password, they
    **cannot** access your files remotely. Disabling the native services completely closes the listening ports
    on the local Wi-Fi interface (`172.x.x.x`), rendering the Mac inaccessible to local network logins.
    Meanwhile, Tailscale intercepts port 22 traffic exclusively over the encrypted mesh and authenticates
    cryptographically based on your Tailscale SSO identity. Stolen Mac passwords are mathematically useless
    without first bypassing your Tailscale Two-Factor Authentication and registering a device onto the mesh.
321
322 **Cross-Platform File Exchange:** To easily browse and exchange files securely, simply use an SFTP client and
    connect to your Mac's MagicDNS name on port 22. Recommended clients: **WinSCP** or **FileZilla** (Windows),
    **Cyberduck** (macOS), and **FE File Explorer** or **Solid Explorer** (iOS/Android).
323
324 ### 9.10 Changing macOS Local Hostname
325 **Context:** Users may wish to change their Mac's computer name/hostname in macOS System Settings.
326 **Effect:** Changing the Mac's hostname will **not** break Nullexit (which relies on internal routing/localhost
    ). However, Tailscale will automatically detect this change and update the machine name on the Tailnet.
327 - The underlying Tailscale `100.x.x.x` IPv4 address will remain exactly the same.
328 - The **MagicDNS name** will change to match the new hostname. Users must remember to update their SFTP/SSH
    clients to use the new MagicDNS address.
329
330 ### 9.11 Logging Architecture
331 For debugging, logs are strictly segmented based on the component's lifecycle:
332 - **`output.log` (Host-side):** Contains all standard error (`stderr`) and verbose output from the host scripts
    (`toggle.sh`, `recover.sh`, `setup.sh`). Since the `rule-compiler` container is ephemeral and deleted
    after running, its logs (and any Python errors from `scripts/sync-rules.py`) are extracted and appended to
    this file before deletion.
333 - **Log Rotation Policy:** On startup, `toggle.sh` checks if `output.log` exceeds **50MB** (`52,428,800`
    bytes). If it does, it performs a metadata-only rename (`mv output.log output.log.old`), preserving history
    while resetting the active log to 0 bytes. This rename-based rotation avoids the heavy disk I/O and CPU
    overhead of rewriting a massive file line-by-line.
334 - **Egress Probe Logging:** The background host-egress leak probe checks if stdout is a TTY (`[ -t 1 ]`) and
    will only print live status updates (using carriage return `\r`) or duplicate console warnings in
    interactive mode. It automatically detects macOS/BSD vs. Linux environments to dynamically construct
    timestamps with date and time (omitting milliseconds on macOS to prevent high CPU utilization from spawning
    sub-second processes).
335 - **`docker logs <container>` (Guest-side):** The persistent containers (`warp`, `tailscale`, `routing-fix`, `
    adguardhome`) use the Docker `json-file` logging driver with a strict `max-size` (1m-10m) to prevent VM
    disk exhaustion. Use standard `docker logs` to view them.
336
337 ### 9.12 Chrome Remote Desktop Connection Failures

```

```

338 **Context:** When attempting to access a remote machine via Chrome Remote Desktop (CRD), the connection fails
    or hangs if Tailscale is active **on the remote device**. The connection works fine even if Tailscale (and
    the exit node) is active **on the local client/viewer device**, but only if Tailscale is disabled on the
    remote machine.
339
340 **Why:** Having Tailscale active on the remote machine creates virtual network interfaces and DNS/routing
    modifications that conflict with Chrome Remote Desktop's WebRTC bindings and signaling traffic.
341
342 **Workaround:**
343 - **Use Tailscale Native RDP/RustDesk (Recommended):** Connect directly to the remote device's Tailscale IP
    (`100.X.Y.Z`) via an RDP or RustDesk client. This is faster and avoids third-party relay conflicts.
344 - **Disable Tailscale on the Remote Device:** If you must use CRD, disable Tailscale on the remote machine
    before connecting.
345
346 ### 9.13 Docker Default Bridge vs Host Wi-Fi Subnet Collision (The 172.17.0.0/16 Subnet Overlap)
347 **Context:** When attempting to ping a local Windows client (`172.17.22.187`) or the default gateway
    (`172.17.0.1`) directly over Wi-Fi, the Mac host returns `No route to host` (displaying a `REJECT` / `!`
    flag in the routing table). Tailscale on the client also drops offline shortly after starting when using
    the exit node, failing to establish direct P2P connections and falling back to slow DERP relays.
348
349 **Root Cause & Subnet Overlap Mechanism:**
350 1. **Docker Default Subnet:** By default, the Docker daemon initializes its default bridge network (`docker0`)
    on the **172.17.0.0/16** IP range.
351 2. **Subnet Conflict:** The host's physical Wi-Fi network happens to be assigned the exact same
    **172.17.0.0/16** range by the local network router.
352 3. **Routing Clash:** Colima launches a Linux VM on the Mac host to run the Docker engine. Because Docker
    inside that VM creates `docker0` and claims the `172.17.0.0/16` subnet, any packet sent to `172.17.x.x`
    from within the containers (such as Tailscale local discovery) is captured by the VM's internal virtual
    bridge interface (which is down/empty) and is blackholed. On the Mac host, macOS dynamically marks routes
    as `REJECT` (!) when local ARP resolution fails, causing local pings to immediately abort with `No route
    to host`.
353 4. **Tailscale Relay Saturation:** Because local peer-to-peer discovery is blackholed, Tailscale falls back to
    public DERP relay servers (e.g. `relay "tor"` in Toronto). When exit-node routing is enabled, all client
    web traffic is routed through the relay. Public relays impose strict rate limits and throttle high-volume
    traffic, resulting in packet drops. This saturation drops Tailscale's control packets, causing the
    connection to time out and showing the client as offline.
354
355 **Fix:**
356 The canonical script for this is `scripts/fix-docker-bridge-collision.sh`. **One command applies the entire fix
    :**
357
358 ```bash
359 bash scripts/fix-docker-bridge-collision.sh          # apply
360 bash scripts/fix-docker-bridge-collision.sh --dry   # preview changes without applying
361 ```
362
363 It auto-detects the host's actual Wi-Fi subnet (no more guessing whether the campus DHCP handed out `172.x`,
    `10.x`, or `192.168.x`), picks a non-conflicting `docker.bip` from a small candidate set (defaulting to
    `172.26.0.1/24` if no overlap is matched), backs up `~/colima/default/colima.yaml` with an ISO-8601
    timestamp, edits the file **in place** (idempotent replaces an existing `docker.bip` if present, appends
    under an existing `docker:` block, or creates a fresh block), restarts Colima, rebinds Wi-Fi DHCP, verifies
    the new default route is no longer Docker's bridge, and re-runs both `./toggle.sh` and `scripts/diagnose-
    host-leak.sh` to print the post-fix verdict.
364
365 For reference, the underlying manual fix is: edit the Colima configuration file (`~/colima/default/colima.yaml
    `) on your Mac to define:
366
367 ```yaml
368 docker:
369   bip: 172.26.0.1/24  # any /24 that does NOT overlap your Wi-Fi subnet
370   ```
371
372 Then, restart Colima to apply the new subnet inside the VM:
373
374 ```bash
375 colima restart
376 ```
377
378 This forces Docker to use `172.26.x.x` for its default bridge, freeing the physical `172.17.x.x` range and
    letting pings and direct Tailscale peer-to-peer connections flow normally.
379
380 ---
381
382 ### 9.14 Cloudflare WARP 24-Second Startup Delay (UDP Session Rate-Limiting)
383 **Context:** When running `toggle.sh` to turn the gateway ON, `docker compose up -d` often hangs for exactly
    24-25 seconds waiting for the `warp` container to become `healthy`. Users might mistakenly blame the Go
    `rule-compiler` processing 400k+ rules for this delay, as Docker prints `rule-compiler Exited 24.3s` at the
    end of the wait.
384
385 **Root Cause:** The `rule-compiler` actually finishes its work in <2 seconds. The 24-second blockage is
    entirely due to Cloudflare WARP's edge server rate-limiting. WireGuard is a stateless protocol with no
    "disconnect" packet. When the toggle is turned OFF, the old UDP session state remains active on Cloudflare's
    backend for 20-30 seconds. When the toggle is instantly turned back ON, Docker starts a new WireGuard
    connection from a new ephemeral UDP source port but uses the **same static cryptographic key** (from `.env
    `). Cloudflare detects this as a potential replay attack or key-sharing abuse, and intentionally drops all

```

packets (rate-limits) for the new connection until the old session's ghost state fully expires (~20-25 seconds).

382 ****Result:**** Gluetun's healthcheck fails repeatedly every 2 seconds until Cloudflare lifts the penalty box, at which point traffic flows and Docker unblocks the rest of the startup sequence. This is a known, expected behavior of reusing static keys rapidly on Cloudflare's network and cannot be bypassed.

383

384 **### 9.15 Background Sudo Failures**

385 ****Context:**** The `--post-wake` script is intentionally designed to skip the ``sudo -v`` interactive prompt because it is launched by ``watcher.sh`` running via ``launchd`` in the background (which has no TTY and cannot accept password input). Thus, it relies entirely on ``sudo -n`` (non-interactive sudo) for all its internal routing changes.

386 ****Problem:**** If the user has not configured the ``/etc/sudoers.d/nullexit`` file properly, any automated background wake event will silently fail: ``sudo -n`` commands drop out, the WARP bypass routes fail to apply, and the ``warp`` container loses internet connectivity.

387

388 **### 9.16 Tailscale Hijacking the Default Route (PHYSICAL_GW Bug)**

389 ****Problem:**** When `--post-wake` runs, it clears the routing table using ``sudo route -n flush`` to ensure a clean slate after the Wi-Fi card roams or wakes up. Because it skips bouncing the Wi-Fi interface (to make the reconnect faster), macOS's DHCP client does not automatically rebuild the physical default route. To counteract this, the script must manually restore the physical default route. Originally, the script tried to capture the physical gateway via ``route get default``. However, because the gateway is already running and Tailscale is active, the default route is often already pointing to ``utunX``. When this happens, ``route get default`` does not output a ``gateway:`` field (it only outputs the ``interface:``), causing the captured ``PHYSICAL_GW`` variable to be empty.

390 When ``PHYSICAL_GW`` is empty, the physical default route is never restored, causing the ``en0`` interface to be isolated from the physical network. The WARP bypass routes then fall back to targeting ``interface en0`` (instead of routing via the gateway), which breaks WireGuard's remote connections completely.

391 ****Fix:**** The script now bypasses the OS routing table entirely and directly queries the hardware DHCP lease (``ipconfig getpacket en0``) to reliably extract the physical router IP, ensuring the physical route is correctly restored even when Tailscale is active.

392

393 **### 9.17 Tailscale MagicDNS Bypassing the AdGuard PREROUTING Intercept**

394 ****Context:**** The ``routing-fix.sh`` script applies an iptables NAT rule (``PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT``) to intercept all incoming DNS queries from connected Tailscale peers and force them into the AdGuard container on port 5335.

395 ****Problem:**** When remote clients (like Android or iOS) have "Use Tailscale DNS settings" turned on, they query Tailscale's MagicDNS IP (``100.100.100.100``). This packet enters the exit node, but the ``tailscaled`` daemon running ``inside`` the gateway container intercepts the ``100.100.100.100`` packet internally. ``tailscaled`` then generates a brand new, local DNS request to the upstream DNS server to resolve the query. Because this new request is generated ``locally`` by the daemon (not arriving externally via the ``tailscale0`` interface), it bypasses the ``PREROUTING`` chain completely. The request glides straight out the WARP tunnel to Cloudflare/Google, entirely bypassing the AdGuard sinkhole.

396 ****Fix:**** The only way to fix this blindspot is to force ``tailscaled`` to use AdGuard as its upstream. In the Tailscale Admin Console, the gateway's Tailscale IPv4 address (e.g., ``100.x.x.x``) must be added as the sole Custom Global Nameserver, and "Override local DNS" must be enabled. Additionally, to block Android devices from bypassing this via Private DNS (DNS-over-TLS), outbound Port 853 traffic is explicitly REJECTED in both ``routing-fix.sh`` and ``post-rules.txt``.

397

398 **### 9.18 Network Bufferbloat & MTU Optimizations (Double-VPN UDP Crash)**

399 ****Symptom:**** When running aggressive UDP bandwidth tests (such as macOS ``networkQuality``, which uses QUIC/HTTP3), the ``nullexit`` tunnel completely crashes or exhibits severe lag (6,000ms+ bufferbloat).

400

401 ****Root Cause:**** The bug is a cascading MTU mismatch. Tailscale generates a UDP packet. Cloudflare WARP (also WireGuard) receives the packet, wraps it in another layer of encryption, pushing the packet size beyond the standard 1500 byte limit. Unlike TCP, UDP packets cannot be resized dynamically via MSS negotiation, resulting in massive IP packet fragmentation. The Apple Virtualization framework (``vz``), which Colima uses for bridged networking, silently drops heavily fragmented UDP packets under high load. This blackholes the entire UDP upload stream, instantly dropping the connection.

402

403 ****Fix (Partial):**** To prevent MTU fragmentation for standard web traffic, ****TCP MSS Clamping**** is strictly enforced via the macOS ``pf`` firewall. By adding ``scrub out all max-mss 1160`` to ``scripts/pf.conf``, macOS unilaterally shrinks all outgoing TCP headers to fit comfortably inside the double-encrypted tunnel, bypassing Path MTU Discovery blackholes. ``TS_DEBUG_MTU=1200`` was also added to ``docker-compose.yml`` to minimize internal fragmentation.

404 ****Unresolved:**** The only way to solve the UDP crash bug natively is to artificially cap the maximum tunnel bandwidth using SQM (``fq_codel``). Because a static bandwidth cap would artificially throttle high-speed networks, ``nullexit`` leaves the bandwidth uncapped, optimizing perfectly for TCP while accepting UDP fragmentation under extreme edge-case load tests.

405 ****Why not Dynamic SQM (Aurate)?**** Implementing an EMA/PID-controlled aurate daemon (like ``sqm-aurate``) requires a constant 100ms polling loop to measure the kernel packet queue, which severely degrades laptop battery life by preventing deep C-state CPU idling. Furthermore, macOS's native firewall (``dummynet``) only supports ``fq_codel`` and does not support the newer Linux ``CAKE`` algorithm, which is the only algorithm that offers kernel-native, event-driven ``aurate-ingress`` (zero polling penalty). Therefore, dynamic SQM is computationally hostile to battery-powered macOS hosts.

406

407 **## 10. Resolved Issues (from README)**

408

409 These bugs and edge cases were discovered and resolved during development.

410

411 **### 10.1 DNS Proxy: TCP Wire Format Mismatch (socat / dnsmasq)**

412


```

413 **Problem:** When the Tailscale data plane is unavailable (DERP relay only), the script falls back to a local
      DNS proxy that forwards queries from host UDP:53 to Docker's TCP:5354 (AdGuard). The `socat` and `dnsmasq`
      approaches both failed.
414
415 - **socat** forwards raw bytes it does not prepend the 2-byte length prefix that DNS-over-TCP requires.
      AdGuard parses the stream incorrectly and returns garbage.
416 - **dnsmasq** tries UDP first to reach the upstream (`127.0.0.1#5354`), but Colima's SSH tunnel only forwards
      TCP. With a single upstream server, dnsmasq doesn't fall back to TCP even with `--timeout=1`.
417
418 **Solution:** Replaced both with a **Python DNS proxy** (`scripts/dns-proxy.py`, ~25 lines) that properly
      handles the DNS-over-TCP wire format: reads a UDP query from the host, prepends the 2-byte length prefix,
      sends it over TCP to AdGuard, strips the prefix from the response, and sends it back over UDP.
419
420 *Architectural Note: Why is the DNS proxy in Python while the SOCKS5 proxy was moved to a compiled Go Docker
      container?
421 The SOCKS5 proxy handles heavy data streaming (e.g., video, downloads). Python introduces massive CPU/battery
      overhead for raw socket streaming, which is why we rely on the compiled Go implementation built directly
      into the `tailscaled` daemon via `TS_SOCKS5_SERVER=:1080`.
422 Conversely, the DNS proxy only handles intermittent UDP queries (a few bytes per minute) generated solely by
      the local host machine. The Python overhead for this is effectively 0.001 Watts. We deliberately kept `dns-
      proxy.py` in Python because it runs natively on the macOS/Linux host rewriting it in Go would force users to
      install a Go compiler (`brew install go`) on their host machine for zero noticeable battery gain. (Note:
      If this architecture is ever adapted to serve as a public DNS resolver for thousands of users or for
      massive automated web-scraping clusters, `dns-proxy.py` must be rewritten in Go to survive the concurrent
      packet flood).
423
424 ### 10.2 Exit Node Routing Conflict & The SOCKS5 Failover
425
426 **Problem:** For days, Tailscale's exit node refused to return traffic back to the client (`rx 0`), leading to
      the assumption that Tailscale's userspace forwarding was bypassing `iptables` and ignoring the WARP `tun0`
      interface. In a desperate attempt to fix this, a **SOCKS5 proxy** (`scripts/socks5-proxy.py`) was
      introduced as a replacement.
427
428 **The Real Root Cause:** Tailscale's userspace forwarding was not the problem. The issue was an environment
      variable in Gluetun (`FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10`) that instructed the VPN container to hijack
      all Tailscale CGNAT traffic and forcibly route it out the unencrypted Docker bridge (`eth0`). This
      blackholed all return packets back to the client.
429
430 **Solution:** Removed the offending `FIREWALL_OUTBOUND_SUBNETS` variable, immediately restoring the Tailscale
      exit node. Instead of removing the SOCKS5 proxy, we repurposed it into a **bulletproof failover** for when
      restrictive Wi-Fi networks block Tailscale UDP ports.
431
432 Architecture after the fix:
433 ```
434 PRIMARY (Exit Node):
435 DNS/TCP/UDP: Apps Tailscale Exit Node gateway container tun0 WARP internet
436
437 FAILOVER (SOCKS5 - if Tailscale is blocked by local Wi-Fi):
438 DNS: Browser 127.0.0.1:53 Python proxy TCP:5354 AdGuard WARP
439 TCP: Apps SOCKS5:1080 gateway container tun0 WARP internet
440 Mesh: SSH/ping 100.x.x.x utun5 WireGuard peers (independent of proxy)
441 ```
442
443 ### 10.3 Pre-Flight Check 1: Wrong `tailscale ping` Flag
444
445 **Problem:** Check 1 of the pre-flight connectivity checks used `--c 1` (double dash) but `tailscale ping`
      accepts `-c 1` (single dash). The invalid flag caused the command to exit with code 1, marking the check as
      FAIL even though the gateway was reachable.
446
447 **Fix:** Changed `--c 1` to `-c 1`.
448
449 ### 10.4 Pre-Flight Check 1: DERP Relay Exit Code
450
451 **Problem:** Even with the correct `-c 1` flag, `tailscale ping` exits with code 1 when the connection goes
      through a DERP relay (`direct connection not established`) even though a pong was successfully received
      (28ms via DERP(tor)). The check treated relayed connections as failures.
452
453 **Fix:** Changed the check from relying on exit code to piping stdout through `grep -q "pong"`. This accepts
      relayed connections as valid (they work fine for the exit node use case), while still failing on true
      timeouts or unreachable peers.
454
455 ### 10.5 Container Default Route Bypasses WARP
456
457 **Problem:** Inside the WARP container's network namespace, the main routing table's default route was via `
      eth0` (Docker bridge), not `tun0` (WARP tunnel). While gluetun uses policy routing (rule 101 table 51820
      `tun0`) for most traffic, traffic originating from the container's own IP (`172.18.0.2`) hits rule 100 (
      from 172.18.0.2 lookup 200), whose default was also via `eth0`. This caused the SOCKS5 proxy's outbound
      connections to bypass WARP.
458
459 **Fix:** Updated the `routing-fix` container to:
460 1. Change the main table's default route to `default dev tun0`

```

```

461 2. Change table 200's default route to `default dev tun0`
462 3. Add a specific route for the WARP WireGuard endpoint (`162.159.192.1`) through `eth0` via the Docker gateway
    in table 200, preventing a tunnel loop
463 4. Re-assert all routes every 30 seconds (gluetun may reset them on health checks)
464 5. Dynamically detect the Docker subnet from `ip route show dev eth0` instead of hardcoding `172.18.0.0/16`
465
466 ### 10.6 IPv6 Exit-Node Leak
467
468 **Problem:** Tailscale supports IPv6, but WARP/glutun is IPv4-only. IPv6 packets forwarded through the exit
    node would leak through the Docker bridge unencrypted.
469
470 **Fix:** Added `ip6tables -A FORWARD -i tailscale0 -j DROP` to `post-rules.txt`, blocking all IPv6 forwarded
    traffic from the Tailscale interface.
471
472 ### 10.7 Hardcoded Docker Subnet in Routing Fix
473
474 **Problem:** The `routing-fix` container hardcoded `172.18.0.0/16` and `172.18.0.1` for the Docker bridge
    subnet and gateway. If Docker's bridge network changed (after `docker network prune`, Colima restart, or on
    a different machine), these routes would be wrong.
475
476 **Fix:** Detection is now dynamic using `ip route show`:
477 - `DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
```

This extracts the actual subnet and gateway directly from the routing table, working with any subnet size
(`/16`, `/24`, `/20`, etc.).

```

478 - `DOCKER_GW=$(ip route show default | awk '{print $3}')
```

479

```

480 This extracts the actual subnet and gateway directly from the routing table, working with any subnet size
    (`/16`, `/24`, `/20`, etc.).
481
482 ### 10.8 SOCKS5 Proxy Threading Race Condition
483
484 **Problem:** The `forward` function in `scripts/socks5-proxy.py` called `select.select([src, dst], [], [])`
    which raised `ValueError: file descriptor cannot be a negative integer (-1)` when one thread closed a
    socket while the other thread was selecting on it.
485
486 **Fix:** Added `ValueError` exception handling around `select.select()` and `recv()` calls. When a file
    descriptor becomes negative, the thread breaks out of the forwarding loop cleanly instead of crashing.
487
488 ### 10.9 Docker Healthcheck: Multi-line Python in CMD Array
489
490 **Problem:** The socks-proxy healthcheck used a `CMD` array with a multi-line Python string. YAML collapsed the
    newlines, causing the `#` comment to comment out the rest of the code and the `assert` statement to be
    mangled into `as`.
491
492 **Fix:** Changed from `["CMD", "python3", "-c", "..."]` to `["CMD-SHELL", "python3 -c '...'"` with a single-
    line Python expression. Uses a real SOCKS5 handshake (sends `[5, 1, 0]`, expects `[5, 0]`) instead of a
    simple TCP port check.
493
494 ### 10.10 Sudo Credential Caching Removed
495
496 **Problem:** The script used `sudo -v` at startup to cache sudo credentials, plus a background `SUDO_KEEPER_PID`
    loop refreshing them every 60 seconds. This required interactive password entry even with NOPASSWD
    sudoers configured, because `sudo -v` itself prompts.
497
498 **Fix:** Removed the entire `sudo -v` + `SUDO_KEEPER_PID` section. All privileged commands now use `sudo -n` (
    non-interactive), which works silently because the user has NOPASSWD rules in `/etc/sudoers.d/nullexit` for
    the specific commands needed (`networksetup`, `python3`, `dscacheutil`, `killall`, `pkill`, `route`, `
    ifconfig`).
499
500 ### 10.11 macOS Application Firewall Silently Blocking AirDrop & Continuity
501
502 **Problem:** Users of complex network extensions (Tailscale, LuLu, Docker, etc.) often run into an issue where
    AirDrop, AirPlay, and Universal Clipboard silently fail, even with the gateway inactive and Wi-Fi/Bluetooth
    correctly configured. The internal Apple Wireless Direct Link (`awdl0`) interface appears healthy, but
    connections never establish.
503
504 **Root Cause:** The built-in macOS Application Firewall has a specific list of explicitly blocked applications.
    Apple's own core networking services (e.g., `/usr/libexec/sharingd`, `/usr/libexec/rapporstd`, `/usr/
    libexec/avconferenced`) can be silently added to the "Block incoming connections" list either through
    accidental "Deny" clicks on permission prompts, execution of aggressive privacy-hardening scripts, or a
    known macOS bug that retains strict blocks even after toggling the "Block all incoming connections" setting
    off. Since these are background daemons, macOS provides no visible error.
505
506 **Fix:** The firewall rules must be cleared via the terminal (or System Settings > Network > Firewall > Options
    ):
507 ```bash
508 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/sharingd
509 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/rapporstd
510 ```
511 Once unblocked, `sharingd` immediately resumes broadcasting mDNS/Bonjour discovery packets, and AirDrop
    restores instantly without requiring a reboot.
512
513 ### 10.12 Hard Reboots Leave Stale Docker Socket in Colima VM

```


514

515 ****Problem:**** If the macOS host machine is subjected to a hard reboot, sudden power loss, or a kernel panic, the Colima VM running in the background is abruptly killed. Upon next boot, running `toggle.sh` resulted in the contradictory output:

516 `Colima is already running.` followed by `Cannot connect to the Docker daemon at unix:///-.colima/default/docker.sock.`

517

518 ****Root Cause:**** The hard crash prevents the inner Docker daemon from executing its graceful shutdown routines. It leaves behind a corrupted `docker.sock` file inside the VM. On boot, the outer VM spins up (hence "already running"), but the inner Docker daemon sees the stale socket, assumes another instance is running, and crashes.

519

520 ****Fix:**** Added an Auto-Healing routine directly into `toggle.sh` (Step 4b). The script now explicitly tests the Docker daemon with `docker ps`. If the daemon is unresponsive despite Colima reporting as active, it assumes a stale socket and automatically runs `colima restart` in the background to cleanly rebuild the VM and socket before proceeding.

521

522 **### 10.13 Graceful Shutdowns Leave DNS Hijacked**

523

524 ****Problem:**** The gateway overrides the macOS host's DNS settings (via `networksetup`) to route queries to the AdGuard container. Because macOS persists `networksetup` changes to disk, shutting down the Mac while the gateway is running causes the Mac to boot up later with hijacked DNS. Since the Colima VM and Docker containers don't auto-start on boot, the user would have no internet access until they manually run the `toggle` script.

525

526 ****Root Cause:**** The `toggle.sh` script exits after the gateway starts. There was no background daemon listening for macOS system shutdown signals to revert the network settings.

527

528 ****Fix:**** Wrapped the background `caffeinate` process (used to prevent system sleep) in a bash `trap` command. The trap listens for `SIGTERM`, `SIGINT`, and `SIGHUP`. When the user initiates a graceful shutdown, macOS sends `SIGTERM` to the background trap, which instantly executes `recover.sh` to flush the host DNS back to `1.1.1.1` right before the machine powers off.

529 ***Note:**** We explicitly use `recover.sh` instead of `toggle.sh` because during a shutdown, the OS limits process cleanup time and is already independently sending termination signals to Docker and Colima. Trying to run `docker compose down` inside a shutdown trap creates a race condition that can hang the script until macOS forcefully `SIGKILL`s it. `recover.sh` abandons container management and surgically flushes the macOS network settings in milliseconds, guaranteeing completion before the OS pulls the plug.

530

531 **### 10.14 Wi-Fi Edge Packet Loss via QUIC (UDP) Fragmentation**

532

533 ****Problem:**** Applications that heavily utilize HTTP/3 (QUIC) over UDP such as Facebook, Instagram, and Google services experience massive stuttering and stalled image loading when the client device is far away from the router (weak Wi-Fi signal). The issue did not occur when close to the router.

534

535 ****Root Cause:**** The `nullexit` gateway uses a double-encrypted tunnel (Tailscale + WARP). To prevent packets from fragmenting when entering these tunnels, a firewall rule in `post-rules.txt` uses `--set-mss 1120` to safely clamp TCP packets. However, because QUIC runs entirely over UDP, it bypasses the TCP MSS handshake completely. QUIC blasts maximum-MTU (1280 byte) UDP packets. The inner `tailscale0` interface (MTU 1200) forces these large UDP packets to fragment into two pieces. When the client is on the edge of Wi-Fi range (where 5-10% packet loss is common), losing *either* UDP fragment destroys the *entire* image frame. This exponential amplification of packet loss stalled QUIC streams.

536

537 ****Fix & Trade-offs:**** Appended an `iptables` rule to `post-rules.txt` that explicitly blocks `UDP --dport 443` (QUIC). When modern web apps detect QUIC is blocked, they instantly fall back to HTTP/2 (TCP 443). Because TCP is routed through the MSS clamp, the packets are resized *before* they leave, entirely eliminating IP fragmentation and restoring high-speed loading over weak Wi-Fi.

538 ***Consequences:**** This adds a minor latency penalty (TCP 3-way handshake) compared to QUIC's zero-RTT connection. It may also force WebRTC video calls (Google Meet/Discord) to fall back to TCP TURN servers, slightly inflating real-time video latency. However, in a constrained double-tunnel mesh, the absolute stability gained from eliminating fragmentation vastly outweighs these trade-offs.

539

540 ---

541

542 **## 11. Deep Dive: Exit Node Return Path Analysis (June 26, 2026)**

543

544 This section preserves the detailed packet-level analysis that led to identifying and fixing the `rx 0` bug.

545

546 **### The exit node was probably never going to work in this container topology**

547

548 Here's the packet path traced by reading `ip rule show` and the routing tables live:

549

550 ****Outbound (Phone-1 internet):****

551 1. Phone-1 sends packet to gateway via Tailscale mesh

552 2. Gateway's `tailscale0` (kernel TUN, `TS_USERSPACE=false`) decrypts packet enters FORWARD chain

553 3. Packet has `src=100.87.42.87` (Phone-1), `dst=some internet IP`

554 4. FORWARD chain (nftables): Rule 1 matches (`-i tailscale0 -j ACCEPT`)

555 5. Routing decision for the forwarded packet. Which ip rule matches?

556 - Rule 98: `to 172.18.0.0/16` no (dst is internet)

557 - Rule 99: `to 100.64.0.0/10` no (dst is internet)

558 - Rule 100: `from 172.18.0.2` ****NO**** (src is `100.87.42.87`, not the container IP!)

559 - Rule 101: `not fwmark 0xca6c lookup 51820` ****YES**** (no fwmark on forwarded packets)

```

560 6. Table 51820: `default dev tun0` packet goes out through WARP
561 7. NAT POSTROUTING: `MASQUERADE on tun0` src becomes WARP IP
562 8. Packet exits through WARP to internet (in theory)
563
564 **Return (internet Phone-1):**
565 1. Return packet arrives on tun0, conntrack de-MASQUERADEs dst back to `100.87.42.87`
566 2. Routing decision for dst=`100.87.42.87`:
567   - Rule 99: `to 100.64.0.0/10 lookup 199` **YES**
568 3. Table 199: `100.64.0.0/10 via 172.18.0.1 dev eth0` **sends it to the Docker gateway!**
569 4. **This is wrong.** The packet should go to tailscale0 (to be re-encrypted and sent to Phone-1), but table
    199 sends it out eth0 to the Docker bridge host.
570
571 **Table 199 is the smoking gun.** It has one route: `100.64.0.0/10 via 172.18.0.1 dev eth0`. This sends ALL
    Tailscale CGNAT return traffic out the Docker bridge to the host not back through tailscale0. The return
    packet leaves the container's network namespace entirely, hits the host's routing stack, and gets dropped
    because the host has no idea what to do with a raw packet for `100.87.42.87`.
572
573 ### Why table 199 exists and why it's wrong for exit node traffic
574
575 Table 199 + the CGNAT rule (pref 99) was designed so that the **container itself** can reach other Tailscale
    peers (e.g., for mesh management, DERP relay). For that use case, routing CGNAT traffic via the Docker
    bridge to the host (which has its own tailscaled) makes sense.
576
577 But for **forwarded exit node traffic**, the return packet needs to go back through tailscale0 inside the
    container not out to the host. The CGNAT rule intercepts the return packet before it can be routed back to
    tailscale0 through the main table or Tailscale's own routing (table 52, which is at pref 5270).
578
579 ### The SOCKS5 path works because it dodges all of this
580
581 SOCKS5 proxy creates connections **from the container's own IP** (`172.18.0.2`). So:
582 - ip rule 100 (`from 172.18.0.2 lookup 200`) matches
583 - Table 200 has `default dev tun0`
584 - Return traffic comes back to `172.18.0.2` on tun0, conntrack de-NATs, proxy sends response to client
585
586 No FORWARD chain involved. No CGNAT rule involved. No table 199. It just works.
587
588 ### Summary of findings
589
590 | Finding | Status | Evidence |
591 |-----|-----|-----|
592 | Table 199 CGNAT route hijacks exit node return traffic | **Fixed** | Changed `lookup 199` to `lookup 52` in `
    scripts/routing-fix.sh`. Return packets now route via tailscale0. |
593 | FORWARD RELATED, ESTABLISHED missing | **Fixed** | Installed `iptables` via apk in `docker-compose.yml`. Rule
    now injects successfully. |
594 | DOCKER_GW variable parsing error | **Fixed** | Added `head -1` in `scripts/routing-fix.sh`. |
595 | `FIREWALL_OUTBOUND_SUBNETS` was the root cause | **Fixed** | Removed from `docker-compose.yml`. Gluetun no
    longer hijacks CGNAT routing. |
596 | Host tailscaled error state from aggressive `tailscale down` | **Mitigated** | Toggle script now detects and
    auto-restarts. |
597
598 ### 11.1 Double-Tunneling MSS Clamping (Stalling Bug)
599
600 **Problem:** Traffic double-wrapped through Tailscale (WireGuard) and WARP (WireGuard) suffered 120 bytes of
    MTU overhead (60 + 60). The default MSS clamp of 1180 was still slightly too large, causing large packets
    to fragment or drop, which resulted in mysterious web page stalls on strict-MSS endpoints.
601
602 **Fix:** Lowered the TCP MSS clamp in `post-rules.txt` to `1120` to guarantee double-tunneled payloads fit
    safely within standard internet MTUs.
603
604 ### 11.2 Linux Native Wi-Fi Bouncing
605
606 **Problem:** The generated `scripts/recover-linux.sh` incorrectly retained the macOS `networksetup` commands,
    which would crash and fail to bounce Wi-Fi on Linux hosts.
607
608 **Fix:** Swapped the macOS logic for native Linux commands. The script now attempts `nmcli radio wifi off/on`
    first, and gracefully falls back to `ip link set wlan0 down/up` if NetworkManager is unavailable.
609
610 ### 11.3 TS_AUTH_ONCE Cloud Footgun & Ephemeral Keys
611
612 **Decision:** The compose file uses `TS_AUTH_ONCE=true` to prevent authentication loops. However, on headless
    cloud VPS deployments, if a standard auth key expires (usually 90 days), the container will silently drop
    off the mesh. We documented this trade-off explicitly in `docker-compose.yml` and recommended the use of **
    Tailscale Ephemeral Keys** for cloud deployments, which automatically clean themselves up and bypass this
    issue.
613
614 ### 11.4 Hardcoded AdGuard Credentials
615
616 **Decision:** AdGuard is deployed with the hardcoded credentials `admin / nullexit`. This is perfectly safe
    behind a NAT on a local macOS laptop. However, if deployed on a cloud host with exposed ports, this becomes
    a severe security risk. We added a stark warning comment in `docker-compose.yml` to ensure cloud users
    change this before deployment.

```

```

617
618 ### 11.5 Routing Fix: 30-second polling vs Netlink Socket
619
620 **Decision:** Instead of building a complex Netlink socket listener (`ip monitor`) to react instantly to
        iptables and routing flushes from Gluetun, we retained the 5-second `sleep` loop in `scripts/routing-fix.sh`.
621
        - **Reasoning:** Building a netlink listener in pure bash on Alpine is incredibly brittle and complex (
            especially for iptables events). The 30-second polling loop is bulletproof. The only trade-off is a
            potential 1-4 second stutter if Gluetun reconnects, which was deemed an acceptable edge case for absolute
            reliability.
622
623 ### 11.6 Cache Poisoning and URL Rot in scripts/sync-rules.py
624
625 **Problem:** If a remote blocklist URL went permanently offline (404 Not Found), the Python `urllib` request
        would return the 404 HTML string. The script would aggressively regex-parse this HTML, find no domains, and
        overwrite the healthy local cache with a 0-domain file. This effectively disabled ad-blocking until the
        URL was fixed.
626
627 **Fix:** Implemented a defensive programming sanity check in `scripts/sync-rules.py`. Before overwriting the
        cache, the script now verifies that the compiled domain count is greater than 10 (and 1 for IP feeds). If
        it falls below this threshold (indicating a catastrophic 404 failure across multiple URLs), it aborts the
        cache overwrite, raises a `ValueError`, and keeps the previous day's healthy cache intact.
628
629 ### 11.7 Cellular Networks & PMTUD Blackholes (The 1280 Byte Limit)
630
631 **Experiment:** Conducted a live packet sweep from the PC-1 host to Phone-1 connected via a cellular network +
        Tailscale DERP relay using `ping -D -s <size>`.
632
633 **Result:**
634 - Packets with a payload of `1252` bytes (+ 28 bytes IP/ICMP headers = **1280 bytes total**) succeeded
        perfectly with ~60ms latency.
635 - Packets with a payload of `1253` bytes (**1281 bytes total**) and above resulted in a silent timeout.
636
637 **Analysis:** The cellular network (or the Tailscale DERP relay) enforces a strict MTU of 1280 bytes.
        Critically, it acts as a "PMTUD Blackhole" - it silently drops packets larger than 1280 bytes instead of
        returning an ICMP "Fragmentation Needed" warning. If the exit node tries to send a standard 1500-byte
        internet packet to the phone over this link, it vanishes, causing web pages to stall permanently.
638
639 **Conclusion:** This empirically validates the absolute necessity of the TCP MSS clamping rule in `post-rules.
        txt`. By artificially clamping the MSS to `1120` (or `1180`), we ensure the TCP payload + headers + double-
        WireGuard overhead never exceeds the strict 1280-byte ceiling of the cellular mesh link.
640
641 ### 11.8 Gluetun nf_tables Parser Crash (Bash Interpolation Failure)
642
643 **Problem:** Attempting to make the TCP MSS dynamically configurable by using a standard bash variable inside `
        post-rules.txt` (`iptables ... --set-mss ${GATEWAY_MSS:-1120}`) caused the Gluetun container to
        catastrophically crash during startup. Gluetun's internal parser executes `post-rules.txt` line by line
        without a shell, meaning the variable was never expanded. It literally attempted to inject the string `${
        GATEWAY_MSS:-1120}` into iptables, resulting in a fatal `bad value for option "--set-mss" error`.
644
645 **Fix:** Removed the bash variable from `post-rules.txt` and reverted it to a pure hardcoded integer. To retain
        dynamic `env` configuration, we moved the interpolation logic to `toggle.sh`. Right before Docker starts,
        the host script parses `GATEWAY_MSS` from the `env` file and uses a cross-platform `sed` command to
        dynamically overwrite the integer directly inside `post-rules.txt`. This perfectly mimics manual
        configuration and completely bypasses Gluetun's strict parser.
646
647 ### 11.9 Seamless Wi-Fi Roaming & The macOS DNS Wipeout Loophole
648
649 **Problem:** The Tailscale, WireGuard, and Colima NAT stacks are natively designed for connectionless roaming
        and will seamlessly survive when the macOS host switches Wi-Fi networks (e.g., roaming from home Wi-Fi to a
        cellular hotspot). However, the internet completely breaks upon network transition. This occurs because
        macOS intentionally wipes out all custom DNS settings (`networksetup -setdnsservers`) and reverts to the
        new network's default DHCP nameserver whenever the active Wi-Fi BSSID changes. Since the Mac is locked to
        the exit node, its DNS requests are swallowed by WARP and dumped onto the public internet, which cannot
        route private DHCP IPs. This results in a total DNS failure.
650
651 **Solution:** Implementing a silent background "DNS Watcher" daemon (`nullexit-dns-watcher`) in `toggle.sh`.
        When the gateway starts, it spawns a background polling loop that checks the active Wi-Fi DNS every 30
        seconds. If macOS resets the DNS during a network change, the watcher instantly detects the drift and
        forcefully re-injects `100.100.21.8` via `networksetup`. When the gateway stops, the watcher process is
        cleanly killed. This allows true, seamless roaming across physical networks without ever dropping the
        gateway state.
652
653 ### 11.10 The Infinite Recursive Routing Loop (Hotspot Paradox)
654
655 **Problem:** A critical network topology crash occurs if the host Mac connects to a Mobile Hotspot (e.g., a
        Windows PC or Phone) that is also connected to the Tailscale mesh and has "Use Exit Node" enabled.
        Because the Mac is hosting the Exit Node, it routes its internet traffic to the Hotspot. The Hotspot
        intercepts the Mac's traffic on its way to the cellular tower and routes it back to the Exit Node (the
        Mac) via Tailscale. The Mac receives it, tries to send it out again, and the Hotspot intercepts it again.
        This creates an infinite, recursive encryption loop that instantly destroys network bandwidth and drops all

```

```

connections.
656
657 **Fix:** This is a physical routing paradox that cannot be resolved in software. The upstream physical router
    providing internet to the Mac **must** be allowed to route its traffic directly to the physical WAN
    interface. The user must manually disable "Use Exit Node" on the upstream device providing the hotspot.
658
659 **Advanced Workaround:** If the upstream hotspot is a Windows machine and the user explicitly wants it to
    remain connected to the Exit Node, a permanent static route can be injected into the Windows routing kernel
    to forcefully bypass the Tailscale WFP interception for WARP packets. By binding the route to the physical
    adapter's Interface Index (IF), it becomes a permanent, roaming-aware bypass.
660
661 On Windows (Admin Command Prompt):
662 ```cmd
663 route print (find Interface List, note the IF number for the active Wi-Fi adapter, e.g. 15)
664 route -p add 162.159.192.1 mask 255.255.255.255 0.0.0.0 IF 15
665 route -p add 162.159.193.1 mask 255.255.255.255 0.0.0.0 IF 15
666 ```
667
668 On Linux (Root Terminal):
669 ```bash
670 ip route add 162.159.192.1 dev wlan0
671 ip route add 162.159.193.1 dev wlan0
672 ```
673
674 On macOS (Root Terminal):
675 ```bash
676 route add -host 162.159.192.1 -interface en0
677 route add -host 162.159.193.1 -interface en0
678 ```
679
680 *(Note: If the upstream router is an iPhone or Android device, you cannot inject static routes without
    jailbreak/root. You must simply disable "Use Exit Node" in the mobile Tailscale app).*
681
682 This punches a tiny hole straight through the paradox, letting the Mac's WARP packets escape to the physical
    internet while keeping the rest of the upstream router's traffic securely trapped in the Tailscale Exit
    Node.
683
684 ### 11.11 AdGuard Filter Redundancy & Memory Optimization
685
686 **Problem:** AdGuard Home does not perform cross-list deduplication in memory. When it loads its own native
    subscription filters (e.g., "AdGuard DNS filter") alongside our massive `compiled_rules.txt`, overlapping
    rules are loaded twice, wasting significant RAM in the Colima VM. The UI would show ~500k rules,
    representing the sum of all lists rather than unique domains.
687
688 **Fix:** Updated `scripts/sync-rules.py` to intelligently cross-reference AdGuard's configuration and
    deduplicate our list against it.
689 1. The script now reads `adguard/conf/AdGuardHome.yaml` to dynamically fetch the exact URLs of any enabled
    native AdGuard filters.
690 2. The parsing engine was upgraded to normalize basic AdGuard syntax (`||domain`) back into raw base domains
    during the build process.
691 3. The script subtracts the native AdGuard domains from our custom blocklist *before* compiling, immediately
    purging ~83,000 completely redundant rules.
692 4. The normalized base domains then pass through our subdomain optimizer, which squashes an additional ~50% of
    the remaining rules.
693
694 This reduces the final compiled output from ~325k to ~281k rules on the `heavy` profile, saving ~45k rules from
    being loaded into memory twice and reducing the overall RAM footprint of the `adguardhome` container.
695
696 ### 11.12 Kernel-Level IP Blocklist (Threat Intelligence Firewall)
697
698 **Problem:** DNS sinkholing cannot stop malware or botnets that bypass DNS entirely by hardcoding direct IP
    addresses (a common technique for C2 communication). These connections pass straight through AdGuard unseen
    .
699
700 **Fix:** Added a second compilation pipeline to `scripts/sync-rules.py` that runs on every startup alongside
    the DNS pipeline.
701 1. The compiler concurrently fetches four curated threat-intelligence feeds: **Feodo Tracker** (abuse.ch
    botnet C2 IPs), **Spamhaus DROP** (IPs allocated to criminal organizations), **Emerging Threats** (
    Proofpoint active C2 and scanners), and **CINS** (active brute-force sources).
702 2. All entries are normalized via Python's `ipaddress` module. Any IP or CIDR that overlaps with RFC1918
    private ranges, loopback, link-local, or the Tailscale CGNAT range (`100.64.0.0/10`) is stripped to prevent
    accidentally locking users out of their own LAN or mesh.
703 3. The cleaned list (16,721 unique IPs/CIDRs) is written as an `ipset restore` file using an **atomic swap
    pattern**: `create_new populate swap with live destroy_new`. This guarantees the live `block_malicious`
    ipset is never empty during a reload.
704 4. `scripts/routing-fix.sh` watches the file's mtime every 30 seconds. On change it runs `ipset restore` and
    idempotently re-injects `FORWARD DROP` rules for both `src` and `dst` into both iptables backends (nftables
    + legacy).
705 5. `docker-compose.yml` mounts `adguard/work/userfilters/` into `routing-fix` as `/userfilters:ro` and adds `
    rule-compiler: service_completed_successfully` to its `depends_on`, eliminating the race condition where
    routing-fix could start before the file existed.

```

706
707 ****Memory cost:**** The entire 16,721-entry ipset costs only ~1.6 MiB of kernel memory negligible in the 600MB VM budget.

708
709 **### 11.13 Standalone Tailscale Daemon Freeze after macOS Sleep/Wake**

710
711 ****Problem:**** When the macOS host goes to sleep and wakes up, the network interfaces and routing tables are rebuilt. The standalone `tailscaled` daemon (installed via Homebrew) occasionally fails to handle this transition and becomes completely frozen/unresponsive. In this state, any command like `tailscale down` or `tailscale status` hangs indefinitely, and all DNS requests sent to the local Tailscale resolver (`100.100.21.8`) time out, causing a total internet blackout on the host.

712
713 ****Fix:**** Enhanced the Tailscale verification and teardown logic in `toggle.sh` and `recover.sh`:

714 1. ****Unresponsive Daemon Detection:**** Instead of assuming the daemon is healthy just because the Unix socket `~/var/run/tailscaled.socket` exists, the scripts now actively run a status check with a 5-second timeout (`run\_with\_timeout 5 tailscale status`). If the check times out (exit code 143), the daemon is classified as unresponsive.

715 2. ****Auto-Recovery Loop:**** If the daemon is unresponsive, the script now automatically attempts to restart the service using `brew services restart tailscale` (falling back to `sudo -n brew services restart tailscale`).

716 3. ****Graceful Retry:**** Once restarted, the script pauses for 3 seconds for the daemon to initialize before proceeding with connection or teardown commands.

717
718 **### 11.14 macOS Sharing Services (AirDrop/AirPlay) Freeze on Network Transitions**

719
720 ****Problem:**** When the gateway is turned on or off, the scripts perform network configuration cleanups which include flushing the routing tables (`route -n flush`), restarting the `mDNSResponder` daemon (`killall -HUP mDNSResponder`), and power-cycling the Wi-Fi interface (`setairportpower` or `ifconfig en0 down/up`). While AirDrop BLE discovery continues to function, the actual file transfers stall at "Waiting..." indefinitely.

721
722 ****Root Cause:**** The rapid tear-down and rebuild of the local routing table and Wi-Fi interface causes Apple's core sharing and connection daemons (`sharingd` and `rapportd`) to lose their socket bindings to the mDNS/Bonjour discovery interface. Instead of reconnecting gracefully, they enter a wedged state and try to route peer-to-peer (AWDL) IPv6/TCP transfer traffic into the default gateway (the Tailscale interface `utun`) instead of the direct wireless link, causing the TLS verification handshake to time out.

723
724 ****Fix:**** Integrated an automatic sharing services reset into both `toggle.sh` and `recover.sh`:

725 1. The script now automatically runs `sudo -n killall sharingd rapportd` at the end of both the ****START**** path and the ****STOP**** path (as well as inside `recover.sh`).

726 2. Killing these daemons forces macOS to immediately relaunch them, binding them fresh to the newly initialized network interfaces and routing tables, allowing AirDrop to work seamlessly while the gateway is active.

727
728 **### 11.15 Unlocking Mode-Locked Output Files Without `chmod` (`unlock-files.sh`)**

729
730 ****The one-command fix:**** `bash scripts/unlock-files.sh` replaces every known locked inode in the project with a fresh writable one. No `chmod`. Covers `.env` (mode `000`), `adguard/work/userfilters/compiled\_rules.txt`, `ip\_blocklist.ipset`, `cache/\*.txt`, `cache/ip/\*.txt`, and `adguard/work/data/filters/\*.txt` (mode `0444`). Run it once after setup or after pulling an old repo; `toggle.sh` and `sync-rules.py` will then work without permission errors.

731
732 ****Context:**** The nullexit project has a strict project-wide policy of `no chmod from scripts` (see README 6). This rule exists because every prior `chmod 0444` (post-write tamper-proof lock) and `chmod 000` (post-toggle `.env` lock) call from `scripts/sync-rules.py` / `toggle.sh` could leave a file permanently unreadable on disk if the script exited early, was killed mid-flight, or simply set a restrictive mode without ever being re-flipped. We've stripped every `chmod` from the codebase. But files ****written** by older versions of those scripts are still on disk in those restrictive modes (`0444` for `compiled\_rules.txt`, `ip\_blocklist.ipset`, `data/filters/<id>.txt`; `000` for `.env` after end-of-toggle lock). Re-running `toggle.sh` or `scripts/sync-rules.py` against those leftovers hits `PermissionError: [Errno 13] Permission denied` immediately.

733
734 ****Why the stale permissions also block `mv`/`rm` of the parent folder:**** The restrictive modes (`000` on `.env`, `0444` on output files) don't just block writes they prevent the kernel from unlinking the file's dentry during a `mv` or `rm` of the ****containing directory**** (`adguard/work/`). macOS enforces this at the VFS layer: you own the directory, but you can't delete a directory that contains entries you can't write to. This made the entire `adguard/work/` tree unmovable/undeletable until the locked inodes inside it were replaced. `unlock-files.sh` resolves this permanently by swapping each locked inode for a fresh `0644` one via atomic rename (`cp` + `mv`), which operates on the directory entry rather than the file contents no `chmod` called, no permission bypass needed.

735
736 ****Problem:**** You (the owner) cannot `cat`, `cp`, `rm`, `>>` redirect, or any normal POSIX write against a `mode=000` file even though you own it the basic mode bits block ALL access for owner, group, and other. The script does not call `chmod` and you want a permanent fix that never relies on a chmod from any future run either.

737
738 ****Solution:**** Replace the locked inode with a fresh readable one. `mv` is atomic; mode bits live in the inode, not the directory entry. Two flavors cover every case we hit on a real machine:

739
740 ****Flavor A** locked file is mode `000` (owner can't even READ it):

741 NOPASSWD sudoers (`/etc/sudoers.d/nullexit`) already includes `usr/bin/python3`. So we read via `sudo python3` (which has the DAC override root has) and pipe the bytes around as base64 in user space, where the

```

    redirect happens with the user's normal umask = `0644`:
742 ```bash
743 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).
    decode())" > /tmp/snap.b64
744 base64 -d < /tmp/snap.b64 > <FILE>.new
745 mv -f <FILE>.new <FILE>
746 ls -la <FILE>          # mode is now 0644
747 ```
748 The old `mode=000` inode is unlinked by `mv`; the new `mode=0644` inode (created by base64-decode via the
    calling user's umask) takes the path. No chmod ever called. The parent directory just needs to be writable
    by you (it always is, since you own it).
749
750 **Flavor B locked file is mode `0444` (you can READ but cannot WRITE; `scripts/sync-rules.py` needs to re-
    write):**
751 You own the file and can read it, you just cannot truncate/overwrite it. The simplest path is to rename it
    aside and let the writer create a fresh inode from scratch (which gets the umask-default `0644`):
752 ```bash
753 mv <FILE> /tmp/old.<FILE>.$$
754 python3 scripts/sync-rules.py          # now writes <FILE> cleanly at mode 0644
755 ```
756 Or apply Flavor A if you'd prefer to preserve the contents while resetting the mode.
757
758 **Why this is the canonical recovery, not a hack:**
759 - Mode bits live in the inode, not the path. `mv -f` replaces the path's inode reference; the old locked inode
    drops out of the directory entirely (dentry eviction) and loses its last link, so the kernel reclaims it.
    The new inode (whatever it is: a freshly-written one, or a base64-decoded copy) gets the path.
760 - The only prerequisite to apply Flavor B's `mv` is: you own the parent directory (so you can unlink entries
    from it) AND you have a NOPASSWD-capable binary that can read the byte stream if mode prohibits user read.
761 - This pattern generalizes. Any time a script ends up producing a "locked file that the next run can't update"
    situation, the same trick applies without a single chmod anywhere.
762
763 **Empirical verification (user machine, July 1, 2026):**
764 - `.env` was `mode=000` (owner $USER, i.e. you): Flavor A unblocked it in 3 commands, no chmod.
765   - Before: `----- 1 $USER staff 899 Jun 28 07:07 .env`
766   - After:  `~rw-r--r--@ 1 $USER staff 899 Jul 1 20:39 .env`
767 - `docker compose config` immediately picked up `TS_AUTHKEY` + `WIREGUARD_PRIVATE_KEY` substitution.
768 - `compiled_rules.txt`, `ip_blocklist.ipset`, `data/filters/1782645604.txt` were `mode=0444`: Flavor B (`mv`-
    aside) unblocked all three.
769 - `scripts/sync-rules.py` then ran clean: 279587 block rules + 171 allow rules + 16710 IP entries compiled; new
    files came out at `0644`.
770 - `toggle.sh` then went end-to-end in 48s (warp / routing-fix / adguardhome / tailscale all healthy, gateway
    mesh-joined, DNS hijack verified single-server).
771
772 **When this trick is appropriate vs. inappropriate:**
773 - You own the parent directory and the file (typical desktop scenario).
774 - The lock is a leftover from a removed `chmod` call that you want off disk forever.
775 - NOPASSWD sudo includes at least one interpreter (`python3` on this system) that can be used to read mode-0
    files. If it does not, add it first: ` ALL=(ALL) NOPASSWD: /usr/bin/python3`.
776 - The file is owned by another user (root, another account) and the lock is intentional respect it; do not
    bypass another user's security boundary.
777 - The parent directory is owned/writable only by another user escalate properly via sudo, or stop and ask the
    user.
778 - You do not actually own the file and there is a legitimate reason for its lock state restore the file via a
    trusted source rather than circumventing the perms.
779
780 **Reusable cookbook:**
781
782 ```bash
783 # Quick diagnostic: figure out which flavor you need.
784 WHOAMI=$(whoami)
785 ls -la <FILE>
786 stat -f 'mode=%Sp owner=%Su group=%Sg' <FILE>
787 # mode=----- or mode=----r----- : you cannot even read it Flavor A.
788 # mode=r--r--r-- but writer fails   : you are blocked from write but can read Flavor B.
789 # mode=rw-r--r--                    : not actually locked at all; unrelated write failure.
790
791 # Flavor A (mode=000):
792 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).
    decode())" > /tmp/snap.b64
793 base64 -d < /tmp/snap.b64 > <FILE>.new && mv -f <FILE>.new <FILE> && rm -f /tmp/snap.b64
794
795 # Flavor B (mode=0444, file is readable):
796 mv <FILE> /tmp/old.<FILE>.$(date +%s)
797 # (let the original writer recreate <FILE>; new file lands at mode 0644)
798
799 # Verify after either flavor:
800 ls -la <FILE>          # mode should be 0644
801 <your normal validation command>
802 ```
803
804

```



```

805 ### 11.16 Gateway Breakage on Lid Close / Wi-Fi Roam Post-Wake + Post-Roam Auto-Recovery
806
807 **Context / Symptom.** Closing the lid (sleep/wake) OR losing + reconnecting Wi-Fi (e.g. in an elevator, caf,
      or roaming between APs) leaves the gateway in a partly-dead state. Symptoms range from a ~30s window of
      dead DNS to a permanent outage that only full `recover.sh` or `toggle.sh` fixes. The root cause is that
      nullexit has no daemon watching macOS sleep/wake or network-state-change events the existing DNS
      Watcher inside `toggle.sh` only runs in-process and is suspended along with the rest of the user shell on
      lid close, so the gateway cannot self-heal after either event.
808
809 **Diagnosis.** Two distinct failure modes share the same root gap.
810
811 * (A) Lid close sleep wake. `caffeinate -i` (used by `toggle.sh`) blocks idle sleep but does not
      block forced sleep from lid close. Closing the lid hard-suspends Colima VM + every container + every
      Tailscale-quit-shells-except-tailscaled. On wake the VM clock needs ~5-30s to resync via NTP, during which
      TLS cert validation fails: Cloudflare WARP `162.159.192.1:2408` rejects the handshake gluetun's
      healthcheck (`interval: 2s, retries: 15`) eventually flips unhealthy Docker `unless-stopped` restarts the
      `warp` container (~30s gap). `mDNSResponder`'s in-memory cache still holds pre-sleep entries (stale A
      records for tailnet hosts) so the first dozen DNS queries after wake time out. `tailscaled` on the host
      often keeps its stale DERP relay preference and routes packets through a dead relay for another 30-60s.
812 * (B) Wi-Fi roam / loss in elevators, captive portals. WARP is a UDP tunnel to Cloudflare's anycast edge.
      Carrier NATs and captive-portal networks silently invalidate the UDP binding on roam and on hotspot-bound
      re-association. macOS `networksetup` should preserve DNS settings on the same Wi-Fi service across a roam
      , but most captive-portal networks DHCP-replace DNS to the captive-portal resolver during the captive-
      portal dance itself, before the user has authenticated so even DNS hijack survives a clean roam and
      quietly breaks on captive-portal handoff.
813
814 **Resolution.** A single launchd LaunchAgent (`com.nullexit.wake-recovery`) runs a long-running shell
      daemon (`scripts/watcher.sh`) that listens for both event sources and, on each fire, executes `bash recover
      .sh --post-wake` a lighter-touch recovery mode that's the opposite of the existing --nuclear semantics
      : it keeps the gateway live while still refreshing every stale subsystem.
815
816 ##### Apple Power Management Primer (for the unfamiliar)
817
818 macOS exposes several relevant surfaces; the right primitive depends on what you actually need to detect. None
      of these are obvious and none of them have a standard splash screen in the docs.
819
820 * `caffeinate <flags>` creates I/O Kit power assertions via `IOPMAssertionCreateWithName`. It does NOT
      block forced sleep (lid close, low-battery shutdown, sleep timer firing on a per-set Energy Settings policy
      ). Flags:
821 * `-i` PreventIdleSleep (the only one `toggle.sh` uses; protects against the OS going idle while the user is
      active but a clamshell close still hard-puts it to sleep)
822 * `-s` PreventSystemSleep (only honored on AC power; the user may be on battery)
823 * `-u` DeclareUserActive (resets the idle timer every 30s by default; equivalent to keeping the cursor
      moving)
824 * `-d` PreventDisplaySleep
825 * `-m` PreventDiskIdleSleep
826 * There is NO `-l` (lid-block) flag in any modern macOS. Lid close is a kernel-firmware-level event that
      ignores user-space assertions.
827 * To prove any of this on live hardware: `pmset -g assertions` lists every active assertion with type /
      process / reason / timeout.
828 * `pmset` is the read-side of the same system:
829 * `pmset -g log | grep -E 'Sleep|Wake|DarkWake'` shows the recent timeline.
830 * `pmset -g history` is the per-day summary.
831 * `pmset -g assertions` is the live assertion list (matches `-i -s -d -u` to processes).
832 * `launchd` does NOT have a native "on-wake" hook. Not in any version of macOS as of Sonoma. The canonical
      recipe is `StartCalendarInterval` with a sub-minute cadence and let launchd queue missed intervals for
      replay-on-wake, or use a third-party daemon (Sleepwatcher). For our use case a long-running shell daemon
      listening to the unified log is more responsive than a polling agent.
833 * `com.apple.system.powermanagement.*` Darwin notifications (`willSleep`, `didWake`, `didDim`, `didUndim`)
      are the C-level signal. From a shell, they are best reached through the unified log with `log stream --
      predicate` see the watcher implementation.
834 * `scutil n.watch` is the canonical CLI for live-following network-state changes. Pairs with `n.add State:/
      Network/Global/IPv4` to get a single tap that fires on any IPv4 link/route/DNS/address change across all
      interfaces. This is what `scripts/watcher.sh`'s network-change listener uses.
835 * `com.apple.networkChange` Darwin notification is the C-level equivalent but has no shell-friendly
      listener; use `scutil` instead.
836 * `SCNetworkReachability` is the Objective-C API used by SOCKS5/HTTP clients to detect route changes per
      target. Never a fit for shell scripts.
837
838 ##### The Fix (component-by-component)
839
840 1. `recover.sh --post-wake` (new flag, non-destructive by design). Adding `--post-wake` to the existing
      nuclear recovery script inverts the semantics. The default mode still tears down Tailscale, resets DNS to
      empty, stops caffeinate, runs `docker compose down`, power-cycles Wi-Fi. The new mode does:
841 * Skip Tailscale disconnect (we WANT it to keep the mesh connection)
842 * Re-hijack DNS to the gateway Tailscale IP read from `ADGUARD_IP.txt` (instead of resetting to empty)
      applies to both `ACTIVE_SERVICE` and `ENO_SERVICE` because the system resolver is scoped to en0
843 * Refresh the exit-node preference with `tailscale up --reset --ssh=true --accept-dns=false --exit-node
      ="$TS_IP" --exit-node-allow-lan-access=true` (the exact flags `toggle.sh` START uses). This re-asserts
      DERP mapping without dropping the mesh

```



```

844 * Inspect `docker inspect --format '{{.State.Health.Status}}' nullexit-warp-1` and only `docker compose
up -d --force-recreate warp` if gluetun is unhealthy this single targeted recreate nudges the UDP NAT
binding back to Cloudflare without disturbing Tailscale or AdGuard
845 * Run the 10.27 sharingd-reset step (existing fix from `toggle.sh` START path) so AirDrop doesn't freeze on
the new IP lease
846 * Verify the gateway with `dig +tcp @127.0.0.1 -p 5354 google.com` (AdGuard via TCP through Colima's SSH
tunnel) and a 4-second `tailscale ping` to the gateway Tailscale IP, instead of the default mode's direct-
to-1.1.1.1 check
847 * Crucially: skip `sudo -v` because the LaunchAgent has no TTY to prompt on; all privileged calls use `
sudo -n` and lean on `/etc/sudoers.d/nullexit` NOPASSWD entries (`networksetup`, `dnsmasq`, `dscacheutil`,
`killall`, `pkill`).
848 2. toggle.sh writes and clears `/tmp/nullexit-gateway-active.marker` (an ISO-8601 UTC timestamp) at the
bottom of the START path and the top of the STOP path / `cleanup_handler`. Two new helpers: `
write_gateway_active_marker` and `clear_gateway_active_marker`. The watcher uses this file as its only
signal that the gateway is live: if `toggle.sh` was stopped (or never started), the watcher no-ops. This
prevents waking-up-only-on-counterfactual events from churning DNS.
849 3. scripts/watcher.sh (long-running daemon, sibling-style source file). Two backgrounded pipelines:
850 * Wake listener: `log stream --predicate 'composedMessage CONTAINS[c] "didWake" OR composedMessage
CONTAINS[c] "Wake from Sleep" OR composedMessage CONTAINS[c] "Waking from" OR composedMessage CONTAINS[c] "
DarkWake"' --style compact --info` piped through `while read; do run_recover "WAKE: ..."; done`. `[c]`
makes the predicate case-insensitive (the unified log writes phrases in mixed case across versions).
851 * Network listener: `(echo "n.add State:/Network/Global/IPv4"; echo "n.watch"; while true; do sleep
86400; done) | scutil 2>/dev/null` piped through a `case` match on `n.state` / `SCEventUpdate`.
852 * run_recover checks the marker, debounces via `/tmp/nullexit-watcher.last-recovery` (default 10s,
configurable via `NULLEXIT_DEBOUNCE_SECONDS`), and shells out to `bash recover.sh --post-wake`. The 10s
debounce prevents a single wake event (which typically emits 2-3 log lines) from triggering multiple
recoveries.
853 * `trap ... TERM INT` `kill 0` cleanly tears down the process group on launchctl `bootout` so the `sleep
86400` inside the scutil pipe also dies.
854 * `wait` blocks indefinitely while both pipelines feed it.
855 4. launchd/com.nullexit.wake-recovery.plist (LaunchAgent in the user domain runs as the console user at
every login without needing sudo).
856 * Label: com.nullexit.wake-recovery
857 * ProgramArguments: ["/bin/bash", "<absolute-path-to-your-nullexit-install>/scripts/watcher.sh"]
858 * RunAtLoad: true starts immediately on `launchctl load` (no need to log out and back in)
859 * KeepAlive: { SuccessfulExit: false, Crashed: false } don't restart on clean SIGTERM exit (so `
launchctl bootout` doesn't get stuck in a relaunch loop). Also don't restart on hard crash: we observed
that macOS Sonoma's sleep/wake suspend-resume path accumulates orphan watcher.sh PIDs because SIGCONT
does not reliably clean up the prior instance's listener grandchildren, and a `KeepAlive.Crashed=true`-
driven relaunch actively races with the still-live prior process. The script enforces single-instance on
its own via a `flock`-style PID-file lock, so a relaunch-on-crash would be skipped by the lock and produce
0 listener coverage until the live instance happened to die. Trade-off: a real crash leaves the user
without auto-recovery until they run `launchctl load -w` manually acceptable because (a) gateway still
works without auto-recovery, (b) a relaunch wouldn't fix whatever caused the crash, and (c) the gateway-
recovery itself is observably broken (no DNS, no exit-node) if it does go down.
860
861 ### 11.17 Host-Side Traffic Leaking Past the Exit Node (with Remote Clients Routing Correctly)
862
863 Symptom toggle.sh reports success. Remote tailnet clients (e.g. an S24 phone) correctly egress through
Cloudflare WARP and see a Cloudflare IP on `whatismyip.com`. But on the HOST Mac running the gateway, `
whatismyip.com` reports the underlying physical ISP's ASN (e.g. `campus_isp` for a university campus Wi-Fi)
. The gateway container itself is healthy; the leak is specifically on the host.
864
865 Root causes Three distinct mechanisms can each produce this exact symptom on the host alone, while leaving
remote clients unaffected. They are mutually rankable so the diagnostic picks the active one
deterministically.
866
867 * (A) SOCKS5 fallback lane active toggle.sh's three Phase-B pre-flight checks (`tailscale ping` AdGuard
via `dig +tcp` WARP internet, `toggle.sh` ~lines 750-820) sometimes flake during the first minute. When ANY
of the three fails, the script sets `SKIP_EXIT_NODE=true` (toggle.sh:849) and instead activates the SOCKS5
fallback path (toggle.sh:894). The SOCKS5 fallback hijacks DNS to `127.0.0.1` (so AdGuard filtering still
works) but modern browsers on macOS do NOT honor the system SOCKS5 proxy they ignore `networksetup -
setsocksfirewallproxy`. Result: DNS gets ad-filtered but actual HTTP/S traffic exits direct through the
campus ISP. This branch is invisible from a remote tailnet client because the client's tunnel is unaffected
.
868
869 * (B) IPv6 leak over campus Wi-Fi The Tailscale exit node and WARP tunnel are both IPv4-only (gluetun + `
post-rules.txt` drop all IPv6 forwarded traffic; devref 10.6). But many university and enterprise APs are
dual-stack the host happily accepts AAAA DNS responses and sends IPv6 traffic straight out `en0`. macOS
happily encrypts that traffic through Tailscale ONLY if Tailscale has IPv6 advertised; with an IPv4-only
exit node, IPv6 traffic skips Tailscale entirely. This branch is invisible from a phone because iOS/Android
Tailscale apps actively sinkhole IPv6 in their VPN profile, but the macOS host's `tailscaled` does not.
870
871 * (C) Route-table freeze and `--accept-routes` reset after `tailscaled` wake/toggle. Running `tailscale up
--reset` clears the exit-node preference and reverts `--accept-routes` back to its default (`false`).
Without explicitly passing `--accept-routes=true`, `tailscaled` will not attempt to install the default
route through the `utun` tunnel interface even if the exit node preference is reconciled. Additionally,
macOS occasionally freezes and fails to apply route-table changes (devref 10.26). In both cases, `netstat -
rn` shows the default route still pointing to the physical Wi-Fi gateway (e.g. `172.17.0.1`) instead of the
Tailscale `utun` interface, causing host traffic to exit direct while remote clients route correctly.
872

```

```

873 **Why remote clients don't see this.** Each remote phone/laptop uses its OWN Tailscale tunnel to reach the
      container's `100.x.x.x:443` over the mesh they're end-to-end encrypted regardless of how the host Mac
      itself is configured. Only the HOST's own egress sockets (HTTP requests from the host's browser, curl, etc
      .) are affected.
874
875 **Resolution.** A single script: `scripts/diagnose-host-leak.sh`. It runs the 7 checks, classifies into one of
      A / B / C / OK, prints a verdict + a ready-to-run fix command on stdout, AND writes the full report to `
      host-leak-diagnostic-<UTC>.txt` so the user can paste the file back instead of scrolling-and-copying from
      the terminal. With `--fix`, the matched remediation is applied and the two checks that could change (warp +
      ipv6) are re-verified. With `--watch` (or `--watch 30`), it runs the full baseline diagnostic then enters
      a continuous loop checking warp/IPv6/default-route every N seconds, alerting only on state changes logs to
      `host-leak-watch.log`.
876
877 ```bash
878 # Diagnose only:
879 bash scripts/diagnose-host-leak.sh
880
881 # Diagnose + apply matched fix + re-verify:
882 bash scripts/diagnose-host-leak.sh --fix
883 ```
884
885 **What it prints.** A 7-section report (`tailscale status`, SOCKS5 state, `cdn-cgi/trace` warp=, IPv6 leak
      probe, host default route, last toggle.sh output.log hints, resolved gateway Tailscale IP), followed by a
      classification block (`Scenario: A | B | C | OK`) with the exact command(s) the user needs to paste into
      their terminal they don't have to figure out which `tailscale up` flags match their environment.
886
887 **What it does NOT do.** It does not touch `toggle.sh` or `toggle.sh`'s pre-flight logic, does not modify `.env`
      (except `--fix` mode appends `GATEWAY_BYPASS_PING=true` when fixing Scenario A), and does not restart
      Docker. It is a read-only diagnostic with an opt-in remediation flag. Safe to run on a healthy gateway the
      worst case is a 30-line report that says "OK".
888
889 **Why cdn-cgi/trace over whatismyip.com for the verdict.** `whatismyip.com`'s ISP database routinely mislabels
      campus IPv6 ranges as residential ISPs (that's why the local ISP's ASN appears even when you're routing
      through Cloudflare). `cdn-cgi/trace` is hosted by Cloudflare ITSELF and reports `warp=on|off` plus the
      exact edge colo serving the request. It is the only public endpoint that can truthfully confirm whether the
      WARP tunnel is in use.
890
891 **Common false alarms.**
892
893 * `warp=on` but whatismyip.com still shows the local ISP. `whatismyip.com`'s ISP-detection database is
      unreliable on academic networks. Trust `cdn-cgi/trace`'s `warp=on` over `whatismyip.com`'s ISP string. Use
      `https://api.ipify.org` (returns just the IP, no ISP DB lookup) as a third confirmatory check.
894
895 * `tailscale status` shows the host online but exit node preference is missing. `tailscale up --reset --
      exit-node=` (empty value) clears any stale preference; `tailscale up --reset --exit-node=<100.x.x.x>` re-
      establishes it. The diagnostic prints the exact command with the resolved TS_IP filled in.
896
897 * Both IPv6 leak AND route-freeze flags set simultaneously. Scenario B outranks Scenario C fixing IPv6
      makes IPv4 the only viable egress, after which the route-freeze usually self-resolves within ~30s (or one
      more `toggle.sh` cycle).
898
899 #### 10.30.1 Host Leak Probe Continuous Sub-Second Egress Monitor
900
901 **What it is.** `scripts/host-leak-probe.sh` is a lightweight background daemon (~1.4 MB RSS, ~01% CPU) that
      polls `https://www.cloudflare.com/cdn-cgi/trace` every 300ms directly from the host via `curl` not via `
      docker compose exec`. This is a fundamentally different vantage point from the WARP Watcher (10.32): the
      WARP Watcher asks the WARP *container* whether it sees `warp=on`; the Host Leak Probe asks what a real
      browser request leaving the host's physical NIC looks like. The two blind spots it covers that the WARP
      Watcher cannot:
902
903 1. Host-side flash-leaks a Cloudflare anycast edge re-route, a browser reusing a stale pooled connection
      across a tunnel state change, or a split-second routing gap during a Gluetun healthcheck restart (see 10.18
      acknowledged 14s window).
904
905 2. Host egress while container is healthy the container can report `warp=on` while the host's own traffic
      exits direct (the three-scenario leak class documented in 10.30).
906
907 **Lifecycle.** Started by `toggle.sh`'s START path (`start_host_leak_probe`) immediately after `
      start_warp_watcher`. Controlled by:
908
909 | Signal path | What happens |
910 |---|---|
911 | `toggle.sh` STOP | `stop_host_leak_probe()` SIGTERM + PID file cleanup |
912 | `toggle.sh` `cleanup_handler` (error/INT/TERM/HUP) | Same |
913 | `recover.sh` (default, non `--post-wake`) | 6d block kill by PID file |
914 | `recover.sh --post-wake` | Left running the gateway is still live |
915
916 PID file: `/tmp/nullexit-host-leak-probe.pid`. Toggle with `HOST_LEAK_PROBE=false` in `.env` to disable at next
      gateway start.
917
918 **Output format.** All events are written to `output.log` (repo root) with UTC timestamps. Only state *changes*
      are logged the probe is silent during normal healthy operation.
919
920 ```

```

```

918 [09:10:26] LEAK warp=off ip=45.23.1.1 prev_warp=on prev_ip=104.28.246.50
919 [09:10:26] ROTATE warp=on ip=104.28.248.11 prev_ip=104.28.246.50
920 [09:10:26] HOST-PROBE failed/timeout (count=1)
921 ```
922
923 curl transport errors (`TLS handshake failed`, `Connection refused`, etc.) are also appended to `output.log`
924 via `2>>output.log` nothing is silenced to `/dev/null`.
925
926 **Grep cheatsheet.**
927 ```bash
928 grep LEAK output.log          # real warp=off events from the host
929 grep ROTATE output.log        # Cloudflare anycast edge rotations (harmless)
930 grep 'HOST-PROBE' output.log  # curl failures / timeouts
931 ```
932
933 **Resource budget.** ~1.4 MB RSS, ~0% CPU at rest, ~3 HTTPS requests/second to Cloudflare. Safe to leave
934 running for hours. Back off the polling interval (`bash scripts/host-leak-probe.sh 1.0`) if you observe
935 probe-failure pile-ups indicating Cloudflare rate-limiting.
936
937 **Detection vs prevention.** This script detects leaks; it does not prevent them. A sub-300ms flash-leak can
938 still slip between two polls undetected. If `grep LEAK output.log` shows confirmed leak events, the next
939 step is a kill-switch (prevention) an `iptables` rule on the host's `en0` that drops non-Tailscale, non-
940 local egress whenever the gateway is active. That is a separate engineering task not yet implemented.
941 * `ProcessType: Background` hint to App Nap not to suspend us.
942 * `StandardOutPath/StandardErrorPath: /tmp/nullexit-watcher.{out,err}.log` separates launchd-level
943 diagnostics from the script's own `/tmp/nullexit-watcher.log` (which `exec >>` redirects).
944
945 ##### Install (one-time)
946
947 The plist lives in the repo at **`launchd/com.nullexit.wake-recovery.plist`** for version control. The
948 installed (live) copy must land in `~/Library/LaunchAgents/` so launchd picks it up on the user session.
949
950 ```bash
951 # Copy the plist to the user-launchd location (run from repo root)
952 cp ./launchd/com.nullexit.wake-recovery.plist \
953 ~/Library/LaunchAgents/
954
955 # Substitute __NULLEXIT_HOME__ with your local install absolute path.
956 # The plist uses WorkingDirectory + a relative program arg, so this
957 # single substitution is the only place it references your machine.
958 sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" \
959 ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
960
961 # Validate the XML
962 plutil -lint ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
963
964 # Load + persist this user session + every future login
965 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
966
967 # Confirm it actually started
968 launchctl list | grep nullexit
969 ```
970
971 To **stop** the watcher (without uninstalling):
972
973 ```bash
974 launchctl bootout gui/$UID/com.nullexit.wake-recovery
975 # or:
976 launchctl unload -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
977 ```
978
979 To **uninstall** completely:
980
981 ```bash
982 launchctl bootout gui/$UID/com.nullexit.wake-recovery 2>/dev/null || true
983 rm ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
984 ```
985
986 ##### Why this fix was preferred over alternatives
987
988 * **Polling with `StartCalendarInterval`** would have given ~30-60s detection latency per wake. The unified-log
989 stream is sub-second.
990 * **Sleepwatcher** would have covered wake events but not network changes. We'd still need a separate `scutil`
991 listener, and introducing a third-party dependency for what amounts to ~150 lines of shell is unjustified.
992 * **Forcing `caffeinate -l`** doesn't exist and the closest equivalent (`caffeinate -s`) only works on AC power
993 . The whole point of the gateway is to stay usable on battery while traveling lid close must NOT silently
994 kill the gateway.
995 * **A kernel extension** is impossible (no entitlement) and out of scope.
996
997 ##### Reproduction recipe

```

```

987
988 Test both events in isolation to confirm the fix actually closes the gap.
989
990 * **(A) Lid-only repro**: with the gateway up, run in a terminal: `pmset displaysleepnow && sleep 30 && pmset
wake` (without physically closing the lid). Watch the wake wake-fires the watcher post-wake routine runs
login should still work in <5s after wake. Compare to baseline pre-fix: pre-fix the gateway would be dead
for 30-90s.
991 * **(B) Roam-only repro**: with the gateway up: `networksetup -setairportpower off && sleep 30 && networksetup
-setairportpower on` (or simply walk out of Wi-Fi range and back). The captive-portal WILL repave DHCP DNS
for a few seconds; the watcher fires on the second `State:/Network/Global/IPv4` change (after the Wi-Fi re-
associates) and the post-wake routine re-hijacks DNS within 2-3s.
992
993 ##### Operative files
994
995 * `recover.sh` added arg parsing; wrapped destructive sections behind `POST_WAKE`; added re-hijack DNS /
refresh exit-node / force-recreate-if-unhealthy path.
996 * `toggle.sh` added `write_gateway_active_marker` / `clear_gateway_active_marker` called at the END of START
and TOP of STOP / cleanup_handler.
997 * `scripts/watcher.sh` new long-running daemon.
998 * `launchd/com.nullexit.wake-recovery.plist` launchd LaunchAgent descriptor.
999 * `~/Library/LaunchAgents/com.nullexit.wake-recovery.plist` the installed copy (one-time copy).
1000
1001 ##### Honest assessment (read before testing)
1002
1003 This fix has two asymmetric halves:
1004
1005 * Network / roam / captive-portal path RELIABLE. ** `scutil n.watch` on `State:/Network/Global/IPv4{6}`
catches every Wi-Fi roam, captive-portal DHCP-rebind, hotspot handoff, and VPN change with zero missed
events. To validate, drop Wi-Fi for 30 s and re-add it; `/tmp/nullexit-watcher.log` will record a `NET:`
trigger and `recover.sh --post-wake` will run within ~10 s of the network coming back.
1006 * Lid close / wake path BEST-EFFORT on macOS Sonoma+. ** Apple *suspends* LaunchAgents process-state during
forced sleep and *resumes* them on wake; it does NOT re-launch them every wake. As a result: (a) `log
stream` is a live subscription events emitted during the agent's suppressed window are DROPPED, not
buffered, so the wake event that prompted the resume is invisible to `log stream` on resume; (b) the "fire
on startup" guard runs once on `launchctl load -w` and after a `KeepAlive` crash-recovery, NOT on every
successive wake; (c) `subsystem == "com.apple.powermanagement"` will catch FUTURE emissions (display dim/
restore, manual sleep/wake) but not the lid-closeopen wake directly.
1007
1008 In practice the lid-wake gap is short, because `toggle.sh`'s existing DNS Watcher already polls every 5 s and
re-hijacks DNS (so DNS recovers within 5 s of any roam or wake), `tailscaled` self-heals via DERP fallback
within 3060 s, and `gluetun` reconnects via its healthcheck retry within 30 s. The user-visible pain on lid
open is ~30 s of wonky DNS, not a permanent outage.
1009
1010 Test the ROAM path FIRST. ** If ROAM works in your testing, the lid-wake gap is acceptable. If lid-wake
handling is critical, the two clean follow-ups are:
1011 1. Sleepwatcher (one canonical install): `brew install sleepwatcher`; drop `recover.sh --post-wake` into
`~/sleep` and `~/wake` so Sleepwatcher invokes it directly on every wake without the launchd propagation
problem.
1012 2. Move the watcher to a LaunchDaemon (root) and use `IOPMSchedulePowerEvent` via a tiny C wrapper not
portable to shell.
1013
1014 A truly reliable shell-only solution to "wake events from inside a LaunchAgent" does not exist on macOS Sonoma
+.
1015
1016 ##### Empirical lessons from deployment
1017
1018 Two findings came up while deploying this fix end-to-end; recording so the next operator doesn't lose an
evening to each.
1019
1020 1. Bash function-call-before-definition gotcha. ** While introducing the gateway-active marker helpers, the
defensive `clear_gateway_active_marker` call was inserted at the very top of `toggle.sh` (right after `set
-e`), but the function definition lived near `PID_FILE="/tmp/nullexit-cafeinate.pid" ~90 lines down. Bash
parses top-to-bottom and resolves names at first invocation, so the call site exited with code 127 (`
clear_gateway_active_marker: command not found`). The gateway stayed unreachable from the START path even
though `bash -n` passed (the parser doesn't catch order-of-resolution errors). Rule: define functions
BEFORE any block that calls them. Verify with `grep -n '^name()\s*{'` followed by `grep -n '^name\s*${`
the latter must appear on a line number AFTER the former.
1021
1022 2. `networksetup -setairportpower off+on` is not a faithful roam reproduction. ** Synthetic Wi-Fi radio-
cycling (`sudo networksetup -setairportpower Wi-Fi off` for 30 s, then back on) is expected to produce a `
NET:` trigger in `scutil n.watch` but produced ZERO during testing because `setairportpower` powers the
radio at the driver level while leaving the network SERVICE in the "up" state from scutil's perspective,
so no `n.state State:/Network/Global/IPv4` event is generated. A real roam (macOS briefly loses the AP and
re-associates into a different one) DOES fire the event because the link actually changes.
1023 Better reproduction recipes in priority order:
1024 * Walk between two real SSIDs via `networksetup -switchtolocation` (or actual physical station movement)
guaranteed to fire the scutil event.
1025 * Force a DHCP rebind on the same SSID with `sudo ipconfig set en0 DHCP` triggers `State:/Network/Global/
IPv4` to update.
1026 * Trust the existing 30-second DNS Watcher inside `toggle.sh` for the DNS path: it re-hijacks DNS
automatically. The post-wake watcher adds clear value mostly for tailscale exit-node re-assertion and warp

```

```

gluetun force-recreate, both of which self-heal within ~30 s anyway.
1027
1028 ### 11.18 Infinite Egress Routing Loops & Subnet Takeover Paradoxes
1029
1030 **Problem:** Two distinct network loops can break host egress connectivity entirely when the exit node is
active:
1031
1032 1. **The Recursive Host-VM Tunnel Loop:**
1033   - **Mechanism:** The host Mac's default route points to the Tailscale exit node (`utun*`). The exit node
container sends its encrypted tunnel packets (destined for the WARP endpoint `162.159.192.1`) back to the
host via the VM NAT. Without a static route exception on the host Mac, the host Mac routes those packets
right back into the exit node (`utun*`), creating an infinite loop that blackouts all host network egress.
1034   - **ARP-Scoping Pitfall:** Simply binding the bypass route to the interface (`route add -host 162.159.192.1
-interface en0`) fails when the default route is deleted or redirected to `utun*`. Since `162.159.192.1` is
not local to the physical link, macOS fails to resolve it via ARP, dropping all tunnel packets.
1035   - **Fix:** Dynamically resolve the active physical gateway IP (e.g. `172.17.0.1`) on startup, and add the
static host routes for the WARP endpoints via the gateway IP directly (`route add -host 162.159.192.1 <
gateway_ip>`). Additionally, because standalone `tailscaled` on macOS does not automatically modify the
system default route via the CLI, we manually redirect the default gateway to `utun*` using `
setup_exit_node_routing`.
1036
1037 2. **The Docker Compose Subnet Takeover Collision:**
1038   - **Mechanism:** To avoid collisions with the host's campus network, Colima's default bridge (`docker.bip`)
is moved (e.g. to `10.200.0.1/24`). However, because `172.17.0.0/16` is now free inside the VM, Docker
Compose's IPAM dynamically takes it over for the project's custom network (`nullexit_default`). Because the
containers receive IPs on `172.17.0.0/16`, the host Mac cannot send local peer discovery packets (e.g.,
Tailscale `magicsock` packets on `172.17.0.2:41641`) directly. The host routing table sends them out `en0`
to the physical Wi-Fi, returning `host is down` or `no route to host`.
1039   - **Fix:** Explicitly configure a custom default network subnet `10.200.1.0/24` in `docker-compose.yml` to
prevent Docker Compose from ever reclaiming the conflicting `172.17/172.18` ranges.
1040
1041 > See also 10.23 (The Hotspot Paradox) for the related infinite loop that occurs when the physical upstream
router is also a Tailscale exit-node client.
1042
1043 ### 11.19 In-Flight WARP Tunnel Liveness Monitor & Statistical Auto-Shutdown
1044
1045 **Why.** nullexit's core promise is that your IP always appears as Cloudflare. If the WARP tunnel silently dies
due to a UDP NAT timeout, a Cloudflare edge rotation, a Colima VM clock skew causing TLS cert rejection,
or any of a dozen other failure modes the host silently falls back to egressing through the physical ISP.
The user sees the gateway as "up" (containers running, Tailscale connected, DNS hijacked) but their real IP
is exposed. This is the exact class of failure that Ingo Blechschmidt documented in his chilling 2024
Linux kernel post-mortem: for over two years, LUKS disk encryption on suspend was silently broken because a
refactored kernel syscall returned success while doing nothing ([source](https://mathstodon.xyz/@iblech
/116769502749142438)). The lesson: **a security mechanism that fails silently is worse than no mechanism at
all** it creates a false sense of safety that prevents the user from taking corrective action.
1046
1047 **Design principle.** The WARP Watcher (`start_warp_watcher()` in `toggle.sh`) is a background daemon that
polls `cdn-cgi/trace` every 30 seconds and applies a **statistically calibrated threshold** before
triggering any safety response. This threshold distinguishes transient network blips (which self-heal
within seconds) from genuine outages (which require intervention). A single `warp-off` reading is
meaningless Docker might be slow, the network might be congested, a container healthcheck might be
restarting. But 6 consecutive off readings (30 continuous seconds of downtime) has vanishingly low
probability of being a false positive: even at an unrealistically high 10% false-positive rate per poll, P
(6 consecutive) = 0.1 0.0001%. In practice, the per-poll false-positive rate is far lower than 10%, making
the real probability astronomically small.
1048
1049 **What it does.**
1050
1051 1. **Always logs.** Every state transition (onoff, offon) is timestamped to `output.log`. The user can `grep
WARP output.log` to audit the tunnel's entire lifetime. This is the "never fail silently" mandate.
1052
1053 2. **Tracks consecutive failures.** A counter increments on each `off` reading and resets to 0 on any `on`
reading. This ensures the watcher never fires on intermittent flapping.
1054
1055 3. **Auto-shuts down at threshold.** When the counter reaches `WARP_FAIL_THRESHOLD` (default 6, configurable in
`.env`), the watcher declares a statistically significant outage and runs `recover.sh` the nuclear
recovery script that disconnects Tailscale, stops Docker containers, resets DNS to `1.1.1.1`, flushes
routes, and power-cycles Wi-Fi. This instantly reverts the host to normal (un-encrypted) internet,
guaranteeing no traffic leaks through a dead tunnel while the user thinks they're protected. A trigger file
at `/tmp/nullexit-warp-shutdown-triggered` prevents double-firing.
1056
1057 4. **Exits cleanly.** After shutdown, the watcher exits (the gateway is dead, there's nothing left to monitor).
`stop_warp_watcher()` is also called by `toggle.sh`'s STOP path and `cleanup_handler` to terminate the
daemon cleanly during normal teardown.
1058
1059 **Configuration.** Set `WARP_FAIL_THRESHOLD=3` in `.env` for a more aggressive 15s timeout, or `
WARP_FAIL_THRESHOLD=12` for a conservative 60s window. The variable is parsed from `.env` using the same `
grep` pattern as `GATEWAY_MSS` and `GATEWAY_BYPASS_PING`.
1060
1061 **Log sample.** After a real outage (or a forced test via `docker compose pause warp`):
1062

```



```

1063 ---
1064 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1065 [2026-07-03T21:15:47Z] WARP SHUTDOWN 6 consecutive failures (threshold=6). Running recover.sh to kill the
1066 gateway and restore normal internet. Your IP is NO LONGER Cloudflare.
1067 [2026-07-03T21:15:52Z] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is now your real ISP IP.
1068 ---
1069 Or, if WARP self-recovers before the threshold:
1070 ---
1071 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1072 [2026-07-03T21:15:32Z] WARP RECOVERED warp=on after 3 consecutive off readings (threshold was 6)
1073 ---
1074
1075
1076 **Relationship to other monitors.** The WARP Watcher is the **innermost safety layer** it runs as a child of `
toggle.sh` and only while the gateway is active. It complements (does not replace) the `scripts/watcher.sh`
daemon (10.29, which handles sleep/wake and Wi-Fi roam) and `scripts/diagnose-host-leak.sh` (10.30, which
is a manual diagnostic tool). The WARP Watcher is the only component that actively verifies end-to-end
tunnel liveness and takes autonomous action when it fails.
1077
1078 ---
1079
1080 ## 12. Recent Updates
1081
1082 Reflects the current branch's state. Each entry one line + commit hash; read the commit message for detail.
1083
1084 - **July 10, 2026 `fix(race): suppress WARP Watcher during post-wake force-recreate`** Fixed a race condition
where switching Wi-Fi caused the host IP to leak as the raw ISP IP (`132.x`) on every network roam. `
recover.sh --post-wake` force-recreating the warp container (~35s) caused the background WARP Watcher to
accumulate 6 consecutive `warp=off` readings and fire nuclear `recover.sh`, killing a gateway that was
healing itself. Fixed by adding a `/tmp/nullexit-warp-inhibit.marker`: `recover.sh` writes it at startup in
`--post-wake` mode (with `trap ... EXIT` for guaranteed cleanup); the WARP Watcher loop in `toggle.sh`
checks it first and resets `consec_off=0` while it exists. See 25 for full post-mortem.
1085
1086 - **July 6, 2026 `fix: resolve edge-case vulnerabilities and system state leaks`** Addressed a suite of
subtle but critical edge cases identified in security review:
1087 1. **DNS Watcher Interface Independence:** Fixed `start_dns_watcher` in `toggle.sh` to dynamically detect the
active interface via `get_active_service()` rather than hardcoding a `grep` for "Wi-Fi", ensuring roaming
on Ethernet/Thunderbolt stays protected.
1088 2. **Robust Lock Verification:** Upgraded the `toggle.sh` lock-file logic to use `ps -p` verification,
preventing permanent lockouts if a crashed script's PID is recycled.
1089 3. **Fail-Closed Crypto:** Changed the `crypto.sh` integrity check in `toggle.sh`/`recover.sh` from an
executable bit check (`[ -x ]`) to a pure file-existence check (`[ -f ]`), guaranteeing execution via `bash
scripts/crypto.sh` even if git strips the `+x` bit.
1090 4. **Strict `iptables` Pinning:** Upgraded `post-rules.txt` FORWARD-chain rules to explicitly pin traffic
between `tailscale0` and `tun0` in both directions, mathematically preventing Tailscale traffic from
escaping through `eth0` during WARP tunnel drops.
1091 5. **Docker Local Network Isolation:** Modified `docker-compose.yml` to explicitly bind the exposed WARP
container ports (`1080`, `5354`) to `127.0.0.1`. This prevents other devices on the local bridged Wi-Fi/LAN
from accessing the SOCKS5 proxy or DNS resolver if Colima provisions a LAN IP.
1092 6. **Routing Verification & Sudoers:** Added a `netstat -nr` check (handling macOS `0/1` syntax) to verify
the default route (`0.0.0.0/1`) actually overrides to Tailscale, and updated the `README.md` and `setup.sh`
sudoers templates to ensure `/usr/bin/python3` is permitted so the fallback DNS proxy doesn't silently
fail.
1093
1094 - **July 4, 2026 `cc789c5` (fix: resolve concurrency, setup variables, CI workflow, and untrack review file)**
Added concurrency protection on `toggle.sh` using `/tmp/nullexit-toggle.lock` to prevent parallel runs.
Initialized `TS_AUTHKEY` defensively in `setup-common.sh` to prevent unbound variable crashes under `set -u`
. Added Go compilation (`go vet` and `go build`) steps to GitHub Actions CI workflow. Ported AdGuard Home
container to cap logs. Fixed macOS BSD `grep` compatibility in `setup-common.sh` using `grep -oE`, resolved
space stripping in `common.sh`, fixed quoting/PID file/routing text in `recover-linux.sh`, and removed
redundant bypass routes block in `recover.sh`.
1095
1096 - **July 2, 2026 `8c5fae1` + `181e1e1` (feat(routing): resolve recursive host-VM tunnel loop & Docker subnet
collision)** Two infinite routing loops fixed. (1) Host-VM tunnel loop: WARP endpoint bypass routes now
injected via physical gateway IP (not interface scoping) to avoid ARP failures under default-route
redirection; `setup_exit_node_routing` added to manually force the host default route to `utun*` since
standalone `tailscaled` CLI on macOS does not update the routing table automatically. (2) Docker Compose
subnet takeover: `docker-compose.yml` now locks the default project network to `10.200.1.0/24` to prevent
IPAM reclaiming `172.17/172.18` after `docker.bip` is moved. `--accept-routes=true` added explicitly to all
`tailscale up --reset` calls. Documented in 10.31.
1097
1098 - **July 4, 2026 `fix: post-wake routing loop & destructive recovery` (networking: routing)** Resolved a
critical fatal bug where waking the Mac from sleep or roaming Wi-Fi permanently broke the internet and
required a full system reboot to fix.
1099 1. **The Routing Loop Bug:** `common.sh`'s `add_warp_bypass_routes` was previously refactored to dynamically
look up the default route without accepting explicit interface arguments. During `post_wake`, Tailscale is
already active, so `route get default` resolved to the Tailscale `utun` interface. This caused Cloudflare
WARP traffic to be routed back *into* Tailscale, creating an infinite recursive routing loop that instantly
killed the internet on every wake. Fixed by making `add_warp_bypass_routes` accept explicit target
arguments and updating `recover.sh` to pass the correct physical interface/gateway.

```

```

1100 2. **The Destructive Recovery Bug:** When users ran `recover.sh` to fix the broken internet, the script
    blindly ran `sudo route -n flush`. On macOS, this doesn't just clear VPN routes it destroys essential system
    routes including loopback (`127.0.0.1`) and multicast (`224.0.0.0`). Bouncing Wi-Fi does not automatically
    restore these internal routes, meaning macOS network services (`configd`, `mDNSResponder`) remained
    permanently wedged until a reboot. Fixed by replacing `route -n flush` with targeted deletions of specific
    gateway routes (`0.0.0.0/1`, `128.0.0.0/1`) and WARP bypass routes.
1101
1102 - **July 2, 2026 `feat: secure execution` (security: script lockdown & auth)** Fixed a major privilege
    escalation vulnerability and added native macOS authentication prompts to `Toggle Gateway.app` and `Recover
    Gateway.app`. Previous setups granted `/usr/bin/python3` blanket `NOPASSWD` sudo access, allowing malware
    to bypass macOS security. We restricted `NOPASSWD` exclusively to `/usr/bin/python3 /Users/<USER>/Developer
    /nullexit/scripts/dns-proxy.py`. Additionally, `scripts/dns-proxy.py` was locked down with `chown root:
    wheel` and `chmod 755` so attackers cannot overwrite the script and exploit the restricted sudo rule. `
    toggle.sh` was also updated to capture `warp` container logs on failure before tearing down the environment
    .
1103 - **July 3, 2026 `fix: resolve numerous system regressions`** Addressed several breakages introduced in
    recent commits:
1104 1. Fixed a truncated line syntax error in `/etc/sudoers.d/nullexit` that broke `NOPASSWD` fallback commands.
1105 2. Fixed a `[Errno 13] Permission denied` error in `sync-rules.py` cache writes by using a temporary file and
    atomic `os.replace()` to bypass inherited mode locks (0444/000). ** (Later reverted see below.) **
1106 3. Fixed a quoting bug in `recover.sh` where `ts_args=""` passed literal escaped quotes.
1107 4. Repaired ANSI color outputs in `diagnose-host-leak.sh` and `fix-docker-bridge-collision.sh` using `%b`
    instead of `%s` and `-e` flags.
1108 5. Fixed `toggle-linux.sh` calling a macOS-only `networksetup` command instead of `resolvectl dns`.
1109 6. Corrected the AdGuard REST API refresh POST in `sync-rules.py` to correctly target port `3000`.
1110 7. Restored the missing `START_GATEWAY=true` variable in `toggle.sh` and purged a stale port `80` mapping
    from `docker-compose.yml`.
1111 8. Verified `output.log` is untracked and safely in `.gitignore`.
1112 - **July 3, 2026 `unlock-files.sh` + revert atomic writes in `sync-rules.py`** The `.tmp` + `os.replace()`
    atomic-write pattern introduced in the commit above was a workaround for stale restrictive file permissions
    (mode `0444`/`000`) left on disk by old `chmod` calls that had already been removed from the codebase.
    Rather than keep the workaround, we created `scripts/unlock-files.sh` a one-shot script that permanently
    fixes the stale permissions by replacing each locked inode with a fresh writable one via atomic rename (`cp
    ` + `mv`), operating on the directory entry instead of the file contents. No `chmod` used. All 5 `.tmp` +
    `os.replace()` sites in `sync-rules.py` were reverted back to direct writes. `scripts/unlock-files.sh`
    covers: `.env` (mode `000`), `adguard/work/userfilters/compiled_rules.txt`, `ip_blocklist.ipset`, `cache/*.
    txt`, `cache/ip/*.txt`, and `adguard/work/data/filters/*.txt` (mode `0444`). Run once after setup or
    pulling an old repo; `toggle.sh` and `sync-rules.py` then work without permission errors. Documented in
    10.28 + 3.
1113 - **July 1, 2026 `c5b92c0` (refactor: personal-leak cleanup)** eliminated every residual personal-username
    leak across source + config files. The launchd `com.nullexit.wake-recovery.plist` now uses
    `WorkingDirectory` + a relative program arg with the `__NULLEXIT_HOME__` sentinel; the install recipe in
    10.29 gains a single `sed -i 's|__NULLEXIT_HOME__|$(pwd)|'` step right after the `cp`. 8 `devref.md`
    prose sites genericized to `~/Library/LaunchAgents/...`, `$USER` for sample output, `**July 1, 2026 `a5e2a5a` (refactor: script-relative paths)** replaced 6 hardcoded `/Users/<user>/<anywhere
    >/nullexit/...` source-code sites (former per-user install-location literals) with script-relative
    resolution. `recover.sh` injects `SCRIPT_DIR=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)` near the top
    and uses it for the post-wake warn (1.321) and the broken-for-60s hint echo (1.446). `scripts/watcher.sh`
    uses the same pattern; `RECOVER=$SCRIPT_DIR/./recover.sh` resolves without launching-path dependency.
1115 - **June 28, 2026 `f8f8e4e` (M1: scripts/ move)** 8 internal-collaborator scripts (routing-fix.sh, socks5-
    proxy.py, dns-proxy.py, sync-rules.py, logger.py, toggle-linux.sh, recover-linux.sh, setup-linux.sh)
    consolidated into a `scripts/` subfolder. The 4 user-facing / orchestrator / config files (`toggle.sh`,
    `recover.sh`, `setup.sh`, `docker-compose.yml`) deliberately stayed at repo root.
1116 - **June 28, 2026 `0875ef3` (ci: glob)** CI `py_compile` line switched from `python3 -m py_compile *.py
    scripts/*.py` to `python3 -m py_compile scripts/*.py` after M1 left root with zero `.py` files (the old
    form was lazy-empty on bash `nullglob=off` runners).
1117 - **June 28, 2026 `25b37a1` (docs: scripts/ prefix)** `devref.md` + `README.md` updated to use the `scripts/`
    prefix for every reference of the 8 moved files. ~22 sites in devref + ~9 sites in README = ~31 patches.
1118
1119 ### End-to-end verification (commit `c5b92c0` cyclone)
1120 Manual START STOP cycle ran clean on macOS after the path relativization + personal-leak cleanup landed:
1121 - `bash toggle.sh` START: exit 0, ~135s (cold Colima boot); marker present (`2026-07-02T03:41:41Z`), all 5
    containers healthy, host DNS hijacked to `100.100.21.8` (no fallback), exit node selected, Cloudflare trace
    through WARP succeeds.
1122 - `bash toggle.sh` STOP: exit 0, ~12s; marker cleared, all nullexit containers gone, host DNS restored to
    `1.1.1.1`, Tailscale disconnected.
1123
1124 ---
1125
1126 ## 13. Privacy Architecture, Threat Model & Upgrades
1127
1128 ### Layered Defense
1129 The gateway stacks four layers targeting different attack surfaces:
1130
1131 - **Layer 1 ISP-Level Surveillance (WARP):** Your ISP sees only an encrypted WireGuard tunnel to a Cloudflare
    IP. In the US, ISPs can legally sell your browsing metadata and are subject to secret NSLs (National
    Security Letters). WARP removes their visibility entirely.
1132 - **Layer 2 DNS Tracking (AdGuard Home):** DNS is the phone book of the internet. By default, queries go to
    your ISP's resolver, giving them a complete log of your traffic. AdGuard intercepts all DNS from mesh
    devices, blocks trackers, and resolves queries through the WARP tunnel.

```



```

1133 - **Layer 3 Device Identity & IP Exposure (Tailscale):** On untrusted networks (coffee shops, public Wi-Fi),
    your device's IP is exposed. Tailscale creates an encrypted WireGuard mesh between your devices, routing
    all traffic to your gateway exit node.
1134 - **Layer 4 Kernel-Level IP Blocking (ipset/iptables):** Sophisticated malware bypasses DNS entirely with
    hardcoded IPs. The `rule-compiler` fetches ~16,700 threat-intelligence IPs/CIDRs (Spamhaus, Feodo Tracker,
    Emerging Threats, CINS) and enforces them as `DROP` rules in the kernel `FORWARD` chain blocking both
    outbound C2 connections and inbound attack traffic.
1135
1136 ### The Documented Threat: Why This Matters
1137 This architecture is calibrated against documented mass-surveillance programs revealed in the Snowden
    disclosures:
1138 - **PRISM:** Bulk collection of communication content from major tech providers.
1139 - **UPSTREAM:** Tapping physical backbone fiber, collecting metadata and content in bulk.
1140 - **Keyscore:** Retroactive search of unencrypted traffic.
1141 - **MUSCULAR:** Infiltration of unencrypted internal datacenter interconnect links.
1142
1143 Passive bulk collection is not paranoia it is a documented reality. By double-encrypting and routing traffic,
    nulloxit prevents your ISP from harvesting your metadata.
1144
1145 ### Trust Assumptions & Shifting
1146 Every component shifts trust rather than eliminating it:
1147 1. **Physical Device Integrity:** You trust your physical devices are not compromised.
1148 2. **Cloudflare Integrity:** You trust Cloudflare not to correlate your WARP tunnel with exit traffic.
1149 3. **Tailscale Integrity:** You trust Tailscale's coordination server not to MITM WireGuard keys.
1150
1151 The goal: no single provider has a complete picture. Your ISP sees an encrypted tunnel. WARP sees traffic with
    no account identity. Tailscale sees node topology but not content.
1152
1153 ### Recommended Upgrades
1154
1155 | Layer | Default | Upgraded |
1156 |---|---|---|
1157 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, proven no-logs, anonymous payment) |
1158 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH (Cloudflare-blind) |
1159 | Coordination | Tailscale (US, OAuth) | Headscale (self-hosted, no third-party) |
1160 | Auth | OAuth / persistent keys | Ephemeral auth keys (no identity persistence) |
1161 | Content | WireGuard asymmetric | WireGuard + PSKs (coordination server blind) |
1162
1163 #### Mullvad (Replacing WARP)
1164 Swedish-based. Police raided their offices in 2023 and left empty-handed no logs exist to seize. Accounts are
    random numbers. Payment by cash or Monero. Replace `VPN_SERVICE_PROVIDER=custom` with `VPN_SERVICE_PROVIDER
    =mullvad` in `docker-compose.yml`.
1165
1166 #### Mullvad DoH Upstreams
1167 Point AdGuard's upstream resolvers to `https://adblock.dns.mullvad.net/dns-query`. Cloudflare WARP only sees
    encrypted HTTPS to Mullvad's IPs completely blind to your DNS queries.
1168
1169 #### Headscale
1170 Self-hosted Tailscale coordination server. Eliminates Tailscale the company, keeping all node topology under
    your control.
1171
1172 #### Ephemeral Auth Keys
1173 Generate in the Tailscale admin panel. Used once to authenticate; node disappears from the admin panel after
    disconnect. Breaks persistent identity linkage.
1174
1175 #### Pre-Shared Keys (PSKs)
1176 Add a WireGuard PSK between peer nodes. Adds a symmetric encryption layer that Tailscale's coordination server
    has no knowledge of, blinding it from reading transit content.
1177
1178
1179 ### Structural Limitations
1180 - **Traffic Analysis / Metadata:** Passive fiber tapping (UPSTREAM) can observe timestamps, data volumes, and
    IP ranges without reading content. Defeating traffic analysis requires mixing networks (Tor) or continuous
    dummy packet padding both heavily degrade performance.
1181 - **Targeted State-Level Adversaries:** Consumer-grade VPNs are not a complete shield against a nation-state
    actively targeting you. This stack is designed to defeat bulk passive surveillance and corporate dragnet
    collection.
1182
1183 ---
1184
1185 ## 14. Per-Device Access Control
1186
1187 Because every mesh device routes internet-bound traffic through the `warp` container's `FORWARD` chain, they
    all appear with their stable `100.x.x.x` Tailscale IP as the `src` address. This gives two powerful
    enforcement surfaces:
1188
1189 ### IP & Port Level (`scripts/routing-fix.sh`)
1190 Write raw `iptables` rules targeting specific devices. The 30-second idempotent loop auto-survives Gluetun
    reconnects:
1191
1192 ```bash

```

```

1193 # Block a specific device from a target subnet
1194 iptables -I FORWARD -s 100.x.x.x -d 203.0.113.0/24 -j DROP
1195
1196 # Restrict a device to only HTTP/HTTPS
1197 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 3000 -j DROP
1198 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 443 -j DROP
1199
1200 # Block a device from internet egress entirely (mesh only)
1201 iptables -I FORWARD -s 100.x.x.x -j DROP
1202
1203 # Time-based (e.g., cut off 10 PM 7 AM)
1204 iptables -I FORWARD -s 100.x.x.x -m time --timestart 22:00 --timestop 07:00 -j DROP
1205 ```
1206
1207 ### DNS Level (AdGuard REST API)
1208 AdGuard Home supports per-client rules keyed by Tailscale IP:
1209
1210 ```bash
1211 curl -X POST http://localhost:80/control/clients/add \
1212   -H "Content-Type: application/json" \
1213   -d '{
1214     "name": "kids-ipad",
1215     "ids": ["100.x.x.x"],
1216     "blocked_services": ["youtube", "tiktok", "instagram"],
1217     "safesearch_enabled": true
1218   }'
1219 ```
1220
1221 ---
1222
1223 ## 15. Packaging nullexit as a macOS Application (.dmg)
1224
1225 nullexit can be packaged into a native `.app` bundle, but this introduces nested virtualization requirements.
1226
1227 nullexit uses Colima (Apple Virtualization Framework `vz`) to run a Linux VM hosting Docker. Installing the
1228   `.app` inside a virtualized macOS environment (Parallels, cloud Mac) means running a Linux VM inside a
1229   macOS VM requiring hardware-level nested virtualization.
1230
1231 ### Hardware Support
1232 - Supported: Apple Silicon M3/M4 (macOS Sequoia 15+), Intel Macs with VMX passthrough.
1233 - Unsupported: M1, M2, A18 Pro. Colima crashes silently; the terminal (`setup.sh`) surfaces crash logs.
1234
1235 ### Build Steps
1236 ```bash
1237 # 1. Verify nested virtualization support
1238 sysctl -a | grep hv_nested_virt_supported # expect: 1
1239
1240 # 2. Compile the AppleScript launcher
1241 osacompile -o "Nullexit.app" "Toggle Gateway.applescript"
1242
1243 # 3. Embed scripts into app resources
1244 mkdir -p "Nullexit.app/Contents/Resources/scripts"
1245 cp toggle.sh setup.sh recover.sh docker-compose.yml "Nullexit.app/Contents/Resources/"
1246
1247 # 4. Package into DMG
1248 hdiutil create -volname "Nullexit Installer" -srcfolder "./Nullexit.app" -ov -format UDZO Nullexit.dmg
1249
1250 # 5. Bypass Gatekeeper (unsigned app)
1251 xattr -cr /Applications/Nullexit.app
1252 ```
1253
1254 Without a paid Apple Developer certificate ($99/yr), users see an "App is damaged" Gatekeeper warning and need
1255 to run step 5.
1256
1257 ---
1258
1259 ## 16. Moonlight/Sunshine + Tailscale Race Condition
1260
1261 When streaming from a remote Windows host running Sunshine over a Tailscale mesh (e.g., while the client is on
1262 a cellular hotspot), a race condition can occur on boot:
1263
1264 1. Both Tailscale and Sunshine are set to start automatically on boot.
1265 2. Tailscale takes a few seconds to fully initialize its `100.x.x.x` virtual adapter and establish a connection
1266 3. Sunshine starts too fast. It launches before Tailscale is fully ready. Since Sunshine only binds to
1267   network interfaces that are active at the exact moment of startup, it misses the Tailscale adapter entirely
1268 4. Moonlight clients (even when configured with the correct Tailscale IP) will fail to connect because Sunshine
1269   isn't listening on that interface.
1270
1271 The "Magic Toggle" Symptom:

```

```

1266 If the user manually enables or disables Wi-Fi on the host PC, it triggers a global Windows network change
      event. Sunshine listens for these events, re-enumerates all network adapters, and says, "Oh, there's a
      Tailscale adapter here!" and finally binds to it. Suddenly, Moonlight can connect.
1267
1268 **The Permanent Fix:**
1269 On the Windows host, open `services.msc`, locate the **Sunshine Service**, and change its Startup type from **
      Automatic** to **Automatic (Delayed Start)**. This forces Windows to wait a minute or two after booting
      before launching Sunshine, ensuring Tailscale's virtual adapter is fully initialized first.
1270
1271 ---
1272
1273 ## 17. Future Work
1274
1275 - **Direct P2P on VM Hosts:** Non-Linux hosts run Docker inside a VM, making true UDP hole-punching impossible.
      The only working solution is a native Linux host (Raspberry Pi, Intel NUC) where Docker runs without VM
      hypervisor translation.
1276 - **Native Linux Deployment:** Benchmark on a Raspberry Pi native container footprint is ~75MB without macOS
      hypervisor overhead.
1277 - **Mesh-Wide Filesystem (SFTP over Tailscale):** Any mesh device passively exposes its filesystem over SFTP.
      Android via Termux + OpenSSH + wakelock; iOS is client-only due to background process limits.
1278 - **Post-Quantum Cryptography (PQC):** Tailscale uses Curve25519, theoretically vulnerable to "harvest now,
      decrypt later" quantum attacks. A future iteration could replace the Tailscale container with raw WireGuard
      + [Rosenpass](https://rosenpass.eu/) to negotiate post-quantum PSKs.
1279
1280 ## 18. Geo-IP Blocking
1281
1282 To block traffic to and from specific countries, the system dynamically downloads country-specific IP ranges
      from [ipdeny.com](http://www.ipdeny.com/ipblocks/data/countries/) and drops them via iptables `FORWARD`
      rules.
1283
1284 To add a new country to the blocklist:
1285 1. Find the 2-letter ISO country code (e.g., `cn` for China, `ru` for Russia, `il` for Israel, `kp` for North
      Korea).
1286 2. Open your `.env` file.
1287 3. Update the `BLOCKED_COUNTRIES` variable: e.g., `BLOCKED_COUNTRIES="kp il cn ru"`.
1288 4. Run `toggle.sh` to restart the gateway and apply the new environment variables to the routing-fix container.
1289
1290 > [!NOTE]
1291 > **Limitations of Geo-IP Blocking:** IP-based blocking is highly effective at blocking servers, direct apps,
      and raw infrastructure physically located and registered in a specific country (e.g., `www.gov.il`).
      However, it will **not** block high-profile websites (e.g., `jpost.com`) that route their traffic through
      global Content Delivery Networks (CDNs) like Google Cloud, Cloudflare, or Fastly. Because the CDN's IP
      addresses are registered in the US or globally, the network traffic physically never routes to the blocked
      country, allowing it to bypass the Geo-IP firewall.
1292
1293 ## 19. Incident Post-Mortems
1294
1295 ### Incident: The Overnight Silent IP Leak (July 2026)
1296
1297 **The Problem:** The host leaked traffic over its raw, unencrypted `eth0` IP continuously for roughly 8 hours,
      completely bypassing the VPN. The `toggle.sh` state and the container liveness checks all reported the
      gateway as healthy and active.
1298
1299 **The Investigation:**
1300 1. **Sleep/Wake Checks:** We initially suspected macOS sleep cycles broke the routing table. A check of `pmset
      -g log` confirmed the laptop literally never slept (0 sleep events recorded since boot), ruling out all
      sleep/wake code paths.
1301 2. **Keepalive Expiry:** We discovered `docker-compose.yml` was missing `
      WIREGUARD_PERSISTENT_KEEPA_LIVE_INTERVAL`. Without a keepalive, standard router NAT tables (which typically
      garbage collect UDP after 30 to 120 seconds of silence) quietly expired the WireGuard port mapping
      overnight.
1302 3. **Container State Masking:** Because WireGuard is stateless, the `warp` container didn't immediately crash.
      It stayed "Up", which meant Docker's healthchecks and our `toggle.sh` liveness checks never noticed the
      tunnel inside it was totally dead.
1303
1304 **The Solution:**
1305 1. Added `WIREGUARD_PERSISTENT_KEEPA_LIVE_INTERVAL=25s` (the WireGuard standard to beat standard 30s NAT
      timeouts) to keep the UDP tunnel permanently pinned open through the router.
1306 2. Injected a **Fail-Closed Kill-Switch** into `routing-fix.sh`: A 30-second loop now actively verifies tunnel
      health by running `curl` over `tun0` to Cloudflare's trace endpoint. If it fails 3 consecutive times, it
      immediately rewrites the default route to `blackhole`, severing all traffic instead of letting it leak
      through `eth0`.
1307 3. Added a listener in `watcher.sh` that detects this fail-closed event and instantly pushes a native macOS UI
      notification to the desktop, alerting the user to manually intervene.
1308
1309 > [!NOTE]
1310 > **Why dual failure systems?**
1311 > The system now relies on two independent failure handlers that cover each other's blind spots:
1312 > 1. **Automated Self-Healing (WARP Watcher + `recover.sh`):** Triggered when the `warp` container logs `warp=
      off`. This happens when the server actively kicks the client. Because the container **knows** it is
      disconnected, it triggers `recover.sh` to safely reboot Docker and reconnect.

```

```

1313 > 2. Fail-Closed Kill-Switch (`routing-fix.sh` + `watcher.sh`):** Triggered when a silent network failure
1314 occurs (e.g., NAT timeouts). The container incorrectly believes it is connected (`warp=on`), so auto-
1315 recovery never fires. Instead of auto-recovering (which risks falling back to raw Wi-Fi mid-process while
1316 the user isn't looking), this kill-switch actively blackholes the internet and forces a manual intervention
1317 via a desktop notification.
1318
1319 ## 20. Design Decisions: Exit Node IP Rotation
1320
1321 A common question for privacy architectures is whether the system should aggressively rotate its public exit IP
1322 (e.g., every 5 minutes, like a Tor circuit or a proxy rotator).
1323
1324 `nullexit` uses Cloudflare WARP via Gluetun. WARP provides anonymity in a crowd** rather than aggressive
1325 rotation. Thousands of users share the same Cloudflare Edge IP simultaneously, making your traffic
1326 indistinguishable from the herd. The IP may occasionally rotate on its own (which `host-leak-probe.sh` logs
1327 as a `ROTATE` event), but it remains generally stable.
1328
1329 Why aggressive IP rotation was intentionally excluded:
1330
1331 1. Broken TCP Connections: Every rotation instantly drops all long-lived TCP sessions (SSH, large downloads
1332 , WebSockets, gaming, Zoom).
1333 2. CAPTCHA & 2FA Hell: Modern web security (e.g., Cloudflare, Akamai) aggressively flags IP-hopping as bot
1334 behavior. Aggressive rotation guarantees the user will be bombarded with "Verify you are human" CAPTCHAs
1335 and "New Login Detected" 2FA emails constantly.
1336 3. WireGuard Architecture: WireGuard is stateless and does not natively support IP hopping on the fly. To
1337 rotate an IP, the VPN client must actively drop the tunnel, request a new endpoint, and re-handshake. This
1338 introduces latency gaps where the kill-switch would repeatedly trigger and blackhole the user's internet.
1339
1340 Verdict: For a daily-driver secure gateway, shared-IP crowding (WARP) is superior to aggressive rotation.
1341 As an added benefit, Cloudflare WARP IPs are actually highly static (tied to the persistent WireGuard
1342 public key generated for your device identity). Because your IP stays static and is part of Cloudflare's
1343 own trusted network, it builds a high "trust score," virtually eliminating CAPTCHA loops while still hiding
1344 you within the massive crowd of other users in that data center. All mesh devices routing through your
1345 gateway will reliably share this exact same trusted IP.
1346
1347 *(Note: The only exception where aggressive IP rotation is genuinely required is for specialized offensive
1348 security tasks like high-volume web scraping or dark web research where avoiding IP bans or active tracking
1349 overrides the need for TCP stability.)*
1350
1351 ## 21. Cloudflare WARP Privacy & Logging
1352
1353 Because `nullexit` uses Cloudflare WARP (via Gluetun) as its upstream exit node, the system inherits Cloudflare
1354 's consumer privacy policy.
1355
1356 What Cloudflare DOES NOT log (Strict No-Logs Policy):
1357
1358 * Browsing History: They do not log the domains or URLs you visit.
1359 * Traffic Destinations: They do not log the IP addresses of the servers you connect to.
1360 * Content: They do not inspect or log the payload/data of your traffic.
1361 * Data Brokering: They explicitly guarantee they will never sell or rent your data to advertisers.
1362
1363 What Cloudflare DOES log (Minimal Telemetry):
1364
1365 * Data Transfer Volume: Total bandwidth used (required to enforce 10GB/mo quotas if you are not using a
1366 paid WARP+ key).
1367 * Performance Metrics: Anonymized connection speeds and latency to optimize their network routing.
1368 * Aggregate Volume: Total traffic volumes to popular destinations in aggregate (e.g., "datacenter X sent 50
1369 TB to Netflix"), stripped of all user IDs.
1370 * Device ID (Public Key): They store the persistent WireGuard public key generated by your container to
1371 authorize the connection.
1372
1373 Privacy Verdict:
1374
1375 Cloudflare WARP provides excellent historical privacy. Because they do not log your destinations, they
1376 cannot comply with historical subpoenas asking what websites you visited yesterday. However, it is not
1377 absolute anonymity. They still know your real ISP IP address while you are actively connected. For daily-
1378 driver privacy against ISPs, hackers, and corporate trackers, it is top-tier. For nation-state evasion, use
1379 Tor.
1380
1381 ---
1382
1383 ## 22. Battery & Power Measurement Quirks
1384
1385 When trying to optimize this framework (or any daemon) for battery life, developers often look for a way to
1386 programmatically measure the exact Wattage consumed by a specific process (e.g., "How many Watts is `toggle
1387 .sh` using?").
1388
1389 This is a hardware impossibility on macOS and Linux.
1390
1391 22.1 Why Activity Monitor is Lying to You
1392
1393 Your machine's logic board only has physical power sensors for the entire CPU package, the GPU, and the RAM.
1394 It knows the CPU is pulling 5W total, but it has no physical hardware capability to know whether Docker is
1395 using 3W and Chrome is using 2W.
1396
1397 The "Energy Impact" score you see in macOS Activity Monitor is a synthetic, heuristic score calculated by `
1398 powerd`. It penalizes apps for waking up the CPU (idle wakes), doing disk I/O, or keeping the screen awake.

```

```

1361         It is a relative score, not actual Wattage.
1362 ### 22.2 The Proper A/B Testing Methodology
1363 If you want to truly know how many Watt-hours or mAh your background daemons (`routing-fix.sh`, `host-leak-
1364 probe.sh`, etc.) are costing you, you must perform a hardware-level A/B drain test:
1365
1366 1. Unplug the laptop, run the gateway, and note your exact battery mAh using:
1367     ```bash
1368     ioreg -l | grep "AppleRawCurrentCapacity"
1369     ```
1370 2. Leave the laptop idle for exactly 1 hour.
1371 3. Run the command again to see exactly how many mAh were drained.
1372 4. Turn the gateway off and repeat the test for another hour.
1373
1374 The delta in mAh drained between the two tests is the true, exact cost of running the software. When optimizing
1375 polling loops (e.g., changing `sleep 5` to `sleep 30`), this is the only reliable way to measure the
1376 actual battery life returned to the user.
1377
1378 ---
1379 ## 23. Architectural Limitations: Host Lockdown Mode
1380
1381 A common privacy feature request is a "Host Lockdown Mode" (Mode 3): A mode where the Docker exit node remains
1382 perfectly functional for remote devices, but the host machine itself is completely blocked from accessing
1383 the internet (to prevent even encrypted host traffic from leaking metadata).
1384
1385 **Why this is impossible on macOS:**
1386 On Linux (e.g., a Raspberry Pi), Docker runs natively. The host's traffic traverses the `OUTPUT` iptables chain
1387 , while Docker traffic traverses the `FORWARD` chain. We could easily drop all `OUTPUT` traffic to kill the
1388 host's internet, and Docker would continue routing traffic perfectly.
1389
1390 However, Docker on macOS runs inside a Linux Virtual Machine (via `qemu` or Apple `Virtualization.framework`).
1391 This VM runs as a standard macOS user-space application. If you configure the macOS firewall (`pf`) to drop
1392 all host outbound traffic, it will indiscriminately drop the VM application's traffic as well, instantly
1393 killing the Cloudflare WARP tunnel and the exit node.
1394
1395 Writing complex macOS `pf` rules to "allow the VM app, allow Cloudflare WARP IPs, allow Tailscale DERP IPs, but
1396 block everything else" is highly fragile and heavily discouraged. Since the current `toggle.sh` default
1397 mode already fully encrypts the host's traffic via the WARP tunnel, the host is already protected from ISP
1398 leaks.
1399
1400 ### 23.1 July 4, 2026: Multi-stage Docker Builds for Go Ports & Teardown Hang Fix
1401 **Symptom:** The Python implementations of `logger.py` and `sync-rules.py` were slow and required a heavy
1402 python runtime inside Alpine containers. Also, `routing-fix` container teardown was hanging for 30s during
1403 `toggle.sh` shutdown.
1404
1405 **Root Cause:**
1406 - Python is slower than Go and installing it dynamically via `apk add` added overhead and complexity to the
1407 containers.
1408 - Docker containers ignore `SIGTERM` if the init process doesn't handle it, causing a full 30s timeout on
1409 shutdown.
1410
1411 **Resolution:**
1412 - Ported `logger` and `sync-rules` to Go using Goroutines for massive concurrent speedups.
1413 - Implemented **Multi-stage Docker builds** (using `golang:1.22-alpine` as builder) to compile the static
1414 binaries inside Docker. The final containers are raw Alpine with zero dependencies.
1415 - Added `--build` to `docker compose up -d` in `toggle.sh` to ensure updates are consistently applied on boot.
1416 - Added `stop_grace_period: 1s` to `routing-fix` in `docker-compose.yml` which eliminates the 30-second
1417 teardown hang instantly.
1418 - Implemented a cryptographic integrity checker (`scripts/crypto.sh`) using HMAC-SHA256 and `NULLEXIT_SEED` in
1419 `.env` to prevent tampering of bash scripts.
1420
1421 ### 23.2 July 4, 2026: Consolidation of Core Bash Scripts (Refactoring)
1422 **Symptom:** Over 200 lines of bash logic were directly duplicated across `toggle.sh` and `recover.sh` (e.g. `
1423 restart_tailscaled_daemon`, `stop_sleep_prevention`, WARP bypass routes). This caused drift when one script
1424 was updated without the other.
1425
1426 **Root Cause:**
1427 - Historical separation of responsibilities allowed `recover.sh` to grow complex alongside `toggle.sh`.
1428
1429 **Resolution:**
1430 - Massively refactored both scripts by extracting all duplicated networking logic, PID lifecycle management,
1431 and environmental configuration down to `scripts/common.sh`.
1432 - Specifically, the `stop_sleep_prevention`, `stop_dns_watcher`, `stop_host_leak_probe`, and `stop_warp_watcher`
1433 functions were collapsed into a single `stop_pidfile_daemon()` abstraction.
1434 - Proxy disable loops were abstracted to `disable_all_proxies()`.
1435 - The `162.159.192.1/.193.1` WARP edge IPs are now dynamically resolved from `.env` instead of hardcoded.
1436
1437 ---
1438
1439 ### 23.3 July 4, 2026: The "Network Unreachable" Host Leak & Post-Wake Container Destructuations
1440 **Symptom:**
1441 1. Running `docker compose config` with dummy keys corrupted the user's `.env` file by appending invalid
1442 WireGuard keys. This caused the `warp` container to become unhealthy on boot, which triggered a full
1443 teardown of the gateway by `toggle.sh`.

```

1415 2. After fixing `.env`, `toggle.sh` successfully booted the gateway, but the host's internet started bypassing the tunnel entirely (a massive host leak). The Cloudflare trace returned `warp=off` and exposed the host's physical ISP IP.

1416 3. Running `recover.sh --post-wake` would violently force-recreate all gateway containers and drop the host's internet connection.

1417

1418 ****Root Cause:****

1419 - ****Bug 1 (The Host Leak):**** In `toggle.sh`, the logic to find the Tailscale `utun` interface used `ifconfig | grep -B4 "inet 100."`. Due to the 4 lines of before-context, this regex was accidentally capturing the interface `*above*` the actual Tailscale interface (e.g. grabbing `utun4` instead of `utun5`). Because `utun4` had no IP address, the subsequent `sudo route add -net 0.0.0.0/1 -interface utun4` silently failed with "Network is unreachable", leaving the host's default route exposed.

1420 - ****Bug 2 (The Post-Wake Destructions):**** The health check in `recover.sh --post-wake` relied on `docker compose exec -T warp curl -sf https://www.cloudflare.com/cdn-cgi/trace`. However, the newer `qmcgaw/glutun` Alpine-based Docker image no longer ships with `curl` pre-installed. The health check failed with `executable file not found in $PATH`, tricking `recover.sh` into thinking the tunnel was completely dead and needed to be forcefully recreated via `docker compose up -d --force-recreate`.

1421 - ****Bug 3 (Restart Flags):**** Previous refactors incorrectly permitted `toggle.sh restart` instead of strictly enforcing `toggle.sh --restart`.

1422

1423 ****Resolution:****

1424 - Cleaned the `.env` file of duplicate dummy entries.

1425 - Replaced the brittle `grep -B4` interface parsing in `toggle.sh` with a strict AWK parser: `awk '/^[a-z0-9]+:/{iface=$1}/inet 100\./{print iface; exit}' | tr -d ':`. This mathematically isolates the exact interface possessing the Tailscale `100.x.x.x` IP address, preventing the `0.0.0.0/1` route from silently failing.

1426 - Changed the health check in `recover.sh` to use `wget -q0- --timeout=3` which is natively installed in the Gluetun image. This stopped the unnecessary destructive recreations of the containers during post-wake events.

1427 - Reverted the `restart` alias in `toggle.sh` to strictly require `--restart`.

1428

1429 **### 23.4 July 4-5, 2026: Tailscale Data-Plane Loop (The DERP Relay Deadlock)**

1430 ****Symptom:****

1431 After bypassing the Tailscale control plane (`192.200.0.0/16`) to fix the "offline" mesh status, the Mac appeared online. However, external peer devices (like the user's phone on cellular) could not successfully ping or SSH into the Mac. `tailscale netcheck` reported `Nearest DERP: unknown (no response to latency probes)`.

1432

1433 ****Root Cause:****

1434 While bypassing the control plane kept the daemon connected to the coordination servers, Tailscale's actual peer-to-peer data plane relies on STUN (for NAT traversal) and DERP relays. These relays span ~80 dynamically allocated IP addresses across multiple cloud providers. Because we were forcing `0.0.0.0/1` through the `utun*` exit node, the Mac's own outbound connection attempts to DERP relays were being routed back into the tunnel. To prevent an infinite loop, Tailscale's daemon dropped its own packets when it saw them returning on the TUN interface. This effectively left the daemon deaf and blind to peer connections.

1435

1436 ****Resolution:****

1437 - Modified `add_warp_bypass_routes` in `scripts/common.sh` to execute a live API fetch (`curl -s https://login.tailscale.com/derpmap/default`) during startup.

1438 - The script parses out all active DERP relay IPv4 addresses (~80 IPs) and adds a physical host bypass route for every single one of them.

1439 - These IPs are written to a temporary file (`/tmp/nullexit-derp-ips.txt`) so they can be cleanly un-routed by `remove_warp_bypass_routes` when the gateway shuts down. Peer connectivity (ping, SSH, SFTP) was fully restored.

1440

1441 **### 23.5 July 5, 2026: Hardcoded WARP Endpoints in docker-compose**

1442 ****Symptom:****

1443 The bash scripts allowed overriding the default Cloudflare WARP IP endpoints via `.env` variables (`WARP_ENDPOINT_1` and `WARP_ENDPOINT_2`) to establish bypass routes. However, `docker-compose.yml` statically hardcoded `162.159.192.1` for the `warp` container's `WIREGUARD_ENDPOINT_IP`.

1444

1445 ****Root Cause & Risk:****

1446 If a user set a custom endpoint in `.env`, the script would successfully bypass the custom IP on the host level. However, Gluetun would still attempt to connect to the hardcoded `162.159.192.1`. Since `162.159.192.1` was no longer explicitly bypassed, its packets would be sucked into the `utun*` exit node, causing an infinite WireGuard routing loop that instantly breaks the gateway.

1447

1448 ****Resolution:****

1449 Modified `docker-compose.yml` to dynamically read the environment variable via `WARP_ENDPOINT_1:-162.159.192.1`, ensuring both the host routing scripts and the container use the exact same endpoint.

1450

1451 **### 23.6 July 6, 2026: Packet Filter (pfctl) Defaulting Local LAN Rules to TCP-Only**

1452 ****Symptom:****

1453 Devices on the exact same local Wi-Fi network as the gateway were experiencing ~500ms latency to the gateway instead of the expected ~80ms.

1454

1455 ****Root Cause:****

1456 When writing the `pf.conf` rules to whitelist local LAN traffic (`pass out quick on $ext_if to { 192.168.0.0/16, ... }`), the rule omitted an explicit protocol. The macOS `pfctl` compiler implicitly applies state tracking to all `pass` rules. However, because state tracking (`keep state`) natively evaluates TCP sessions, `pfctl` automatically appended the `flags S/SA` modifier to the compiled rule in

the kernel. This silently restricted the whitelist to ****TCP only****, causing all local UDP traffic to be dropped by the default-deny kill-switch. Since Tailscale relies on UDP port 41641 for direct P2P connections, the local devices were unable to hole-punch and were forced to fall back to routing traffic through a remote Tailscale DERP relay, massively inflating latency.

****Resolution:****
Split the single implicit rule into explicit ``proto tcp``, ``proto udp``, and ``proto icmp`` rules in ``scripts/pf.conf`` to guarantee the macOS compiler allows all three core transport protocols on the local subnet without mutating the rule into a TCP-only lock.

23.7 July 8, 2026: Network Watcher Event Mismatch, Sudo-in-Background Crash, and Loop-Prevention

****Symptom:****

1. Switching Wi-Fi or waking the Mac from sleep did not trigger a post-wake recovery (``recover.sh --post-wake``) automatically.
2. The network connection would eventually get completely sinkholed/blocked. If the user tried to restart or wait, it did not self-heal.
3. During the recovery attempt, checking the public IP on the host would return the unencrypted campus/ISP IP (starting with ``132.``) instead of the encrypted tunnel IP.

****Root Cause:****

- ****Bug 1 (Network Watcher Mismatch):**** In ``scripts/watcher.sh``, the network change listener watched ``State:/Network/Global/IPv4`` changes via ``scutil n.watch`` but matched them against ``*n.state!*SCEventUpdate*``. On macOS, the actual output is ``changedKey [0] = State:/Network/Global/IPv4``. Because the pattern did not match, all network switches were silently ignored.
- ****Bug 2 (Sudo Caching Background Crash):**** When the background WARP Watcher eventually detected the broken tunnel (after 6 failures/30s), it triggered the default nuclear ``recover.sh`` (with ``POST_WAKE=false``). Because there was no active TTY in the background context, the ``sudo -v`` caching check aborted instantly with a terminal required error. Due to ``set -e``, ``recover.sh`` died immediately before resetting DNS, disabling the packet-filter killswitch, or stopping containers, leaving the host network completely sinkholed.
- ****Bug 3 (Post-Wake Infinite Loop):**** Once the watcher patterns were fixed, ``recover.sh --post-wake`` triggered successfully on network changes. However, because the script deleted the default routing entries at the start, spent ~15 seconds rebuilding 88 DERP relay bypass routes, and re-added the default routes at the end, this execution time exceeded the watcher's 10-second debounce threshold. Furthermore, the watcher recorded the ``start`` timestamp in the debounce file. Consequently, the route changes made by the script itself triggered the watcher again immediately after completion, trapping the system in an infinite loop of recovery runs. During this loop, the VPN routes were constantly deleted, resulting in the host leaking traffic through the real ISP interface (an IP starting with ``132.``) for 15s out of every cycle.

****Resolution:****

- ****Fixed Watcher Matching:**** Updated ``scripts/watcher.sh`` to match ``*changedKey*`` and ``*State:/Network/Global/IPv*`` to correctly catch network changes.
- ****Bypassed Background Sudo:**** Added ``--auto`` and ``--non-interactive`` flags to ``recover.sh`` and added a TTY check ``[ -t 0 ]`` to skip interactive ``sudo -v`` authentication when running headlessly in the background. Updated the WARP Watcher call in ``toggle.sh`` to pass ``--auto``.
- ****Prevented Infinite Loops:**** Modified ``scripts/watcher.sh`` to write the ****completion**** timestamp of ``recover.sh`` to ``DEBOUNCE_FILE`` (instead of only the start timestamp). This ensures all buffered network config events generated during route reconstruction are evaluated against the end-of-run timestamp and successfully debounced, breaking the infinite loop.
- ****Centralized & Timestamped Logs:**** Redirected ``watcher.sh`` stdout/stderr from ``/tmp/nullexit-watcher.log`` to the repository's main ``output.log`` and updated ``scripts/common.sh``'s ``step``, ``ok``, ``warn``, ``fail``, and ``die`` helpers to prefix logs with a local ``[YYYY-MM-DD HH:MM:SS]`` timestamp.

24. Censorship-Resistant Transport & Egress Compartmentalization

24.1 Why this is needed
WARP can be blocked from deployment countries with strict internet controls. The most likely cause isn't that "encrypted traffic is blocked" in general it's that WireGuard has a recognizable handshake signature, and ``WIREGUARD_ENDPOINT_IP=162.159.192.1`` on ``WIREGUARD_ENDPOINT_PORT=2408`` is a fixed, easily-enumerable target (Cloudflare's known WARP range). That combination is exactly what IP/ASN-based and DPI-based blocking is good at catching.

24.2 Important correction: Gluetun's `SHADOWSOCKS=on` is the wrong direction
Gluetun's built-in Shadowsocks option runs a Shadowsocks ****server**** inside the already-connected VPN tunnel, so other LAN devices can reach out through Gluetun's ****existing**** connection via the SS protocol. It is not a way to make Gluetun itself connect ****outbound**** through a Shadowsocks server Gluetun only speaks OpenVPN or WireGuard as its actual transport. There is no ``VPN_TYPE=shadowsocks``. Confirmed against Gluetun's own maintainer/community discussion: people have asked for a "Shadowsocks-client" mode specifically to chain through Gluetun, and the standing answer is "run a separate Shadowsocks client container."

24.3 Diagnose before building anything
The right fix depends entirely on what's actually being blocked:

- Point Gluetun at a ****different provider/IP/ASN**** (any other Gluetun-supported WireGuard/OpenVPN provider, or a self-hosted WireGuard server) and test. If that gets through, the block is Cloudflare/WARP-specific, not a general WireGuard block ****no new component needed****, just swap ``VPN_SERVICE_PROVIDER`` / the endpoint.
- If WireGuard fails regardless of endpoint, the block is protocol-level (DPI fingerprinting the handshake) proceed to obfuscation.

24.4 Path A: Swap WARP for a different Gluetun-native transport

```

1493 Simplest fix, only applies if 24.3 shows the block is Cloudflare-specific. Change `VPN_SERVICE_PROVIDER` / `
      VPN_TYPE` / endpoint in `docker-compose.yml` and in the now-centralized `WARP_ENDPOINT_*` values in `.env`.
      Nothing else in the stack needs to change.
1494
1495 ### 24.5 Path B: Add an obfuscation hop
1496 Needed if WireGuard itself is being fingerprinted/blocked. Two ways to slot it in:
1497
1498 **B1 Wrap (recommended):** Run a Shadowsocks (or obfs4/v2ray) client as a new sidecar container. Point Gluetun
      's `WIREGUARD_ENDPOINT_IP` at `127.0.0.1:<local-port>` instead of Cloudflare directly; the sidecar relays
      that traffic to a Shadowsocks server you run yourself abroad. Keeps Gluetun's kill-switch, healthchecks,
      and the entire rest of the stack (Tailscale, AdGuard, ipset, `routing-fix.sh`) untouched, since none of
      them know the transport changed underneath `warp`.
1499
1500 **B2 Replace:** Swap the `warp` service entirely for a Shadowsocks-client + `tun2socks` container that owns
      the network namespace instead of Gluetun. More invasive loses Gluetun's built-in kill-switch/healthcheck,
      which would need to be rebuilt by hand (see Section 24.9 for the pluggable architecture to support this
      natively).
1501
1502 **Recommendation: B1.** Smaller surface area, preserves the self-healing architecture already built and
      documented.
1503
1504 ### 24.6 Fingerprinting caveat
1505 Plain Shadowsocks can still be caught by active-probing DPI in more aggressive censorship environments. If
      plain SS gets blocked too, the next step up is an obfuscation plugin (`v2ray-plugin`, `Cloak`) or a newer
      protocol designed against active probing (VLESS+Reality, Hysteria2). Test plain SS first don't over-build
      before confirming it's needed.
1506
1507 ### 24.7 AmneziaWG (The High-Speed Alternative)
1508 If you want to maintain the bare-metal speeds of WireGuard while bypassing DPI (e.g. in Egypt or Russia), the
      best alternative to Shadowsocks/V2Ray is AmneziaWG. AmneziaWG is a fork of WireGuard that pads the
      handshake packets with randomized "junk" data, entirely destroying the 148-byte DPI signature that
      firewalls look for.
1509 - **Pros:** Because it runs entirely as UDP without TCP-over-TCP overhead, it completely avoids "TCP meltdown"
      latency spikes associated with Shadowsocks/V2Ray wrappers.
1510 - **Cons:** You cannot use Cloudflare WARP. Furthermore, it requires completely ripping Gluetun out of the `
      nullexit` stack and building a custom Docker container, meaning you lose Gluetun's built-in kill-switches
      and health-check logic.
1511
1512 ### 24.8 Infrastructure Costs & Privacy
1513 To use either Shadowsocks (Path B) or AmneziaWG, you cannot use the free Cloudflare WARP infrastructure,
      because Cloudflare servers only speak standard WireGuard. The software for both custom protocols is 100%
      free and open-source, but you must host the remote server infrastructure yourself.
1514 - **Free Tier (Oracle Cloud):** You can host your remote server on Oracle Cloud's "Always Free" tier for $0.
      However, this routes your highly sensitive, censorship-evading traffic directly through Oracle's massive
      corporate data broker. If privacy is the core objective, handing all your metadata to Oracle defeats the
      purpose.
1515 - **Paid Tier (Hetzner / Mullvad):** The recommended path is to rent a standard ~$4/month Virtual Private
      Server (VPS) in a free country (e.g., Hetzner in Germany, subject to strict EU GDPR privacy laws) and self-
      host the server. Alternatively, Mullvad VPN (5/month) provides native v2ray/Shadowsocks bridges built into
      their network, allowing you to bypass censorship with a strict zero-logs policy. It is highly recommended
      to pay with untrackable payment methods to maintain complete anonymity, which these providers typically
      support without requiring local residency.
1516
1517 ### 24.9 The Future: Egress Compartmentalization (Pluggable Architecture)
1518 To natively support invasive replacement paths like **Path B2** or **AmneziaWG** (Section 24.7) without
      breaking the core `nullexit` routing and monitoring logic, the architecture must be refactored into a
      modular, "pluggable" design.
1519
1520 1. **The Egress Plugin System:** Currently, WARP-specific logic (like `cdn-cgi/trace` health checks and
      hardcoded interface names) is scattered across `toggle.sh`, `scripts/routing-fix.sh`, and `scripts/host-
      leak-probe.sh`. This should be abstracted into a `scripts/plugins/` directory, where each egress method (`
      warp.sh`, `v2ray.sh`) implements standardized functions: `get_egress_interface()`, `check_health()`, and `
      get_bypass_ips()`.
1521 2. **Dynamic Docker Compose:** Based on an `EGRESS_TYPE` variable in `.env`, dynamic compose file generation
      would spin up only the required egress containers (e.g., skipping the `warp` container when `EGRESS_TYPE=
      v2ray` is set).
1522 3. **Agnostic Routing & Watchers:** `routing-fix.sh` would dynamically read the `EGRESS_INTERFACE` from the
      active plugin to apply `iptables` rules, and the background watchers in `toggle.sh` would become an
      agnostic `Egress Watcher` that invokes `check_health()` periodically.
1523
1524 This compartmentalization transforms `nullexit` into a universal, censorship-resistant gateway where the entire
      egress layer can be swapped by changing a single `.env` variable and running `./toggle.sh --restart`.
1525
1526 ---
1527
1528 ## 25. Incident Post-Mortem: WARP Watcher Race vs Post-Wake (July 10, 2026)
1529
1530 ### Symptom
1531 After switching Wi-Fi networks, the host IP appeared as the raw ISP IP (`132.x.x.x`) instead of a Cloudflare/
      WARP IP. The gateway appeared to complete post-wake recovery successfully in the logs, but nuclear shutdown
      fired immediately after and killed everything.

```

```

1532
1533 ### Root Cause
1534 A fatal race condition between two concurrent processes:
1535
1536 1. **Wi-Fi roam** `watcher.sh` fires `recover.sh --post-wake`
1537 2. **Post-wake** finds the `warp` container missing/dead calls `docker compose up -d --force-recreate` (~35
    seconds)
1538 3. **In parallel**, the WARP Watcher (running as a child of `toggle.sh`) polls the warp container every 5s
    during failures
1539 4. While the container is being recreated it returns `warp=off` for those 35 seconds 7 consecutive failures
    **WARP SHUTDOWN fires**, calling nuclear `recover.sh`
1540 5. Nuclear `recover.sh` tears down Tailscale, resets DNS to 1.1.1.1, stops all containers **gateway dead, IP
    exposed**
1541 6. The post-wake `docker compose up` finishes at ~35s mark, container healthy but the gateway is already gone
1542
1543 The evidence in `output.log`:
1544 ```
1545 [2026-07-10T06:07:27Z] NET: ... bash recover.sh --post-wake
1546 [2026-07-10T06:07:47Z] WARP DOWN cdn-cgi/trace reports warp=off watcher starts counting
1547 ...
1548 add net 0.0.0.0: gateway utun5 post-wake successfully re-routes at 02:08:22
1549 ...
1550 [2026-07-10T06:09:04Z] WARP SHUTDOWN 6 consecutive failures watcher fires nuclear
1551 ```
1552
1553 ### Fix (July 10, 2026)
1554 Added a **WARP Watcher inhibit marker** (`/tmp/nullexit-warp-inhibit.marker`):
1555
1556 - `recover.sh --post-wake` writes the marker at startup **before** any container operations
1557 - The WARP Watcher loop checks for the marker at the top of every iteration; if present it resets `consec_off
    =0` and sleeps 5s (skips the poll entirely)
1558 - `recover.sh` removes the marker at the very end (on both success and failure via `trap cleanup_inhibit EXIT`)
1559 - This guarantees the watcher never counts failures during the intentional container-down window of a post-wake
    force-recreate
1560
1561 The marker file path is hardcoded to `/tmp/nullexit-warp-inhibit.marker` in both `toggle.sh` and `recover.sh`.
1562
1563 ## 26. Incident Post-Mortem: Startup Pre-Flight Check Tailscale Race (July 10, 2026)
1564
1565 ### Symptom
1566 Immediately after startup via `toggle.sh`, the pre-flight connectivity check `[1/3] Gateway reachable via
    Tailscale` returned `FAIL`, forcing `toggle.sh` to skip the full exit-node activation and fall back to
    SOCKS5/local DNS proxy mode.
1567
1568 ### Root Cause
1569 A timing race condition during startup. In `toggle.sh`, the host joins the mesh using `tailscale up --reset` (
    Phase A) right before performing the pre-flight check. Because `tailscale up` resets and reconfigures the
    host's `tailscaled` daemon, the local daemon must fetch the latest netmap asynchronously from the
    coordination servers.
1570 This network map propagation and route establishment takes a few seconds. Running `tailscale ping` immediately
    after `tailscale up` returns (with no delay) causes the check to fail because the local daemon has not yet
    discovered the gateway node `100.100.21.8`.
1571
1572 ### Fix (July 10, 2026)
1573 Upgraded pre-flight Check 1 in both `toggle.sh` and `scripts/toggle-linux.sh` to use a 5-attempt retry loop
    with a 1-second delay between checks. This allows up to 5 seconds for local tailnet routing paths to settle
    , ensuring the gateway is correctly reached and a pong is received before deciding whether to enable exit-
    node routing.
1574
1575 ## 27. Incident Post-Mortem: Concurrency Collision between Nuclear Teardown and Post-Wake (July 10, 2026)
1576
1577 ### Symptom
1578 When the background WARP Watcher triggers a nuclear shutdown (due to a real or simulated tunnel failure), the
    gateway is completely torn down. However, the network configuration changes made during the teardown
    trigger a `State:/Network/Global/IPv4` network change event. The LaunchAgent-based network watcher (
    `watcher.sh`) intercepts this event and concurrently spawns `recover.sh --post-wake` to heal/restore the
    gateway. This causes `docker compose down` (from nuclear teardown) and `docker compose up` (from post-wake)
    to run in parallel, resulting in container errors like `dependency failed to start: container warp exited
    (0)` and the gateway becoming wedged.
1579
1580 ### Root Cause
1581 A lack of coordination/locking between the lifecycle control scripts. While `toggle.sh` used `/tmp/nullexit-
    toggle.lock` to prevent concurrent `toggle.sh` runs, `recover.sh` (both nuclear and `--post-wake` modes)
    did not utilize or check any lock files.
1582
1583 ### Fix (July 10, 2026)
1584 Implemented a shared lifecycle lock file (`/tmp/nullexit-toggle.lock`) across both scripts:
1585 - **`recover.sh` (all modes)**: Checks `/tmp/nullexit-toggle.lock` at startup. If the lock is held by another
    running lifecycle script (`toggle.sh` or `recover.sh`):
1586 - In `--post-wake` mode, it logs a skip message to `output.log` and exits cleanly (`exit 0`). This safely
    prevents the auto-recovery agent from recreating containers during a teardown.

```

```

1587 - In nuclear recovery mode, it exits with an error.
1588 - **`recover.sh` Lock Cleanup**: Cleans up the lock file upon completion using `trap cleanup_recover EXIT` (
    which checks if the lock file belongs to the current PID).
1589 - **`toggle.sh` update**: The lock check in `toggle.sh` was updated to look for both `toggle.sh` and `recover.
    sh` in the running processes to enforce proper exclusion.
1590
1591 ## 28. Incident Post-Mortem: Infinite Auto-Recovery Loop after WARP Watcher Nuclear Shutdown (July 10, 2026)
1592
1593 ### Symptom
1594 When the background WARP Watcher triggered a nuclear shutdown (due to a tunnel outage), it successfully
    executed `recover.sh` (nuclear). However, immediately after the teardown completed, the system started
    spawning `recover.sh --post-wake` and rebuilding the containers again. This created an infinite loop of
    tearing down and auto-restarting the gateway, keeping the host's internet route permanently wedged.
1595
1596 ### Root Cause
1597 An active-state marker file leak. The LaunchAgent-based network watcher (`watcher.sh`) only fires `recover.sh
    --post-wake` when `/tmp/nullexit-gateway-active.marker` is present on disk. While `toggle.sh` (STOP path)
    correctly deleted this marker, the nuclear `recover.sh` script did not. When the WARP Watcher ran `recover.
    sh` (nuclear), the containers were stopped but the active-state marker was left on disk. The network
    changes from the teardown then triggered `watcher.sh`, which saw the marker, believed the gateway was still
    supposed to be active, and triggered `--post-wake` to start it back up.
1598
1599 ### Fix (July 10, 2026)
1600 Modified the start of `recover.sh` to explicitly delete `/tmp/nullexit-gateway-active.marker` if `POST_WAKE` is
    `false` (nuclear mode). This ensures that any subsequent network configuration changes made during the
    teardown are correctly ignored by `watcher.sh` because the marker file is gone.
1601
1602 ## 29. Incident Post-Mortem: Pre-Flight Check False Negatives due to VPN Firewall STUN Blocks (July 10, 2026)
1603
1604 ### Symptom
1605 Immediately after a cold boot or full restart of the gateway, the pre-flight connectivity check `[1/3] Gateway
    reachable via Tailscale` failed, causing `toggle.sh` to skip the full exit-node configuration and fall back
    to `SOCKS5/local DNS proxy mode`. However, manually running `tailscale ping 100.100.21.8` immediately after
    startup succeeded in `ims`.
1606
1607 ### Root Cause
1608 An asynchronous Tailscale handshake delay caused by firewall restrictions. Inside the container network
    namespace, the `warp` container (gluetun) implements a strict firewall that blocks all outbound UDP traffic
    to the public internet except to the designated WireGuard endpoint. Because of this, the `tailscale`
    container's STUN requests to public STUN servers are blocked, causing the local daemon to report `UDP is
    blocked, trying HTTPS`.
1609 Without UDP STUN capability, Tailscale is forced to fall back to a slower DERP-assisted handshake (relayed via
    HTTPS/TCP). This negotiation and direct-path punch-through takes roughly 30 to 35 seconds to establish on
    cold boots (after a full `tailscale up --reset`). Since the pre-flight check only waited 15 seconds (15
    attempts with 1-second sleeps), the check timed out and aborted before the path completed.
1610
1611 ### Fix (July 10, 2026)
1612 Increased the pre-flight ping retry limit from 15 to 45 attempts (45 seconds) in both `toggle.sh` and `scripts/
    toggle-linux.sh`. This provides sufficient time for the DERP-assisted handshake to complete on cold boots,
    while still passing immediately (in 1-2 seconds) on warm starts or once the path is established.
1613
1614 ## 30. Incident Post-Mortem: Docker Image Build DNS Failures on Immediate Restart (July 10, 2026)
1615
1616 ### Symptom
1617 When executing `toggle.sh --restart`, the script successfully stops the gateway but then immediately fails
    during startup during `docker compose build` with DNS resolution errors:
1618 `failed to do request: Head "https://registry-1.docker.io/...": dial tcp: lookup registry-1.docker.io on
    192.168.5.1:53: no such host`.
1619
1620 ### Root Cause
1621 A race condition between physical link negotiation and virtual machine startup. The STOP path of the restart
    command invokes `cleanup_network_state` which power-cycles the host's physical Wi-Fi interface (`en0`) to
    flush stale routing states. Immediately after, `toggle.sh` starts the gateway boot sequence. Because `
    toggle.sh` did not verify the status of the physical interface, it proceeded to boot Colima and build
    Docker containers while `en0` was still negotiating a DHCP lease. Consequently, the host (and by extension,
    the VM bridge) had no active internet gateway/DNS path, causing Docker's registry check to fail.
1622
1623 ### Fix (July 10, 2026)
1624 1. Created a unified `wait_for_dhcp_settle` helper function in `scripts/common.sh` that polls `ipconfig
    getpacket` and `ifconfig` until a valid IP and router are assigned. To handle typical macOS Wi-Fi
    reassociation and DHCP negotiation latency (which commonly takes 15-20 seconds after a power-cycle), this
    loop was configured to poll up to 60 times (30 seconds total).
1625 2. Injected a call to `wait_for_dhcp_settle` at the beginning of the `START` action in `toggle.sh` (right
    before checking Colima status) to block container builds until the host's physical network is online.
1626 3. Refactored `recover.sh` to reuse the unified `wait_for_dhcp_settle` function.
1627
1628 ## 31. Incident Post-Mortem: PF Kill-Switch Bricks Host Internet in SOCKS5 Fallback Mode (July 10, 2026)
1629
1630 ### Symptom
1631 When the pre-flight checks fail (e.g. because host Tailscale was unauthenticated/logged out), the script falls
    back to `SOCKS5 proxy mode` but the host immediately loses all internet connectivity and tailscaled reports

```

```

1632     You are logged out... connect: no route to host`. Even running `sudo tailscale up` to log in fails because
1633     the connection to the control plane times out.
1634 ### Root Cause
1635 An architectural mismatch. The macOS Packet Filter (PF) Kill-Switch works at the IP level: it blocks all
1636     outbound traffic on the physical interface (`en0`) except to explicitly whitelisted VPN endpoints,
1637     expecting all other traffic to go through the virtual VPN interface (`utun*`).
1638 In SOCKS5 fallback mode, the exit-node (`utun*`) is NOT enabled; instead, only specific applications (like
1639     browsers) use the localhost SOCKS5 proxy, while all other host traffic (including the `tailscaled` daemon's
1640     UDP packets) continues to go through `en0`. Because the script still enabled the PF Kill-Switch during
1641     SOCKS5 fallback, it blocked all non-SOCKS5 traffic on `en0`, completely bricking the host's internet and
1642     locking the Tailscale daemon out of the control plane.
1643 ### Fix (July 10, 2026)
1644 Modified `toggle.sh` to remove the `enable_killswitch` call from the SOCKS5 fallback path. The firewall kill-
1645     switch is now strictly reserved for exit-node VPN mode, ensuring SOCKS5 fallback leaves the host's direct
1646     internet route open for system daemons to authenticate.
1647 ## 32. Incident Post-Mortem: Host Tailscale Logouts due to Aggressive reset Flags (July 10, 2026)
1648 ### Symptom
1649 Every time the user runs `toggle.sh` or `recover.sh` which disconnects/reconnects the host client, the host
1650     client randomly gets logged out, forcing them to re-run `sudo tailscale up` to authenticate.
1651 ### Root Cause
1652 An overly aggressive configuration reset. The host-side `tailscale up` calls (used in `
1653     disconnect_tailscale_host` and the startup/enable exit node paths) all passed the `--reset` flag. On macOS,
1654     `--reset` resets the entire configuration state and authentication token cache of the daemon. Since the
1655     script does not pass a `TS_AUTHKEY` to the host's commands (which are meant for the user's private
1656     interactive client), the `--reset` flag effectively logged the user out of their tailnet, requiring
1657     interactive re-login.
1658 ### Fix (July 10, 2026)
1659 Removed the `--reset` flag from all host-side `tailscale up` invocations in `toggle.sh` and `scripts/common.sh
1660    `. This ensures that exit-node state transitions (enabling/disabling) are handled safely while preserving
1661     the host client's authenticated session.
1662 ## 33. Incident Post-Mortem: Discarded Docker Exec Errors in logs (July 10, 2026)
1663 ### Symptom
1664 When the `warp` container is stopped, dead, or failing to connect to its endpoints, the `toggle.sh` and `
1665     recover.sh` connectivity health checks fail, but no details are printed to `output.log`. The logs only show
1666     generic `FAIL` outputs, making it hard to see if the failure was a network timeout, Docker daemon error,
1667     or a missing binary.
1668 ### Root Cause
1669 Overly broad error silencing. The `docker compose exec` commands in the health checks (Check 3 in `toggle.sh`
1670     and the traffic check in `recover.sh`) both redirected stderr to `/dev/null` (`&>/dev/null` and `2>/dev/
1671     null` respectively), completely throwing away any diagnostic error messages produced by Docker or `wget`.
1672 ### Fix (July 10, 2026)
1673 Redirected stderr of the `docker compose exec` check commands to `output.log` (`2>> output.log`). This ensures
1674     that any command execution failures or connection timeout details are logged.
1675 ## 34. Architectural Design: How nullexit Mimics Commercial VPNs during Roaming
1676 The `nullexit` roaming/network recovery subsystem replicates the exact engineering mechanics used by commercial
1677     , premium VPN clients (like Mullvad or NordVPN) to transition across Wi-Fi networks safely and seamlessly.
1678 ### The 4-Step Transition Cycle
1679 When your Mac disconnects from one Wi-Fi and associates with another, `nullexit` coordinates the following
1680     events:
1681 1. **Firewall Lock (Kill-Switch)**: The macOS Packet Filter (PF) anchor `com.apple/nullexit` remains locked on
1682     the physical interface (`en0`), blocking all direct outbound IPv4 and IPv6 traffic. This guarantees that
1683     your real ISP IP or unencrypted DNS requests **never leak** onto the new network while the connection is
1684     rebuilding.
1685 2. **System Event Interception**: A launchd LaunchAgent (`watcher.sh`) listens to the macOS System
1686     Configuration framework for the `State:/Network/Global/IPv4` key changes, spawning `recover.sh --post-wake`
1687     in under 500ms when a change is detected.
1688 3. **Bypass Routing**: The script dynamically resolves the new physical network's IP gateway (e.g.
1689     `192.168.137.1`), cleans up stale bypass routes from the previous network, and adds static bypass routes
1690     pointing to Cloudflare's WARP endpoints and the Tailscale control plane directly via the new gateway. This
1691     allows the VPN client to negotiate its handshake outside of the tunnel.
1692 4. **Hole-Punching / NAT Re-negotiation**: The script runs a target check on the `warp` container's tunnel. If
1693     the WireGuard connection is wedged by the network switch, it force-recreates the container to establish a
1694     fresh NAT binding to Cloudflare, restoring connection in ~15 seconds.
1695 ## 35. Incident Post-Mortem: macOS Tailscale Flag Requirement Abort (July 10, 2026)
1696 ### Symptom

```


1677 When the `toggle.sh` stop trap or the roaming watcher triggered, the gateway failed to shut down or disconnect properly. The `output.log` showed the error: `Error: changing settings via 'tailscale up' requires mentioning all non-default flags. To proceed, either re-run your command with --reset...`

1678

1679 **### Root Cause**

1680 In a previous attempt to stop host-side logouts (Section 32), we removed the `--reset` flag from all `tailscale up` invocations. However, if the Tailscale daemon on macOS is already running with non-default flags (like `--ssh` or `--accept-routes`), invoking `tailscale up` with only a subset of flags (like `--exit-node=`) triggers a strict safety check in the Tailscale CLI that aborts the command completely. Because the command aborted, exit-node routing remained stuck.

1681

1682 **### Fix (July 10, 2026)**

1683 Migrated all specific preference changes in `toggle.sh` and `scripts/common.sh` from `tailscale up` to a two-step sequence:

1684 1. `tailscale up` (with no flags) to safely ensure the interface is brought up using the current authenticated state.

1685 2. `tailscale set --exit-node=...` to modify the specific target preference cleanly without triggering the missing flags error or requiring `--reset`.

1686

1687 **## 36. Incident Post-Mortem: macOS SNAT Endpoint Poisoning & The P2P Blackhole (July 10, 2026)**

1688

1689 **### Symptom**

1690 When a Tailscale client (like an S24) connected to the exact same Wi-Fi network as the macOS gateway, it completely lost internet connectivity. However, if the S24 connected to a Windows PC's hotspot on the same network, the tunnel worked flawlessly.

1691

1692 **### Root Cause**

1693 Claude's critique correctly dismantled the port collision theory: the gateway daemon wasn't failing to bind, and macOS wasn't load-balancing ports. The real culprit was **macOS SNAT Source Port Mangling poisoning Tailscale's path discovery**.

1694

1695 Here is the exact step-by-step of the "infinite loop" blackhole:

1696 1. ****The P2P Handshake:**** The S24 and the Mac are on the same Wi-Fi (`192.168.1.x`). The S24 discovers the Mac's local IP and sends a UDP hole-punch packet to `192.168.1.5:41641`. Colima successfully forwards this to the gateway container.

1697 2. ****The Bypass Rule:**** The gateway replies. Because `routing-fix.sh` forces all Tailscale UDP traffic out `eth0` to bypass WARP, the reply goes to the macOS host.

1698 3. ****macOS SNAT Mangling:**** macOS routes the reply out `en0` to the S24. Because the packet crosses from the Colima VM bridge to the Wi-Fi interface, macOS applies Source NAT (SNAT). Critically, macOS **randomizes** the source port (e.g., changes it from `41641` to `58291`).

1699 4. ****Endpoint Poisoning:**** The S24 receives the authenticated Tailscale packet from `192.168.1.5:58291`. Tailscale's `magicsock` is designed to handle NAT dynamically, so it assumes the Mac has roamed to port `58291`! The S24 updates its endpoint and starts sending all its encrypted internet traffic to `192.168.1.5:58291`.

1700 5. ****The Blackhole:**** macOS has no port forward for `58291`. It drops 100% of the S24's outbound data packets. The connection is completely dead.

1701

1702 **### Why did the Hotspot work?**

1703 When the S24 connected to the PC hotspot, it was behind the PC's NAT. The S24 sent the packet, PC NATted it, and the Mac replied (mangled to `58291`). When the reply hit the PC, the PC's strict NAT dropped it because the source port (`58291`) didn't match the destination port it originally sent to (`41641`). Because the mangled packet was dropped, the S24 never poisoned its endpoint! It safely gave up on P2P, fell back to the 100% reliable TCP DERP relay, and the internet worked perfectly.

1704

1705 **### Fix & Resolution (July 10, 2026)**

1706

1707 ****Phase 1 Port disambiguation:**** Changed the gateway container's Tailscale listen port from the default `41641` to `41642` across `docker-compose.yml`, `post-rules.txt`, and `routing-fix.sh`. This prevents any bind conflict with the macOS host's native Tailscale daemon.

1708

1709 ****Phase 2 RFC1918 DROP rules:**** Updated `routing-fix.sh`'s `add\_tailscale\_p2p\_bypass()` to explicitly drop all outgoing Tailscale UDP packets destined for private subnets (`192.168.0.0/16`, `10.0.0.0/8`, `172.16.0.0/12`) when `TAILSCALE\_ALLOW\_LAN\_P2P=false`. This surgically kills the local P2P hole-punch attempt **before** it can trigger the macOS gvproxy SNAT mangling. The S24 receives a clean failure and immediately locks onto the public DERP relay with 0% packet loss.

1710

1711 ****Phase 3 Automatic network detection:**** Because `TAILSCALE\_ALLOW\_LAN\_P2P=false` breaks P2P on trusted hotspots that genuinely don't have AP isolation, a `.env` toggle and a full auto-detection system were implemented:

1712

1713 - ****`.env` toggle:**** `TAILSCALE\_ALLOW\_LAN\_P2P=true/false` static fallback, used only if auto-detection cannot run.

1714 - ****`watcher.sh` auto-detection** (`detect\_lan\_p2p\_mode`): ****** On every network change, `watcher.sh` runs two checks:

1715 1. ****WPA2-Enterprise detection:**** `airport -I` is used to read the current Wi-Fi `link auth` field. If the auth type does not contain `-psk` (e.g., it is plain `wpa2` = 802.1x), the network is classified as enterprise and P2P is forced off.

1716 2. ****AP Isolation probe:**** For WPA2-Personal networks, a short `ping` is sent to the default gateway. If the gateway responds (it always should on a real home/hotspot network), P2P is allowed. If not, AP isolation is inferred and P2P is forced off.

1717 - The result (`true` or `false`) is written to `lan\_p2p\_detected` in the repo root.


```

1718 - **`routing-fix.sh` dynamic reading:** `add_tailscale_p2p_bypass()` re-reads `.lan_p2p_detected` on every 30-
second loop iteration and atomically adds or removes the RFC1918 DROP rules. **No container restart is
needed when switching networks.**
1719
1720 ### Known Unknowns (Pending Verification)
1721 - The gvpnproxy's SNAT source port mangling theory is mechanistically sound and backed by observed behavior (
gvpnproxy's UDP proxy changelog has documented edge cases in this exact code path), but has not yet been
confirmed via `tcpdump -i en0 udp portrange 40000-60000` while triggering a P2P handshake from the phone. A
packet capture cross-referenced with `tailscale ping` endpoint reports on the phone side would confirm the
theory definitively.
1722 - Claude's recommendation: run `tcpdump -i en0 udp port 41642 or portrange 40000-60000` on the Mac while
triggering the handshake from the phone to see whether the reply leaves with a different source port than
`41642`.
1723
1724 ## 37. TODO
1725
1726 **Verify Wi-Fi Roaming:** Investigate and test whether switching Wi-Fi networks now successfully and smoothly
recovers the gateway end-to-end without any tailscale flag errors or dead tunnels. (Pending user
verification).
1727 **Fix Diagnostic Script Check 5/8:** ~Investigate why `diagnose-host-leak.sh` check 5/8 (`Host default route
`) sometimes reports "default route goes via physical Wi-Fi Tailscale route assertion failed" on macOS
even though the `warp=on` egress checks confirm that traffic is successfully tunneling.~` **Closed** fixed
by switching check 5 from `netstat -rn | grep default` to `route -n get 1.1.1.1`. macOS Tailscale uses
interface-scoped routes and does not replace the DHCP default route entry. See 36 Known Unknowns.
1728
1729 ## 38. Observation: AdGuard Returns Two Different Sinkhole IPs (Unverified)
1730
1731 > **Status: Observed but not formally verified. Needs a deeper audit of the rule-compiler sources to confirm.**
1732
1733 ### Observation (July 10, 2026)
1734 When querying AdGuard Home for blocked ad domains, some domains resolve to `0.0.0.0` and others resolve to
`127.0.0.1`. Both are blocked, both cause connection failure, but the different responses were unexpected.
1735
1736 ...
1737 doubleclick.net 0.0.0.0 (blocked)
1738 ads.google.com 127.0.0.1 (blocked)
1739 pagead2.googlesyndication.com 0.0.0.0 (blocked)
1740 scorecardresearch.com 127.0.0.1 (blocked)
1741 ...
1742
1743 ### Likely Explanation (Unverified)
1744 There are three historical schools of thought on the "correct" DNS sinkhole response, and AdGuard faithfully
preserves whichever the source blocklist used:
1745
1746 | Sinkhole IP | Convention | Origin |
1747 |---|---|---|
1748 | `0.0.0.0` | AdBlock-style lists (`\|\|domain`) | Modern DNS-level blocking; AdGuard's native format. Fails
fast OS doesn't attempt a TCP connection. Works for both IPv4 and IPv6 without a separate rule. |
1749 | `127.0.0.1` | Hosts-file-style lists (`127.0.0.1 domain`) | 90s-era `/etc/hosts` tradition. Steven Black's
hosts list (500k+ domains) still uses this. |
1750 | `NXDOMAIN` | Purist / Pi-hole default | Returns "domain doesn't exist." Semantically most accurate but
requires browsers to handle NXDOMAIN gracefully. |
1751
1752 The dual-response behavior in this project is because `rule-compiler` pulls from both AdBlock-format sources (
Hagezi, uBlock origin lists) and hosts-format sources (Spamhaus DROP, Steven Black, Feodo Tracker), and
AdGuard returns whatever sinkhole IP the matching rule specifies.
1753
1754 ### To Verify
1755 Run `docker compose exec tailscale cat /etc/hosts` to confirm no hosts-file rules are being injected at the
container level, and check which specific AdGuard rule matched each domain via the AdGuard query log at `
http://100.100.21.8:3000/#logs`.
1756
1757 ## 39. Observation: AGY CLI Appears to Generate Native macOS Notification Pop-ups (Unverified)
1758
1759 > **Status: Observed but source unverified. macOS security logs ruled out system-level origin.**
1760
1761 ### Observation (July 10, 2026)
1762 While running a DNS blocklist verification via the Antigravity CLI (`agy`), a command was proposed that
included `malware.wicar.org` (a known-safe DNS blocklist test domain) in a `dig` loop. Shortly after, a
macOS-style notification popup appeared describing a "malicious" or "blocked" event. The AGY CLI terminal
session then closed.
1763
1764 ### Investigation
1765 - **macOS XProtect / Gatekeeper logs:** Empty. Zero events in the 30-minute window around the incident.
1766 - **macOS Endpoint Security / System Policy logs:** Empty.
1767 - **Broad `log show` search for "malicious", "malware", "blocked":** Empty.
1768 - **Conclusion:** macOS itself did not generate the notification. No system security subsystem fired.
1769
1770 ### Likely Explanation (Unverified)
1771 The notification appears to have originated from **AGY CLI's own internal safety system**. AGY CLI is a native
macOS application (not just a terminal process) and has access to the macOS Notification Center API (`

```

```

UNUserNotificationCenter`). When its guardrails detected a domain containing the word "malware" (`malware.
wicar.org`) in a proposed shell command, it likely:
1772 1. Blocked the command from executing
1773 2. Posted a native macOS-style notification via the Notification Center
1774
1775 This is surprising behavior for a CLI tool most terminal programs don't send GUI notifications but AGY CLI
appears to bridge both worlds: it runs in a terminal but is packaged as a native app with full system API
access. The notification would be indistinguishable from a system security alert at a glance.
1776
1777 ### Notes
1778 - `malware.wicar.org` is the DNS/AV equivalent of the EICAR test file a deliberately benign domain that
triggers blocklist/AV systems to verify they work. It was included in the test set to confirm the AdGuard
blocklist was catching it. AGY's safety system is not aware of this distinction.
1779 - For future DNS blocklist testing, use only ad/tracking domains (e.g., `doubleclick.net`, `adnxs.com`) to
avoid triggering AGY's guardrails.
1780
1781 ## 40. Apple Silently Removed the `airport` Binary in macOS Sonoma 14.4 (March 2024)
1782
1783 ### What happened
1784 `watcher.sh`'s `detect_lan_p2p_mode()` was written to use the `airport` CLI tool at:
1785 ```
1786 /System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport
1787 ```
1788 This path returned `exit 127: no such file or directory`. The function silently fell back to `reason="no-wifi"`
even while the Mac was actively connected to WPA2 Enterprise Wi-Fi.
1789
1790 ### Why Apple removed it
1791 `airport` was never a real public binary it lived inside a **private framework** and was always technically
unsupported. Apple deprecated it quietly years ago and finally removed it in **macOS Sonoma 14.4 (March
2024)**. The removal is part of a broader privacy clampdown on Wi-Fi metadata:
1792 - Starting in **macOS 14.5**, BSSIDs and other Wi-Fi association details are **redacted** (`<redacted>`) in
most tools, including the official replacement `wdutil`
1793 - Apple's official recommendation is to use the **CoreWLAN framework** (Swift/Objective-C) with proper
entitlements useless for a bash script
1794 - The official CLI replacement `wdutil` requires `sudo` for most useful output also useless for a background
LaunchAgent
1795
1796 ### Fix
1797 Replaced `airport -I | awk '/link auth/'` with:
1798 ```bash
1799 system_profiler SPAirPortDataType | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF
; exit}'
1800 ```
1801 This returns human-readable strings like `WPA2 Enterprise`, `WPA2 Personal`, or `None` for the currently
associated network without `sudo`, and without any removed private binary.
1802
1803 **Confirmed working output on this machine:**
1804 ```
1805 P2P detect: security='Enterprise' allow=false (reason: 802.1x-enterprise)
1806 ```
1807
1808 ### Risk: `system_profiler` may not survive forever either
1809 `system_profiler SPAirPortDataType` still works as of macOS Sequoia (15.x) without sudo and without redaction.
However, given Apple's trajectory on Wi-Fi privacy, this could be locked down in a future release. If `
security` comes back empty on a future macOS version while the Mac is actively on Wi-Fi, the function will
silently fall back to `allow=false` (safe default), but the detection will be broken again. The long-term
fix would be a small Swift helper using CoreWLAN that we compile once and bundle in the repo.
1810
1811 ## 41. Fixed: Watcher.sh Suppressed Exit Code Bug
1812
1813 ### Observation
1814 The `scripts/watcher.sh` script is a long-running daemon that invokes `recover.sh --post-wake` when it detects
system wake or network roam events. Previously, it contained the following bug:
1815 ```bash
1816 bash "$RECOVER" --post-wake
1817 local exit_code=$?
1818 ...
1819 return $exit_code || true
1820 ```
1821 The `|| true` mistakenly swallowed the actual `$exit_code` returned by `recover.sh`, causing `run_recover()` to
always return `0` (success), masking any underlying failures in the wake recovery process.
1822
1823 ### Fix
1824 The `|| true` statement was removed from the return line:
1825 ```bash
1826 return $exit_code
1827 ```
1828 Since `watcher.sh` explicitly omits `set -e` (to prevent pipe breakages from killing the daemon), removing `||
true` safely allows the underlying exit code to propagate without risking daemon termination. The watcher
daemon was then reloaded (`launchctl kickstart -k`) to pick up the script changes in memory.
1829
1830 ## 42. Network Readiness Race Condition on `--restart` (Fixed)

```

```

1830
1831 **Problem:** When running `./toggle.sh --restart`, the script tears down the gateway and immediately begins the
      startup sequence. On systems with Wi-Fi (or any wireless interface), the OS takes a moment to re-associate
      with the network after the teardown's DNS/routing changes. If the startup sequence ran before the OS had a
      default route, `get_active_service` would return nothing, routing bypass targets would be wrong, and DNS/
      WARP setup would silently fail resulting in a partially configured gateway that self-heals only once the `
      watcher.sh` re-runs.
1832
1833 **Root cause:** The original code had no gate before the first network-dependent call (`ACTIVE_SERVICE=$(
      get_active_service)` at the top of the start branch). This was a pure timing race.
1834
1835 **First fix (Wi-Fi specific):** A polling loop on `ipconfig getifaddr en0` was added to wait for an IPv4
      address on `en0` before proceeding. This worked but was brittle it only handled Wi-Fi and would fail for
      Ethernet, USB-C adapters, or iPhone USB tethering.
1836
1837 **Generalized fix:** Replaced the `en0` check with `route get default | grep 'gateway:'`. This detects a valid
      default gateway on any interface, covering all connection types. The loop polls every 2 seconds with a
      60-second hard timeout (then proceeds with a warning rather than hanging forever). The output prints the
      detected gateway IP so failures are easy to diagnose in logs.
1838
1839 ```bash
1840 # In toggle.sh, before ACTIVE_SERVICE=$(get_active_service)
1841 while true; do
1842     if route get default 2>/dev/null | grep -q 'gateway: '; then
1843         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1844         echo " Network ready (default gateway: $_gw)."
1845         break
1846     fi
1847     ...
1848 done
1849 ```
1850
1851 **Why not ping?** Pinging an IP (e.g. `1.1.1.1`) during restart is unreliable because DNS may still be hijacked
      from the previous session, and the exit node routing may not yet be cleared, causing the ping to loop back
      through the tunnel. `route get default` is a pure local kernel query no packets sent.
1852
1853 ## 43. Low Gateway Throughput (DERP Relay & MTU Fragmentation) Fixed
1854
1855 **Problem:** The host Mac's internet throughput through the gateway dropped significantly (e.g., from 34Mbps
      raw to 2.1Mbps via gateway). This was caused by a combination of two bottlenecks:
1856
1857 **1. Tailscale DERP Relay Bottleneck:**
1858 Because the host Mac and the Docker container operate behind the same public IP, standard hole-punching for
      Tailscale P2P failed, forcing traffic through Tailscale's NYC DERP relay. Normally, `routing-fix.sh`
      explicitly drops container Tailscale UDP packets destined for local subnets (when `TAILSCALE_ALLOW_LAN_P2P=
      false` in `.env` or auto-detected). We nuke these P2P connections on purpose to prevent the severe macOS `
      gvproxy` SNAT endpoint poisoning issue we faced earlier (see `devref.md 36`). However, this strict policy
      unintentionally blocked direct communication between the container and the Mac host itself via the Docker
      bridge network.
1859
1860 **Fix:** Tailscale's control plane registers the Mac host using its physical IP (e.g., `172.17.52.223`), not
      the Docker bridge IP. If this physical IP falls within the dropped local subnets (like `172.16.0.0/12`),
      the P2P handshake is dropped. To solve this natively, `scripts/common.sh` now features a `write_host_ips()`
      function that actively grabs all physical IPv4 addresses on the Mac and writes them to `.host_ips`. `
      routing-fix.sh` dynamically reads this file every 30 seconds and injects priority `RETURN` rules for those
      exact physical IPs. This guarantees direct container-to-host P2P over the local bridge (bypassing DERP)
      while continuing to block external LAN devices (like phones) to prevent the 36 SNAT poisoning bug.
1861
1862 **2. MTU Double-Encapsulation Fragmentation:**
1863 The host's Tailscale interface (`utun5`) defaulted to an MTU of 1280. The WARP tunnel inside the container also
      uses an MTU of 1280. When a full 1280-byte Tailscale packet from the host entered the container and was
      encapsulated by WARP (adding ~60 bytes of overhead), the resulting packet exceeded 1280 bytes, leading to
      severe fragmentation and performance degradation.
1864
1865 **Fix:** In `toggle.sh`'s `setup_exit_node_routing` function, the host's Tailscale `utun` interface MTU is
      explicitly lowered to `1200` (configurable via `HOST_MTU` in `.env`). This ensures that host-originated
      packets can comfortably fit inside the container's WARP tunnel encapsulation without fragmenting.

```

5 diagrams.md

```

1 # nullexit Flow Diagrams & Architecture
2
3 ---
4
5 ## 1. System Architecture What's Running & Where
6

```

```

7  mermaid
8  graph TB
9      subgraph INTERNET[" Internet / Cloudflare Edge"]
10         CF["Cloudflare WARP\n(Exit Point)\nwarp=on"]
11     end
12
13     subgraph TAILNET[" Tailscale Mesh (100.x.x.x)"]
14         PHONE[" Phone / Laptop\n(Tailnet Client)"]
15     end
16
17     subgraph MACHOST[" macOS Host"]
18         direction TB
19         PFFIREWALL["macOS pf Firewall\n(Kill-Switch & TCP MSS Clamping)"]
20         TSDAEMON["tailscaled\n(brew service)\nhost Tailscale"]
21         HOSTDNS["Host DNS\n Gateway IP\n(hijacked by toggle.sh)"]
22         HOSTSOCKS["Host SOCKS5\n127.0.0.1:1080\n(fallback only)"]
23
24         subgraph MONITORS[" Background Daemons (toggle.sh children)"]
25             WARPWATCH["WARP Watcher\n(every 30s, via docker exec)\n output.log"]
26             LEAKPROBE["Host Leak Probe\n(every 300ms, via curl from HOST)\n output.log"]
27             DNSWATCHER["DNS Watcher\n(re-hijacks if drift detected)\n output.log"]
28             CAFFEINATE["caffeinate -i\n(sleep prevention)"]
29         end
30     end
31
32     subgraph COLIMAVM[" Colima VM (Linux, 600MB RAM)"]
33         subgraph NETNS["warp container network namespace (shared by ALL containers)"]
34             GLUETUN["warp / Gluetun\nWireGuard tun0\n(owns the namespace)"]
35             TS["tailscale container\nAdvertises exit node\ntailscale0 interface"]
36             SOCKS["tailscale\nSOCKS5\nport 1080"]
37             ADGUARD["adguardhome\nDNS sinkhole\nport 5335"]
38             ROUTINGFIX["routing-fix sidecar\nGeo-IP Blocking & Go Logger\n(writes blocked.log)"]
39         end
40     end
41
42     subgraph LAUNCHD[" launchd (always-on)"]
43         WATCHER["scripts/watcher.sh\n(sleep/wake + Wi-Fi roam\n+ LAN P2P auto-detection)"]
44     end
45
46     subgraph RECOVERY[" Recovery"]
47         RECOVER["recover.sh\n(nuclear reset)"]
48         RECOVERPOST["recover.sh --post-wake\n(light refresh)"]
49     end
50
51     %% Traffic flow
52     PHONE -->|"WireGuard / Tailscale mesh"| TSDAEMON
53     TSDAEMON -->|"exit-node route utun"| TS
54     TS -->|"FORWARD tun0 (MASQUERADE)"| GLUETUN
55     GLUETUN -->|"WireGuard UDP (macOS Host Egress)"| PFFIREWALL
56     PFFIREWALL -->|"TCP MSS Clamped / Kill-Switch check"| CF
57
58     %% DNS
59     HOSTDNS -->|"UDP:53 TCP:5354"| ADGUARD
60     ADGUARD -->|"DNS upstream via tun0"| CF
61
62     %% Monitoring
63     WARPWATCH -->|"docker exec warp wget cdn-cgi/trace"| GLUETUN
64     LEAKPROBE -->|"curl cdn-cgi/trace (HOST NIC)"| CF
65     DNSWATCHER -->|"networksetup -getdnsservers"| HOSTDNS
66
67     %% Logging
68     ROUTINGFIX -->|"writes"| BLOCKEDLOG["blocked.log\n(Host-side)"]
69
70     %% Recovery triggers
71     WARPWATCH -->|"6 consecutive warp=off"| RECOVER
72     WATCHER -->|"sleep/wake / roam"| RECOVERPOST
73     RECOVERPOST -->|"if gateway unhealthy"| RECOVER
74     WATCHER -->|"writes .lan_p2p_detected"| ROUTINGFIX
75
76     style MONITORS fill:#1a1a2e,color:#e0e0ff
77     style COLIMAVM fill:#0d2137,color:#e0e0ff
78     style NETNS fill:#0a1628,color:#e0e0ff
79     style RECOVERY fill:#2d0a0a,color:#ffe0e0
80     style INTERNET fill:#0a2d1a,color:#e0ffe0
81     style TAILNET fill:#1a2d0a,color:#e8ffe0
82     style LAUNCHD fill:#2d1a2d,color:#ffe0ff
83
84  ---
85  ---
86
87  ## 2. toggle.sh Full START Flow

```

```

88
89 ```mermaid
90 flowchart TD
91     START(["./toggle.sh"])
92     CHECK{"is_gateway_active?\n(containers running OR\nDNS 1.1.1.1)"}
93
94     START --> CHECK
95     CHECK -->|"YES already ON"| STOPFLOW[" See STOP Flow"]
96     CHECK -->|"NO start it"| S1
97
98     S1["1. Reset DNS 1.1.1.1\n(prevent deadlocks)"]
99     S2["2. tailscale down\n(prevent exit-node routing during boot)"]
100    S3["3. Check Colima VM\n(colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged)"]
101    S4["4. Configure VM swap\n(400MB, prevent OOM)"]
102    S5["5. Clean corrupted AdGuard config\n(prevent container crash loop)"]
103    S6["6. docker compose up -d --build\n(compile Go threat rules & boot containers)"]
104    S7["7. Poll tailscale status\n(wait for 'offers exit node'\nup to 60s)"]
105    TSREADY{"container\nTailscale ready?"}
106    ABORT([" ABORT\n cleanup_handler"])
107    S8["8. Resolve gateway Tailscale IP\n(ADGUARD_IP.txt or docker exec)"]
108    S9["9. Verify tailscaled running\n(auto-start if needed)"]
109    S10["10. tailscale up --reset\n--exit-node=\n--accept-routes=true\n--ssh=true --accept-dns=false"]
110
111    PF1{"Pre-flight 1/3\ntailscale ping gateway"}
112    PF2{"Pre-flight 2/3\ndig +tcp AdGuard DNS"}
113    PF3{"Pre-flight 3/3\nWARP internet check"}
114    ALLPASS{"All 3 pass?"}
115
116    EXITPATH[" EXIT NODE PATH\n tailscale up --exit-node=\n force_dns_to_gateway (3 retry)\n SOCKS5 disabled"]
117    SOCKSPATH[" SOCKS5 FALLBACK PATH\n macOS SOCKS5 127.0.0.1:1080\n DNS proxy 127.0.0.1:53\n No exit node (\n SKIP_EXIT_NODE=true)"]
118
119    DAEMONS["Start background daemons\n caffeinate -i (sleep prevention)\n DNS Watcher\n WARP Watcher\n Host\n Leak Probe\n(all write output.log)"]
120
121    RULECOMPILE["Rule compiler status\n(log rule/IP counts)"]
122    WRITEMARKER["write_gateway_active_marker\n(/tmp/nullexit-gateway-active.marker)"]
123    DONE([" Gateway UP"])
124
125    S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7
126    S7 --> TSREADY
127    TSREADY -->|"NO (40 NoState)"| ABORT
128    TSREADY -->|"YES"| S8
129    S8 --> S9 --> S10
130    S10 --> PF1 --> PF2 --> PF3
131    PF1 & PF2 & PF3 --> ALLPASS
132    ALLPASS -->|"YES"| EXITPATH
133    ALLPASS -->|"NO"| SOCKSPATH
134    EXITPATH --> DAEMONS
135    SOCKSPATH --> DAEMONS
136    DAEMONS --> RULECOMPILE --> WRITEMARKER --> DONE
137
138    style ABORT fill:#5c1010,color:#fff
139    style EXITPATH fill:#0a3d1a,color:#c0ffc0
140    style SOCKSPATH fill:#3d2d00,color:#ffe0a0
141    style DAEMONS fill:#0a1f3d,color:#c0d8ff
142    style DONE fill:#0a3d1a,color:#fff
143
144
145 ---
146
147 ## 3. toggle.sh STOP Flow
148
149 ```mermaid
150 flowchart TD
151     STOP(["./toggle.sh\n(gateway already ON)"])
152
153     ST1["1. tailscale down\n(disconnect from mesh)"]
154     ST2["2. docker compose down -t 5\n(stop all containers)"]
155     ST3["3. Reset DNS 1.1.1.1\n(networksetup on all services)"]
156     ST4["4. stop_local_dns_proxy\n(kill Python UDP proxy)"]
157     ST5["5. stop_pidfile_daemon\n(kill caffeinate, rm PID)"]
158     ST6["6. stop_pidfile_daemon\n(kill DNS Watcher, rm PID)"]
159     ST7["7. stop_pidfile_daemon\n(kill WARP Watcher, rm PID)"]
160     ST8["8. stop_pidfile_daemon\n(kill Leak Probe, rm PID)"]
161     ST9["9. clear_gateway_active_marker\n(rm /tmp/nullexit-gateway-active.marker)"]
162     ST10["10. cleanup_network_state\n disable_all_proxies\n Flush DNS cache (dscacheutil)\n Flush stale routes\n n Power-cycle Wi-Fi (offon)"]
163
164     STOP --> ST1 --> ST2 --> ST3 --> ST4 --> ST5 --> ST6 --> ST7 --> ST8 --> ST9 --> ST10
165     ST10 --> DONE([" Gateway DOWN\nInternet restored"])

```

```

166 STOP --> ST1 --> ST2 --> ST3 --> ST4
167 ST4 --> ST5 --> ST6 --> ST7 --> ST8 --> ST9 --> ST10 --> DONE
168
169 style DONE fill:#0a3d1a,color:#fff
170
171
172 ---
173
174 ## 4. Monitoring Layer The 4 Background Daemons
175
176 ```mermaid
177 graph LR
178   subgraph DAEMONS["Background Daemons (all start with gateway, stop on teardown)"]
179     direction TB
180
181     subgraph D1[" Sleep Prevention"]
182       CAFF["caffeinate -i\nPID: /tmp/nullexit-caffeinate.pid\n\nKeeps Mac awake while\ngateway is active"]
183     end
184
185     subgraph D2[" DNS Watcher"]
186       DNSW["Polls networksetup every ~30s\nPID: /tmp/nullexit-dns-watcher.pid\n\nIf DNS drifts from gateway IP\nre-hijacks automatically\noutput.log"]
187     end
188
189     subgraph D3[" WARP Watcher"]
190       WARPW["Polls every 30s via:\nncurl cdn-cgi/trace\nPID: /tmp/nullexit-warp-watcher.pid\n\nCounts consecutive warp=off\nWARP_FAIL_THRESHOLD (default 6=30s)\nruns recover.sh (nuclear)\noutput.log"]
191     end
192
193     subgraph D4[" Host Leak Probe"]
194       LEAKP["Polls every 300ms via:\nncurl cdn-cgi/trace (HOST NIC directly)\nPID: /tmp/nullexit-host-leak-probe.pid\n\nLogs ONLY on state change:\n LEAK / ROTATE / HOST-PROBE failed\nAll errors output.log\n(HOST_LEAK_PROBE=true in .env)"]
195     end
196   end
197
198   subgraph BLIND["What each monitor can/can't see"]
199     B1["WARP Watcher sees:\n Container WARP tunnel state\n Host NIC traffic\n Flash leaks < 5s"]
200     B2["Host Leak Probe sees:\n Host NIC egress (browser path)\n Sub-second transitions\n Container internals"]
201   end
202
203   D3 -->|"covers"| B1
204   D4 -->|"covers"| B2
205
206   style D1 fill:#1a2d0a
207   style D2 fill:#0a1a2d
208   style D3 fill:#2d0a2d
209   style D4 fill:#2d1a00
210   style BLIND fill:#1a1a1a,color:#aaa
211
212
213 ---
214
215 ## 5. Traffic Flow Data Path (Normal Operation)
216
217 ```mermaid
218 sequenceDiagram
219   participant PHONE as Phone<br/>(Tailnet client)
220   participant TS_HOST as tailscaled<br/>(macOS host)
221   participant TS_CONTAINER as tailscale<br/>(container)
222   participant GLUETUN as Gluetun/warp<br/>(container)
223   participant CF as Cloudflare<br/>WARP Edge
224   participant INTERNET as Internet
225
226   Note over PHONE,INTERNET: Normal EXIT NODE path (all 3 pre-flights passed)
227
228   PHONE->>TS_HOST: WireGuard packet<br/>(encrypted, UDP 41641)
229   TS_HOST->>TS_CONTAINER: route via utun* tailscale0
230   TS_CONTAINER->>GLUETUN: FORWARD chain<br/>(iptables MASQUERADE on tun0)
231   GLUETUN->>TS_HOST: WireGuard tunnel (tun0) egresses macOS Host
232   Note over TS_HOST: macOS pf Firewall applies<br/>Kill-Switch & TCP MSS Clamping
233   TS_HOST->>CF: re-encrypted UDP packet (MSS 1160)
234   CF->>INTERNET: Plain HTTPS from<br/>Cloudflare IP
235
236   Note over PHONE,INTERNET: DNS Resolution path
237   PHONE->>TS_HOST: DNS query
238   TS_HOST->>TS_CONTAINER: UDP:53 TCP:5354 AdGuard :5335
239   TS_CONTAINER->>GLUETUN: AdGuard upstream DNS via tun0

```



```

240 GLUETUN-->>CF: Encrypted DNS query
241 CF-->>GLUETUN: DNS response
242 GLUETUN-->>TS_CONTAINER: response
243 TS_CONTAINER-->>TS_HOST: response
244 TS_HOST-->>PHONE: DNS answer
245 ---
246 ---
247 ---
248
249 ## 6. recover.sh Decision Tree
250
251 mermaid
252 flowchart TD
253     INVOKE(["recover.sh called"])
254     MODE{"--post-wake\nflag?"}
255
256     subgraph POSTWAKE["--post-wake (light refresh, gateway stays UP)"]
257         PW1["Re-hijack DNS gateway IP"]
258         PW2["Flush DNS cache"]
259         PW3["Refresh tailscale exit-node"]
260         PW4["Force-recreate warp container\n(if Gluetun healthcheck failed)"]
261         PW5["Leave: containers, Wi-Fi,\ncaffeinate, DNS watcher,\nHost Leak Probe untouched"]
262     end
263
264     subgraph NUCLEAR["Default (nuclear gateway torn down)"]
265         N0["Sudo keep-alive loop (background)"]
266         N1["1. tailscale down\n(disconnect from mesh)"]
267         N2["2. Reset DNS 1.1.1.1\n(ALL network services)"]
268         N3["3. disable_all_proxies\n(all interfaces)"]
269         N4["4. Flush DNS cache\n(dscacheutil -flushcache)"]
270         N5["5. Flush stale routes\n(WARP endpoint routes)"]
271         N6a["6a. docker compose down -t 30\n(stop all containers)"]
272         N6b["6b. stop_pidfile_daemon\n(kill caffeinate)"]
273         N6c["6c. stop_pidfile_daemon\n(kill DNS Watcher)"]
274         N6d["6d. stop_pidfile_daemon\n(kill Leak Probe)"]
275         N7["7. Power-cycle Wi-Fi\n(networksetup offsleep 2on)"]
276         N8["8. Verify internet working\n(curl ifconfig.io)"]
277     end
278
279     DONE_PW(["Gateway refreshed\n(still running)"])
280     DONE_NUC(["Internet restored\n(gateway fully stopped)"])
281
282     INVOKE --> MODE
283     MODE -->|"yes"| PW1
284     PW1 --> PW2 --> PW3 --> PW4 --> PW5 --> DONE_PW
285     MODE -->|"no"| N0
286     N0 --> N1 --> N2 --> N3 --> N4 --> N5
287     N5 --> N6a --> N6b --> N6c --> N6d --> N7 --> N8 --> DONE_NUC
288
289     style POSTWAKE fill:#0a2d1a,color:#c0ffc0
290     style NUCLEAR fill:#2d0505,color:#ffc0c0
291     style DONE_PW fill:#0a3d1a,color:#fff
292     style DONE_NUC fill:#3d1a00,color:#ffe0c0
293 ---
294 ---
295 ---
296
297 ## 7. Failure Paths & Self-Healing
298
299 mermaid
300 flowchart LR
301     subgraph EVENTS["Failure / Event"]
302         E1["WARP tunnel drops\n(warp=off)"]
303         E2["Mac wakes from sleep\nor Wi-Fi roams"]
304         E3["DNS drifts from\ngateway IP"]
305         E4["toggle.sh crashes /\nCtrl-C / SIGTERM"]
306         E5["Host-side flash leak\ndetected (HOST NIC)"]
307         E6["Silent NAT timeout\n(ping stops working)"]
308     end
309
310     subgraph DETECTORS["Detector"]
311         D1["WARP Watcher\n(30s poll, docker exec)"]
312         D2["scripts/watcher.sh\n(launchd, always on)"]
313         D3["DNS Watcher\n(30s poll, networksetup)"]
314         D4["cleanup_handler\n(ERR/INT/TERM/HUP trap)"]
315         D5["Host Leak Probe\n(300ms poll, HOST curl)"]
316         D6["routing-fix.sh\n(30s curl over tun0)"]
317     end
318
319     subgraph ACTIONS["Response"]
320         A1["threshold consecutive off\nrecover.sh (nuclear)"]

```

```

321     A2["recover.sh --post-wake\n(gentle refresh)"]
322     A3["Re-hijack DNS\n(networksetup setdnsservers)"]
323     A4["Stop all daemons\nReset DNS 1.1.1.1\nTear down Tailscale"]
324     A5["Log LEAK to output.log"]
325     A6["Blackhole internet traffic\n+ Drop TUNNEL_FAILED_CLOSED.marker"]
326     A7["Push macOS Desktop Notification\n(via watcher.sh listener)"]
327 end
328
329 E1 --> D1 --> A1
330 E2 --> D2 --> A2
331 E3 --> D3 --> A3
332 E4 --> D4 --> A4
333 E5 --> D5 --> A5
334 E6 --> D6 --> A6
335 A6 -.->|"marker detected"| D2
336 D2 --> A7
337
338 style EVENTS fill:#2d1010,color:#ffcccc
339 style DETECTORS fill:#0a1a2d,color:#cce0ff
340 style ACTIONS fill:#0a2d0a,color:#ccffcc
341 ---
342
343 ---
344
345 ## 8. output.log Who Writes What
346
347 All events converge into a single `output.log` at the repo root.
348
349 | Writer | Event Types | Format |
350 |-----|-----|-----|
351 | `toggle.sh` | All startup/shutdown steps, errors, pre-flight results | Plain text with timestamps |
352 | `recover.sh` | Every recovery step (nuclear or post-wake) | Plain text |
353 | **WARP Watcher** | `WARP DOWN`, `WARP RECOVERED`, `WARP SHUTDOWN` | `[UTC timestamp]` prefix |
354 | **Host Leak Probe** | `LEAK`, `ROTATE`, `HOST-PROBE failed/timeout` | `[HH:MM:SS.mmm]` prefix |
355 | **DNS Watcher** | DNS re-hijack events | Appended inline |
356 | `docker compose` | Container logs on failure (last 100 lines of warp) | Dumped on ERR path |
357 | `routing-fix.sh` | (via `nullexit-logger` inside container) | Structured |
358
359 **Grep cheatsheet:**
360 ```bash
361 grep LEAK output.log # Host-side warp-off events (real leaks)
362 grep ROTATE output.log # Cloudflare edge IP rotations (harmless)
363 grep 'HOST-PROBE' output.log # curl failures from probe
364 grep 'WARP DOWN' output.log # Container-side tunnel drops
365 grep 'WARP SHUTDOWN' output.log # Auto-recovery triggered
366 grep 'WARP RECOVERED' output.log # Self-healed before threshold
367 ```

```

6 toggle.sh

```

1 #!/bin/bash
2 # toggle.sh nullexit gateway toggle script for macOS
3 # Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.
4
5 set -e
6
7 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
8
9 # Enforce Cryptographic Script Integrity
10 if [ -f "scripts/crypto.sh" ]; then
11     if ! bash scripts/crypto.sh --verify; then
12         exit 1
13     fi
14 fi
15
16 # Define log file
17 LOG_FILE="$PWD/output.log"
18
19 # Rotate log file if it exceeds 50MB (52428800 bytes)
20 if [ -f "$LOG_FILE" ]; then
21     log_size=$(stat -f%z "$LOG_FILE" 2>/dev/null || stat -c%s "$LOG_FILE" 2>/dev/null || echo 0)
22     if [ "$log_size" -gt 52428800 ]; then
23         mv "$LOG_FILE" "${LOG_FILE}.old" 2>/dev/null || rm -f "$LOG_FILE"
24         echo "[$(date '+%Y-%m-%d %H:%M:%S')] Log file exceeded 50MB; rotated to output.log.old" > "$LOG_FILE"
25     fi
26 fi

```

```

27
28 # --- MAIN EXECUTION LOGGING ---
29 echo -e "\n===== " >> "$LOG_FILE"
30 echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh invoked" >> "$LOG_FILE"
31 echo "===== " >> "$LOG_FILE"
32
33 # (Previously this script chmod 600'd .env at start to "unlock" it for docker compose,
34 # and chmod 000'd it on exit. That pattern broke docker compose on macOS/Colima when
35 # the umask produced 0600 the script removed its own ability to read .env before it could even proceed.)
36
37 # Start execution timer
38 TOGGLE_START_TIME=$SECONDS
39
40 if [[ "$1" == "--restart" ]]; then
41     echo "Executing Gateway Restart Sequence..."
42     # We must source common.sh here temporarily to check is_gateway_active before we proceed
43     source "$SCRIPT_DIR/scripts/common.sh"
44     if is_gateway_active; then
45         echo "Gateway is currently running. Stopping it first..."
46         bash "$0"
47         echo "Gateway stopped. Now starting it..."
48     else
49         echo "Gateway is already stopped. Starting it..."
50     fi
51     bash "$0"
52     exit 0
53 fi
54
55 # Global flag to track if the script completed successfully
56 SUCCESS_RUN=false
57
58 # Global variable to track the currently running background command PID
59 CURRENT_BG_PID=""
60
61 # Source common bash functions
62 source "$SCRIPT_DIR/scripts/common.sh"
63
64 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
65 LOCK_FILE="/tmp/nullexit-toggle.lock"
66 if [ -f "$LOCK_FILE" ]; then
67     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
68     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
69         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
70             die "Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running."
71         fi
72     fi
73 fi
74 echo "$$" > "$LOCK_FILE"
75
76 # Helper function to run a GUI command as the logged-in console user
77 # (Prevents permission failures if the script is run with sudo)
78 run_gui_cmd() {
79     local console_user
80     console_user=$(stat -f '%Su' /dev/console 2>> output.log || echo "$SUDO_USER")
81     if [ -z "$console_user" ] || [ "$console_user" = "root" ]; then
82         console_user=$(logname 2>> output.log || echo "$USER")
83     fi
84
85     if [ -n "$console_user" ] && [ "$console_user" != "root" ] && [ "$EUID" -eq 0 ]; then
86         sudo -u "$console_user" "$@"
87     else
88         "$@"
89     fi
90 }
91
92 # Active-state marker: written at end of START path so external watchers
93 # (see launchd/com.nullexit.wake-recovery.plist + scripts/watcher.sh +
94 # devref 10.29) know the gateway is currently up. They only fire
95 # recover.sh --post-wake when this file is present. Cleared at the top
96 # of the STOP path and inside cleanup_handler on any error/signal path
97 # so a half-broken gateway never re-triggers post-wake refreshes.
98 write_gateway_active_marker() {
99     date -u +%FT%TZ > /tmp/nullexit-gateway-active.marker
100     # Set the watcher debounce timestamp to now to prevent a redundant post-startup recovery run.
101     date +%s > /tmp/nullexit-watcher.last-recovery 2>/dev/null || true
102 }
103
104 clear_gateway_active_marker() {
105     rm -f /tmp/nullexit-gateway-active.marker
106     rm -f "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/TUNNEL_FAILED_CLOSED.marker"
107 }

```

```

108
109 # Defensive: clear any stale marker from a prior crashed/aborted run. If a
110 # previous toggle.sh wrote the marker but was OOM-killed in the middle of the
111 # START block (before the END-of-START write) or if the user rebooted mid
112 # session this prevents the watcher from firing post-wake against a
113 # non-running gateway on every wake event. Placed AFTER the function
114 # definitions so bash resolves the call at parse time.
115 clear_gateway_active_marker
116
117 PID_FILE="$PID_CAFFEINATE"
118
119 # Start macOS caffeinate to prevent sleep while gateway is running
120 start_sleep_prevention() {
121     if ! command -v caffeinate >/dev/null 2>&1; then
122         echo " [Warning] caffeinate tool not found. Sleep prevention unavailable."
123         return 1
124     fi
125
126     # Stop any existing sleep prevention process first to avoid duplicates
127     stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"
128
129     echo " Enabling system sleep prevention (caffeinate)..."
130     # Run caffeinate wrapped in a bash trap to prevent idle system sleep.
131     # Critically, this traps macOS shutdown signals (SIGTERM) and automatically
132     # flushes hijacked DNS back to normal right before the Mac powers off.
133     # We do not use recover.sh here because it is a heavy recovery script (managing
134     # containers, restarting tailscaled, etc.) which takes too long and gets
135     # terminated by macOS before it can reset the DNS. Instead, we surgically and
136     # instantly restore normal DNS and proxy settings here.
137     nohup bash -c "
138     trap '
139         echo \"[Shutdown Trap] Reverting network settings...\" >> \"\$PWD/output.log\" 2>&1
140         kill \$! 2>/dev/null || true
141         sudo -n networksetup -setdnsservers \"\$ACTIVE_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
142         sudo -n networksetup -setdnsservers \"\$ENO_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
143         sudo -n networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 || true
144         sudo -n networksetup -setsearchdomains \"\$ENO_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 || true
145         sudo -n networksetup -setsocksfirewallproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 ||
146     true
147         sudo -n networksetup -setsocksfirewallproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
148         sudo -n networksetup -setwebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
149         sudo -n networksetup -setwebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
150         sudo -n networksetup -setsecurewebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
151         sudo -n networksetup -setsecurewebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
152         if command -v tailscale >/dev/null 2>&1; then
153             tailscale up >> \"\$PWD/output.log\" 2>&1 || true
154             tailscale set --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 || true
155             TS_DOWN_ARGS=\"\"
156             if [ \"\${KILL_SWITCH}\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
157             tailscale down \$TS_DOWN_ARGS >> \"\$PWD/output.log\" 2>&1 &
158         fi
159         sleep 1
160         echo \"[Shutdown Trap] Cleanup complete.\" >> \"\$PWD/output.log\" 2>&1
161         exit 0
162     ' SIGTERM SIGINT SIGHUP
163     caffeinate -i &
164     wait \$!
165     \" >> output.log 2>&1 &
166     local caffe_pid=$!
167     echo \"$caffe_pid\" > \"$PID_FILE"
168     echo " Sleep prevention active (PID $caffe_pid). Your Mac won't sleep while the gateway is running."
169 }
170
171
172 DNS_WATCHER_PID_FILE="$PID_DNS_WATCHER"
173 WARP_WATCHER_PID_FILE="/tmp/nullexit-warp-watcher.pid"
174
175 start_dns_watcher() {
176     local target_ip=$1
177     stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
178     echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
179     nohup bash -c "
180     source \"\$SCRIPT_DIR/scripts/common.sh\"
181     trap 'exit 0' SIGTERM SIGINT SIGHUP
182     while true; do
183         ACTIVE_IF=$(get_active_service)
184         if [ -n \"\$ACTIVE_IF\" ]; then
185             CURRENT_DNS=$(networksetup -getdnsservers \"\$ACTIVE_IF\" >/dev/null)
186             if [ \"\$CURRENT_DNS\" != \"\$target_ip\" ]; then
187                 networksetup -setdnsservers \"\$ACTIVE_IF\" \"\$target_ip\" >/dev/null 2>&1

```

```

188     fi
189     fi
190     sleep 30
191 done
192 " >> output.log 2>&1 &
193 echo $! > "$DNS_WATCHER_PID_FILE"
194 }
195
196
197
198 # Background WARP tunnel liveness monitor. Polls cdn-cgi/trace every 30s
199 # while healthy, but accelerates to polling every 5s if a failure is detected.
200 # Always logs state transitions to output.log. When warp=off persists for
201 # WARP_FAIL_THRESHOLD consecutive polls (default 6 = 30s of downtime), the watcher
202 # triggers a nuclear recovery: runs recover.sh to tear down the entire
203 # gateway disconnecting Tailscale, stopping containers, resetting DNS,
204 # and power-cycling Wi-Fi. Note: This minimizes exposure window, but
205 # only the pf kill switch guarantees absolutely zero leakage on drop.
206 #
207 # The threshold (default 6) is statistically chosen: even at an
208 # unrealistically high 10% false-positive rate per poll, P(6 consecutive
209 # false positives) = 0.1^6 0.0001%. Real-world false-positive rates
210 # are far lower, making 30s of continuous downtime overwhelmingly likely
211 # to be a genuine outage.
212 #
213 # Configurable via .env: WARP_FAIL_THRESHOLD=3 (15s, more aggressive)
214 start_warp_watcher() {
215     stop_warp_watcher
216     # Parse threshold from .env (same pattern as GATEWAY_MSS, GATEWAY_BYPASS_PING, etc.)
217     local threshold
218     threshold=$(read_env_var WARP_FAIL_THRESHOLD)
219     if [ -z "$threshold" ]; then
220         threshold="{WARP_FAIL_THRESHOLD:-6}"
221     fi
222     local trigger_file="/tmp/nullexit-warp-shutdown-triggered"
223     rm -f "$trigger_file"
224     echo " Starting background WARP Watcher (polling every 30s [accelerating to 5s on failure], shutdown after $
225         {threshold} consecutive failures)..."
226     nohup bash -c "
227         trap 'exit 0' SIGTERM SIGINT SIGHUP
228         last_state='on'
229         consec_off=0
230         threshold='$threshold'
231         trigger_file='$trigger_file'
232         out_log='$PWD/output.log'
233         recover_bin='$PWD/recover.sh'
234         inhibit_file='/tmp/nullexit-warp-inhibit.marker'
235         while true; do
236             # Inhibit check
237             # recover.sh --post-wake writes this marker while force-recreating the
238             # warp container. During that window, the container is intentionally
239             # down don't count it as a failure or we'll fire nuclear recover.sh
240             # and kill a gateway that was in the process of healing itself.
241             if [ -f "\${inhibit_file}" ]; then
242                 consec_off=0
243                 sleep 5
244                 continue
245             fi
246             state='off'
247             if docker compose exec -T warp wget -qO- --timeout=5 \
248                 https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
249                 | grep -q 'warp=on'; then
250                 state='on'
251             fi
252             # State-transition logging
253             if [ "\${state}" != "\${last_state}" ]; then
254                 if [ "\${state}" = 'off' ]; then
255                     echo "[\$(date -u +%FT%TZ)] WARP DOWN cdn-cgi/trace reports warp=off" >> "\${out_log}"
256                 fi
257                 last_state="\${state}"
258             fi
259             # Consecutive-failure tracking + auto-shutdown
260             if [ "\${state}" = 'off' ]; then
261                 consec_off=$((consec_off + 1))
262                 if [ "\${consec_off}" -ge "\${threshold}" ] && [ ! -f "\${trigger_file}" ]; then
263                     touch "\${trigger_file}"
264                     echo "[\$(date -u +%FT%TZ)] WARP SHUTDOWN \${consec_off} consecutive failures (threshold=\${threshold})
265                     . Running recover.sh to kill the gateway and restore normal internet. Your IP is NO LONGER Cloudflare.\" >>

```

```

267     \\\$out_log\
268     # Nuclear recovery: tear down the entire gateway.
269     # recover.sh resets DNS 1.1.1.1, disconnects Tailscale,
270     # stops Docker containers, flushes routes, power-cycles Wi-Fi.
271     bash \\\$recover_bin\ --auto >> \\\$out_log\ 2>&1 || true
272     echo \\\$(date -u +%FT%TZ)] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is now
your real ISP IP.\ >> \\\$out_log\
273     exit 0
274 fi
275 else
276     if [ \\\$consec_off\ -gt 0 ]; then
277         echo \\\$(date -u +%FT%TZ)] WARP RECOVERED warp=on after \\\$consec_off consecutive off readings (
threshold was \\\$threshold)\ >> \\\$out_log\
278         fi
279         consec_off=0
280     fi
281     if [ \\\$state\ = \off\ ]; then
282         sleep 5
283     else
284         sleep 30
285     fi
286 done
287 " >> output.log 2>&1 &
288 echo $! > "$WARP_WATCHER_PID_FILE"
289 }
290
291 stop_warp_watcher() {
292     if [ -f "$WARP_WATCHER_PID_FILE" ]; then
293         local wp
294         wp=$(cat "$WARP_WATCHER_PID_FILE")
295         if [ -n "$wp" ] && kill -0 "$wp" 2>/dev/null; then
296             echo " Stopping background WARP Watcher (PID $wp)..."
297             kill "$wp" 2>/dev/null || true
298         fi
299         rm -f "$WARP_WATCHER_PID_FILE"
300     fi
301 }
302
303
304
305
306
307 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
308 cleanup_handler() {
309     local exit_code=$?
310     local trigger_type="$1"
311     local line_no="$2"
312
313     if [ "$SUCCESS_RUN" = "false" ]; then
314         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
315         if [ -n "$ACTIVE_SERVICE" ]; then
316             reset_dns
317         fi
318
319         if [ -n "$CURRENT_BG_PID" ]; then
320             echo "Terminating active command (PID $CURRENT_BG_PID)..."
321             kill -15 "$CURRENT_BG_PID" 2>> output.log || true
322         fi
323
324         # Disconnect host Tailscale and close the GUI application on failure
325         disconnect_tailscale_host
326
327         # Stop local DNS proxy if running
328         stop_local_dns_proxy
329
330         # Stop sleep prevention
331         stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
332         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
333         stop_warp_watcher
334         clear_gateway_active_marker
335
336         # Capture warp logs on failure for debugging before teardown
337         if [ "$trigger_type" = "ERR" ]; then
338             echo -e "\n--- WARP FAILURE LOGS ---" >> output.log
339             docker compose logs warp --tail=100 >> output.log 2>&1 || true
340             echo "-----" >> output.log
341         fi
342
343         # Best-effort container teardown on failure

```



```

344     docker compose down --remove-orphans -t 30 2>> output.log || true
345
346     # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
347     if [ -n "$ACTIVE_SERVICE" ]; then
348         cleanup_network_state
349     fi
350
351
352     if [ "$trigger_type" = "ERR" ]; then
353         echo -e "\n=====
354         echo "ERROR: Script failed at line $line_no with exit code $exit_code."
355         echo "=====
356         echo "Troubleshooting steps:"
357         echo "1. Run 'colima status' to verify the VM is active."
358         echo "2. Run 'docker compose ps' to check container health."
359         echo "3. Run 'docker compose logs' to view application logs."
360         echo "4. Run 'tailscale status' to check host Tailscale status."
361         echo "=====
362     fi
363     rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
364 fi
365 }
366
367 trap 'cleanup_handler ERR $LINENO' ERR
368 trap 'cleanup_handler INT' INT
369 trap 'cleanup_handler TERM' TERM
370 trap 'cleanup_handler HUP' HUP
371
372 # NOPASSWD sudo
373 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
374 # needed (networksetup, dnsmasq, dscacheutil, killall, pkill, socat).
375 # All privileged calls below use `sudo -n` which will run without prompting.
376 # No credential caching loop is needed.
377
378 # Add Homebrew and standard paths
379 setup_standard_path
380
381 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
382 # Uses a Python DNS proxy (handles TCP wire format correctly).
383 # Python3 is built into macOS no external dependencies.
384 DNS_PROXY_BIN=""
385 if command -v python3 >> output.log 2>&1; then
386     DNS_PROXY_BIN="python3"
387 elif command -v python >> output.log 2>&1; then
388     DNS_PROXY_BIN="python"
389 fi
390
391 # Path to the DNS proxy script (sibling of this script)
392 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/dns-proxy.py"
393
394 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
395 # (`brew install tailscale` + `brew services start tailscale`) NOT the
396 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
397 # to launch, no menu-bar click to perform, and no scutil Network Service to
398 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
399 # if you ran the install with sudo) and the control socket is always available.
400 TS_BIN=""
401 if command -v tailscale >> output.log 2>&1; then
402     TS_BIN="tailscale"
403 fi
404
405 # Baseline: disable Tailscale DNS management at the daemon level.
406 # `tailscale set` writes a persistent preference. However, `tailscale up --reset`
407 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
408 # re-enable `--accept-dns=true` and nullify this `set`.
409 # Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
410 # `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
411 #
412 # This initial `set` is still useful as a baseline for non-reset operations
413 # (e.g. if the user runs `tailscale up` manually without --reset) and covers
414 # the brief window between this line and the first `--reset` call.
415 #
416 # Runs once at script start while tailscaled is still connected (if it was
417 # left running from a previous toggle), so the pref takes effect before any
418 # disconnection or reconnection logic.
419 if [ -n "$TS_BIN" ]; then
420     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
421 fi
422
423 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
424 # *entire* teardown no scutil tunnel stop, no Tailscale.app GUI quit, no pkill.

```

```

425 # tailscaled keeps running (as a LaunchAgent / LaunchDaemon), which is intentional:
426 # the next `tailscale up` then reconnects in seconds rather than waiting for a
427 # slow .app launch + Network Extension activation. The `tailscale down` call also
428 # retains the user's auth state, so we don't force a browser re-login every cycle.
429 #
430 # Defined early (before the if/else branches and referenced by cleanup_handler's
431 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
432
433
434 # Wait for Network Readiness
435 # On --restart, the gateway tears down while the host network interface may
436 # still be reconnecting (Wi-Fi, Ethernet, USB tethering, etc.). If we proceed
437 # too early, get_active_service returns nothing, routing targets the wrong
438 # interface, and DNS/WARP setup silently fails.
439 #
440 # We use `route get default` to detect a valid default gateway rather than
441 # checking a specific interface (e.g. en0) this generalizes across all
442 # connection types: Wi-Fi, Ethernet, USB-C adapters, iPhone tethering, etc.
443 # The moment the OS has any routable default gateway, we proceed.
444 _net_wait_secs=0
445 _net_timeout=60
446 echo "Waiting for network connectivity before starting..."
447 while true; do
448     if route get default 2>/dev/null | grep -q 'gateway: '; then
449         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
450         echo "Network ready (default gateway: $_gw)."
451         break
452     fi
453     if [ "$_net_wait_secs" -ge "$_net_timeout" ]; then
454         echo " [Warning] No default gateway after ${_net_timeout}s proceeding anyway."
455         break
456     fi
457     sleep 2
458     _net_wait_secs=$(($_net_wait_secs + 2))
459 done
460
461 ACTIVE_SERVICE=$(get_active_service)
462
463 # Resolve the service name for en0 (usually "Wi-Fi") macOS scutil DNS resolver
464 # is commonly scoped to en0, so per-service DNS changes on other interfaces are
465 # ignored unless en0's service is also updated.
466 ENO_SERVICE=$(get_en0_service)
467
468 # Helper to add host-side bypass routes for the WARP WireGuard endpoints.
469 # This prevents an infinite recursive routing loop (tunnel loop) where
470 # the container's WireGuard packets exit the VM, reach the host, match the
471 # default route (the exit node), and get routed back into the tunnel.
472 # We route via the gateway IP (not interface) to ensure packets are routed
473 # properly even when the host's default route changes to the Tailscale interface.
474
475 # Helper to manually override the host's default route to point through the
476 # Tailscale utun* interface. Standalone tailscaled on macOS (Homebrew version)
477 # lacks the platform-specific integration to override the default gateway
478 # automatically when --exit-node is used.
479 setup_exit_node_routing() {
480     local ts_iface
481     ts_iface=$(ifconfig | awk '/^[a-z0-9]+:/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':')
482
483     if [ -n "$ts_iface" ]; then
484         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
485         local host_mtu
486         host_mtu=$(read_env_var HOST_MTU)
487         host_mtu=${host_mtu:-1200}
488
489         # Lower the host Tailscale MTU (defaults to 1200) to prevent fragmentation when passing
490         # through the container's WARP tunnel (which also has an MTU of 1280).
491         sudo -n ifconfig "$ts_iface" mtu "$host_mtu" >> output.log 2>&1 || true
492
493         # DO NOT delete the physical default route! It crashes Colima.
494         # Use the /1 trick to mathematically override the default route.
495         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
496         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
497         sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
498         sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
499
500         if netstat -nr | grep -E -q "^(0\.\0\.\0/1|0/1)[[:space:]]+.*$ts_iface"; then
501             echo " [ ] Default route successfully overridden via $ts_iface."
502         else
503             echo " [!] Failed to verify exit node routing on $ts_iface."
504         fi
505     else

```

```

506     echo "[Warning] Could not detect Tailscale utun interface for host routing."
507 fi
508 }
509
510 # Helper to restore host DNS to a clean state (used on script start, on failure,
511 # and after a successful STOP). Without this, a successful toggle-off leaves the
512 # host's resolver pinned to a now-dead gateway IP every lookup stalls for macOS's
513 # DNS timeout (~5s) before falling through to whatever next server is configured.
514 # With it, we always leave the host in `1.1.1.1 / no search domain`.
515 #
516 # We set DNS on BOTH the active service AND the en0 service because macOS's scutil DNS
517 # resolver is commonly scoped to en0 (Wi-Fi). A per-service change on e.g.
518 # "USB 10/100 LAN" is ignored by the system resolver unless the en0 service is also updated.
519 reset_dns() {
520     if [ -n "$ACTIVE_SERVICE" ]; then
521         for dns_svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
522             networksetup -setsearchdomains "$dns_svc" "Empty" 2>> output.log || true
523             networksetup -setdnsservers "$dns_svc" 1.1.1.1 2>> output.log || true
524         done
525     fi
526 }
527
528 # Helper to nuke leftover network state after teardown (proxy settings, routing
529 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
530 # problem where stale state kills internet after the gateway stops.
531 cleanup_network_state() {
532     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
533
534     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
535     for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
536         disable_all_proxies "$svc"
537     done
538     echo " Proxies disabled."
539
540     # 2. Flush DNS cache
541     if command -v dscacheutil >> output.log 2>&1; then
542         sudo -n dscacheutil -flushcache 2>> output.log || true
543     fi
544     sudo -n killall -HUP mDNSResponder 2>> output.log || true
545     echo " DNS cache flushed."
546
547     # 3. Flush stale routing table entries
548     if command -v route >> output.log 2>&1; then
549         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
550         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
551     fi
552     remove_warp_bypass_routes
553     disable_killswitch
554     echo " Routing table and firewall flushed."
555
556     # 4. Power-cycle Wi-Fi to clear any lingering interface state
557     WIFI_PORT=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2
558 }')
559     if [ -n "$WIFI_PORT" ]; then
560         sudo -n networksetup -setairportpower "$WIFI_PORT" off 2>> output.log || true
561         sleep 2
562         sudo -n networksetup -setairportpower "$WIFI_PORT" on 2>> output.log || true
563         echo " Wi-Fi power-cycled (interface $WIFI_PORT)."
564         sleep 3
565     else
566         echo " Wi-Fi interface not detected; bouncing en0..."
567         sudo -n ifconfig en0 down 2>> output.log || true
568         sudo -n ifconfig en0 up 2>> output.log || true
569     fi
570
571     # Force restart sharing services to prevent AirDrop / AirPlay discovery freezes
572     # after network interface/DNS changes.
573     echo " Resetting macOS sharing services (AirDrop/AirPlay)..."
574     reset_sharing_services
575
576     echo "Network state cleanup complete."
577 }
578
579 # SOCKS5 Proxy (WARP Tunnel)
580 # When Tailscale's exit node can't route through WARP (due to userspace
581 # forwarding), we use a SOCKS5 proxy running inside the warp container's
582 # network namespace. Connections created by this proxy go through the
583 # kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale's
584 # userspace exit node which bypasses it.
585 #
586 # The SOCKS5 proxy is exposed on localhost:1080 via Docker port mapping.

```

```

586 # `networksetup -setsocksfirewallproxy` on macOS routes system TCP traffic
587 # through the proxy, which then goes through WARP.
588 #
589 # IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
590 # They must use --socks5-hostname or the ALL_PROXY env variable explicitly.
591
592 SOCKS_PROXY_PORT=1080
593
594 enable_socks_proxy() {
595     local svc="$1"
596     if [ -z "$svc" ]; then
597         svc="$ACTIVE_SERVICE"
598     fi
599
600     echo -n " Enabling system-wide SOCKS5 proxy on $svc (localhost:$SOCKS_PROXY_PORT)... "
601     sudo -n networksetup -setsocksfirewallproxy "$svc" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log || true
602     sudo -n networksetup -setsocksfirewallproxystate "$svc" on 2>> output.log || true
603
604     # Also set on en0 service for macOS resolver consistency
605     if [ "$svc" != "$ENO_SERVICE" ]; then
606         sudo -n networksetup -setsocksfirewallproxy "$ENO_SERVICE" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log ||
607             true
608         sudo -n networksetup -setsocksfirewallproxystate "$ENO_SERVICE" on 2>> output.log || true
609     fi
610
611     # IMPORTANT: Exclude local LAN and mDNS traffic from the proxy so P2P works natively
612     # without source-IP masking from the Colima VM bridge.
613     sudo -n networksetup -setproxybypassdomains "$svc" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
614     172.16.0.0/12 "*.local" 2>> output.log || true
615     if [ "$svc" != "$ENO_SERVICE" ]; then
616         sudo -n networksetup -setproxybypassdomains "$ENO_SERVICE" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
617         172.16.0.0/12 "*.local" 2>> output.log || true
618     fi
619
620     echo "done."
621     echo " All TCP traffic now routed through gateway -> WARP tunnel -> internet."
622     echo " (CLI tools: export ALL_PROXY=socks5://127.0.0.1:$SOCKS_PROXY_PORT)"
623 }
624
625 disable_socks_proxy() {
626     for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
627         sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
628     done
629     echo " SOCKS5 proxy disabled."
630 }
631
632 # Local DNS Proxy (Python)
633 # When the tailnet data plane cannot establish, we use a Python DNS proxy.
634 # It listens on UDP:53, receives DNS queries, forwards them over TCP to
635 # AdGuard at 127.0.0.1:5354 (Docker port mapping), and returns the response.
636 # The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
637 # correctly unlike socat which just forwards raw bytes.
638 #
639 # Python3 is built into macOS no external dependencies.
640 DNS_PROXY_PID=""
641
642 # Kill any leftover DNS proxy process
643 stop_local_dns_proxy() {
644     if [ -n "$DNS_PROXY_PID" ]; then
645         kill "$DNS_PROXY_PID" 2>/dev/null || true
646         wait "$DNS_PROXY_PID" 2>/dev/null || true
647         DNS_PROXY_PID=""
648     fi
649
650     # Clean up any stale Python DNS proxy processes
651     sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
652 }
653
654 # Start local DNS proxy (Python)
655 start_local_dns_proxy() {
656     if [ -z "$DNS_PROXY_BIN" ]; then
657         echo " Python not found. Python3 is built into macOS this shouldn't happen."
658         return 1
659     fi
660
661     if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
662         echo " DNS proxy script not found at $DNS_PROXY_SCRIPT"
663         return 1
664     fi
665
666     # Kill any leftover process first
667     stop_local_dns_proxy

```

```

664
665 echo -n " Starting local DNS proxy via Python (UDP:53 TCP:5354)... "
666 sudo -n "$DNS_PROXY_BIN" "$DNS_PROXY_SCRIPT" &
667 DNS_PROXY_PID=$!
668 disown "$DNS_PROXY_PID" 2>/dev/null || true
669
670 # Give it a moment to bind
671 sleep 0.5
672
673 if kill -0 "$DNS_PROXY_PID" 2>/dev/null; then
674     echo "started (PID $DNS_PROXY_PID)."
675
676     # Hijack host DNS to localhost
677     echo -n " Hijacking host DNS to 127.0.0.1 for ad-blocking... "
678     networksetup -setsearchdomains "$ACTIVE_SERVICE" "Empty" 2>/dev/null || true
679     networksetup -setsearchdomains "$ENO_SERVICE" "Empty" 2>/dev/null || true
680     if ! networksetup -setdnsservers "$ACTIVE_SERVICE" 127.0.0.1; then
681         echo "FAILED (networksetup error check permissions)."
682         stop_local_dns_proxy
683         return 1
684     fi
685     networksetup -setdnsservers "$ENO_SERVICE" 127.0.0.1 2>/dev/null || true
686     sudo -n dscacheutil -flushcache 2>/dev/null || true
687     sudo -n killall -HUP mDNSResponder 2>/dev/null || true
688
689     # Verify DNS actually works through the proxy
690     echo -n " Verifying DNS resolution... "
691     if dig @127.0.0.1 google.com +short +timeout=5 &>/dev/null; then
692         echo "ok."
693         return 0
694     else
695         echo "FAILED (DNS queries not reaching AdGuard)."
696         echo " Restoring DNS to 1.1.1.1..."
697         reset_dns
698         stop_local_dns_proxy
699         return 1
700     fi
701 else
702     echo "FAILED (could not bind port 53 is it in use?)."
703     DNS_PROXY_PID=""
704     return 1
705 fi
706 }
707
708 # Helper to FORCIBLY hijack host DNS to a single server (the gateway's AdGuard IP)
709 # and VERIFY via `networksetup -getdnsservers` that the only name server is exactly
710 # $1. Returns 0 iff the live state matches; non-zero if it can't be confirmed.
711 #
712 # This deliberately does NOT append a 1.1.1.1 fallback: macOS's resolver queries
713 # the list in order and falls back to the next entry on timeout, so on a
714 # misbehaving AdGuard (filter compile crash, OOM kill, etc.) the resolver still
715 # leaks to 1.1.1.1 and silently bypasses ad-blocking. With no fallback, a broken
716 # gateway manifests as a *visible* DNS outage that prompts the user to
717 # investigate which is the correct failure mode.
718 force_dns_to_gateway() {
719     local target_ip="$1"
720     if [ -z "$target_ip" ] || [ -z "$ACTIVE_SERVICE" ]; then
721         echo " [force_dns] ERROR: target_ip or ACTIVE_SERVICE is empty" >&2
722         return 1
723     fi
724
725     local attempt
726     for attempt in 1 2 3; do
727         echo -n " [force_dns] Attempt $attempt/3: setting DNS to $target_ip on \"${ACTIVE_SERVICE}\" + \"
728             $ENO_SERVICE\"... "
729
730         # Apply to BOTH the active service AND the en0 service the scutil DNS resolver
731         # is scoped to en0 (Wi-Fi) and ignores per-service changes that don't include it.
732         # Fail fast on ACTIVE_SERVICE; best-effort on ENO_SERVICE (it may not exist).
733         networksetup -setsearchdomains "$ACTIVE_SERVICE" "ts.net" || true
734         if ! networksetup -setdnsservers "$ACTIVE_SERVICE" "$target_ip"; then
735             echo "FAILED (networksetup error check permissions)"
736             sleep 1
737             continue
738         fi
739         networksetup -setsearchdomains "$ENO_SERVICE" "ts.net" || true
740         networksetup -setdnsservers "$ENO_SERVICE" "$target_ip" || true
741
742         local entries
743         entries=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log \

```

```

743 | awk '/^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)
744 \.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/ {print}'
745 if echo "$entries" | grep -qFx "$target_ip"; then
746     echo "VERIFIED"
747     return 0
748 fi
749 local current
750 current=$(echo "$entries" | tr '\n' ' ' | sed 's/ $//')
751 echo "NOT YET (current DNS: [${current:-none}])"
752
753 if [ "$attempt" = "3" ]; then sleep 2; else sleep 1; fi
754 done
755
756 echo " [force_dns] FAILED after 3 attempts"
757 return 1
758 }
759
760 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
761 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
762 # `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
763 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
764
765
766 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
767 reset_dns
768
769
770
771
772 echo "Checking Gateway Status..."
773 HIJACK_HOST=$(read_env_var GATEWAY_HIJACK_HOST | tr '[:upper:]' '[:lower:]')
774
775 if is_gateway_active; then
776     echo -e "\n=====
777     echo "Gateway is RUNNING. Stopping it now..."
778     echo -e "=====
779
780 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
781 if [[ "$HIJACK_HOST" != "false" ]]; then
782     disconnect_tailscale_host
783     echo ""
784 fi
785
786 echo "Stopping Docker containers..."
787 docker compose down --remove-orphans -t 30
788
789 # Only stop Colima if the user explicitly opted in (false by default) to prevent breaking their other Docker
790 # dev projects
791 STOP_COLIMA=$(read_env_var STOP_COLIMA_ON_EXIT | tr '[:upper:]' '[:lower:]')
792 if [[ "$STOP_COLIMA" == "true" ]]; then
793     echo -e "\nStopping Colima VM to free up host RAM and battery..."
794     run_with_timeout 30 colima stop >> output.log 2>&1 || echo "Warning: Failed to stop Colima gracefully."
795 else
796     echo -e "\nLeaving Colima running. (Set STOP_COLIMA_ON_EXIT=true in .env to change this)"
797 fi
798
799 if [[ "$HIJACK_HOST" != "false" ]]; then
800     # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
801     # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
802     # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
803     echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)..."
804     reset_dns
805
806 # 3. Stop local DNS proxy if running
807 stop_local_dns_proxy
808
809 # Stop background daemons
810 stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
811 stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
812 stop_warp_watcher
813 clear_gateway_active_marker
814
815 # 4. Nuke leftover network state so internet actually works after teardown
816 cleanup_network_state
817 fi
818
819 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
820 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
821 else
822     START_GATEWAY=true

```



```

822 echo "[$(date '+%Y-%m-%d %H:%M:%S')] Action: STARTUP GATEWAY" >> "$LOG_FILE"
823 echo -e "\n=====
824 echo "Gateway is STOPPED. Starting it now..."
825 echo -e "=====
826
827 # 1. Prevent host exit-node deadlock during VM / Container startup
828 if [ "$HIJACK_HOST" != "false" ]; then
829     disconnect_tailscale_host
830 fi
831
832 # 2. Wait for physical DHCP lease to settle (crucial for restarts/wake-up/roaming)
833 wait_for_dhcp_settle
834
835 # 3. Boot Colima VM if it is not already running
836 echo -e "\nChecking Colima VM status..."
837 if ! run_with_timeout 15 colima status >> output.log 2>&1; then
838     echo "Colima is not running. Starting Colima (600MB RAM allocation, vz VM, network address, bridged)..."
839     run_with_timeout 120 colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged
840 else
841     echo "Colima is already running."
842 fi
843
844 # 3b. Auto-detect and fix Docker Subnet Collisions (e.g. if the user travels to a 172.17.x.x Wi-Fi)
845 if [ -f "scripts/fix-docker-bridge-collision.sh" ]; then
846     if bash scripts/fix-docker-bridge-collision.sh --dry | grep -q "collision detected"; then
847         echo -e "\n[!] WARNING: Docker Subnet Collision Detected with your current Wi-Fi!"
848         echo -e "    Auto-fixing Colima config to prevent internet jitter..."
849         bash scripts/fix-docker-bridge-collision.sh --skip-toggle || true
850     fi
851 fi
852
853 # 3c. Configure swap file inside the VM to prevent OOM on low-memory limits
854 # We also set vm.swappiness=10 so the Linux kernel strictly prefers physical RAM
855 # and avoids unnecessarily wearing out the SSD with proactive background swapping.
856 if ! run_with_timeout 15 colima ssh -- grep -q 'swapfile' /proc/swaps >> output.log 2>&1; then
857     echo "Configuring 400MB swap file inside the VM to prevent OOM..."
858     run_with_timeout 30 colima ssh -- sudo sh -c "if [ ! -f /swapfile ]; then dd if=/dev/zero of=/swapfile bs=1
M count=400 status=none && mkswap /swapfile; fi && swapon /swapfile && sysctl vm.swappiness=10" >> output.
log 2>&1 || echo "Warning: Failed to enable swap file inside the VM."
859 fi
860
861 # 4. Clean up corrupted AdGuardHome configurations
862 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
863     rm -f "adguard/conf/AdGuardHome.yaml"
864     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
865 fi
866
867 # 4b. Auto-heal stale Docker socket from hard reboots
868 if ! run_with_timeout 15 docker ps >/dev/null 2>&1; then
869     echo "Docker daemon is unresponsive (likely a stale socket from a crash). Auto-healing Colima..."
870     run_with_timeout 120 colima restart >> output.log 2>&1
871 fi
872
873 # 4c. Inject TCP MSS clamp from .env into post-rules.txt before starting
874 if [ -f .env ] && grep -q "^GATEWAY_MSS=" .env; then
875     GATEWAY_MSS=$(read_env_var GATEWAY_MSS)
876     if [ "$GATEWAY_MSS" =~ ^[0-9]+$ ]; then
877         # macOS sed syntax
878         sed -i '' "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null || \
879         # Linux sed fallback
880         sed -i "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null
881     fi
882 fi
883
884 # 5. Start compose services
885 write_host_ips
886 echo -e "\nStarting Docker containers..."
887 docker compose up -d --build
888
889 # Log the output of the rule compiler for debugging before removing it
890 docker compose logs rule-compiler >> output.log 2>&1
891 # Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
892 docker compose rm -s -f rule-compiler >> output.log 2>&1
893
894 RULE_COUNT=$(grep "! Total Block Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
895 NATIVE_COUNT=$(grep "! Native AdGuard Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
896
897 if [ "$RULE_COUNT" != "0" ] && [ "$RULE_COUNT" != "" ]; then
898     if [ "$NATIVE_COUNT" != "0" ] && [ "$NATIVE_COUNT" != "" ]; then

```

```

899     TOTAL_COUNT=$((RULE_COUNT + NATIVE_COUNT))
900     # Format with commas for readability (e.g. 441,578)
901     if command -v printf &> /dev/null; then
902         FORMATTED_RULE=$(printf "%'d" "$RULE_COUNT")
903         FORMATTED_TOTAL=$(printf "%'d" "$TOTAL_COUNT")
904         echo "   DNS: Compiled $FORMATTED_RULE unique custom rules (Total active in AdGuard: ~$FORMATTED_TOTAL
rules)."
905     else
906         echo "   DNS: Compiled $RULE_COUNT unique custom rules (Total active in AdGuard: ~$TOTAL_COUNT rules)."
907     fi
908 else
909     echo "   DNS: Compiled and loaded $RULE_COUNT optimized rules."
910 fi
911 fi
912
913 # Report IP blocklist compilation results
914 IP_COUNT=$(grep "^# Entries:" adguard/work/userfilters/ip_blocklist.ipset 2>/dev/null | awk -F': ' '{print $2
}' || echo "0")
915 if [ "$IP_COUNT" != "0" ] && [ "$IP_COUNT" != "" ]; then
916     if command -v printf &> /dev/null; then
917         FORMATTED_IP=$(printf "%'d" "$IP_COUNT")
918         echo "   IP:  Loaded $FORMATTED_IP threat intelligence IPs/CIDRs into kernel firewall."
919     else
920         echo "   IP:  Loaded $IP_COUNT threat intelligence IPs/CIDRs into kernel firewall."
921     fi
922 fi
923
924 # 6. Wait for the gateway container's Tailscale connection to be ready
925 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
926 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
927
928 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
929 echo "   (This can take 30-60s  Tailscale goes through: NoState Starting Running Online)"
930 connected=false
931 consecutive_failures=0
932 for i in {1..60}; do
933     # Run tailscale status and capture its output so the user can see what's happening.
934     # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
935     ts_output=$(run_with_timeout 5 docker compose exec -T tailscale tailscale status 2>&1 || true)
936
937     if echo "$ts_output" | grep -q "offers exit node"; then
938         connected=true
939         break
940     fi
941
942     # Track consecutive failures to detect a stuck state.
943     # If we see 40+ consecutive 'NoState' responses, give up early
944     # the daemon is alive but can't connect to the coordination server.
945     if echo "$ts_output" | grep -q "NoState"; then
946         consecutive_failures=$((consecutive_failures + 1))
947         if [ "$consecutive_failures" -ge 40 ]; then
948             echo ""
949             echo "   [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
950             echo "   Tailscale daemon is running but can't reach the coordination server."
951             break
952         fi
953     else
954         consecutive_failures=0
955     fi
956
957     # Print a condensed status every 10 seconds so the user can see progress
958     if [ $((i % 10)) -eq 0 ]; then
959         ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
960         echo "   [attempt $i/60] $ts_short"
961         elif [ $((i % 5)) -eq 0 ]; then
962             echo -n "$[i]"
963         else
964             echo -n "."
965         fi
966     sleep 1
967 done
968 echo ""
969
970 if [ "$connected" = "true" ]; then
971     echo "Gateway container is online on the Tailscale mesh."
972 elif [ "$BYPASS_PING" = "true" ]; then
973     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
974 else
975     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
976     echo "   The container was seen in the following states during the wait:" >&2
977     echo "$ts_output" | sed 's/~/ /' >&2

```

```

978     echo "" >&2
979     echo " This could mean:" >&2
980     echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
981     echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
982     echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
983     echo "" >&2
984     echo " Diagnose manually:" >&2
985     echo " docker compose exec -T tailscale tailscale status" >&2
986     echo " docker compose exec -T tailscale tailscale netcheck" >&2
987 cleanup_handler ERR $LINENO
988 exit 1
989 fi
990
991 # 6b. Resolve the gateway's Tailscale IP.
992 # Primary: static ADGUARD_IP.txt (fast, no docker-exec dependency).
993 # Fallback: dynamic query from the container (handles IP changes after re-auth).
994 #
995 # CRITICAL: `docker compose exec -T` on macOS/Colima injects carriage
996 # returns (\r) into captured output. If $TS_IP ends with \r, then
997 # `networksetup -setdnsservers` silently rejects the malformed IP and
998 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
999 # `tr -d '\r'` strips the CR; `awk 'NR==1{print $1; exit}'` takes only
1000 # the first field of the first line (preserving the old `head -1` guard).
1001 TS_IP=""
1002 TS_IP=$(read_adguard_ip || true)
1003 if [ -n "$TS_IP" ]; then
1004     echo "Using static IP from ADGUARD_IP.txt: $TS_IP"
1005 elif [ "$connected" = "true" ]; then
1006     TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' |
1007         awk 'NR==1{print $1; exit}' || true)
1008     if [ -n "$TS_IP" ]; then
1009         echo "[Fallback] Resolved gateway Tailscale IP dynamically: $TS_IP"
1010     fi
1011 fi
1012
1013 # 7. Connect host Mac to Tailscale mesh.
1014 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
1015 # system LaunchDaemon via 'brew services start tailscale'), the previous five-
1016 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
1017 # no menu bar item to click, and no Accessibility permission required.
1018 #
1019 # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
1020 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
1021 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
1022 # are impossible without the host on the mesh).
1023 #
1024 # We connect in two phases:
1025 #   Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
1026 #   Phase B Set exit node only after pre-flight checks confirm the gateway works
1027 HOST_ON_MESH=false
1028 SKIP_EXIT_NODE=false
1029 if [ "$HIJACK_HOST" = "false" ]; then
1030     echo -e "\n[Info] HIJACK_HOST is false (Headless Mode). Host networking will remain untouched."
1031     SKIP_EXIT_NODE=true
1032 elif [ -n "$TS_BIN" ]; then
1033     echo -n "Verifying tailscaled is reachable"
1034     daemon_ready=false
1035     status_exit=0
1036     for i in {1..10}; do
1037         # Test if the daemon responds to status check.
1038         # If status returns non-zero, check if it was a normal "stopped/disconnected" state
1039         # (exit code 1 is normal for disconnected). If the command exited quickly (not killed by timeout),
1040         # and the error is just "Tailscale is stopped", then the daemon is alive and ready.
1041         # But if it timed out (exit code 143) or is completely unresponsive, it's wedged.
1042         status_exit=0
1043         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
1044             daemon_ready=true
1045             break
1046         else
1047             status_exit=$?
1048             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
1049                 daemon_ready=true
1050                 break
1051             fi
1052         fi
1053         echo -n "."
1054         sleep 1
1055     done
1056     echo ""
1057
1058     # If the daemon was unresponsive or socket check failed, attempt auto-restart

```

```

1058 if [ "$daemon_ready" != "true" ]; then
1059     if [ "$status_exit" -eq 143 ]; then
1060         warn "tailscaled daemon is wedged/unresponsive. Restarting it..."
1061     else
1062         warn "tailscaled daemon is not running. Attempting to start it..."
1063     fi
1064     restart_tailscaled_daemon
1065
1066     # Re-verify
1067     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
1068         daemon_ready=true
1069     fi
1070 fi
1071
1072 if [ "$daemon_ready" = "true" ]; then
1073     echo "tailscaled is responsive (running as a system service)."

```

```

1139     SKIP_EXIT_NODE=true
1140 fi
1141 elif command -v nslookup &>/dev/null; then
1142     if nslookup -port=5354 google.com 127.0.0.1 &>/dev/null; then
1143         echo "PASS"
1144     else
1145         echo "FAIL"
1146         SKIP_EXIT_NODE=true
1147     fi
1148 else
1149     echo "SKIP (dig/nslookup not available)"
1150 fi
1151
1152 # Check 3: Does the WARP container have external internet?
1153 echo -n " [3/3] WARP container internet... "
1154 tmp_err=$(mktemp)
1155 if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace >/dev/null 2>
1156 "$tmp_err"; then
1157     echo "PASS"
1158 else
1159     echo "FAIL"
1160     if [ -s "$tmp_err" ]; then
1161         err_msg=$(cat "$tmp_err" | tr -d '\r')
1162         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Check 3] WARP container check failed: $err_msg" >> output.log
1163     fi
1164     SKIP_EXIT_NODE=true
1165 fi
1166 rm -f "$tmp_err"
1167
1168 # Act based on check results
1169 if [ "$SKIP_EXIT_NODE" != "true" ]; then
1170     echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
1171     $TS_BIN up || true
1172     if $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node="$TS_IP" --exit-node-allow-
1173     lan-access=true; then
1174         echo "Exit node enabled."
1175         add_warp_bypass_routes
1176         setup_exit_node_routing
1177         enable_killswitch
1178     else
1179         echo "[Warning] Failed to set exit node."
1180         SKIP_EXIT_NODE=true
1181     fi
1182 fi
1183
1184 if [ "$SKIP_EXIT_NODE" = "true" ]; then
1185     warn "Pre-flight checks failed EXIT NODE + DNS HIJACK SKIPPED."
1186     echo " Host stays on Tailscale mesh but your internet routes through your normal connection."
1187     echo " Troubleshoot with:"
1188     echo "   docker compose logs warp | tail -20"
1189     echo "   docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
1190     echo " Falling back to SOCKS5 proxy for traffic routing..."
1191 fi
1192 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
1193     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
1194     SKIP_EXIT_NODE=true
1195 fi
1196
1197 # 8. Apply DNS Hijacking.
1198 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
1199 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
1200 # AdGuard is also available at localhost:5354 for manual configuration in apps.
1201 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
1202     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
1203     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
1204     if force_dns_to_gateway "$TS_IP"; then
1205         echo "DNS hijack verified: single server = $TS_IP."
1206     else
1207         echo "[Warning] DNS hijack could not be verified."
1208         echo " If DNS is broken, run manually:"
1209         echo "   networksetup -setdnsservers \"\$ACTIVE_SERVICE\" $TS_IP"
1210         echo "   networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" ts.net"
1211     fi
1212 elif [ -n "$TS_IP" ] && [ "$HIJACK_HOST" != "false" ]; then
1213     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
1214     echo " Trying local DNS proxy for ad-blocking..."
1215     if start_local_dns_proxy; then
1216         echo " Ad-blocking active via local DNS proxy through AdGuard."
1217     else
1218         echo " Local DNS proxy unavailable. DNS stays at 1.1.1.1."
1219     fi

```

```

1218     echo "  AdGuard available at http://localhost:80 (port 5354 for DNS via TCP)."
```

```

1219 fi
```

```

1220
```

```

1221 # Enable SOCKS5 proxy for traffic routing through WARP
```

```

1222 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
```

```

1223 echo -e "\n Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
```

```

1224 echo "  (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
```

```

1225 enable_socks_proxy "$ACTIVE_SERVICE"
```

```

1226
```

```

1227 # Verify the SOCKS5 proxy works through WARP
```

```

1228 echo -n "  Verifying SOCKS5 proxy routes through WARP... "
```

```

1229 if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/
```

```

1230     trace 2>/dev/null | grep -q "warp=on"; then
```

```

1231     echo "ok (warp=on)."
```

```

1232     echo "  All TCP traffic now encrypted through Cloudflare WARP tunnel."
```

```

1233 else
```

```

1234     echo "FAILED (proxy not routing through WARP)."
```

```

1235     echo "  Disabling SOCKS5 proxy internet stays on direct connection."
```

```

1236     disable_socks_proxy
```

```

1237     echo "  SOCKS proxy available at localhost:$SOCKS_PROXY_PORT for manual configuration."
```

```

1238 fi
```

```

1239 echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
```

```

1240 fi
```

```

1241
```

```

1242 # 9. Verify host connectivity
```

```

1243 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
```

```

1244     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
```

```

1245         echo "Host is online on the Tailscale mesh."
```

```

1246     else
```

```

1247         echo "[Warning] Host is not responding on Tailscale."
```

```

1248     fi
```

```

1249 fi
```

```

1250
```

```

1251 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
```

```

1252 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
```

```

1253
```

```

1254 # Prevent system from going to sleep while gateway is running
```

```

1255 start_sleep_prevention
```

```

1256
```

```

1257 if [[ "$HIJACK_HOST" != "false" ]]; then
```

```

1258     if [ -n "$TS_IP" ] && [ "$TS_IP" != "1.1.1.1" ]; then
```

```

1259         start_dns_watcher "$TS_IP"
```

```

1260     fi
```

```

1261
```

```

1262     # Start WARP liveness monitor (logs to output.log on warp flip)
```

```

1263     start_warp_watcher
```

```

1264 fi
```

```

1265
```

```

1266 # Reset sharing services after network configuration to prevent AirDrop freezes
```

```

1267 echo "Resetting macOS sharing services (AirDrop/AirPlay)..."
```

```

1268 reset_sharing_services
```

```

1269
```

```

1270 # Tell external watchers (post-wake / network-change) the gateway is up.
```

```

1271 write_gateway_active_marker
```

```

1272
```

```

1273 # Local Network Surveillance Check
```

```

1274 echo -e "\n"
```

```

1275 echo "Local Network Surveillance Check"
```

```

1276 echo ""
```

```

1277 # Count populated ARP cache entries to detect network scanning or high congestion
```

```

1278 # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
```

```

1279 ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
```

```

1280 if [ "$ARP_COUNT" -gt 15 ]; then
```

```

1281     warn "High ARP activity detected ($ARP_COUNT devices in cache)."
```

```

1282     warn "This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)."
```

```

1283     echo -e "  [] Your traffic remains fully encrypted and invisible.\n"
```

```

1284 else
```

```

1285     echo -e "  [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
```

```

1286 fi
```

```

1287 fi
```

```

1288
```

```

1289 SUCCESS_RUN=true
```

```

1290
```

```

1291 # Final DNS state summary
```

```

1292 if [ -n "$ACTIVE_SERVICE" ]; then
```

```

1293     FINAL_DNS=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log || true)
```

```

1294     echo -e "\n"
```

```

1295     echo -e "DNS STATE: $FINAL_DNS"
```

```

1296     echo -e ""
```

```

1297 fi
```



```

1298
1299 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
1300     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
1301         echo -e "\n[Warning] One or more blacklist URLs failed to download (404/Offline)."
1302         echo "    The gateway gracefully fell back to yesterday's cached blacklist."
1303         echo "    (Run 'docker logs rule-compiler' later to investigate which link died.)"
1304     fi
1305 fi
1306
1307 rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
1308 if [ -t 0 ]; then
1309     read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
1310     if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
1311         echo "Reversing gateway state..."
1312         exec bash "$0"
1313     fi
1314 fi
1315
1316 echo -e "\nYou can close this terminal window now."

```

7 recover.sh

```

1 #!/bin/bash
2 # recover.sh nullexit KILL-SWITCH recovery script for macOS
3 # Fixes internet when toggle.sh leaves you stranded.
4 #
5 # This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
6 #   1. Disconnecting host Tailscale from the mesh (exit-node off)
7 #   2. Resetting DNS to default on ALL network services
8 #   3. Disabling rogue proxy settings (SOCKS, web, secure web)
9 #   4. Flushing macOS DNS cache
10 #   5. Flushing stale routing table entries
11 #   6. Stopping gateway Docker containers
12 #   7. Power-cycling Wi-Fi
13 #   8. Verifying internet is working
14 #
15 # Usage:  double-click Recover-Gateway.applescript
16 #          OR  ./recover.sh
17 #          OR  ./recover.sh --post-wake    (lighter-touch refresh, keeps gateway live)
18
19 set -e
20
21 # Resolve script directory (used for path-relative refs)
22 # recover.sh lives at the repo root; SCRIPT_DIR lets us launch toggle.sh
23 # (and reference any other repo-root file) from the same directory no
24 # matter where the user invoked recover.sh from.
25 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
26 cd "$SCRIPT_DIR" || exit 1
27
28 # Enforce Cryptographic Script Integrity
29 if [ -f "scripts/crypto.sh" ]; then
30     if ! bash scripts/crypto.sh --verify; then
31         exit 1
32     fi
33 fi
34
35 # Argument parsing
36 # --post-wake: lightweight refresh after sleep/wake or Wi-Fi roam.
37 #             Keeps the gateway live while HUPping DNS caches, refreshing the
38 #             Tailscale exit-node leak, re-hijacking host DNS, and force-
39 #             recreating the warp container if its gluetun healthcheck failed.
40 #             Does NOT tear down Docker, Tailscale, sleep prevention, or Wi-Fi.
41 #             Called from scripts/watcher.sh on every wake / network-change event
42 #             while /tmp/nullexit-gateway-active.marker is present (i.e. the
43 #             gateway is currently up). See devref.md 10.29 for the why.
44 POST_WAKE=false
45 AUTO_MODE=false
46 for arg in "$@"; do
47     if [ "$arg" = "--post-wake" ]; then
48         POST_WAKE=true
49     elif [ "$arg" = "--auto" ] || [ "$arg" = "--non-interactive" ]; then
50         AUTO_MODE=true
51     fi
52 done
53
54 rm -f "$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker"

```

```

55
56 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
57 LOCK_FILE="/tmp/nullexit-toggle.lock"
58 if [ -f "$LOCK_FILE" ]; then
59     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
60     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
61         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
62             if [ "$POST_WAKE" = "true" ]; then
63                 echo "[$(date -u +%FT%TZ)] POST-WAKE skipped: another lifecycle script (PID $LOCK_PID) is running." >>
64                     output.log
65                 exit 0
66             else
67                 echo "Error: Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running." >&2
68                 exit 1
69             fi
70         fi
71     fi
72     echo "$$" > "$LOCK_FILE"
73
74 # Clear the active-state marker in nuclear recovery mode since the gateway is being
75 # completely torn down. This prevents scripts/watcher.sh from triggering post-wake
76 # recoveries in response to network changes caused by the teardown itself.
77 if [ "$POST_WAKE" = "false" ]; then
78     rm -f "/tmp/nullexit-gateway-active.marker"
79 fi
80
81 # WARP Watcher inhibit marker: written immediately in --post-wake mode so the
82 # background WARP Watcher (in toggle.sh) skips failure counting while we are
83 # intentionally tearing down / recreating the warp container. Without this, the
84 # watcher accumulates 6 consecutive warp=off readings during force-recreate (~35s)
85 # and fires nuclear recover.sh, killing a gateway that would have been healthy.
86 WARP_INHIBIT_FILE="/tmp/nullexit-warp-inhibit.marker"
87 if [ "$POST_WAKE" = "true" ]; then
88     touch "$WARP_INHIBIT_FILE"
89     echo "[$(date -u +%FT%TZ)] POST-WAKE started WARP Watcher inhibited to prevent spurious shutdown during warp
90         recreate." >> output.log
91 fi
92
93 # Ensure the inhibit marker and lock file are always removed when the script exits (success or error)
94 cleanup_recover() {
95     rm -f "$WARP_INHIBIT_FILE"
96     if [ -f "$LOCK_FILE" ] && [ "$(cat "$LOCK_FILE" 2>/dev/null)" = "$$" ]; then
97         rm -f "$LOCK_FILE"
98     fi
99 }
100 trap cleanup_recover EXIT
101
102 # Source common formatting and helper functions
103 source "$SCRIPT_DIR/scripts/common.sh"
104
105 # Always add Homebrew path
106 setup_standard_path
107
108 echo ""
109 if [ "$POST_WAKE" = "true" ]; then
110     echo -e "${YELLOW}${BOLD}${NC}"
111     echo -e "${YELLOW}${BOLD} Nullexit POST-WAKE / POST-ROAM LIGHT ${NC}"
112     echo -e "${YELLOW}${BOLD} (keeps the gateway live, refreshes) ${NC}"
113     echo -e "${YELLOW}${BOLD}${NC}"
114     echo ""
115     echo "Re-hijacking DNS, refreshing Tailscale exit-node, force-recreating"
116     echo "warp if unhealthy, and resetting sharing services. The gateway stays"
117     echo "up. docker compose / Wi-Fi / caffeinate are NOT touched."
118     echo ""
119 else
120     echo -e "${RED}${BOLD}${NC}"
121     echo -e "${RED}${BOLD} Nullexit Gateway RECOVERY MODE ${NC}"
122     echo -e "${RED}${BOLD}${NC}"
123     echo ""
124     echo "This script will tear down the gateway and restore normal internet."
125     echo ""
126 fi
127
128 # Cache sudo credentials upfront (default mode ONLY)
129 # Skip in --post-wake: the LaunchAgent-launched watcher has no TTY, so `sudo -v`
130 # would block forever waiting for a password. All post-wake commands use `sudo -n`
131 # which is non-blocking and relies on /etc/sudoers.d/nullexit NOPASSWD entries.
132 SUDO_KEEPER_PID=""
133 if [ "$POST_WAKE" = "false" ] && [ "$AUTO_MODE" = "false" ] && [ -t 0 ]; then
134     echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"

```

```

134 sudo -v
135 (
136     while true; do
137         sudo -n true 2>/dev/null
138         sleep 60
139     done
140 ) &
141 SUDO_KEEPER_PID=$!
142 trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
143 fi
144
145 # Resolve physical default interface and gateway upfront
146 PHYSICAL_IFACE=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{
147     getline; print $2; exit}')
148 [ -z "$PHYSICAL_IFACE" ] && PHYSICAL_IFACE="en0"
149
150 if [ "$POST_WAKE" = "true" ]; then
151     wait_for_dhcp_settle
152 else
153     # Quick resolution in non-post-wake mode (non-blocking)
154     PHYSICAL_GW=$(ipconfig getpacket "$PHYSICAL_IFACE" 2>> output.log | awk -F'[{}]' '/router/{print $2}')
155 fi
156
157 # 1. Disconnect host Tailscale (default) / Refresh exit-node (post-wake)
158 if [ "$POST_WAKE" = "true" ]; then
159     step "Refreshing host Tailscale exit-node routing (post-wake)"
160     if command -v tailscale >> output.log 2>&1; then
161         # Re-issue the SAME exit-node up command toggle.sh START uses. This refreshes
162         # the DERP relay mapping and re-asserts the exit-node preference without
163         # dropping the host's mesh connection (which `tailscale down` would).
164         TS_IP=""
165         for src in ADGUARD_IP.txt; do
166             [ -f "$src" ] || continue
167             TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
168             [ -n "$TS_IP" ] && break
169         done
170
171         if [ -n "$TS_IP" ]; then
172             step "Re-establishing exit node $TS_IP via 'tailscale set --exit-node'"
173             # `tailscale set` only mutates the named preference (exit-node here).
174             # Crucially it does NOT call --reset, which would re-apply ALL flags and
175             # trigger a sub-second route-table re-evaluation cycle each post-wake.
176             # On already-connected nodes, the lighter `set` keeps the existing tunnel
177             # alive while only changing the exit-node preference.
178             if run_with_timeout 15 tailscale set --exit-node="$TS_IP" \
179                 --exit-node-allow-lan-access=true \
180                 >> output.log 2>&1; then
181                 ok "Exit-node $TS_IP re-asserted via 'tailscale set'"
182             else
183                 warn "tailscale set --exit-node=$TS_IP didn't respond; falling back to mesh-only"
184                 run_with_timeout 10 tailscale set --exit-node= \
185                     >> output.log 2>&1 || true
186             fi
187         else
188             warn "ADGUARD_IP.txt missing cannot re-assert exit node"
189             run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= \
190                 >> output.log 2>&1 || true
191         fi
192     else
193         warn "tailscale CLI not found"
194     fi
195 else
196     step "Disconnecting host Tailscale from mesh"
197     disconnect_tailscale_host "tailscale"
198 fi
199
200 # 2. Reset DNS to default on ALL network services (default) / Re-hijack gateway DNS (post-wake)
201 if [ "$POST_WAKE" = "true" ]; then
202     step "Re-hijacking host DNS to gateway IP (post-wake / post-roam)"
203     TS_IP=""
204     for src in ADGUARD_IP.txt; do
205         [ -f "$src" ] || continue
206         TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
207         [ -n "$TS_IP" ] && break
208     done
209
210     if [ -n "$TS_IP" ]; then
211         # Detect the active network service (using helper in common.sh)
212         ACTIVE_SVC=$(get_active_service)
213         ENO_SVC=$(get_en0_service)

```

```

214 networksetup -setsearchdomains "$ACTIVE_SVC" "ts.net" 2>> output.log || true
215 networksetup -setdnsservers "$ACTIVE_SVC" "$TS_IP" 2>> output.log || true
216 networksetup -setsearchdomains "$ENO_SVC" "ts.net" 2>> output.log || true
217 networksetup -setdnsservers "$ENO_SVC" "$TS_IP" 2>> output.log || true
218
219 HITS=$(networksetup -getdnsservers "$ACTIVE_SVC" 2>> output.log | grep -Fx "$TS_IP" || true)
220 if [ -n "$HITS" ]; then
221     ok "DNS re-hijacked to $TS_IP on services: $ACTIVE_SVC, $ENO_SVC"
222 else
223     warn "networksetup accepted but DNS read-back didn't include $TS_IP"
224 fi
225 else
226     warn "ADGUARD_IP.txt missing cannot re-hijack DNS (user must run toggle.sh START)"
227 fi
228 else
229     step "Resetting DNS to default on all network services (empty = DHCP)"
230
231 # Get ALL network services (handles spaces in names, skips disabled ones)
232 SERVICES=$(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$" ||
233     true)
234
235 if [ -z "$SERVICES" ]; then
236     # Fallback to common names
237     SERVICES="Wi-Fi Ethernet Thunderbolt Ethernet USB 10/100 LAN"
238 fi
239
240 RESET_COUNT=0
241 while IFS= read -r service; do
242     # Strip leading asterisk (disabled services)
243     service_clean=$(echo "$service" | sed 's/^\\*/')
244     [ -z "$service_clean" ] && continue
245
246     # Check if this service has a hardware port (skip VPN/service-only entries)
247     if networksetup -listallhardwareports 2>> output.log | grep -A1 "Port: $service_clean$" | grep -q "Device:"
248     || [ "$service_clean" = "Wi-Fi" ]; then
249         networksetup -setsearchdomains "$service_clean" "Empty" 2>> output.log
250         # Set to "empty" so macOS uses DHCP-assigned DNS (not hardcoded 1.1.1.1)
251         sudo networksetup -setdnsservers "$service_clean" "empty" 2>> output.log
252     ) && RESET_COUNT=$((RESET_COUNT + 1)) || true
253 fi
254 done <<< "$SERVICES"
255
256 ok "DNS reset on $RESET_COUNT service(s)"
257 fi
258
259 # 3. Disable rogue proxy settings (default only gateway uses proxy in non-exit-node path)
260 if [ "$POST_WAKE" = "false" ]; then
261     step "Disabling proxy settings (SOCKS, web, secure web)"
262     for svc in $(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$" ||
263         echo "Wi-Fi"); do
264         svc_clean=$(echo "$svc" | sed 's/^\\*/')
265         [ -z "$svc_clean" ] && continue
266         disable_all_proxies "$svc_clean"
267     done
268     ok "Proxy settings cleared"
269 else
270     step "Proxy settings: leaving untouched (post-wake keeps gateway SOCKS wiring alive)"
271 fi
272
273 # 4. Flush macOS DNS cache (always safe and useful in both modes)
274 step "Flushing DNS cache"
275 if command -v dscacheutil >> output.log 2>&1; then
276     flush_dns_cache
277     ok "DNS cache flushed (mDNSResponder HUP)"
278 else
279     warn "dscacheutil not found"
280 fi
281
282 # 5. Clear stale routing table (always safe in both modes)
283 step "Flushing stale routing table entries"
284 # Resolve physical default gateway IP before flushing
285 # We cannot trust `route get default` here because Tailscale may have already hijacked
286 # the default route to utunX. We must read the actual DHCP router assignment from the hardware.
287 # PHYSICAL_IFACE and PHYSICAL_GW resolved upfront
288
289 # Instead of full route -n flush (which destroys macOS loopback/multicast and wedges the OS until reboot),
290 # we cleanly delete the Tailscale overrides and Cloudflare WARP bypass routes in ALL modes.
291 # This ensures tailscaled isn't blocked from reaching its control plane when waking from sleep.
292 sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true

```

```

292 sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
293 remove_warp_bypass_routes
294 ok "Routing overrides cleared"
295
296 if [ "$POST_WAKE" = "false" ]; then
297     disable_killswitch
298     ok "Routing table flush completed via targeted deletes and firewall disabled"
299 else
300     ok "Routing overrides cleared (post-wake mode to preserve Colima bridge100)"
301 fi
302
303 # In post-wake mode we don't bounce Wi-Fi, so we MUST manually restore the physical default route
304 if [ "$POST_WAKE" = "true" ] && [ -n "$PHYSICAL_GW" ]; then
305     # Ensure physical gateway is set just in case it was dropped
306     sudo -n route add default "$PHYSICAL_GW" >> output.log 2>&1 || true
307 fi
308
309 # CONDITIONAL BYPASS ROUTE RESTORATION:
310 # If this was a post-wake recovery and the exit node is active, the route flush
311 # just wiped the static bypass routes for the WARP endpoints. We must re-add
312 # them immediately to prevent a routing loop when Tailscale starts sending traffic.
313 if [ "$POST_WAKE" = "true" ]; then
314     if [ -z "${ACTIVE_IF:-}" ]; then
315         ACTIVE_IF=$(route get default 2>> output.log | awk '/interface:/{print $2; exit}')
316         if [ -z "$ACTIVE_IF" ] || [[ "$ACTIVE_IF" =~ ^tun ]] || [[ "$ACTIVE_IF" =~ ^tun ]]; then
317             for i in en0 en1 en2 en3; do
318                 if ifconfig "$i" 2>> output.log | grep -q 'status: active'; then
319                     ACTIVE_IF="$i"; break
320                 fi
321             done
322         fi
323         [ -z "$ACTIVE_IF" ] && ACTIVE_IF="en0"
324     fi
325
326     echo "Re-adding host bypass routes for Cloudflare WARP endpoints..."
327     remove_warp_bypass_routes
328     if [ -n "${PHYSICAL_GW:-}" ]; then
329         add_warp_bypass_routes "$PHYSICAL_GW"
330     else
331         add_warp_bypass_routes "$ACTIVE_IF"
332     fi
333
334     # Also re-apply the default route pointing to the Tailscale interface
335     ts_iface=$(ifconfig 2>> output.log | grep -B4 "inet 100." | grep -E '[a-z0-9]+' | cut -d: -f1 | head -n 1)
336     if [ -n "$ts_iface" ]; then
337         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
338         # DO NOT delete the physical default route! It crashes Colima.
339         # Use the /1 trick to mathematically override the default route.
340         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
341         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
342         sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
343         sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
344         enable_killswitch
345     fi
346 fi
347
348 # 6. Stop gateway Docker containers (default) / Force-recreate warp if unhealthy (post-wake)
349 if [ "$POST_WAKE" = "true" ]; then
350     step "Checking Colima VM state"
351     colima_status=$(colima status 2>&1)
352     if echo "$colima_status" | grep -qi "running"; then
353         ok "Colima VM is running"
354         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima status check passed." >> output.log
355     else
356         warn "Colima VM is NOT running or unresponsive"
357         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima is unhealthy or dead! Output: $colima_status" >> output.log
358         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Attempting to restart Colima..." >> output.log
359         colima restart >> output.log 2>&1 || warn "Failed to restart Colima"
360     fi
361
362     step "Checking warp container health (gluetun UDP tunnel)"
363     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Inspecting warp container state..." >> output.log
364     # The warp container is the most failure-prone link in the chain at wake/roam:
365     # its UDP NAT binding to Cloudflare's edge (162.159.192.1:2408) silently dies
366     # during a Wi-Fi roam. We verify the tunnel is actually passing traffic via curl.
367     WARP_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
368     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container status: $WARP_STATE" >> output.log
369
370     if [ "$WARP_STATE" = "missing" ]; then

```

```

371 warn "warp container is missing leaving gateway containers alone (full relaunch requires \"${SCRIPT_DIR}/
toggle.sh\")"
372 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container missing from Docker engine. Requires
full toggle.sh launch." >> output.log
373 else
374 tmp_err=$(mktemp)
375 if docker compose exec -T warp wget -q0- --timeout=3 https://www.cloudflare.com/cdn-cgi/trace 2>"$tmp_err"
| grep -q "warp=on"; then
376 ok "warp container is healthy and actively tunneling traffic (no recreate needed)"
377 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container passed live traffic healthcheck (
Cloudflare trace successful)." >> output.log
378 else
379 warn "warp container tunnel is dead (Docker status: $WARP_STATE) force-recreating all gateway containers
to restore network namespace"
380 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container is DEAD or STUCK. Forcing recreation
of gateway stack..." >> output.log
381 if [ -s "$tmp_err" ]; then
382 err_msg=$(cat "$tmp_err" | tr -d '\r')
383 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Healthcheck failure details: $err_msg" >> output.
log
384 fi
385 docker compose up -d --force-recreate >> output.log 2>&1 || warn "force-recreate failed"
386
387 NEW_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
388 ok "warp container health after recreate: $NEW_STATE"
389 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container health after recreation: $NEW_STATE"
>> output.log
390 fi
391 rm -f "$tmp_err"
392 fi
393 else
394 step "Stopping gateway Docker containers"
395 if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
396 if [ -f "docker-compose.yml" ]; then
397 docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
398 else
399 warn "docker-compose.yml not found in current directory"
400 fi
401 else
402 warn "Docker not available skipping"
403 fi
404 fi
405
406 # 6b. Stopping sleep prevention (caffeinate) default only
407 if [ "$POST_WAKE" = "false" ]; then
408 step "Stopping sleep prevention"
409 stop_pidfile_daemon "$PID_CAFFEINATE" "system sleep prevention"
410 else
411 step "Sleep prevention: leaving untouched (caffeinate already re-acquired by parent shell)"
412 fi
413
414 # 6c. Stopping DNS Watcher default only
415 if [ "$POST_WAKE" = "false" ]; then
416 step "Stopping DNS Watcher"
417 stop_pidfile_daemon "$PID_DNS_WATCHER" "background DNS Watcher"
418 else
419 step "DNS Watcher: leaving untouched (it keeps re-hijacking DNS on every roam)"
420 fi
421
422 # 7. Power-cycle Wi-Fi default only
423 if [ "$POST_WAKE" = "false" ]; then
424 step "Power-cycling Wi-Fi"
425 bounce_wifi_interfaces
426 else
427 step "Wi-Fi: leaving untouched (post-wake keeps the current Wi-Fi association)"
428 fi
429
430 # 8. Verify (default checks direct internet; post-wake verifies the gateway)
431 if [ "$POST_WAKE" = "true" ]; then
432 step "Verifying gateway is healthy after post-wake refresh"
433
434 # Wait a moment for tailscale + warp healthcheck to settle
435 sleep 3
436
437 GATEWAY_OK=false
438 if command -v dig >> output.log 2>&1; then
439 # AdGuard is exposed at localhost:5354. If this resolves, the warp tunnel
440 # is alive AND DNSCache has picked up the post-wake reset.
441 if dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5 >> output.log 2>&1; then
442 ok "Gateway DNS (AdGuard via localhost:5354) works"
443 GATEWAY_OK=true

```



```

444     else
445         warn "Gateway DNS not responding yet"
446     fi
447 fi
448
449 # Direct host gateway check via Tailscale ping (relayed pong counts as success)
450 if command -v tailscale >> output.log 2>&1; then
451     TS_IP=""
452     [ -f ADGUARD_IP.txt ] && TS_IP=$(read_adguard_ip || true)
453     if [ -n "$TS_IP" ] && tailscale ping --until-direct=false -c 1 --timeout 4s "$TS_IP" 2>> output.log | grep
454         -q pong; then
455         ok "Gateway $TS_IP reachable via Tailscale"
456         GATEWAY_OK=true
457     else
458         warn "Tailscale ping to gateway did not pong exit-node may still be re-establishing"
459     fi
460 fi
461
462 if [ "$GATEWAY_OK" = "true" ]; then
463     echo -e " ${GREEN}${BOLD} Gateway is healthy after post-wake refresh.${NC}"
464 else
465     echo -e " ${YELLOW}${BOLD} Gateway recovery is in-progress.${NC}"
466     echo "     The route may need another 30s before queries succeed."
467     echo "     If it stays broken for >60s, run: bash \"${SCRIPT_DIR}/toggle.sh\""
468 fi
469 else
470     step "Verifying internet connectivity"
471
472     # Wait a moment for the network to settle
473     sleep 2
474
475     INTERNET_OK=false
476
477     # Try DNS resolution first
478     if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
479         ok "DNS resolution works (google.com via 1.1.1.1)"
480         INTERNET_OK=true
481     elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
482         ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
483         INTERNET_OK=true
484     else
485         warn "DNS resolution check failed will still try ping/curl"
486     fi
487
488     # Try actual HTTP connectivity
489     if command -v curl >> output.log 2>&1; then
490         if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
491             ok "Internet reachable via HTTP"
492             INTERNET_OK=true
493         elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
494             ok "Internet reachable via HTTP (plain)"
495             INTERNET_OK=true
496         else
497             warn "HTTP check failed"
498         fi
499     elif command -v ping >> output.log 2>&1; then
500         if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
501             ok "Internet reachable via ping"
502             INTERNET_OK=true
503         else
504             warn "Ping check failed"
505         fi
506     fi
507 fi
508
509 # Summary
510 echo ""
511 echo -e "${BOLD}${NC}"
512 if [ "$POST_WAKE" = "true" ]; then
513     echo -e "${BOLD}POST-WAKE SUMMARY${NC}"
514 else
515     echo -e "${BOLD}RECOVERY SUMMARY${NC}"
516 fi
517 echo -e "${BOLD}${NC}"
518
519 # Show final DNS state
520 FINAL_DNS=$(networksetup -getdnsservers "Wi-Fi" 2>/dev/null || echo "N/A")
521 echo " DNS (Wi-Fi): $FINAL_DNS"
522
523 FINAL_DNS2=$(networksetup -getdnsservers "Ethernet" 2>/dev/null || true)
524 if [[ -z "$FINAL_DNS2" || "$FINAL_DNS2" == *"not a recognized"* || "$FINAL_DNS2" == *"Error"* ]]; then

```

```

524 FINAL_DNS2="N/A (Not configured)"
525 fi
526 echo "   DNS (Ethernet): $FINAL_DNS2"
527
528 echo ""
529
530 if [ "$POST_WAKE" = "false" ]; then
531     if [ "$INTERNET_OK" = "true" ]; then
532         echo -e "   ${GREEN}${BOLD} Internet is working.${NC}"
533     else
534         echo -e "   ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
535         echo "       This might mean:"
536         echo "       - Your Wi-Fi is disconnected from the router"
537         echo "       - Tailscale is still interfering (try restarting tailscaled:"
538         echo "         brew services restart tailscale)"
539         echo "       - You need to re-connect to Wi-Fi in the menu bar"
540         echo "       - Try toggling Wi-Fi off/on in the menu bar"
541         echo ""
542         echo "       Or try manually:"
543         echo "         networksetup -setdnsservers Wi-Fi empty"
544         echo "         tailscale down"
545         echo "         sudo route delete -net 0.0.0.0/1"
546         echo "         sudo route delete -net 128.0.0.0/1"
547         echo "         brew services restart tailscale"
548     fi
549 fi
550
551 # Reset sharing services to prevent AirDrop freezes after interface changes
552 # This is the SAME step in BOTH modes (post-wake inherits the previous fix from 10.27).
553 echo -e "   Resetting macOS sharing services (AirDrop/AirPlay)..."
554 reset_sharing_services
555
556 echo ""
557 if [ "$POST_WAKE" = "true" ]; then
558     echo -e "${YELLOW}${BOLD}Post-wake refresh complete.${NC}"
559     # Remove the WARP Watcher inhibit marker now that the gateway is fully healthy.
560     # The watcher will resume normal liveness monitoring on its next 5s poll.
561     rm -f "$WARP_INHIBIT_FILE"
562     echo "[$(date -u +%FT%TZ)] POST-WAKE complete  WARP Watcher inhibit released." >> output.log
563 else
564     echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
565 fi
566 echo ""

```

8 setup.sh

```

1  #!/bin/bash
2  # setup.sh nullexit one-shot setup script (macOS)
3  # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4  set -euo pipefail
5
6  # Source common formatting and helper functions
7  source "$(dirname "$0")/scripts/common.sh"
8
9  # Run common installation logic
10 source "$(dirname "$0")/scripts/setup-common.sh"
11
12 # Compile macOS Shortcuts
13 echo -e "\n${BOLD}Compiling macOS Desktop Shortcuts...${NC}"
14 if command -v osacompile &> /dev/null; then
15     osacompile -o "Toggle Gateway.app" "Toggle-Gateway.applescript" >/dev/null 2>&1
16     osacompile -o "Recover Gateway.app" "Recover-Gateway.applescript" >/dev/null 2>&1
17     echo "   Created 'Toggle Gateway.app' and 'Recover Gateway.app'"
18 fi
19
20 # Done
21 echo ""
22 echo -e "${GREEN}${BOLD}${NC}"
23 echo -e "${GREEN}${BOLD}           Setup complete!           ${NC}"
24 echo -e "${GREEN}${BOLD}${NC}"
25 echo ""
26 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
27 echo "   https://login.tailscale.com/admin/machines"
28 echo "   Find your gateway  '...'  'Edit route settings'  enable Exit Node"
29 echo ""
30

```

```

31 echo -e "${YELLOW}${BOLD}Sudoers / Background Execution Requirements:${NC}"
32 echo "  To allow silent, non-interactive execution of the firewall and network routes (especially during sleep/
33 wake recovery), you must configure passwordless sudo."
34 echo "  Run this exact command in your terminal:"
35 echo "    echo \"\$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/
36 bin/killall, /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/brew,
37 /usr/local/bin/brew, /usr/bin/python3 \$PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 \$PWD/scripts/
38 dns-proxy.py, /usr/local/bin/python3 \$PWD/scripts/dns-proxy.py\" | sudo tee /etc/sudoers.d/nullexit"
39 echo ""
40 if [[ -n "${TS_IP:-}" ]]; then
41   echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"
42   echo ""
43 fi
44 echo -e "${BOLD}To update your block/allow rules:${NC}"
45 echo "  python3 scripts/sync-rules.py"
46 echo ""
47 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
48 echo "  macOS:  double-click the 'Toggle Gateway' app icon!"
49 echo "  Windows: double-click Toggle-Gateway.bat"
50 echo ""
51 if [[ "$OSTYPE" == "darwin"* ]]; then
52   echo -e "${YELLOW}${BOLD>Note on macOS Tailscale Sandboxing:${NC}"
53   echo "  The Tailscale GUI app (tailscale-app) installed on macOS is sandboxed."
54   echo "  While you cannot run a Tailscale SSH server on this Mac host, you can"
55   echo "  still SSH outbound to other devices on the mesh using their MagicDNS"
56   echo "  addresses directly (e.g. ssh user@device.tailnet-name.ts.net)."
57   echo ""
58 fi

```

9 scripts/toggle-linux.sh

```

1 #!/bin/bash
2 # toggle-linux.sh nullexit gateway toggle script for Linux
3 # Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.
4
5 set -e
6
7
8 # Start execution timer
9 TOGGLE_START_TIME=$SECONDS
10
11 if [[ "$1" == "restart" ]] || [[ "$1" == "--restart" ]]; then
12   echo "Executing Gateway Restart Sequence..."
13   # We must source common.sh here temporarily to check is_gateway_active before we proceed
14   source "$(dirname "${BASH_SOURCE[0]}")/common.sh"
15   if is_gateway_active; then
16     echo "Gateway is currently running. Stopping it first..."
17     bash "$0"
18     echo "Gateway stopped. Now starting it..."
19   else
20     echo "Gateway is already stopped. Starting it..."
21   fi
22   bash "$0"
23   exit 0
24 fi
25
26 # Global flag to track if the script completed successfully
27 SUCCESS_RUN=false
28
29 # Global variable to track the currently running background command PID
30 CURRENT_BG_PID=""
31
32 # Source common bash functions
33 source "$(dirname "${BASH_SOURCE[0]}")/common.sh"
34
35
36 PID_FILE="/tmp/nullexit-cafeinate.pid"
37
38 # Start macOS caffeineinate to prevent sleep while gateway is running
39 start_sleep_prevention() {
40   if ! command -v systemd-inhibit >/dev/null 2>&1; then
41     echo " [Warning] systemd-inhibit tool not found. Sleep prevention unavailable."
42     return 1

```

```

43 fi
44
45 # Stop any existing sleep prevention process first to avoid duplicates
46 stop_sleep_prevention
47
48 echo " Enabling system sleep prevention (systemd-inhibit)..."
49 # Run systemd-inhibit wrapped in a bash trap to prevent idle system sleep.
50 # Critically, this traps shutdown signals (SIGTERM) and automatically
51 # flushes hijacked DNS back to normal right before the PC powers off.
52 # We do not use recover-linux.sh here because it is a heavy recovery script
53 # which takes too long and gets terminated before it can reset the DNS. Instead,
54 # we surgically and instantly restore normal DNS settings here.
55 nohup bash -c "
56     trap '
57         echo \"[Shutdown Trap] Reverting network settings...\\" >> \"\$PWD/output.log\" 2>&1
58         kill \$! 2>/dev/null || true
59         sudo -n resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
60         sudo -n resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
61         resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
62         resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
63         if command -v tailscale >/dev/null 2>&1; then
64             tailscale up --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 &
65             TS_DOWN_ARGS=""
66             if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS="--accept-risk=lose-ssh\"; fi
67             tailscale down \"\$TS_DOWN_ARGS\" >> \"\$PWD/output.log\" 2>&1 &
68         fi
69         sleep 1
70         echo \"[Shutdown Trap] Cleanup complete.\\" >> \"\$PWD/output.log\" 2>&1
71         exit 0
72     ' SIGTERM SIGINT SIGHUP
73     systemd-inhibit --what=sleep --who=null --why="gateway running\" --mode=block sleep infinity &
74     wait \$!
75     " >> output.log 2>&1 &
76     local caffe_pid=$!
77     echo "$caffe_pid" > "$PID_FILE"
78     echo " Sleep prevention active (PID $caffe_pid). Your PC won't sleep while the gateway is running."
79 }
80
81 # Stop sleep prevention when gateway is stopped
82 stop_sleep_prevention() {
83     if [ -f "$PID_FILE" ]; then
84         local caffe_pid
85         caffe_pid=$(cat "$PID_FILE")
86         if [ -n "$caffe_pid" ] && kill -0 "$caffe_pid" 2>/dev/null && ps -p "$caffe_pid" -o command= 2>/dev/null |
87             grep -q -E "systemd-inhibit|recover-linux.sh"; then
88             echo " Stopping system sleep prevention (systemd-inhibit wrapper PID $caffe_pid)..."
89             kill "$caffe_pid" 2>/dev/null || true
90         fi
91         rm -f "$PID_FILE"
92     fi
93 }
94
95 DNS_WATCHER_PID_FILE="/tmp/nullexit-dns-watcher.pid"
96
97 start_dns_watcher() {
98     local target_ip=$1
99     stop_dns_watcher
100     echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
101     nohup bash -c "
102         trap 'exit 0' SIGTERM SIGINT SIGHUP
103         while true; do
104             if [ -n \"\$ACTIVE_SERVICE\" ]; then
105                 CURRENT_DNS=$(resolvectl dns \"\$ACTIVE_SERVICE\" 2>/dev/null | grep -oE
106                 '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head -1)
107                 if [ \"\$CURRENT_DNS\" != \"\$target_ip\" ]; then
108                     resolvectl dns \"\$ACTIVE_SERVICE\" \"\$target_ip\" >/dev/null 2>&1 || true
109                 fi
110             fi
111             sleep 30
112         done
113         " >> output.log 2>&1 &
114     echo $! > "$DNS_WATCHER_PID_FILE"
115 }
116
117 stop_dns_watcher() {
118     if [ -f "$DNS_WATCHER_PID_FILE" ]; then
119         local watcher_pid
120         watcher_pid=$(cat "$DNS_WATCHER_PID_FILE")
121         if [ -n "$watcher_pid" ] && kill -0 "$watcher_pid" 2>/dev/null; then
122             echo " Stopping background DNS Watcher (PID $watcher_pid)..."
123             kill -9 "$watcher_pid" 2>/dev/null || true
124         fi
125     fi
126 }

```

```

122     fi
123     rm -f "$DNS_WATCHER_PID_FILE"
124 fi
125 }
126
127 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
128 cleanup_handler() {
129     local exit_code=$?
130     local trigger_type="$1"
131     local line_no="$2"
132
133     if [ "$SUCCESS_RUN" = "false" ]; then
134         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
135         if [ -n "$ACTIVE_SERVICE" ]; then
136             reset_dns
137         fi
138
139         if [ -n "$CURRENT_BG_PID" ]; then
140             echo "Terminating active command (PID $CURRENT_BG_PID)..."
141             kill -15 "$CURRENT_BG_PID" 2>> output.log || true
142         fi
143
144         # Disconnect host Tailscale and close the GUI application on failure
145         disconnect_tailscale_host
146
147         # Stop local DNS proxy if running
148         stop_local_dns_proxy
149
150         # Stop sleep prevention
151         stop_sleep_prevention
152         stop_dns_watcher
153
154         # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
155         if [ -n "$ACTIVE_SERVICE" ]; then
156             cleanup_network_state
157         fi
158
159         if [ "$trigger_type" = "ERR" ]; then
160             echo -e "\n=====
161             echo "ERROR: Script failed at line $line_no with exit code $exit_code."
162             echo "=====
163             echo "Troubleshooting steps:"
164             echo "1. Run 'colima status' to verify the VM is active."
165             echo "2. Run 'docker compose ps' to check container health."
166             echo "3. Run 'docker compose logs' to view application logs."
167             echo "4. Run 'tailscale status' to check host Tailscale status."
168             echo "=====
169         fi
170     fi
171 }
172 }
173
174 trap 'cleanup_handler ERR $LINENO' ERR
175 trap 'cleanup_handler INT' INT
176 trap 'cleanup_handler TERM' TERM
177 trap 'cleanup_handler HUP' HUP
178
179 # NOPASSWD sudo
180 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
181 # needed (networksetup, dnsmasq, dscacheutil, killall, pkill, socat).
182 # All privileged calls below use `sudo -n` which will run without prompting.
183 # No credential caching loop is needed.
184
185 # Add Homebrew and standard paths
186 export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
187
188 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
189 # Uses a Python DNS proxy (handles TCP wire format correctly).
190 # Python3 is built into macOS no external dependencies.
191 DNS_PROXY_BIN=""
192 if command -v python3 >> output.log 2>&1; then
193     DNS_PROXY_BIN="python3"
194 elif command -v python >> output.log 2>&1; then
195     DNS_PROXY_BIN="python"
196 fi
197
198 # Path to the DNS proxy script (sibling of this script)
199 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/dns-proxy.py"
200
201 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula

```

```

202 # (`brew install tailscale` + `systemctl start tailscaled`) NOT the
203 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
204 # to launch, no menu-bar click to perform, and no scutil Network Service to
205 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
206 # if you ran the install with sudo) and the control socket is always available.
207 TS_BIN=""
208 if command -v tailscale >> output.log 2>&1; then
209     TS_BIN="tailscale"
210 fi
211
212 # Baseline: disable Tailscale DNS management at the daemon level.
213 # `tailscale set` writes a persistent preference. However, `tailscale up --reset`
214 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
215 # re-enable `--accept-dns=true` and nullify this `set`.
216 # Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
217 # `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
218 #
219 # This initial `set` is still useful as a baseline for non-reset operations
220 # (e.g. if the user runs `tailscale up` manually without --reset) and covers
221 # the brief window between this line and the first `--reset` call.
222 #
223 # Runs once at script start while tailscaled is still connected (if it was
224 # left running from a previous toggle), so the pref takes effect before any
225 # disconnection or reconnection logic.
226 if [ -n "$TS_BIN" ]; then
227     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
228 fi
229
230 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
231 # *entire* teardown no need to mess with system network managers.
232 # tailscaled keeps running (as a systemd service), which is intentional:
233 # the next `tailscale up` then reconnects in seconds rather than waiting for a
234 # slow daemon launch. The `tailscale down` call also retains the user's
235 # auth state, so we don't force a browser re-login every cycle.
236 #
237 # Defined early (before the if/else branches and referenced by cleanup_handler's
238 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
239 restart_tailscaled_daemon() {
240     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
241     echo " [Tailscale Recovery] Attempting to restart tailscaled service via systemctl..."
242     if run_with_timeout 15 sudo -n systemctl restart tailscaled >> output.log 2>&1; then
243         echo " [Tailscale Recovery] Successfully restarted tailscaled service."
244     else
245         echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
246     fi
247     sleep 3
248 }
249 disconnect_tailscale_host() {
250     if [ -n "$TS_BIN" ]; then
251         echo "Disconnecting host Tailscale from mesh (tailscaled stays running as system service)..."
252         local ts_args=""
253         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
254         if ! run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1; then
255             restart_tailscaled_daemon
256             run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1 || true
257         fi
258     fi
259 }
260
261 # Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
262 get_active_service() {
263     # Get the interface used for default internet routing
264     local iface
265     iface=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
266     if [ -z "$iface" ]; then
267         echo ""
268         return 1
269     fi
270     echo "$iface"
271 }
272
273 ACTIVE_SERVICE=$(get_active_service)
274
275 # Resolve the service name for en0 (usually "Wi-Fi") macOS scutil DNS resolver
276 # is commonly scoped to en0, so per-service DNS changes on other interfaces are
277 # ignored unless en0's service is also updated.
278 # Helper to restore host DNS to a clean state (used on script start, on failure,
279 # and after a successful STOP). Without this, a successful toggle-off leaves the
280 # host's resolver pinned to a now-dead gateway IP every lookup stalls for macOS's
281 # DNS timeout (~5s) before falling through to whatever next server is configured.
282 # With it, we always leave the host in `1.1.1.1 / no search domain`.

```

```

283 reset_dns() {
284     if [ -n "$ACTIVE_SERVICE" ]; then
285         resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
286         resolvectl revert "$ACTIVE_SERVICE" 2>/dev/null || true
287     fi
288 }
289
290 # Helper to nuke leftover network state after teardown (proxy settings, routing
291 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
292 # problem where stale state kills internet after the gateway stops.
293 cleanup_network_state() {
294     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
295
296     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
297     # (Skipped for Linux since we don't set global SOCKS proxy)
298     echo " Proxies disabled."
299
300     # 2. Flush DNS cache
301     if command -v resolvectl >> output.log 2>&1; then
302         sudo -n resolvectl flush-caches 2>> output.log || true
303     elif command -v systemd-resolve >> output.log 2>&1; then
304         sudo -n systemd-resolve --flush-caches 2>> output.log || true
305     fi
306     echo " DNS cache flushed."
307
308     # 3. Flush stale routing table entries
309     if command -v ip >> output.log 2>&1; then
310         sudo -n ip route flush cache >> output.log 2>&1 || true
311     fi
312     echo " Routing table flushed."
313
314     # 4. Power-cycle Wi-Fi to clear any lingering interface state
315     echo " Restarting network interface..."
316     sudo -n ip link set dev "$ACTIVE_SERVICE" down 2>> output.log || true
317     sleep 1
318     sudo -n ip link set dev "$ACTIVE_SERVICE" up 2>> output.log || true
319     echo " Interface power-cycled ($ACTIVE_SERVICE)."
320     sleep 3
321
322     echo "Network state cleanup complete."
323 }
324
325 # SOCKS5 Proxy (WARP Tunnel)
326 # When Tailscale's exit node can't route through WARP (due to userspace
327 # forwarding), we use a SOCKS5 proxy running inside the warp container's
328 # network namespace. Connections created by this proxy go through the
329 # kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale's
330 # userspace exit node which bypasses it.
331 #
332 # The SOCKS5 proxy is exposed on localhost:1080 via Docker port mapping.
333 # `networksetup -setsocksfirewallproxy` on macOS routes system TCP traffic
334 # through the proxy, which then goes through WARP.
335 #
336 # IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
337 # They must use --socks5-hostname or the ALL_PROXY env variable explicitly.
338
339 SOCKS_PROXY_PORT=1080
340
341 enable_socks_proxy() {
342     local svc="$1"
343     echo " (Linux global SOCKS proxy configuration is desktop-environment specific, skipping.)"
344     echo " SOCKS5 proxy is active and listening on 127.0.0.1:$SOCKS_PROXY_PORT"
345 }
346
347 disable_socks_proxy() {
348     local svc="$1"
349     echo " (Linux global SOCKS proxy configuration skipped.)"
350 }
351
352 # Local DNS Proxy (Python)
353 # When the tailnet data plane cannot establish, we use a Python DNS proxy.
354 # It listens on UDP:53, receives DNS queries, forwards them over TCP to
355 # AdGuard at 127.0.0.1:5354 (Docker port mapping), and returns the response.
356 # The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
357 # correctly unlike socat which just forwards raw bytes.
358 #
359 # Python3 is built into macOS no external dependencies.
360 DNS_PROXY_PID=""
361
362 # Kill any leftover DNS proxy process
363 stop_local_dns_proxy() {

```



```

364 if [ -n "$DNS_PROXY_PID" ]; then
365     kill "$DNS_PROXY_PID" 2>/dev/null || true
366     wait "$DNS_PROXY_PID" 2>/dev/null || true
367     DNS_PROXY_PID=""
368 fi
369 # Clean up any stale Python DNS proxy processes
370 sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
371 }
372
373 # Start local DNS proxy (Python)
374 start_local_dns_proxy() {
375     if [ -z "$DNS_PROXY_BIN" ]; then
376         echo " Python not found. Python3 is built into macOS this shouldn't happen."
377         return 1
378     fi
379
380     if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
381         echo " DNS proxy script not found at $DNS_PROXY_SCRIPT"
382         return 1
383     fi
384
385     # Kill any leftover process first
386     stop_local_dns_proxy
387
388     echo -n " Starting local DNS proxy via Python (UDP:53 TCP:5354)... "
389     sudo -n "$DNS_PROXY_BIN" "$DNS_PROXY_SCRIPT" &
390     DNS_PROXY_PID=$!
391     disown "$DNS_PROXY_PID" 2>/dev/null || true
392
393     # Give it a moment to bind
394     sleep 0.5
395
396     if kill -0 "$DNS_PROXY_PID" 2>/dev/null; then
397         echo "started (PID $DNS_PROXY_PID)."

```

```

445 sudo -n resolvectl domain "$ACTIVE_SERVICE" "-ts.net" 2>/dev/null || true
446 if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" "$target_ip"; then
447     echo "FAILED (resolvectl error)"
448 else
449     # Verify
450     entries=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+')
451     if [ "$entries" == "$target_ip" ]; then
452         echo "VERIFIED"
453         return 0
454     else
455         echo "MISMATCH (Resolver refused lock)"
456     fi
457 fi
458 sleep 2
459 done
460 return 1
461 }
462
463 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
464 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
465 # `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
466 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
467 # Capture current DNS before resetting so we can check if it was hijacked
468 INITIAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head
469 -1 || true)
470
471 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
472 reset_dns
473
474 # Local Network Surveillance Check
475 echo -e "\n"
476 echo "Local Network Surveillance Check"
477 echo ""
478 # Count populated ARP cache entries to detect network scanning or high congestion
479 if command -v ip >/dev/null 2>&1; then
480     ARP_COUNT=$(ip neigh | grep -iv 'incomplete' | wc -l | tr -d ' ')
481 else
482     # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
483     ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
484 fi
485 if [ "$ARP_COUNT" -gt 15 ]; then
486     echo -e " \033[1;33m[!] WARNING: High ARP activity detected ($ARP_COUNT devices in cache).\033[0m"
487     echo -e " \033[1;33m[!] This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)
488     .\033[0m"
489     echo -e " [] Your traffic remains fully encrypted and invisible.\n"
490 else
491     echo -e " [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
492 fi
493
494 echo "Checking Gateway Status..."
495 if is_gateway_active; then
496     echo -e "\n=====
497     echo "Gateway is RUNNING. Stopping it now..."
498     echo -e "=====
499
500 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
501 disconnect_tailscale_host
502 echo ""
503
504 echo "Stopping Docker containers..."
505 docker compose down -t 5
506
507 echo -e "\nDocker daemon handles container teardown automatically."
508
509 # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
510 # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
511 # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
512 echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
513 reset_dns
514
515 # 3. Stop local DNS proxy if running
516 stop_local_dns_proxy
517
518 # Stop sleep prevention
519 stop_sleep_prevention
520 stop_dns_watcher
521
522 # 4. Nuke leftover network state so internet actually works after teardown
523 cleanup_network_state

```

```

524 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
525 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
526 else
527 echo -e "\n=====
528 echo "Gateway is STOPPED. Starting it now..."
529 echo -e "=====\\n"
530
531 # 1. Prevent host exit-node deadlock during VM / Container startup
532 disconnect_tailscale_host
533
534 echo -e "\\nUsing native Linux Docker..."
535
536 # 4. Clean up corrupted AdGuardHome configurations
537 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
538 rm -f "adguard/conf/AdGuardHome.yaml"
539 echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
540 fi
541
542
543
544 # 5. Start compose services
545 write_host_ips
546 echo -e "\\nStarting Docker containers..."
547 docker compose up -d
548
549 # Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
550 docker compose rm -s -f rule-compiler >> output.log 2>&1
551
552 # 6. Wait for the gateway container's Tailscale connection to be ready
553 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
554 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
555
556 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
557 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
558 connected=false
559 consecutive_failures=0
560 for i in {1..60}; do
561 # Run tailscale status and capture its output so the user can see what's happening.
562 # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
563 ts_output=$(run_with_timeout 5 docker compose exec -T tailscale tailscale status 2>&1 || true)
564
565 if echo "$ts_output" | grep -q "offers exit node"; then
566 connected=true
567 break
568 fi
569
570 # Track consecutive failures to detect a stuck state.
571 # If we see 40+ consecutive 'NoState' responses, give up early
572 # the daemon is alive but can't connect to the coordination server.
573 if echo "$ts_output" | grep -q "NoState"; then
574 consecutive_failures=$((consecutive_failures + 1))
575 if [ "$consecutive_failures" -ge 40 ]; then
576 echo ""
577 echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
578 echo " Tailscale daemon is running but can't reach the coordination server."
579 break
580 fi
581 else
582 consecutive_failures=0
583 fi
584
585 # Print a condensed status every 10 seconds so the user can see progress
586 if [ $((i % 10)) -eq 0 ]; then
587 ts_short=$(echo "$ts_output" | head -1 | tr -d '\\r\\n')
588 echo " [attempt $i/60] $ts_short"
589 elif [ $((i % 5)) -eq 0 ]; then
590 echo -n "$[i]"
591 else
592 echo -n " ."
593 fi
594 sleep 1
595 done
596 echo ""
597
598 if [ "$connected" = "true" ]; then
599 echo "Gateway container is online on the Tailscale mesh."
600 elif [ "$BYPASS_PING" = "true" ]; then
601 echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
602 else
603 echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
604 echo " The container was seen in the following states during the wait:" >&2

```

```

605 echo "$ts_output" | sed 's/^/ /' >&2
606 echo "" >&2
607 echo " This could mean:" >&2
608 echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
609 echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
610 echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
611 echo "" >&2
612 echo " Diagnose manually:" >&2
613 echo " docker compose exec -T tailscale tailscale status" >&2
614 echo " docker compose exec -T tailscale tailscale netcheck" >&2
615 cleanup_handler ERR $LINENO
616 exit 1
617 fi
618
619 # 6b. Resolve the gateway's Tailscale IP.
620 # Primary: static ADGUARD_IP.txt (fast, no docker-exec dependency).
621 # Fallback: dynamic query from the container (handles IP changes after re-auth).
622 #
623 # CRITICAL: `docker compose exec -T` on macOS/Colima injects carriage
624 # returns (\r) into captured output. If $TS_IP ends with \r, then
625 # `networksetup -setdnsservers` silently rejects the malformed IP and
626 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
627 # `tr -d '\r'` strips the CR; `awk 'NR==1{print $1; exit}'` takes only
628 # the first field of the first line (preserving the old `head -1` guard).
629 TS_IP=""
630 TS_IP=$(cat ADGUARD_IP.txt 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
631 if [ -n "$TS_IP" ]; then
632     echo "Using static IP from ADGUARD_IP.txt: $TS_IP"
633 elif [ "$connected" = "true" ]; then
634     TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' |
635         awk 'NR==1{print $1; exit}' || true)
636     if [ -n "$TS_IP" ]; then
637         echo "[Fallback] Resolved gateway Tailscale IP dynamically: $TS_IP"
638     fi
639 fi
640
641 # 7. Connect host Mac to Tailscale mesh.
642 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
643 # system LaunchDaemon via 'systemctl start tailscaled'), the previous five-
644 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
645 # no menu bar item to click, and no Accessibility permission required.
646 #
647 # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
648 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
649 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
650 # are impossible without the host on the mesh).
651 #
652 # We connect in two phases:
653 # Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
654 # Phase B Set exit node only after pre-flight checks confirm the gateway works
655 HOST_ON_MESH=false
656 SKIP_EXIT_NODE=false
657 if [ -n "$TS_BIN" ]; then
658     echo -n "Verifying tailscaled is reachable"
659     daemon_ready=false
660     for i in {1..10}; do
661         # `tailscale status` returns exit code 1 when the daemon is alive but
662         # disconnected ("Tailscale is stopped."). We check the daemon socket
663         # directly as a fallback if the socket exists, tailscaled is running
664         # even if the host isn't connected to the mesh yet.
665         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
666             daemon_ready=true
667             break
668         fi
669         echo -n "."
670         sleep 1
671     done
672     echo ""
673
674     if [ "$daemon_ready" != "true" ]; then
675         echo -e "\033[0;33m[!] tailscaled daemon is not running. Attempting to start it...\033[0m"
676
677         # Try system-wide daemon (with timeout sudo may prompt for password)
678         if [ "$daemon_ready" != "true" ]; then
679             if run_with_timeout 10 sudo systemctl start tailscaled 2>> output.log; then
680                 echo "Started tailscaled as system LaunchDaemon."
681                 for i in {1..15}; do
682                     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
683                         daemon_ready=true
684                         break
685                     fi

```

```

685         sleep 1
686     done
687     fi
688 fi
689 fi
690
691 if [ "$daemon_ready" = "true" ]; then
692     echo "tailscaled is responsive (running as a system service)."
```

693

```

694     # Phase A: Join mesh without exit node
695     echo "Connecting host to Tailscale mesh (no exit node yet)..."
696     if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node=; then
697         HOST_ON_MESH=true
698         echo "Host is on Tailscale mesh."
699     else
700         echo -e "\033[0;33m[!] tailscale up failed even though the daemon is running.\033[0m"
701         echo " This usually means the host hasn't authenticated with your tailnet yet."
702         echo " Run this command in a terminal to authenticate:"
703         echo ""
704         echo "         sudo tailscale up"
705         echo ""
706         echo " A browser will open. Log in to your Tailscale account, then re-run toggle-linux.sh."
707     fi
708 else
709     echo -e "\033[0;31m[!] tailscaled could NOT be started automatically.\033[0m"
710     echo " Run this in a terminal to start and authenticate Tailscale:"
711     echo ""
712     echo "         sudo systemctl start tailscaled"
713     echo "         sudo tailscale up"
714     echo ""
715     echo " Then re-run toggle-linux.sh."
716 fi
717 else
718     echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
719 fi
720
721 # Phase B: Pre-flight checks + enable exit node only if safe
722 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
723     echo -e "\n"
724     echo "Pre-flight connectivity check"
725     echo ""
726
727     # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
728     # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
729     # ("direct connection not established") even though the pong was received.
730     # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
731     echo -n " [1/3] Gateway reachable via Tailscale... "
732     ping_ok=false
733     # The host tailscaled needs a few seconds to propagate the new network map and
734     # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
735     for attempt in {1..45}; do
736         if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
737             ping_ok=true
738             break
739         fi
740         sleep 1
741     done
742     if [ "$ping_ok" = "true" ]; then
743         echo "PASS"
744     else
745         echo "FAIL"
746         SKIP_EXIT_NODE=true
747     fi
748
749     # Check 2: Can we reach AdGuard via localhost (exposed port 5354)?
750     # Colima's SSH tunnel only forwards TCP (not UDP), so we use +tcp.
751     echo -n " [2/3] AdGuard DNS via localhost... "
752     if command -v dig &>/dev/null; then
753         if dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5 &>/dev/null; then
754             echo "PASS"
755         else
756             echo "FAIL"
757             SKIP_EXIT_NODE=true
758         fi
759     elif command -v nslookup &>/dev/null; then
760         if nslookup -port=5354 google.com 127.0.0.1 &>/dev/null; then
761             echo "PASS"
762         else
763             echo "FAIL"
764             SKIP_EXIT_NODE=true
765         fi
766     fi

```

```

766 else
767     echo "SKIP (dig/nslookup not available)"
768 fi
769
770 # Check 3: Does the WARP container have external internet?
771 echo -n " [3/3] WARP container internet... "
772 if docker compose exec -T warp wget -q0- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace &>/dev/null;
773 then
774     echo "PASS"
775 else
776     echo "FAIL"
777     SKIP_EXIT_NODE=true
778 fi
779
780 # Act based on check results
781 if [ "$SKIP_EXIT_NODE" != "true" ]; then
782     echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
783     if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node="$TS_IP" --exit-node-allow-lan-access=
784     true; then
785         echo "Exit node enabled."
786     else
787         echo "[Warning] Failed to set exit node."
788         SKIP_EXIT_NODE=true
789     fi
790 fi
791
792 if [ "$SKIP_EXIT_NODE" = "true" ]; then
793     echo -e "\n\033[0;33m[] Pre-flight checks failed EXIT NODE + DNS HIJACK SKIPPED.\033[0m"
794     echo " Host stays on Tailscale mesh but your internet routes through your normal connection."
795     echo " Troubleshoot with:"
796     echo "   docker compose logs warp | tail -20"
797     echo "   docker compose exec -T warp wget -q0- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
798     echo ""
799     echo " Falling back to SOCKS5 proxy for traffic routing..."
800 fi
801 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
802     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
803     SKIP_EXIT_NODE=true
804 fi
805
806 # 8. Apply DNS Hijacking.
807 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
808 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
809 # AdGuard is also available at localhost:5354 for manual configuration in apps.
810 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
811     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
812     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
813     if force_dns_to_gateway "$TS_IP"; then
814         echo "DNS hijack verified: single server = $TS_IP."
815         start_dns_watcher "$TS_IP"
816     else
817         echo "[Warning] DNS hijack could not be verified."
818         echo " If DNS is broken, run manually:"
819         echo "   networksetup -setdnsservers \"$ACTIVE_SERVICE\" $TS_IP"
820         echo "   networksetup -setsearchdomains \"$ACTIVE_SERVICE\" ts.net"
821     fi
822 elif [ -n "$TS_IP" ]; then
823     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
824     echo " Trying local DNS proxy for ad-blocking..."
825     if start_local_dns_proxy; then
826         echo " Ad-blocking active via local DNS proxy through AdGuard."
827     else
828         echo " Local DNS proxy unavailable. DNS stays at 1.1.1.1."
829         echo " AdGuard available at http://localhost:80 (port 5354 for DNS via TCP)."
830     fi
831 fi
832
833 # Enable SOCKS5 proxy for traffic routing through WARP
834 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
835 echo -e "\n Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
836 echo " (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
837 enable_socks_proxy "$ACTIVE_SERVICE"
838
839 # Verify the SOCKS5 proxy works through WARP
840 echo -n " Verifying SOCKS5 proxy routes through WARP... "
841 if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/
842 trace 2>/dev/null | grep -q "warp=on"; then
843     echo "ok (warp=on)."
844     echo " All TCP traffic now encrypted through Cloudflare WARP tunnel."
845 else
846     echo "FAILED (proxy not routing through WARP)."
847     echo " Disabling SOCKS5 proxy internet stays on direct connection."

```

```

844     disable_socks_proxy
845     echo "  SOCKS proxy available at localhost:${SOCKS_PROXY_PORT} for manual configuration."
846   fi
847 else
848     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
849 fi
850
851 # 9. Verify host connectivity
852 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
853     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
854         echo "Host is online on the Tailscale mesh."
855     else
856         echo "[Warning] Host is not responding on Tailscale."
857     fi
858 fi
859
860 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
861 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
862
863 # Prevent system from going to sleep while gateway is running
864 start_sleep_prevention
865 fi
866
867 SUCCESS_RUN=true
868
869 # Final DNS state summary
870 if [ -n "$ACTIVE_SERVICE" ]; then
871     FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr '
872 ' ' || true)
873     echo -e "\n"
874     echo -e "DNS STATE: $FINAL_DNS"
875     echo -e ""
876 fi
877
878 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
879     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
880         echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
881         echo "  The gateway gracefully fell back to yesterday's cached blocklist."
882         echo "  (Run 'docker logs rule-compiler' later to investigate which link died.)"
883     fi
884 fi
885
886 echo -e "\n"
887 read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
888 if [ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]; then
889     echo "Reversing gateway state..."
890     exec bash "$0"
891 fi
892
893 echo -e "\nYou can close this terminal window now."

```

10 scripts/recover-linux.sh

```

1  #!/bin/bash
2  # recover.sh nullexit KILL-SWITCH recovery script for Linux
3  # Fixes internet when toggle.sh leaves you stranded.
4  #
5  # This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
6  #   1. Disconnecting host Tailscale from the mesh (exit-node off)
7  #   2. Resetting DNS to default on ALL network services
8  #   3. Disabling rogue proxy settings (SOCKS, web, secure web)
9  #   4. Flushing macOS DNS cache
10 #   5. Flushing stale routing table entries
11 #   6. Stopping gateway Docker containers
12 #   7. Power-cycling Wi-Fi
13 #   8. Verifying internet is working
14 #
15 # Usage: double-click Recover-Gateway.applescript
16 #        OR ./recover.sh
17
18 set -e
19
20 # Source common formatting and helper functions
21 source "$(dirname "$0")/common.sh"
22
23 # Always add Homebrew path

```



```

24 export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"
25
26 echo ""
27 echo -e "${RED}${BOLD}${NC}"
28 echo -e "${RED}${BOLD}      Nullexit Gateway  RECOVERY MODE      ${NC}"
29 echo -e "${RED}${BOLD}${NC}"
30 echo ""
31 echo "This script will tear down the gateway and restore normal internet."
32 echo ""
33
34 # Cache sudo credentials upfront
35 # Prevents the script from hanging halfway through when it needs to run
36 # privileged commands (route flush, Wi-Fi power cycle, etc.).
37 echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
38 sudo -v
39 (
40     while true; do
41         sudo -n true 2>/dev/null
42         sleep 60
43     done
44 ) &
45 SUDO_KEEPER_PID=$!
46 trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
47
48 # 1. Disconnect host Tailscale
49 step "Disconnecting host Tailscale from mesh"
50 if command -v tailscale >> output.log 2>&1; then
51     # Check if actually connected first (with timeout tailscaled could be wedged)
52     if run_with_timeout 5 tailscale status >> output.log 2>&1; then
53         # Also explicitly reset any exit-node so it doesn't linger.
54         if run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1;
55             then
56                 ok "Exit-node preference cleared"
57             else
58                 warn "tailscale up --reset didn't respond (tailscaled may be wedged)"
59             fi
60             ts_args=""
61             if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
62             if run_with_timeout 10 tailscale down $ts_args >> output.log 2>&1; then
63                 ok "Tailscale disconnected"
64             else
65                 warn "tailscale down didn't respond"
66             fi
67             warn "tailscaled not reachable skipping (timeout after 5s)"
68         fi
69     else
70         warn "tailscale CLI not found skipping"
71     fi
72
73 # 2. Reset DNS to default on ALL network services
74 step "Resetting DNS to default on active interface"
75
76 ACTIVE_SERVICE=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
77 if [ -n "$ACTIVE_SERVICE" ]; then
78     sudo resolvectl revert "$ACTIVE_SERVICE" 2>> output.log || true
79     ok "DNS reset on $ACTIVE_SERVICE"
80 else
81     warn "Could not determine active network interface"
82 fi
83
84 # 3. Disable rogue proxy settings
85 step "Disabling proxy settings (SOCKS, web, secure web)"
86 echo " (Linux global SOCKS proxy configuration skipped.)"
87 ok "Proxy settings cleared"
88
89
90 # 4. Flush macOS DNS cache
91 step "Flushing DNS cache"
92 if command -v resolvectl >> output.log 2>&1; then
93     sudo resolvectl flush-caches 2>> output.log || true
94     ok "DNS cache flushed"
95 else
96     warn "resolvectl not found"
97 fi
98
99 # 5. Clear stale routing table
100 step "Flushing stale routing table entries"
101 sudo ip route flush cache >> output.log 2>&1 || true
102 ok "Routing table flushed"
103

```

```

104 # 6. Stop gateway Docker containers
105 step "Stopping gateway Docker containers"
106 if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
107     if [ -f "docker-compose.yml" ]; then
108         docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
109     else
110         warn "docker-compose.yml not found in current directory"
111     fi
112 else
113     warn "Docker not available skipping"
114 fi
115
116 # 6b. Stopping sleep prevention (systemd-inhibit)
117 step "Stopping sleep prevention"
118 PID_FILE="/tmp/nullexit-cafeinate.pid"
119 if [ -f "$PID_FILE" ]; then
120     INHIBIT_PID=$(cat "$PID_FILE")
121     if [ -n "$INHIBIT_PID" ] && kill -0 "$INHIBIT_PID" 2>/dev/null; then
122         kill "$INHIBIT_PID" 2>/dev/null || true
123         ok "Sleep prevention stopped (PID $INHIBIT_PID)"
124     else
125         warn "Sleep prevention process not running, cleaning up stale PID file"
126     fi
127     rm -f "$PID_FILE"
128 else
129     ok "Sleep prevention was not active"
130 fi
131
132 # 7. Power-cycle Wi-Fi
133 step "Power-cycling Wi-Fi"
134 if command -v nmcli &>/dev/null; then
135     sudo nmcli radio wifi off 2>> output.log || true
136     sleep 2
137     sudo nmcli radio wifi on 2>> output.log || true
138     ok "Wi-Fi bounced via nmcli"
139     sleep 3
140 else
141     warn "nmcli not found. Falling back to ip link..."
142     WIFI_IFACE=$(ip link | grep -E '[0-9]+: (wl[a-zA-Z0-9]+|wlan[0-9]+):' | awk -F': ' '{print $2}')
143     if [ -n "$WIFI_IFACE" ]; then
144         sudo ip link set "$WIFI_IFACE" down 2>> output.log || true
145         sleep 2
146         sudo ip link set "$WIFI_IFACE" up 2>> output.log || true
147         ok "Wi-Fi bounced via ip link (interface $WIFI_IFACE)"
148         sleep 3
149     else
150         warn "No Wi-Fi interface detected."
151     fi
152 fi
153
154 # 8. Verify internet connectivity
155 step "Verifying internet connectivity"
156
157 # Wait a moment for the network to settle
158 sleep 2
159
160 INTERNET_OK=false
161
162 # Try DNS resolution first
163 if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
164     ok "DNS resolution works (google.com via 1.1.1.1)"
165     INTERNET_OK=true
166 elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
167     ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
168     INTERNET_OK=true
169 else
170     warn "DNS resolution check failed will still try ping/curl"
171 fi
172
173 # Try actual HTTP connectivity
174 if command -v curl >> output.log 2>&1; then
175     if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
176         ok "Internet reachable via HTTP"
177         INTERNET_OK=true
178     elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
179         ok "Internet reachable via HTTP (plain)"
180         INTERNET_OK=true
181     else
182         warn "HTTP check failed"
183     fi
184 elif command -v ping >> output.log 2>&1; then

```

```

185 if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
186     ok "Internet reachable via ping"
187     INTERNET_OK=true
188 else
189     warn "Ping check failed"
190 fi
191 fi
192
193 # Summary
194 echo ""
195 echo -e "${BOLD}${NC}"
196 echo -e "${BOLD}RECOVERY SUMMARY${NC}"
197 echo -e "${BOLD}${NC}"
198
199 # Show final DNS state
200 FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr '
201 ' ' ' ' || echo "N/A")
202 echo "    DNS ($ACTIVE_SERVICE): $FINAL_DNS"
203
204 echo ""
205
206 if [ "$INTERNET_OK" = "true" ]; then
207     echo -e "    ${GREEN}${BOLD} Internet is working.${NC}"
208 else
209     echo -e "    ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
210     echo "        This might mean:"
211     echo "        - Your Wi-Fi is disconnected from the router"
212     echo "        - Tailscale is still interfering (try restarting tailscaled:"
213     echo "          systemctl restart tailscaled)"
214     echo "        - You need to re-connect to Wi-Fi in the menu bar"
215     echo "        - Try toggling Wi-Fi off/on in the menu bar"
216     echo ""
217     echo "        Or try manually:"
218     echo "        resolvectl revert $ACTIVE_SERVICE"
219     extra_args=""
220     if is_kill_switch_enabled; then extra_args="--accept-risk=lose-ssh"; fi
221     echo "        tailscale down $extra_args"
222     echo "        sudo ip route flush cache"
223     echo "        systemctl restart tailscaled"
224 fi
225
226 echo ""
227 echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
228 echo ""

```

11 scripts/setup-linux.sh

```

1 #!/bin/bash
2 # scripts/setup-linux.sh nullexit one-shot setup script (Linux)
3 # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4 set -euo pipefail
5
6 # Source common formatting and helper functions
7 source "$(dirname "$0")/common.sh"
8
9 # Run common installation logic
10 source "$(dirname "$0")/setup-common.sh"
11
12 # Compile Linux Desktop Shortcuts
13 echo -e "\n${BOLD}Creating Linux Desktop Shortcuts...${NC}"
14 DIR="$(pwd)"
15 cat <<EOF > "Toggle Gateway.desktop"
16 [Desktop Entry]
17 Version=1.0
18 Name=Toggle Gateway
19 Comment=Start or Stop Nullexit Gateway
20 Exec=bash -c "cd '$DIR' && ./scripts/toggle-linux.sh; read -p 'Press Enter to close...' dummy"
21 Icon=utilities-terminal
22 Terminal=true
23 Type=Application
24 Categories=Network;Security;
25 EOF
26
27 cat <<EOF > "Recover Gateway.desktop"
28 [Desktop Entry]
29 Version=1.0

```

```

30 Name=Recover Gateway
31 Comment=Emergency DNS and Network Recovery
32 Exec=bash -c "cd '$DIR' && ./scripts/recover-linux.sh; read -p 'Press Enter to close...' dummy"
33 Icon=utilities-terminal
34 Terminal=true
35 Type=Application
36 Categories=Network;Security;
37 EOF
38 echo "    Created 'Toggle Gateway.desktop' and 'Recover Gateway.desktop'"
39
40 # Done
41 echo ""
42 echo -e "${GREEN}${BOLD}${NC}"
43 echo -e "${GREEN}${BOLD}                Setup complete!                ${NC}"
44 echo -e "${GREEN}${BOLD}${NC}"
45 echo ""
46 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
47 echo "    https://login.tailscale.com/admin/machines"
48 echo "    Find your gateway  '...'  'Edit route settings'  enable Exit Node"
49 echo ""
50
51 if [[ -n "${TS_IP:-}" ]]; then
52     echo -e "${BOLD}AdGuard Home dashboard:${NC}    http://${TS_IP}:3000    (username: admin)"
53     echo ""
54 fi
55
56 echo -e "${BOLD}To update your block/allow rules:${NC}"
57 echo "    python3 scripts/sync-rules.py"
58 echo ""
59 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
60 echo "    Linux: double-click the 'Toggle Gateway' desktop icon!"
61 echo "    (or run ./scripts/toggle-linux.sh in terminal)"
62 echo ""

```

12 docker-compose.yml

```

1 services:
2   warp:
3     image: qmcgaw/gluetun:v3.41.1
4     container_name: warp
5     hostname: nullexit-gateway
6     cap_add:
7       - NET_ADMIN
8     devices:
9       - /dev/net/tun:/dev/net/tun
10    ports:
11      - "127.0.0.1:5354:5335/udp"    # AdGuard DNS (5335 is used by SSH on host, use 5354 instead)
12      - "127.0.0.1:5354:5335/tcp"    # AdGuard DNS (TCP)
13      - "41642:41642/udp"    # Tailscale WireGuard (direct connection to host)
14      - "127.0.0.1:1080:1080/tcp"    # SOCKS5 proxy (routed through WARP tunnel)
15    environment:
16      - TZ=America/New_York
17      - VPN_SERVICE_PROVIDER=custom
18      - VPN_TYPE=wireguard
19      - WIREGUARD_PRIVATE_KEY=${WIREGUARD_PRIVATE_KEY}
20      - WIREGUARD_PUBLIC_KEY=${WIREGUARD_PUBLIC_KEY}
21      - WIREGUARD_ADDRESSES=${WIREGUARD_ADDRESSES}
22      - WIREGUARD_ENDPOINT_IP=${WARP_ENDPOINT_1:-162.159.192.1}
23      - WIREGUARD_ENDPOINT_PORT=2408
24      # Set to 25s (WireGuard default) to beat standard 30s hardware NAT/firewall UDP timeouts
25      - WIREGUARD_PERSISTENT_KEEPAIVE_INTERVAL=25s
26      # MTU Fix (Critical): Prevent packet fragmentation from double-encryption
27      - WIREGUARD_MTU=1280
28      # Give WARP more time to establish the connection before internal restart (default is 6s which is too
29      # short)
30      - HEALTH_VPN_DURATION_INITIAL=30s
31      # Use plaintext DNS resolvers instead of DNS-over-TLS (DoT) to prevent startup failures on networks where
32      # port 853 is blocked
33      - DNS_UPSTREAM_RESOLVER_TYPE=plain
34      - DNS_UPSTREAM_PLAIN_ADDRESSES=1.1.1.1:53,8.8.8.8:53
35      # Allow local network traffic to bypass WARP so Tailscale can establish fast, direct peer-to-peer
36      # connections (no DERP relays) on the LAN
37      - FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8
38      # Built-in DNS blocking for ads, malicious domains, and tracking
39      # NOTE: This is the massive "Hagezi-style" built-in blocklist.
40      # You can turn these to 'on' for maximum security if you have spare RAM

```

```

38     # or are running this on a cloud exit node with adequate autonomous resources (requires approx 1-2GB of
    RAM).
39     # Disabled intentionally: AdGuard Home provides the DNS filtering layer instead.
40     # If AdGuard fails, there is no fallback filtering, but the healthcheck
41     # dependency chain ensures Tailscale won't come up if AdGuard is down.
42     - BLOCK_MALICIOUS=off
43     - BLOCK_SURVEILLANCE=off
44     - BLOCK_ADS=off
45 dns:
46     - 1.1.1.1
47 sysctls:
48     - net.ipv4.ip_forward=1
49 volumes:
50     - ./post-rules.txt:/iptables/post-rules.txt:ro
51 restart: unless-stopped
52 healthcheck:
53     test: ["CMD-SHELL", "/gluetun-entrypoint healthcheck"]
54     interval: 2s
55     timeout: 5s
56     retries: 15
57     start_period: 60s
58 logging:
59     driver: "json-file"
60     options:
61         max-size: "10m"
62         max-file: "3"
63
64 tailscale:
65     image: tailscale/tailscale:v1.98.4
66     container_name: tailscale
67     network_mode: service:warp
68     depends_on:
69         warp:
70             condition: service_healthy
71         adguardhome:
72             condition: service_healthy
73 cap_add:
74     - NET_ADMIN
75     - NET_RAW
76     - SYS_MODULE
77 sysctls:
78     - net.ipv4.ip_forward=1
79     - net.ipv6.conf.all.forwarding=1
80 environment:
81     - TZ=America/New_York
82     - TS_EXTRA_ARGS=--advertise-exit-node
83     - PORT=41642
84     - TS_USERSPACE=false
85     - TS_AUTHKEY=${TS_AUTHKEY}
86     - TS_STATE_DIR=/var/lib/tailscale
87     # (Note: For headless/cloud deployments, you can bypass this footgun by
88     # using Tailscale Ephemeral Keys which auto-clean themselves).
89     - TS_AUTH_ONCE=true
90     # MTU Optimization: Force Tailscale packets to be smaller than WARP's 1280 MTU
91     # to prevent WireGuard-in-WireGuard fragmentation, completely eliminating bufferbloat
92     - TS_DEBUG_MTU=1200
93     # Enable Tailscale's built-in SOCKS5 proxy to route out of the WARP tunnel
94     - TS_SOCKS5_SERVER=:1080
95 devices:
96     - /dev/net/tun:/dev/net/tun
97 volumes:
98     - tailscale_state:/var/lib/tailscale
99 restart: unless-stopped
100 logging:
101     driver: "json-file"
102     options:
103         max-size: "10m"
104         max-file: "3"
105
106 routing-fix:
107     image: nullexit-routing-fix:v1.0.0
108     build:
109         context: ./scripts
110         dockerfile: Dockerfile.routing-fix
111     container_name: routing-fix
112     network_mode: service:warp
113     # NOTE: Deliberately NOT sharing tailscale's PID namespace the shared
114     # namespace caused routing-fix to be killed whenever tailscale restarted
115     # (e.g. during gluetun health-check flaps), which dropped all routing
116     # table changes and left traffic leaking through eth0 instead of tun0.
117 cap_add:

```

```

118     - NET_ADMIN
119     - SYSLOG
120 depends_on:
121     warp:
122         condition: service_healthy
123         rule-compiler:
124             condition: service_completed_successfully
125 environment:
126     - TZ=America/New_York
127     - WARP_ENDPOINT=${WARP_ENDPOINT_1:-162.159.192.1}
128     - BLOCKED_COUNTRIES=${BLOCKED_COUNTRIES}
129     - TAILSCALE_ALLOW_LAN_P2P=${TAILSCALE_ALLOW_LAN_P2P:-false}
130 volumes:
131     - ./scripts/routing-fix.sh:/routing-fix.sh:ro
132     # SECURITY: ./adguard/work/ is bind-mounted into this container.
133     # Never write to it from the host or another container. The
134     # single allowed writer is `python3 sync-rules.py` manual
135     # edits corrupt the AdGuard cache + BoltDB session store.
136     - ./adguard/work/userfilters:/userfilters:ro
137     - ./adguard/work:/adguard_work:ro
138     - ./app
139 restart: unless-stopped
140 stop_grace_period: 1s
141 logging:
142     driver: "json-file"
143     options:
144         max-size: "1m"
145         max-file: "1"
146
147 adguardhome:
148     # WARNING: AdGuard is pre-configured with hardcoded credentials (admin/nullexit).
149     # This is fine for a local Mac running behind a NAT, but if you are deploying
150     # this on a public cloud VPS with port 80/3000 exposed, change this immediately!
151     image: adguard/adguardhome:v0.107.77
152     container_name: adguardhome
153     network_mode: service:warp
154     depends_on:
155         warp:
156             condition: service_healthy
157             rule-compiler:
158                 condition: service_completed_successfully
159     environment:
160         - TZ=America/New_York
161     healthcheck:
162         test: ["CMD-SHELL", "nc -z 127.0.0.1 5335 || exit 1"]
163         interval: 2s
164         timeout: 5s
165         retries: 10
166     volumes:
167         # SECURITY: ./adguard/work/ is bind-mounted into this container.
168         # Never write to it from the host or another container. The
169         # single allowed writer is `python3 sync-rules.py` manual
170         # edits corrupt the AdGuard cache + BoltDB session store.
171         - ./adguard/work:/opt/adguardhome/work
172         - ./adguard/conf:/opt/adguardhome/conf
173     restart: unless-stopped
174     stop_grace_period: 2s
175     logging:
176         driver: "json-file"
177         options:
178             max-size: "10m"
179             max-file: "3"
180
181 rule-compiler:
182     image: nullexit-rule-compiler:v1.0.0
183     build:
184         context: ./scripts
185         dockerfile: Dockerfile.rule-compiler
186     container_name: rule-compiler
187     volumes:
188         - ./app
189
190 volumes:
191     tailscale_state:
192
193 networks:
194     default:
195         ipam:
196             config:
197                 - subnet: 10.200.1.0/24

```

13 scripts/common.sh

```
1 #!/bin/bash
2 # common.sh shared bash utilities for nullexit scripts
3
4 # Formatting
5 RED='\033[0;31m'
6 GREEN='\033[0;32m'
7 YELLOW='\033[1;33m'
8 BLUE='\033[0;34m'
9 BOLD='\033[1m'
10 NC='\033[0m'
11
12 # Constants & Environment
13 export MARKER_FILE="/tmp/nullexit-gateway-active.marker"
14 export PID_CAFFEINATE="/tmp/nullexit-cafeinate.pid"
15 export PID_DNS_WATCHER="/tmp/nullexit-dns-watcher.pid"
16 export DEFAULT_WARP_ENDPOINT_1="162.159.192.1"
17 export DEFAULT_WARP_ENDPOINT_2="162.159.193.1"
18
19 setup_standard_path() {
20     export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
21 }
22
23 step() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ] ${BLUE}${BOLD} ${*}${NC}"; }
24 ok() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${GREEN} ${*}${NC}"; }
25 warn() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${YELLOW} ${*}${NC}"; }
26 fail() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${RED} ${*}${NC}"; }
27 die() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ] ${RED} ${*}${NC}\n"; exit 1; }
28
29 # Helper Functions
30
31 # Pure-bash timeout (no dependency on GNU coreutils' timeout)
32 # Runs a command with a safety cutoff so a wedged daemon can't hang the script.
33 run_with_timeout() {
34     local timeout_sec="$1"
35     shift
36     if [ $# -eq 0 ]; then return 1; fi
37
38     "$@" &
39     local cmd_pid=$!
40     CURRENT_BG_PID="$cmd_pid"
41
42     (
43         sleep "$timeout_sec"
44         if kill -0 "$cmd_pid" 2>> output.log; then
45             echo -e "\n[Timeout] Command '$*' exceeded $timeout_sec seconds. Terminating..." >&2
46             kill -15 "$cmd_pid" 2>> output.log || true
47             sleep 2
48             if kill -0 "$cmd_pid" 2>> output.log; then
49                 kill -9 "$cmd_pid" 2>> output.log || true
50             fi
51         fi
52     ) &
53     local watcher_pid=$!
54
55     # Disable set -e temporarily to capture exit status of wait
56     set +e
57     wait "$cmd_pid" 2>> output.log
58     local exit_status=$?
59     set -e
60
61     # Kill the watcher since the command finished
62     kill "$watcher_pid" 2>> output.log && wait "$watcher_pid" 2>> output.log || true
63
64     CURRENT_BG_PID=""
65     return "$exit_status"
66 }
67
68 # Check if the gateway is active (either containers are running, or host DNS is hijacked)
69 is_gateway_active() {
70     # 1. Check if containers are running (suppress stderr to avoid errors if docker is down)
71     if run_with_timeout 15 docker compose ps --status running 2>/dev/null | grep -q 'warp'; then
72         echo "[ $(date '+%Y-%m-%d %H:%M:%S') ] is_gateway_active: TRUE (warp container is running)" >> "$LOG_FILE"
73         return 0
74     fi
75
76     # 2. Check if host DNS was hijacked (not 1.1.1.1 and not default/empty)
77     local current_dns=""
```



```

78 local active_svc=""
79 if [[ "$OSTYPE" == "darwin"* ]]; then
80     active_svc=$(get_active_service)
81     current_dns=$(networksetup -getdnsservers "$active_svc" 2>> output.log || true)
82 else
83     current_dns=$(resolvectl dns 2>/dev/null | awk '/Global|Link/ {for(i=4;i<=NF;i++) print $i}' | head -n1)
84 fi
85
86 if [[ -n "$current_dns" && "$current_dns" != "1.1.1.1" && ! "$current_dns" =~ "There aren't any DNS Servers"
87 ]]; then
88     echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (DNS was hijacked: $current_dns)" >> "$LOG_FILE"
89     return 0
90 fi
91
92 # 3. Check if SOCKS proxy is enabled (macOS only)
93 if [[ "$OSTYPE" == "darwin"* ]]; then
94     local socks_proxy
95     socks_proxy=$(networksetup -getsocksfirewallproxy "$active_svc" 2>> output.log || true)
96     if echo "$socks_proxy" | grep -q "Enabled: Yes"; then
97         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (SOCKS proxy enabled)" >> "$LOG_FILE"
98         return 0
99     fi
100 fi
101
102 echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: FALSE (Containers down, DNS clean, SOCKS disabled)"
103 >> "$LOG_FILE"
104 return 1
105 }
106
107 # Reads an environment variable from a file (default: .env), strips quotes and whitespace
108 read_env_var() {
109     local var_name="$1"
110     local env_file="${2:-.env}"
111     grep -E "~${var_name}=" "$env_file" 2>/dev/null | cut -d '=' -f2- | tr -d "\"\\'" | sed -e 's/[[:space:]]*//'\n' -e 's/[[:space:]]*//'\n' || echo ""
112 }
113
114 # Load KILL_SWITCH variable from .env (defaults to false if not found/empty)
115 export KILL_SWITCH
116 KILL_SWITCH=$(read_env_var "KILL_SWITCH" 2>/dev/null | tr '[:upper:]' '[:lower:]')
117 if [ -z "$KILL_SWITCH" ]; then
118     KILL_SWITCH="false"
119 fi
120
121 # Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
122 get_active_service() {
123     local iface
124     iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
125
126     # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
127     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
128         for i in en0 en1 en2 en3; do
129             if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "
130                 inet "; then
131                 iface="$i"
132                 break
133             fi
134         done
135     fi
136
137     # Default fallback if still empty or not enX
138     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
139         iface="en0"
140     fi
141
142     # Map interface (e.g., en0) to service name (e.g., Wi-Fi)
143     local service
144     service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B 1 "Device: $iface" | head -n 1 | sed
145         -E 's/^\([0-9\+]+\)\s* //' )
146
147     if [ -n "$service" ]; then
148         echo "$service"
149     else
150         echo "Wi-Fi"
151     fi
152 }
153
154 get_en0_service() {
155     local service

```

```

152 service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B1 "Device: en0" | head -1 | sed -E 's
    /\^[0-9\.*]+\)/' || true)
153 if [ -z "$service" ]; then
154     echo "Wi-Fi"
155 else
156     echo "$service"
157 fi
158 }
159
160 get_warp_endpoint_1() { read_env_var "WARP_ENDPOINT_1" ".env" | grep -Eo '[0-9\.]+' || echo "
    $DEFAULT_WARP_ENDPOINT_1"; }
161 get_warp_endpoint_2() { read_env_var "WARP_ENDPOINT_2" ".env" | grep -Eo '[0-9\.]+' || echo "
    $DEFAULT_WARP_ENDPOINT_2"; }
162
163 restart_tailscaled_daemon() {
164     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
165     echo " [Tailscale Recovery] Attempting to restart tailscaled service..."
166
167     if run_with_timeout 15 brew services restart tailscale >> output.log 2>&1; then
168         echo " [Tailscale Recovery] Successfully restarted tailscaled (user service)."
169     elif run_with_timeout 15 sudo -n brew services restart tailscale >> output.log 2>&1; then
170         echo " [Tailscale Recovery] Successfully restarted tailscaled (system service)."
171     else
172         echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
173     fi
174     sleep 3
175 }
176
177 disconnect_tailscale_host() {
178     local ts_bin="${1:-tailscale}"
179     if command -v "$ts_bin" >> output.log 2>&1; then
180         echo " Disconnecting host Tailscale from mesh..."
181
182         # Check if actually connected first (with timeout tailscaled could be wedged)
183         local status_ok=false
184         if run_with_timeout 5 "$ts_bin" status >> output.log 2>&1; then
185             status_ok=true
186         else
187             local status_exit=$?
188             # If it was a quick exit code 1 (disconnected), it is still reachable
189             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
190                 status_ok=true
191             fi
192         fi
193
194         if [ "$status_ok" = "false" ]; then
195             restart_tailscaled_daemon
196         fi
197
198         # Explicitly reset any exit-node so it doesn't linger.
199         "$ts_bin" up >> output.log 2>&1 || true
200         if run_with_timeout 10 "$ts_bin" set --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1; then
201             echo " [] Exit-node preference cleared."
202         else
203             echo " [!] tailscale up didn't respond (tailscaled may be wedged)"
204         fi
205
206         local ts_args=""
207         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
208         if run_with_timeout 10 "$ts_bin" down $ts_args >> output.log 2>&1; then
209             echo " [] Tailscale disconnected."
210         else
211             echo " [!] tailscale down didn't respond"
212         fi
213     else
214         echo " [!] tailscale CLI not found skipping"
215     fi
216 }
217
218 stop_pidfile_daemon() {
219     local pid_file="$1"
220     local label="$2"
221     if [ -f "$pid_file" ]; then
222         local pid
223         pid=$(cat "$pid_file")
224         if kill -0 "$pid" 2>/dev/null; then
225             echo " Stopping $label (PID $pid)..."
226             sudo -n kill "$pid" 2>> output.log || kill "$pid" 2>> output.log || true
227             sleep 0.5
228             if kill -0 "$pid" 2>/dev/null; then
229                 sudo -n kill -9 "$pid" 2>> output.log || kill -9 "$pid" 2>> output.log || true

```

```

230     fi
231     fi
232     rm -f "$pid_file"
233 fi
234 }
235
236 disable_all_proxies() {
237     local svc="$1"
238     if [ -n "$svc" ]; then
239         sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
240         sudo -n networksetup -setwebproxystate "$svc" off 2>> output.log || true
241         sudo -n networksetup -setsecurewebproxystate "$svc" off 2>> output.log || true
242     fi
243 }
244
245 flush_dns_cache() {
246     if [[ "$OSTYPE" == "darwin"* ]]; then
247         sudo -n dscacheutil -flushcache 2>> output.log || true
248         sudo -n killall -HUP mDNSResponder 2>> output.log || true
249     else
250         resolvectl flush-caches 2>> output.log || true
251     fi
252 }
253
254 wait_for_dhcp_settle() {
255     # Resolve physical interface (Wi-Fi or Ethernet)
256     local physical_iface
257     physical_iface=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{
258         getline; print $2; exit}')
259     [ -z "$physical_iface" ] && physical_iface="en0"
260
261     echo -n "Waiting for physical network interface ($physical_iface) DHCP lease to settle..."
262     local settled=false
263     for attempt in {1..60}; do
264         PHYSICAL_GW=$(ipconfig getpacket "$physical_iface" 2>> output.log | awk -F'[]' '/router/{print $2}')
265         local local_ip
266         local_ip=$(ifconfig "$physical_iface" 2>/dev/null | awk '/inet/{print $2}')
267
268         if [ -n "$PHYSICAL_GW" ] && [[ "$PHYSICAL_GW" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
269             [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
270             [[ "$local_ip" != "169.254.*" ]]; then
271             echo "settled (Interface IP: $local_ip, Router IP: $PHYSICAL_GW)."
272             settled=true
273             break
274         fi
275
276         if [ "$attempt" -gt 15 ] && [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
277             [[ "$local_ip" != "169.254.*" ]]; then
278             local fallback_gw
279             fallback_gw=$(route get default 2>> output.log | awk '/gateway:/{print $2}')
280             if [ -n "$fallback_gw" ] && [[ "$fallback_gw" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
281                 PHYSICAL_GW="$fallback_gw"
282                 fi
283                 echo "static/settled detected (Interface IP: $local_ip, Gateway IP: ${PHYSICAL_GW:-unknown})."
284                 settled=true
285                 break
286             fi
287             echo -n "."
288             sleep 0.5
289         done
290         if [ "$settled" = "false" ]; then
291             echo "timed out or no active link. Proceeding anyway."
292         fi
293     }
294
295     reset_sharing_services() {
296         if [[ "$OSTYPE" == "darwin"* ]]; then
297             sudo -n killall sharingd rapportd 2>> output.log || true
298         fi
299     }
300
301     read_adguard_ip() {
302         if [ -f "ADGUARD_IP.txt" ]; then
303             tr -d '\r' < "ADGUARD_IP.txt" | awk 'NR==1{print $1;exit}'
304         fi
305     }
306
307     is_kill_switch_enabled() {
308         [ "$KILL_SWITCH" = "true" ]
309     }

```

```

309 add_warp_bypass_routes() {
310     local target="${1:-}"
311     local ep1
312     ep1=$(get_warp_endpoint_1)
313     local ep2
314     ep2=$(get_warp_endpoint_2)
315
316     if [[ "$OSTYPE" == "darwin"* ]]; then
317         local routing_arg=""
318         local msg_via=""
319         if [ -n "$target" ]; then
320             if [[ "$target" =~ ^en[0-9]+$ || "$target" == "bridge"* ]]; then
321                 routing_arg="-interface $target"
322                 msg_via="interface $target"
323             else
324                 routing_arg="$target"
325                 msg_via="gateway $target"
326             fi
327         else
328             local gateway_ip
329             gateway_ip=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
330             if [ -z "$gateway_ip" ]; then
331                 local iface
332                 iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
333                 if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
334                     iface="en0"
335                 fi
336                 routing_arg="-interface $iface"
337                 msg_via="interface $iface"
338             else
339                 routing_arg="$gateway_ip"
340                 msg_via="gateway $gateway_ip"
341             fi
342         fi
343
344         echo -e "\nAdding host bypass routes for Cloudflare WARP endpoints via $msg_via..."
345         sudo -n route delete -host "$ep1" 2>/dev/null || true
346         sudo -n route delete -host "$ep2" 2>/dev/null || true
347         sudo -n route add -host "$ep1" $routing_arg >> output.log 2>&1 || true
348         sudo -n route add -host "$ep2" $routing_arg >> output.log 2>&1 || true
349
350         echo -e "Adding host bypass routes for Tailscale control plane via $msg_via..."
351         sudo -n route delete -net 192.200.0.0/24 2>/dev/null || true
352         sudo -n route add -net 192.200.0.0/24 $routing_arg >> output.log 2>&1 || true
353
354         echo -e "Adding host bypass routes for Tailscale DERP relays via $msg_via..."
355         local derp_ips
356         derp_ips=$(curl -s --connect-timeout 5 https://login.tailscale.com/derpmap/default | grep -oE '"IPv4"[[:space:]]*:[[:space:]]*"([0-9.]+)"' | cut -d'"' -f4 | grep -E '^([0-9]+\.[0-9]+\.[0-9]+\.[0-9])+$' || true)
357         if [ -n "$derp_ips" ]; then
358             echo "$derp_ips" > /tmp/nullexit-derp-ips.txt
359             for ip in $derp_ips; do
360                 sudo -n route delete -host "$ip" 2>/dev/null || true
361                 sudo -n route add -host "$ip" $routing_arg >> output.log 2>&1 || true
362             done
363         fi
364     else
365         local gateway_ip="$target"
366         if [ -z "$gateway_ip" ]; then
367             gateway_ip=$(ip route show default 2>/dev/null | awk '/default/ {print $3}')
368         fi
369         if [ -n "$gateway_ip" ]; then
370             sudo ip route add "$ep1" via "$gateway_ip" >> output.log 2>&1 || true
371             sudo ip route add "$ep2" via "$gateway_ip" >> output.log 2>&1 || true
372         fi
373     fi
374 }
375
376 remove_warp_bypass_routes() {
377     local ep1
378     ep1=$(get_warp_endpoint_1)
379     local ep2
380     ep2=$(get_warp_endpoint_2)
381
382     if [[ "$OSTYPE" == "darwin"* ]]; then
383         echo -e "\nRemoving host bypass routes for Cloudflare WARP endpoints..."
384         sudo -n route delete -host "$ep1" >> output.log 2>&1 || true
385         sudo -n route delete -host "$ep2" >> output.log 2>&1 || true
386         echo -e "Removing host bypass routes for Tailscale control plane..."
387         sudo -n route delete -net 192.200.0.0/24 >> output.log 2>&1 || true
388         if [ -f /tmp/nullexit-derp-ips.txt ]; then

```

```

389     echo -e "Removing host bypass routes for Tailscale DERP relays..."
390     while read -r ip; do
391         sudo -n route delete -host "$ip" >> output.log 2>&1 || true
392     done < /tmp/nullexit-derp-ips.txt
393     rm -f /tmp/nullexit-derp-ips.txt
394     fi
395 else
396     sudo ip route del "$ep1" >> output.log 2>&1 || true
397     sudo ip route del "$ep2" >> output.log 2>&1 || true
398 fi
399 }
400
401 bounce_wifi_interfaces() {
402     if [[ "$OSTYPE" == "darwin"* ]]; then
403         local wifi_port
404         wifi_port=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2}')
405         if [ -n "$wifi_port" ]; then
406             sudo -n networksetup -setairportpower "$wifi_port" off 2>> output.log || true
407             sleep 2
408             sudo -n networksetup -setairportpower "$wifi_port" on 2>> output.log || true
409             echo "Wi-Fi bounced (interface $wifi_port)"
410             sleep 3
411         else
412             echo "Could not detect Wi-Fi interface. Bouncing fallback interfaces..."
413             set +e
414             for iface in en0 en1 en2 en3 en4 en5; do
415                 if ifconfig "$iface" >> output.log 2>&1; then
416                     sudo -n ifconfig "$iface" down 2>> output.log || true
417                     sudo -n ifconfig "$iface" up 2>> output.log || true
418                 fi
419             done
420             set -e
421             echo "Fallback interfaces bounced"
422         fi
423     fi
424 }
425
426 get_active_interface() {
427     local iface
428     iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
429
430     # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
431     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
432         for i in en0 en1 en2 en3; do
433             if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "
434                 inet "; then
435                 iface="$i"
436                 break
437             fi
438         done
439     fi
440
441     # Default fallback if still empty or not enX
442     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
443         iface="en0"
444     fi
445     echo "$iface"
446 }
447
448 enable_killswitch() {
449     if [[ "$OSTYPE" == "darwin"* ]]; then
450         if ! is_kill_switch_enabled; then
451             return 0
452         fi
453
454         local ext_if
455         ext_if=$(get_active_interface)
456         local ep1
457         ep1=$(get_warp_endpoint_1)
458         local ep2
459         ep2=$(get_warp_endpoint_2)
460         local pf_conf_path
461         pf_conf_path="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/pf.conf"
462
463         echo -e "\nEnabling macOS Packet Filter (PF) Kill-Switch on $ext_if..."
464
465         # Enable PF and capture the reference token
466         local token_out
467         token_out=$(sudo -n pfctl -E 2>> output.log || true)

```

```

468 local token
469 token=$(echo "$token_out" | awk '/Token/{print $NF}')
470 if [ -n "$token" ]; then
471     echo "$token" > /tmp/nullexit-pf-token.txt
472     echo " [] PF enabled with token $token."
473 else
474     echo " [!] WARNING: Could not capture PF reference token (may already be enabled or sudo failed)."
475 fi
476
477 # Load the rules into the anchor
478 if sudo -n pfctl -D "ext_if=$ext_if" -a com.apple/nullexit -f "$pf_conf_path" >> output.log 2>&1; then
479     echo " [] Ruleset loaded into anchor com.apple/nullexit."
480 else
481     echo " [!] ERROR: Failed to load ruleset into anchor."
482 fi
483
484 # Populate tables in the anchor
485 sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T flush >> output.log 2>&1 || true
486 sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$sep1" >> output.log 2>&1 || true
487 sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$sep2" >> output.log 2>&1 || true
488
489 if [ -f /tmp/nullexit-derp-ips.txt ]; then
490     local derp_ips
491     derp_ips=$(tr '\n' ' ' < /tmp/nullexit-derp-ips.txt)
492     if [ -n "$derp_ips" ]; then
493         sudo -n pfctl -a com.apple/nullexit -t derp_relays -T flush >> output.log 2>&1 || true
494         sudo -n pfctl -a com.apple/nullexit -t derp_relays -T add $derp_ips >> output.log 2>&1 || true
495     fi
496 fi
497 echo " [] Kill-switch tables populated."
498 fi
499 }
500
501 disable_killswitch() {
502     if [[ "$OSTYPE" == "darwin"* ]]; then
503         if ! is_kill_switch_enabled; then
504             return 0
505         fi
506         echo -e "\nDisabling macOS PF Kill-Switch..."
507
508         # Flush rules from anchor com.apple/nullexit
509         sudo -n pfctl -a com.apple/nullexit -F all >> output.log 2>&1 || true
510
511         # Release the enable reference if token is present
512         if [ -f /tmp/nullexit-pf-token.txt ]; then
513             local token
514             token=$(cat /tmp/nullexit-pf-token.txt)
515             if [ -n "$token" ]; then
516                 sudo -n pfctl -X "$token" >> output.log 2>&1 || true
517                 echo " [] Released PF enable reference token $token."
518             fi
519             rm -f /tmp/nullexit-pf-token.txt
520         else
521             echo " [] Rules flushed from anchor com.apple/nullexit."
522         fi
523     fi
524 }
525
526
527 write_host_ips() {
528     # Gather all active IPv4 addresses on the host and write to .host_ips
529     # routing-fix.sh will read this file to explicitly whitelist them for direct Tailscale P2P
530     local repo_dir
531     repo_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
532
533     if [[ "$OSTYPE" == "darwin"* ]]; then
534         ifconfig | awk '/inet /{print $2}' | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
535     else
536         ip -4 addr show 2>/dev/null | awk '/inet /{print $2}' | cut -d/ -f1 | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
537     fi
538 }

```

14 scripts/watcher.sh

```
1 #!/bin/bash
```

```

2 # scripts/watcher.sh nullexit post-wake / post-roam recovery watcher
3 #
4 # Long-running daemon. Launched by launchd as a LaunchAgent at
5 # ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
6 # (label com.nullexit.wake-recovery).
7 #
8 # Two independent listeners run as backgrounded pipelines:
9 #
10 # (1) WAKE listener `log stream` live-follows the macOS unified log for
11 # power-management events (lid closewake, manual wake, DarkWake from
12 # clamshell). Each event triggers run_recover (with a global debounce so
13 # a single wake that emits 2-3 log lines only causes ONE post-wake run).
14 #
15 # (2) NETWORK listener `scutil n.watch` on `State:/Network/Global/IPv4`
16 # fires on every Wi-Fi roam, ethernet swap, hotspot join, captive-portal
17 # re-bind, VPN up/down, etc. Same debounce.
18 #
19 # Both listeners call run_recover, which:
20 # - checks /tmp/nullexit-gateway-active.marker (only act when toggle.sh left
21 # the gateway up)
22 # - debounces (10s default) so multiple events don't pile up
23 # - shells out to `bash <repo>/recover.sh --post-wake` directly (no GUI auth prompt needed due to NOPASSWD
24 # sudoers)
25 #
26 # See devref.md 10.29 for the full Apple power management primer and the
27 # rationale behind using `log stream` + `scutil n.watch` from a shell daemon.
28 #
29 # NOTE: errexit (`set -e`) is intentionally NOT enabled. This is a long-running
30 # daemon, not a discrete-step script, and errexit actively breaks the reconnect
31 # loops below: any failed pipe (log stream on rotation, scutil on state churn)
32 # would kill the outer subshell before `echo "reconnecting..."; sleep 5;` runs,
33 # AND the parent's final `wait` would propagate a non-zero child exit and kill
34 # the entire watcher. Every error path is already wrapped in `|| true` /
35 # `2>/dev/null` fallbacks.
36 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
37 source "$SCRIPT_DIR/common.sh"
38 setup_standard_path
39 LOG="$SCRIPT_DIR/./output.log"
40 # Resolve our own directory; recover.sh lives one level up at the repo root.
41 # This makes the daemon install-agnostic no need to template the path
42 # in launchd, no need to keep a deploy-specific hardcoded location in sync.
43 RECOVER="$SCRIPT_DIR/./recover.sh"
44 MARKER="$MARKER_FILE"
45 DEBOUNCE_FILE="/tmp/nullexit-watcher.last-recovery"
46 DEBOUNCE_SECONDS="${NULLEXIT_DEBOUNCE_SECONDS:-10}"
47
48 exec >> "$LOG" 2>&1
49
50 # Single-instance lock
51 # Without this, repeated lid-closewake cycles under macOS Sonoma+ accumulate
52 # orphan watcher.sh processes: launchd suspends on sleep and SIGCONT-resumes
53 # on wake, and KeepAlive.Crashed=true caused relaunch on any non-zero exit;
54 # the listener grandchildren (log stream | while ... and (echo n.add ...;
55 # sleep 86400) | scutil | while ...) stay alive because pipe producers don't
56 # reliably respond to plain SIGTERM if the consumer is in a half-closed state.
57 #
58 # We solve it with a PID-file lock (POSIX-portable; macOS does NOT ship `flock`).
59 # The trap below removes the lock file on every exit so a stale lock never
60 # outlives a crashed watcher. On launch: read the existing PID, check it is
61 # alive AND is actually a `scripts/watcher.sh` process; if so, exit 0; else
62 # overwrite with our PID and proceed. The check-then-write window has a TOCTOU
63 # race but is microseconds wide and only matters for hand-launched duplicate
64 # test invocations; production is single-launch via launchctl load -w.
65 LOCK_FILE="/tmp/nullexit-watcher.lock"
66 LOCK_PID=""
67 if [ -e "$LOCK_FILE" ]; then
68     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
69     if [ -n "$LOCK_PID" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
70         # Verify the holder is actually a scripts/watcher.sh process (not some
71         # unrelated process that got the recycled PID). The exact-launchd pattern
72         # `bash <abs-path>/scripts/watcher.sh` is hard to accidentally match with
73         # a `grep scripts/watcher.sh` invocation because we anchor on `^bash`
74         # followed by whitespace and end-of-line on the script path.
75         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -qE "(^[[[:space:]])bash[[:space:]]+.*scripts/watcher\\.sh$"; then
76             echo "[$(date -u +%FT%TZ)] watcher.sh: another instance holds $LOCK_FILE (pid=$LOCK_PID), exiting"
77             exit 0
78         fi
79     fi
80 fi

```



```

81 echo $$ > "$LOCK_FILE"
82
83 echo "[$(date -u +%FT%TZ)] watcher.sh start (pid=$$ ppid=$PPID user=$(id -un) lock=acquired)"
84
85 # Treat launchd-resume as a wake signal. Apple's launchd SUSPENDS LaunchAgents
86 # during system sleep and restores them on wake; during that suspended gap,
87 # macOS's 'log stream' cannot catch the wake event that prompted the resume.
88 # The next thing that happens after wake IS our own startup, so we always
89 # fire one post-wake run unconditionally here. The marker check + debounce
90 # in run_recover() make this idempotent: if the gateway isn't up or we just
91 # debounced, it no-ops. Without this, the FIRST wake-after-install goes
92 # unseen and the watcher appears broken until the SECOND wake.
93 # Helper: run recover.sh --post-wake if the gateway is active
94 run_recover() {
95     local why="$1"
96     if [ ! -f "$MARKER" ]; then
97         echo "[$(date -u +%FT%TZ)] $why no $MARKER, skip"
98         return 0
99     fi
100     local now last
101     now=$(date +%s)
102     last=$(cat "$DEBOUNCE_FILE" 2>/dev/null || echo 0)
103     if [ "$((now - last))" -lt "$DEBOUNCE_SECONDS" ]; then
104         echo "[$(date -u +%FT%TZ)] $why debounced ($(now - last)s since last, threshold ${DEBOUNCE_SECONDS}s)"
105         return 0
106     fi
107     echo "$now" > "$DEBOUNCE_FILE"
108     echo "[$(date -u +%FT%TZ)] $why bash $RECOVER --post-wake"
109     bash "$RECOVER" --post-wake
110     local exit_code=$?
111     echo "$(date +%s)" > "$DEBOUNCE_FILE"
112     echo "[$(date -u +%FT%TZ)] $why exit=$exit_code (now=$(date +%s))"
113     return $exit_code
114 }
115
116 if [ -f "$MARKER" ]; then
117     run_recover "initial post-wake (launchd resume = wake signal)"
118 fi
119
120 # LAN P2P Auto-Detection
121 # Detects whether the current network allows safe Tailscale LAN P2P by:
122 # 1. Checking WPA2-Enterprise (802.1x) via airport -I always forces P2P=false
123 #    because enterprise networks almost universally have AP Client Isolation.
124 # 2. AP Isolation probe sends a broadcast ping and checks if ANY LAN host responds.
125 #    If no LAN hosts respond on a non-empty network, AP Isolation is active.
126 # Writes 'true' or 'false' to .lan_p2p_detected in the repo root.
127 # routing-fix.sh reads this file dynamically every 30s loop iteration.
128 P2P_OVERRIDE_FILE="$SCRIPT_DIR/../../lan_p2p_detected"
129
130
131 detect_lan_p2p_mode() {
132     local reason="unknown" allow="false"
133
134     # Step 1: Detect Wi-Fi security type via system_profiler (airport binary removed in macOS 15+)
135     # SPAirPortDataType lists all known networks; the first "Security:" line is the current association.
136     local security
137     security=$(system_profiler SPAirPortDataType 2>/dev/null \
138         | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}')
139
140     if [ -z "$security" ]; then
141         # Not on Wi-Fi or system_profiler unavailable fail safe
142         reason="no-wifi" allow="false"
143     elif echo "$security" | grep -qi "Enterprise"; then
144         # WPA2/WPA3 Enterprise = 802.1x always AP isolated
145         reason="802.1x-enterprise" allow="false"
146     elif echo "$security" | grep -qi "Personal\|None\|Open"; then
147         # WPA2/WPA3 Personal or open probe for AP isolation
148         local gw
149         gw=$(route -n get 1.1.1.1 2>/dev/null | awk '/gateway:/{print $2}')
150         if [ -z "$gw" ]; then
151             reason="no-default-route" allow="false"
152         else
153             local responses
154             responses=$(ping -c 3 -t 1 -q "$gw" 2>/dev/null | awk '/packets transmitted/{print $4}')
155             if [ "${responses:-0}" -gt 0 ]; then
156                 reason="wpa-personal-lan-reachable" allow="true"
157             else
158                 reason="wpa-personal-ap-isolated" allow="false"
159             fi
160         fi
161     else

```

```

162     # Unknown security type fail safe
163     reason="unknown-security-type" allow="false"
164 fi
165
166 echo "[$(date -u +%FT%TZ)] P2P detect: security='${security:-none}' allow=${allow} (reason: ${reason})"
167 echo "$allow" > "$P2P_OVERRIDE_FILE"
168 write_host_ips
169 }
170
171 # Run once on startup
172 detect_lan_p2p_mode
173
174 # Listener 1: WAKE events via unified log
175 # Predicate picks up:
176 # * "didWake"           regular kernel wake events (lid open, RTC alarm)
177 # * "Wake from Sleep"   powerd-driven normal wake
178 # * "DarkWake"          clamshell-display wake that didn't fully wake the
179 #                        Mac (relevant when lid opens during display sleep)
180 # * "Waking from"       generic catch-all for variations in newer macOS
181 # `[c]` makes the match case-insensitive (the unified log uses mixed-case
182 # phrases). --info reduces noise by dropping debug/trace log levels.
183 # Subsystem-based predicate catches every powermanagement emission (willSleep,
184 # didWake, didDim, didUndim, DarkWake, etc.) regardless of message phrasing
185 # across kernel versions. Phrasing-based predicates ('didWake', 'Wake from Sleep')
186 # are fragile across macOS releases. We accept the extra log noise because the
187 # run_recover() debounce + marker check filter out anything that isn't a real
188 # gateway-relevant event.
189 (
190     # Reconnect loop: macOS rotates the unified log buffer periodically; when
191     # that happens, this `log stream` subscriber's pipe EOFs, the inner
192     # `while read` consumer terminates, and without this wrapper the listener
193     # would be silently dead for the rest of the watcher's lifetime.
194     while :; do
195         log stream --predicate \
196             'subsystem == "com.apple.powermanagement"' \
197             --style compact --info 2>/dev/null \
198             | while IFS= read -r line; do
199                 [ -z "$line" ] && continue
200                 run_recover "WAKE: $(echo "$line" | head -c 100)"
201             done
202         echo "[$(date -u +%FT%TZ)] log stream subshell exited; reconnecting in 5s"
203         sleep 5
204     done
205 ) &
206
207 # Listener 2: NETWORK state changes via scutil
208 # `scutil n.watch` on a state key prints SCEventUpdate lines whenever the key
209 # changes. We watch Global/IPv4 because that's the umbrella that catches link,
210 # address, route, and DNS changes for ALL interfaces in one namespace.
211 # The pipe is kept alive by an `sleep 86400` loop so scutil never sees EOF
212 # from us (which would silently terminate the watch).
213 (
214     # Reconnect loop: scutil can exit noisily on certain state changes (rare,
215     # but observed). Without the wrapper, the network listener's pipe EOFs and
216     # every subsequent roam goes unseen until launchd restarts us.
217     while :; do
218         (
219             # Watch both IPv4 and IPv6 umbrella state IPv6-only or dual-stack
220             # networks never fire on the IPv4 key alone, and the user's first roam
221             # in a hotel / airport / on cellular hotspot is often IPv6-first under
222             # NAT64.
223             echo "n.add State:/Network/Global/IPv4"
224             echo "n.add State:/Network/Global/IPv6"
225             echo "n.watch"
226             while true; do sleep 86400; done
227         ) | script -q /dev/null scutil 2>/dev/null \
228         | while IFS= read -r line; do
229             case "$line" in
230                 *changedKey*|*State:/Network/Global/IPv*|*n.state*|*SCEventUpdate*)
231                     detect_lan_p2p_mode
232                     run_recover "NET: $(echo "$line" | head -c 100)"
233                     ;;
234                 *) ;;
235             esac
236         done
237         echo "[$(date -u +%FT%TZ)] scutil pipe exited; reconnecting in 5s"
238         sleep 5
239     done
240 ) &
241 # Listener 3: TUNNEL_FAILED_CLOSED.marker Watcher
242 # Polls for the fail-closed marker dropped by the routing-fix container.

```

```

243 (
244     last_notified=0
245     while ;; do
246         if [ -f "$SCRIPT_DIR/../../TUNNEL_FAILED_CLOSED.marker" ]; then
247             now=$(date +%s)
248             # Notify once every 5 minutes (300s) if it stays failed
249             if [ "$((now - last_notified))" -ge 300 ]; then
250                 osascript -e 'display notification "VPN Tunnel Health Check Failed. Internet traffic is blackholed to
prevent IP leak." with title "NullExit Alert"' || true
251                 last_notified=$now
252             fi
253         else
254             last_notified=0
255         fi
256         sleep 3
257     done
258 ) &
259
260 # Lifetime
261 # `wait` blocks until both background pipelines exit (which they normally
262 # never do). launchctl `bootout`/SIGTERM triggers the trap, which kills the
263 # whole process group so the sleep-forever in the scutil pipe also dies.
264 # We catch TERM/INT/HUP for explicit signals and EXIT for any normal-exit path
265 # (so listeners always get cleaned up even if `exit 0` runs somewhere).
266 # `rm -f $LOCK_FILE` first so a stale lock can never block the next launch.
267 # `pkill -TERM -P $$` then targets direct listener-children, then `kill 0`
268 # takes care of any backgrounded group-mates not reachable via the parent.
269 trap 'echo "[$(date -u +%FT%TZ)] watcher.sh terminating (signal/exit=$?)"; rm -f "$LOCK_FILE" 2>/dev/null ||
true; pkill -TERM -P $$ 2>/dev/null || true; kill 0 2>/dev/null || true; exit 0' TERM INT HUP EXIT
270 wait

```

15 scripts/crypto.sh

```

1  #!/usr/bin/env bash
2  # crypto.sh - Cryptographic Integrity Check
3
4  set -euo pipefail
5  cd "$(dirname "$0")/.."
6
7  if [ "$#" -ne 1 ] || { [ "$1" != "--sign" ] && [ "$1" != "--verify" ]; }; then
8      echo "Usage: $0 --sign | --verify"
9      exit 1
10 fi
11
12 if [ ! -f .env ]; then
13     echo "Error: .env not found."
14     exit 1
15 fi
16
17 SEED=$(grep "^NULLEXIT_SEED=" .env | cut -d '=' -f2)
18 if [ -z "$SEED" ]; then
19     echo "Error: NULLEXIT_SEED not found in .env."
20     exit 1
21 fi
22
23 if [ "$1" == "--sign" ]; then
24     echo "Generating cryptographic signatures using seed..."
25     rm -f .signatures
26     for file in toggle.sh recover.sh scripts/common.sh scripts/routing-fix.sh scripts/toggle-linux.sh scripts/
recover-linux.sh scripts/diagnose-host-leak.sh scripts/watcher.sh scripts/pf.conf post-rules.txt docker-
compose.yml; do
27         if [ -f "$file" ]; then
28             hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
29             echo "$file:$hash" >> .signatures
30             echo " [Signed] $file"
31         else
32             echo " [Skipped] $file not found"
33         fi
34     done
35     echo "Signatures saved to .signatures"
36     exit 0
37 fi
38
39 if [ "$1" == "--verify" ]; then
40     if [ ! -f .signatures ]; then
41         echo "Error: .signatures file missing. Run $0 --sign first."

```

```

42     exit 1
43 fi
44 FAIL=0
45 while IFS=: read -r file expected_hash; do
46     if [ -z "$file" ]; then continue; fi
47     if [ ! -f "$file" ]; then
48         echo "[FAIL] $file is missing!"
49         FAIL=1
50         continue
51     fi
52     actual_hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
53     if [ "$actual_hash" != "$expected_hash" ]; then
54         echo "[FAIL] $file has been modified! Hash mismatch."
55         FAIL=1
56     fi
57 done < .signatures
58
59 if [ "$FAIL" -eq 1 ]; then
60     echo ""
61     echo "CRITICAL: Script integrity verification failed!"
62     echo "One or more core files have been modified without authorization."
63     echo "To authorize these changes, run: ./scripts/crypto.sh --sign"
64     echo ""
65     exit 1
66 fi
67 exit 0
68 fi

```

16 .aiignore

```

1 # Environment variables and secrets
2 .env

```

17 .gitignore

```

1 # Environment variables and secrets
2 .env
3
4 # wgcf generated files containing keys and account info
5 wgcf-account.toml
6 wgcf-profile.conf
7
8 # Tailscale persistent state containing node identity
9 tailscale-state/
10
11 # Binaries
12 wgcf
13
14 # macOS App bundles
15 *.app/
16 *.app
17
18 # AdGuard Home persistent state
19 adguard/
20 ADGUARD_IP.txt
21 logs.txt
22
23 #
24 stability_incident_report.md
25 output.log
26 .DS_Store
27 *.desktop
28 blocked.log
29 wtf.md
30 host-leak-diagnostic-*.txt
31 nullexit_review.md
32 nullexit_unified.pdf
33 nullexit_unified.tex
34 nullexit_unified.txt
35
36 nullexit_unified.synctex.gz
37

```

```

38 # Runtime state files
39 .host_ips
40 .lan_p2p_detected
41 .signatures
42 TUNNEL_FAILED_CLOSED.marker

```

18 .host_ips

```

1 172.17.52.223
2 100.109.94.19
3 192.168.64.1

```

19 Recover-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; bash recover.sh"
10 end tell

```

20 Toggle-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; ./toggle.sh"
10 end tell

```

21 benchmarks/benchmark.sh

```

1 #!/usr/bin/env bash
2
3 # benchmark.sh
4 # Automates the full benchmarking suite (both Python download test and Lighthouse HTML test)
5 # Runs the suite with Gateway ON, toggles OFF, runs again, then toggles back ON.
6
7 set -e
8
9 # Change to project root so we can access toggle.sh reliably
10 cd "$(dirname "$0")/.."
11
12 # Ensure jq is installed
13 if ! command -v jq &> /dev/null; then
14     echo "Error: 'jq' is not installed. Please run 'brew install jq'"
15     exit 1
16 fi
17
18 run_lighthouse() {
19     local url=$1
20     local file_prefix=$2
21     echo " [Lighthouse] Testing $url ..."
22
23     # Run lighthouse headlessly, silencing standard output

```

```

24 npx -y lighthouse "$url" --chrome-flags="--headless" --only-categories=performance --output=json --output-
    path="${file_prefix}.json" >/dev/null 2>&1
25
26 # Parse the results
27 local score=$(jq -r '.categories.performance.score' "${file_prefix}.json")
28 local tti=$(jq -r '.audits["interactive"].displayValue' "${file_prefix}.json")
29 local si=$(jq -r '.audits["speed-index"].displayValue' "${file_prefix}.json")
30
31 # Convert score to percentage
32 local score_pct=$(awk "BEGIN {print $score*100}")
33
34 echo "    -> Performance Score: ${score_pct}%"
35 echo "    -> Time to Interactive: $tti"
36 echo "    -> Speed Index: $si"
37
38 # Cleanup
39 rm -f "${file_prefix}.json"
40 }
41
42 echo "=====
43 echo "    PHASE 1: Testing with Gateway ON"
44 echo "=====
45 # 1. Run raw Python download test
46 python3 benchmarks/test_load.py
47
48 # 2. Run Lighthouse HTML render test
49 echo ""
50 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
51 run_lighthouse "https://www.cnet.com/" "on_cnet"
52 run_lighthouse "https://www.independent.co.uk/" "on_ind"
53
54
55 echo ""
56 echo "=====
57 echo "    PHASE 2: Toggling Gateway OFF"
58 echo "=====
59 ./toggle.sh
60 echo "Waiting 10 seconds for macOS Wi-Fi to stabilize after DNS reset..."
61 sleep 10
62
63
64 echo ""
65 echo "=====
66 echo "    PHASE 3: Testing with Gateway OFF"
67 echo "=====
68 # 1. Run raw Python download test
69 python3 benchmarks/test_load.py
70
71 # 2. Run Lighthouse HTML render test
72 echo ""
73 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
74 run_lighthouse "https://www.cnet.com/" "off_cnet"
75 run_lighthouse "https://www.independent.co.uk/" "off_ind"
76
77
78 echo ""
79 echo "=====
80 echo "    PHASE 4: Restoring Gateway ON"
81 echo "=====
82 ./toggle.sh
83
84 echo ""
85 echo "=====
86 echo " BENCHMARKS COMPLETE!"
87 echo "You can scroll up to compare Phase 1 (AdGuard ON) vs Phase 3 (AdGuard OFF)."
88 echo "=====

```

22 benchmarks/test_load.py

```

1 import urllib.request
2 import urllib.error
3 import time
4 from html.parser import HTMLParser
5 import concurrent.futures
6
7 class ResourceParser(HTMLParser):

```

```

8     def __init__(self):
9         super().__init__()
10        self.urls = set()
11
12    def handle_starttag(self, tag, attrs):
13        attrs = dict(attrs)
14        url = None
15        if tag in ['img', 'script']:
16            url = attrs.get('src')
17        elif tag == 'link' and attrs.get('rel') == 'stylesheet':
18            url = attrs.get('href')
19
20        if url and url.startswith('http'):
21            self.urls.add(url)
22
23    def fetch_url(url):
24        try:
25            req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
26            urllib.request.urlopen(req, timeout=3)
27            return True
28        except Exception:
29            return False
30
31    def measure_url(target_url):
32        print(f"\nTesting {target_url} Load...")
33        start_total = time.time()
34
35        # Fetch main HTML
36        try:
37            req = urllib.request.Request(target_url, headers={'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36'})
38            response = urllib.request.urlopen(req, timeout=5)
39            html = response.read().decode('utf-8', errors='ignore')
40        except Exception as e:
41            print(f"Failed to fetch {target_url}: {e}")
42            return
43
44        parser = ResourceParser()
45        parser.feed(html)
46        urls = list(parser.urls)
47        print(f"Found {len(urls)} subresources to fetch (scripts, images, css)...")
48
49        # Concurrently fetch subresources
50        success_count = 0
51        fail_count = 0
52        max_threads = min(64, max(1, len(urls)))
53        with concurrent.futures.ThreadPoolExecutor(max_workers=max_threads) as executor:
54            results = list(executor.map(fetch_url, urls))
55
56        success_count = sum(1 for r in results if r)
57        fail_count = len(results) - success_count
58
59        total_time = time.time() - start_total
60        print(f"Time taken: {total_time:.2f} seconds")
61        print(f"Successfully fetched: {success_count}, Failed/Blocked: {fail_count}")
62
63    urls_to_test = [
64        "https://www.dailymail.co.uk/home/index.html",
65        "https://www.cnet.com/",
66        "https://www.independent.co.uk/"
67    ]
68
69    for u in urls_to_test:
70        measure_url(u)

```

23 black_list.txt

```

1 ad.doubleclick.net
2 adclick.g.doubleclick.net
3 adeventtracker.spotify.com
4 adfstat.yandex.ru
5 ads-api.tiktok.com
6 ads-api.twitter.com
7 ads-fa.spotify.com
8 ads-sg.tiktok.com
9 ads.tiktok.com

```



```

10 adserver.unityads.unity3d.com
11 adsfs.oppomobile.com
12 an.facebook.com$important
13 connect.facebook.net$important
14 pixel.facebook.com$important
15 analytics.query.yahoo.com
16 analytics.s3.amazonaws.com
17 analytics.spotify.com
18 analyticsengine.s3.amazonaws.com
19 appmetrica.yandex.ru
20 audio2.spotify.com
21 bdapi-ads.realmemobile.com
22 bdapi-in-ads.realmemobile.com
23 bounceexchange.com
24 bs.serving-sys.com
25 business-api.tiktok.com
26 ck.ads.oppomobile.com
27 crashdump.spotify.com
28 data.ads.oppomobile.com
29 data.mistat.india.xiaomi.com
30 data.mistat.rus.xiaomi.com
31 data.mistat.xiaomi.com
32 desktop.spotify.com
33 doubleclick.net
34 ds.metric.spotify.com
35 gemini.yahoo.com
36 googleads.g.doubleclick.net
37 googleadservices.com
38 googlesyndication.com
39 grs.hicloud.com
40 gtm.spotify.com
41 iot-eu-logser.realmme.com
42 iot-logser.realmme.com
43 log.byteoversea.com
44 log.fc.yahoo.com
45 log.spotify.com
46 m.doubleclick.net
47 mediavisor.doubleclick.net
48 metrics.spotify.com
49 metrika.yandex.ru
50 omaze.com
51 pagead2.googlesyndication.com
52 securepubads.g.doubleclick.net
53 static.doubleclick.net
54 stats.g.doubleclick.net
55 tracking.rus.miui.com
56 udcms.yahoo.com
57 v.jwpcdn.com
58 weblb-wg.gslb.spotify.com
59 webview.unityads.unity3d.com
60 diagassets.apple.com
61 iphonesubmissions.apple.com
62 radarsubmissions.apple.com
63 metrics.apple.com
64 xp.apple.com
65 heads-fa.spotify.com
66 heads4.spotify.com
67 pixel.spotify.com
68 segs.spotify.com
69 audio-fa.spotify.com
70 events.data.microsoft.com
71 datarouter-prod.ak.epicgames.com
72 nel.cloudflare.com
73 eu-mobile.events.data.microsoft.com
74 stocks-analytics-events.apple.com

```

24 cloud-init.yaml

```

1 #cloud-config
2 # Generic Cloud-Init Bootstrap Script for Remote Exit Nodes (Ubuntu/Debian)
3 # Paste this into the "User Data" or "Initialization Script" section when creating a VPS.
4
5 package_update: true
6 package_upgrade: true
7
8 packages:

```

```

9 - apt-transport-https
10 - ca-certificates
11 - curl
12 - gnupg
13 - lsb-release
14 - git
15 - python3
16 - python3-pip
17
18 runcmd:
19 # 1. Install Docker natively
20 - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-
  archive-keyring.gpg
21 - echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/
  docker.list > /dev/null
22 - sudo apt-get update
23 - sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
24
25 # 2. Add default user (ubuntu) to docker group
26 - usermod -aG docker ubuntu || true
27 - usermod -aG docker debian || true
28
29 # 3. Create the installation directory
30 - mkdir -p /opt/nullexit
31 - chown -R ubuntu:ubuntu /opt/nullexit || chown -R debian:debian /opt/nullexit
32
33 # 4. Enable IPv4 and IPv6 forwarding for VPN routing
34 - echo "net.ipv4.ip_forward=1" >> /etc/sysctl.d/99-tailscale.conf
35 - echo "net.ipv6.conf.all.forwarding=1" >> /etc/sysctl.d/99-tailscale.conf
36 - sysctl -p /etc/sysctl.d/99-tailscale.conf
37
38 final_message: "The remote exit node has been fully bootstrapped. SSH in, clone the repo to /opt/nullexit, add
  your .env, and run ./setup-linux.sh!"

```

25 launchd/com.nullexit.wake-recovery.plist

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5   <key>Label</key>
6   <string>com.nullexit.wake-recovery</string>
7
8   <!-- WorkingDirectory + relative program arg keeps this plist install-agnostic.
9        The source-of-truth for the install path is __NULLEXIT_HOME__, which
10       setup.sh or the user must substitute with their absolute nullexit
11       install root at install time. Program arg is `scripts/watcher.sh`
12       RELATIVE to WorkingDirectory, so launchd's evaluated cwd IS the
13       install root and BASH_SOURCE inside scripts/watcher.sh still
14       resolves to the real <install>/scripts/watcher.sh, preserving the
15       SCRIPT_DIR pattern. After install-time `sed -i ' ' 's|__NULLEXIT_HOME__|<abs_path>|'`,
16       the plist is portable to any clone location. -->
17   <key>WorkingDirectory</key>
18   <string>__NULLEXIT_HOME__</string>
19
20   <key>ProgramArguments</key>
21   <array>
22     <string>/bin/bash</string>
23     <string>scripts/watcher.sh</string>
24   </array>
25
26   <!-- Start the daemon immediately on launchctl load (instead of waiting
27        for the next user login). Combined with keepAlive below this means
28        once installed: load once, runs forever across logouts/reboots. -->
29   <key>RunAtLoad</key>
30   <true/>
31
32   <!-- Restart policy:
33        SuccessfulExit=false    SIGTERM / clean exit does NOT relaunch (so
34                                `launchctl bootout` cleanly stops, not loops).
35        Crashed=false          HARD crashes ALSO do NOT relaunch. We've seen
36                                that repeated sleep/wake cycles on macOS
37                                Sonoma+ accumulate orphan watcher.sh PIDs
38                                because SIGCONT/resume doesn't reliably kill
39                                the prior instance. The watcher.sh script

```

```

40         now enforces single-instance via a manual
41         PID-file and process-identity check at the
42         top, so a re-launch on crash is
43         actively dangerous (it would skip the lock
44         and exit; better to surface the crash in
45         logs than to silently 0-process). -->
46     <key>KeepAlive</key>
47     <dict>
48         <key>SuccessfulExit</key>
49         <false/>
50         <key>Crashed</key>
51         <false/>
52     </dict>
53
54     <!-- Background hint to App Nap so it doesn't get suspended when
55         the user is inactive. We're the entire reason this LaunchAgent
56         needs to NOT nap. -->
57     <key>ProcessType</key>
58     <string>Background</string>
59
60     <!-- launchd-level stdout/stderr (the bash launcher shell). The script
61         itself writes to /tmp/nullexit-watcher.log via exec >>. These
62         separate channels help us tell "launchd crashed" from "the bash
63         wrapper piped to recover.sh errored". -->
64     <key>StandardOutPath</key>
65     <string>/tmp/nullexit-watcher.out.log</string>
66     <key>StandardErrorPath</key>
67     <string>/tmp/nullexit-watcher.err.log</string>
68 </dict>
69 </plist>

```

26 post-rules.txt

```

1 iptables -A FORWARD -i tailscale0 -o tun0 -j ACCEPT
2 iptables -A FORWARD -i tun0 -o tailscale0 -j ACCEPT
3 iptables -A INPUT -i tailscale0 -j ACCEPT
4 iptables -A INPUT -i eth0 -p udp --dport 41642 -j ACCEPT
5 iptables -t nat -A POSTROUTING -o tun0 -j MASQUERADE
6 # Clamp MSS to 1120. With Tailscale overhead on top of WARP (each WireGuard encapsulation adds ~60 bytes),
7 # the safe MSS for double-tunneled traffic is 1120 to prevent strict-MSS endpoints from stalling.
8 iptables -t mangle -A POSTROUTING -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1120
9
10 iptables -t nat -A PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT --to-port 5335
11 iptables -t nat -A PREROUTING -i tailscale0 -p tcp --dport 53 -j REDIRECT --to-port 5335
12
13 # Also redirect DNS from Docker bridge interface so the host can reach AdGuard via localhost:53
14 iptables -t nat -A PREROUTING -i eth0 -p udp --dport 53 -j REDIRECT --to-port 5335
15 iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 53 -j REDIRECT --to-port 5335
16
17 # Block IPv6 exit-node traffic to prevent leakage (Gluetun/WARP is IPv4-only)
18 ip6tables -A FORWARD -i tailscale0 -j DROP
19
20 # Block DNS-over-TLS (Port 853) to force Android Private DNS to fallback to Port 53
21 iptables -A FORWARD -i tailscale0 -p tcp --dport 853 -j REJECT --reject-with tcp-reset
22 iptables -A FORWARD -i tailscale0 -p udp --dport 853 -j REJECT
23
24 # Block QUIC (UDP 443) to force Facebook/Google to fallback to TCP.
25 # UDP bypasses TCP MSS clamping, causing fragmentation. Over weak Wi-Fi (far from gateway),
26 # dropping a single fragment destroys the entire packet, stalling image downloads.
27 iptables -A FORWARD -i tailscale0 -p udp --dport 443 -j REJECT

```

27 scripts/Dockerfile.routing-fix

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY logger/ ./logger/
4 RUN cd logger && go build -o logger main.go
5
6 FROM alpine:3.20
7 RUN apk add --no-cache iptables iproute2 curl ipset
8 COPY --from=builder /app/logger/logger /usr/local/bin/nullexit-logger
9 COPY routing-fix.sh /routing-fix.sh

```

```
10 CMD ["sh", "/routing-fix.sh"]
```

28 scripts/Dockerfile.rule-compiler

```
1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY rule-compiler/ ./rule-compiler/
4 RUN cd rule-compiler && go build -o sync-rules main.go
5
6 FROM alpine:3.20
7 COPY --from=builder /app/rule-compiler/sync-rules /usr/local/bin/sync-rules
8 WORKDIR /app
9 CMD ["sync-rules"]
```

29 scripts/Toggle-Gateway.bat

```
1 <# :
2 @echo off
3 powershell -NoProfile -ExecutionPolicy Bypass -Command "Invoke-Expression ([System.IO.File]::ReadAllText('%~f0
4 ') -replace '^(?s).*?#s*>s*\r?\n','')
5 exit /b
6 #>
7 # Toggle-Gateway.bat (Hybrid Batch-PowerShell Toggler) nullexit Gateway Toggler for Windows
8 # Mirrors the sophisticated error handling, checks, and automation of toggle.sh
9
10 # 1. Administrator Check & Elevation
11 $isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([
12     Security.Principal.WindowsBuiltInRole]::Administrator)
13 if (-not $isAdmin) {
14     Write-Host "Requesting Administrator privileges..." -ForegroundColor Yellow
15     Start-Process PowerShell -ArgumentList "-NoProfile -ExecutionPolicy Bypass -Command `Invoke-Expression ([
16         System.IO.File]::ReadAllText('$PSCmdPath') -replace '^(?s).*?#s*>s*\r?\n','')`"" -Verb RunAs
17     Exit
18 }
19
20 # Set console output encoding to UTF8 for clean symbols
21 [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
22 Clear-Host
23
24 Write-Host "" -ForegroundColor Green
25 Write-Host "      nullexit Windows Toggler" -ForegroundColor Green
26 Write-Host "" -ForegroundColor Green
27 Write-Host ""
28
29 # 2. Helpers
30 function Reset-HostDNS {
31     Write-Host "Restoring default DNS (DHCP/DHCP-assigned DNS)..." -ForegroundColor Cyan
32     $ActiveAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
33     foreach ($adapter in $ActiveAdapters) {
34         Write-Host "    -> Resetting DNS on adapter: $($adapter.Name)" -ForegroundColor Gray
35         Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ResetServerAddresses -ErrorAction
36         SilentlyContinue
37     }
38 }
39
40 function Hijack-HostDNS ($ip) {
41     Write-Host "Hijacking DNS to route through AdGuard ($ip)..." -ForegroundColor Yellow
42     $ActiveAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
43     foreach ($adapter in $ActiveAdapters) {
44         Write-Host "    -> Pointing DNS to $ip on adapter: $($adapter.Name)" -ForegroundColor Gray
45         Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses $ip -ErrorAction
46         SilentlyContinue
47     }
48 }
49
50 function Get-HostTailscalePath {
51     $paths = @(
52         "$env:ProgramFiles\Tailscale\tailscale.exe",
53         "$env:ProgramFiles (x86)\Tailscale\tailscale.exe",
54         "tailscale.exe"
55     )
56     foreach ($path in $paths) {
```

```

52     if (Test-Path $path) { return $path }
53 }
54     return $null
55 }
56
57 # 3. Detect State
58 # Prevent deadlocks by resetting DNS to default temporarily during checks
59 Reset-HostDNS
60
61 Write-Host "Checking Docker daemon status..." -ForegroundColor Cyan
62 & docker info >$null 2>&1
63 if ($LASTEXITCODE -ne 0) {
64     Write-Host "Docker is not running. Starting Docker Desktop..." -ForegroundColor Yellow
65     $dockerPath = "C:\Program Files\Docker\Docker\Docker Desktop.exe"
66     if (Test-Path $dockerPath) {
67         Start-Process $dockerPath
68         Write-Host "Waiting for Docker daemon to become responsive..." -ForegroundColor Gray
69         $timeout = 60
70         while ($timeout -gt 0) {
71             Start-Sleep -Seconds 2
72             & docker info >$null 2>&1
73             if ($LASTEXITCODE -eq 0) {
74                 Write-Host "Docker is ready!" -ForegroundColor Green
75                 break
76             }
77             $timeout -= 2
78         }
79         if ($timeout -le 0) {
80             Write-Error "Docker failed to start in time. Aborting."
81             Exit 1
82         }
83     } else {
84         Write-Error "Docker Desktop executable not found at standard path. Please start Docker manually."
85         Exit 1
86     }
87 }
88
89 # Detect if gateway is already running
90 $runningWarp = docker compose ps --status running --format json | ConvertFrom-Json -ErrorAction
    SilentlyContinue | Where-Object { $_.Service -eq "warp" }
91 $isGatewayActive = ($null -ne $runningWarp)
92
93 # 4. Execution
94 if ($isGatewayActive) {
95     # STOP PATH
96     Write-Host "Gateway is currently RUNNING. Disabling now..." -ForegroundColor Yellow
97     Write-Host ""
98
99     # Disconnect host Tailscale from exit node if installed
100     $tsCli = Get-HostTailscalePath
101     if ($null -ne $tsCli) {
102         Write-Host "Disconnecting host Tailscale from exit node..." -ForegroundColor Cyan
103         & $tsCli up --exit-node= --accept-dns=true >$null 2>&1
104     }
105
106     Write-Host "Stopping Docker containers..." -ForegroundColor Cyan
107     docker compose down -t 5
108
109     Reset-HostDNS
110     Write-Host "Gateway has been successfully STOPPED." -ForegroundColor Green
111 } else {
112     # START PATH
113     Write-Host "Gateway is currently STOPPED. Enabling now..." -ForegroundColor Yellow
114     Write-Host ""
115
116     # Run Rules Compiler if python is available
117     Get-Command python -ErrorAction SilentlyContinue >$null
118     if ($LASTEXITCODE -eq 0) {
119         Write-Host "Compiling DNS blocklists..." -ForegroundColor Cyan
120         python scripts/sync-rules.py
121     } else {
122         Write-Host "Python not found in PATH. Skipping rule pre-compilation." -ForegroundColor Yellow
123     }
124
125     # Clean up corrupted AdGuardHome configurations
126     $confFile = "adguard\conf\AdGuardHome.yaml"
127     if (Test-Path $confFile) {
128         $fileSize = (Get-Item $confFile).Length
129         if ($fileSize -eq 0) {
130             Remove-Item $confFile -Force

```

```

131     Write-Host "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop." -
132     ForegroundColor Yellow
133   }
134 }
135
136 Write-Host "Starting Docker containers..." -ForegroundColor Cyan
137 docker compose up -d
138
139 Write-Host "Waiting for container's Tailscale connection to offer Exit Node..." -ForegroundColor Cyan
140 Write-Host " (This can take up to 60 seconds..." -ForegroundColor Gray
141
142 $elapsed = 0
143 $maxWait = 60
144 $resolvedIP = $null
145
146 while ($elapsed -lt $maxWait) {
147   $statusJson = docker compose exec -T tailscale tailscale status --json 2>$null
148   if ($null -ne $statusJson) {
149     $status = $statusJson | ConvertFrom-Json -ErrorAction SilentlyContinue
150     if ($null -ne $status -and $null -ne $status.Self) {
151       # Check if it has a valid Tailscale IP and is running
152       if ($status.Self.Online -and $status.Self.TailscaleIPs.Count -gt 0) {
153         $resolvedIP = $status.Self.TailscaleIPs[0]
154         break
155       }
156     }
157   }
158   Start-Sleep -Seconds 2
159   $elapsed += 2
160 }
161
162 if ($null -ne $resolvedIP) {
163   Write-Host "Tailscale IP resolved: $resolvedIP" -ForegroundColor Green
164   Write-Host ""
165
166   # Configure host Tailscale client to route through this exit node
167   $tsCli = Get-HostTailscalePath
168   if ($null -ne $tsCli) {
169     Write-Host "Configuring host Tailscale to use Exit Node ($resolvedIP)..." -ForegroundColor Cyan
170     & $tsCli up --exit-node=$resolvedIP --accept-dns=false >$null 2>&1
171   }
172
173   # Hijack DNS
174   Hijack-HostDNS $resolvedIP
175   Write-Host ""
176   Write-Host "Gateway has been successfully STARTED." -ForegroundColor Green
177 } else {
178   Write-Host "Warning: Could not resolve gateway Tailscale IP. DNS hijacking skipped." -ForegroundColor
179   Red
180   # Fallback to ADGUARD_IP.txt if it exists
181   if (Test-Path "ADGUARD_IP.txt") {
182     $fallbackIP = Get-Content "ADGUARD_IP.txt" -TotalCount 1
183     if (-not [string]::IsNullOrEmpty($fallbackIP)) {
184       Write-Host "[Fallback] Using static IP from ADGUARD_IP.txt: $fallbackIP" -ForegroundColor
185       Yellow
186       Hijack-HostDNS $fallbackIP
187       Write-Host "Gateway STARTED (with static fallback IP)." -ForegroundColor Green
188     }
189   } else {
190     Write-Host "Gateway started in offline/unauthenticated state. Run setup.sh if this is a fresh setup
191     ." -ForegroundColor Yellow
192   }
193 }
194
195 Write-Host ""
196 Write-Host "Press any key to exit..."
197 [void]$Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")

```

30 scripts/diagnose-host-leak.sh

```

1 #!/bin/bash
2 # scripts/diagnose-host-leak.sh nullexit host-routing diagnostic
3 #
4 # Symptom this addresses: tests like https://www.whatismyip.com on the
5 # *HOST MAC* return the underlying physical ISP/ASN

```

```

6 # local ISP / campus network") even though the nullexit gateway is active and remote
7 # tailnet clients correctly egress via Cloudflare WARP. The container and
8 # the gateway itself are healthy only the host's own routing is leaking.
9 #
10 # This script runs 7 checks in sequence, classifies the result into ONE
11 # of the three known leak scenarios, writes the full report to a
12 # timestamped file in the project root, and prints a verdict + a
13 # ready-to-run fix command on stdout. Pass --fix to apply the matching
14 # remediation in the same invocation and re-verify egress afterwards.
15 # See devref.md 10.30 for the deep-dive rationale.
16 #
17 # Usage:
18 #   bash scripts/diagnose-host-leak.sh           # diagnose + write report
19 #   bash scripts/diagnose-host-leak.sh --fix      # diagnose + fix + re-verify
20 #   bash scripts/diagnose-host-leak.sh --watch    # full baseline then loop checks every 60s
21 #   bash scripts/diagnose-host-leak.sh --watch 30 # same, with custom interval (seconds)
22 #   bash scripts/diagnose-host-leak.sh --help    # this message
23 #
24 # Multiple runs produce timestamped files (one per run) so historical
25 # reports can be diffed. The newest file is the most recent.
26
27 set -uo pipefail
28 # NOTE: errexit (`set -e`) is intentionally NOT enabled. Several checks
29 # below are EXPECTED to fail and that's diagnostic data (e.g. `tailscale
30 # status` returns non-zero when the daemon is wedged). Every command
31 # pipes to a helper that records the exit code without killing the script.
32
33 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
34 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
35 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
36 source "$SCRIPT_DIR/common.sh"
37 LOG_FILE="$PROJECT_ROOT/output.log"
38 REPORT_FILE="$PROJECT_ROOT/host-leak-diagnostic-$TIMESTAMP.txt"
39
40 # Argument parsing
41 DO_FIX=false
42 DO_WATCH=false
43 WATCH_INTERVAL=60
44 WATCH_LOG="$PROJECT_ROOT/host-leak-watch.log"
45 case "${1:-}" in
46   --fix) DO_FIX=true ;;
47   --watch)
48     DO_WATCH=true
49     # Optional second argument: custom interval in seconds
50     if [ -n "${2:-}" ] && [[ "$2" =~ ^[0-9]+$ ]]; then
51       WATCH_INTERVAL="$2"
52     fi
53   ;;
54   --help|-h)
55     sed -n '2,31p' "$0"
56     exit 0
57   ;;
58   *)
59     : # default: diagnose only
60   ;;
61 *)
62   echo "Unknown argument: $1" >&2
63   echo "Run with --help for usage." >&2
64   exit 2
65 ;;
66 esac
67
68 # Colours (auto-disabled when stdout is not a tty, e.g. piped)
69 if [ -t 1 ]; then
70   RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
71   BOLD='\033[1m'; NC='\033[0m'
72 else
73   RED=''; GREEN=''; YELLOW=''; BOLD=''; NC=''
74 fi
75
76 # Output helpers write to BOTH stdout and the report file
77 exec 3>> "$REPORT_FILE" || {
78   printf '\n[FATAL] cannot open %s for write\n' "$REPORT_FILE" >&2
79   exit 1
80 }
81
82 section() {
83   local title="$1"
84   printf '\n%b %s %b\n' "$BOLD" "$title" "$NC" >&3
85   printf '\n%b %s %b\n' "$BOLD" "$title" "$NC"
86 }
87
88 line() { printf ' %b\n' "$1" >&3; printf ' %b\n' "$1"; }

```



```

87 ok()      { line "${GREEN}${NC} $1"; }
88 warn()    { line "${YELLOW}${NC} $1"; }
89 fail()     { line "${RED}${NC} $1"; }
90 note()     { line "(no ${1})"; }
91
92 # Watch-mode header (only shown when entering watch loop)
93 if [ "$DO_WATCH" = "true" ]; then
94     echo ""
95     echo ""
96     echo " n u l l e x i t   w a t c h   m o d e "
97     echo ""
98     echo " Interval:          ${WATCH_INTERVAL}s"
99     echo " Watch log:          $WATCH_LOG"
100    echo ""
101 else
102    echo ""
103    echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c "
104    echo ""
105    echo " Timestamp (UTC): $TIMESTAMP"
106    echo " Project root:    $PROJECT_ROOT"
107    echo " Report file:     $REPORT_FILE"
108    echo " Mode:           $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only (pass --
        fix to remediate)")"
109    echo ""
110    echo "" >&3
111    echo "" >&3
112    echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c " >&3
113    echo "" >&3
114    echo " Timestamp (UTC): $TIMESTAMP" >&3
115    echo " Mode:           $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only")" >&3
116 fi
117
118 # Always add Homebrew to PATH (same as toggle.sh / recover.sh)
119 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
120
121 # Pure-bash timeout (macOS lacks GNU `timeout`)
122 run_with_timeout() {
123     local timeout_sec="$1"
124     shift
125     if [ "$#" -eq 0 ]; then return 1; fi
126     "$@" &
127     local cmd_pid=$!
128     (
129         sleep "$timeout_sec"
130         if kill -0 "$cmd_pid" 2>> "$LOG_FILE"; then
131             kill -9 "$cmd_pid" 2>> "$LOG_FILE" || true
132         fi
133     ) >> "$LOG_FILE" 2>&1 &
134     local watcher_pid=$!
135     set +e
136     wait "$cmd_pid"
137     local exit_status=$?
138     set -uo pipefail
139     kill "$watcher_pid" 2>> "$LOG_FILE" || true
140     return "$exit_status"
141 }
142
143 # Resolve the active network service (same heuristic as toggle.sh)
144 resolve_active_service() {
145     get_active_service
146 }
147
148 #
149 # Quick-check functions (used by --watch loop; return simple status strings)
150 # Each echoes its result so the caller can capture and compare.
151 #
152
153 quick_warp() {
154     # Returns: "on", "off", "unknown"
155     local out
156     out=$(run_with_timeout 8 curl -s --max-time 6 \
157         https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null || echo "")
158     local warp
159     warp=$(printf '%s' "$out" | awk -F=' ' '{print $2; exit}')
160     printf '%s' "${warp:-unknown}"
161 }
162
163 quick_ipv6() {
164     # Returns: the IPv6 address if leaking, or empty string if clean
165     local out
166     out=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>/dev/null || echo "")

```

```

167 if [ -n "$out" ] && printf '%s' "$out" | grep -qE '[0-9a-fA-F:]+$'; then
168     printf '%s' "$out"
169 fi
170 }
171
172 quick_default_route() {
173     # Returns: "utun" if default route is via Tailscale utun*,
174     #           "wifi" if via physical Wi-Fi (192.168.x.x),
175     #           "other" otherwise
176     local route
177     route=$(netstat -rn 2>/dev/null | awk '/^default/ {print $0; exit}')
178     if [ -z "$route" ]; then
179         printf '%s' "none"
180     elif printf '%s' "$route" | grep -qE 'utun[0-9]+'; then
181         printf '%s' "utun"
182     elif printf '%s' "$route" | grep -qE '^(default|0\.\0\.\0\.\0).*\b192\.\168\.'; then
183         printf '%s' "wifi"
184     else
185         printf '%s' "other"
186     fi
187 }
188
189 # Watch loop (runs after baseline diagnostic when --watch is passed)
190 run_watch_loop() {
191     local baseline_warp="$1"
192     local baseline_default="$2"
193     local baseline_ipv6_leak="$3"
194     local interval="$4"
195
196     echo ""
197     echo ""
198     echo " Baseline established. Monitoring every ${interval}s..."
199     echo " warp:      $baseline_warp"
200     echo " default:    $baseline_default"
201     echo ""
202
203     # Safety: warn if interval is very short (avoids hammering APIs)
204     if [ "$interval" -lt 30 ]; then
205         printf '%b' Short interval (%ss) rate-limiting risk on external APIs%b\n' "$YELLOW" "$interval" "$NC"
206         printf ' Consider 30s or longer for production use.\n'
207     fi
208     printf '[%s] WATCH START warp=%s default=%s interval=%ss\n' \
209         "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$baseline_warp" "$baseline_default" "$interval" \
210         >> "$WATCH_LOG"
211
212     local iteration=0
213     local last_warp="$baseline_warp"
214     local last_default="$baseline_default"
215     local last_ipv6_ok=$(( [ "$baseline_ipv6_leak" = "false" ] && echo true || echo false ))
216
217     trap 'printf "\n%bWatch stopped by user. Log at: %s%b\n" "$YELLOW" "$WATCH_LOG" "$NC"; exit 0' INT TERM
218
219     while true; do
220         iteration=$((iteration + 1))
221
222         local warp_now ipv6_now default_now
223         warp_now=$(quick_warp)
224         ipv6_now=$(quick_ipv6)
225         default_now=$(quick_default_route)
226
227         local ts
228         ts=$(date -u +%H:%M:%S)
229
230         # WARP check
231         if [ "$warp_now" != "$last_warp" ]; then
232             if [ "$warp_now" = "off" ] || [ "$warp_now" = "unknown" ]; then
233                 printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
234                 printf '%b[%s] %bWARP FLIPPED: %s %s%b YOUR IP IS LEAKING!\n' \
235                     "$RED" "$ts" "$BOLD" "$last_warp" "$warp_now" "$NC"
236                 printf '[%s] ALERT warp flipped %s%b\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
237             else
238                 printf '\n%b[%s] warp recovered: %s %s%b\n' \
239                     "$GREEN" "$ts" "$last_warp" "$warp_now" "$NC"
240                 printf '[%s] OK warp recovered %s%b\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
241             fi
242             last_warp="$warp_now"
243         fi
244
245         # IPv6 check
246         if [ -n "$ipv6_now" ]; then
247             if [ "$last_ipv6_ok" = true ]; then

```

```

248     printf '\n\b ALERT %b\n' "$RED$BOLD" "$NC"
249     printf '%b[%s] %bIPv6 LEAK DETECTED %s\b\n' \
250         "$RED" "$ts" "$BOLD" "$ipv6_now" "$NC"
251     printf "[%s] ALERT IPv6 leak %s\n" "$ts" "$ipv6_now" >> "$WATCH_LOG"
252     last_ipv6_ok=false
253 fi
254 else
255     if [ "$last_ipv6_ok" = false ]; then
256         printf '%b[%s] %bIPv6 leak resolved\b\n' "$GREEN" "$ts" "$NC"
257         printf "[%s] OK IPv6 resolved\n" "$ts" >> "$WATCH_LOG"
258     fi
259     last_ipv6_ok=true
260 fi
261
262 # Default route check
263 if [ "$default_now" != "$last_default" ]; then
264     if [ "$default_now" != "utun" ]; then
265         printf '\n\b ALERT %b\n' "$RED$BOLD" "$NC"
266         printf '%b[%s] %bDEFAULT ROUTE CHANGED: %s %s\b traffic may bypass WARP!\n' \
267             "$RED" "$ts" "$BOLD" "$last_default" "$default_now" "$NC"
268         printf "[%s] ALERT route changed %s%s\n" "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
269     else
270         printf '%b[%s] default route recovered: %s %s\b\n' \
271             "$GREEN" "$ts" "$last_default" "$default_now" "$NC"
272         printf "[%s] OK route recovered %s%s\n" "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
273     fi
274     last_default="$default_now"
275 fi
276
277 # Heartbeat (every 10th iteration or if all OK)
278 if [ $((iteration % 10)) -eq 0 ]; then
279     printf "[%s] #d warp=%s ipv6=%s route=%s\n" \
280         "$ts" "$iteration" "$warp_now" "${ipv6_now:-none}" "$default_now" >> "$WATCH_LOG"
281     printf '\r%b[%s] #d warp=%s route=%s (Ctrl+C to stop)%b\n' \
282         "$GREEN" "$ts" "$iteration" "$warp_now" "$default_now" "$NC"
283 fi
284
285     sleep "$interval"
286 done
287 }
288
289 #
290 # Begin main diagnostic (skipped in non-watch modes if we're just watching)
291 #
292
293 ACTIVE_SERVICE=$(resolve_active_service)
294 ENO_SERVICE=$(networksetup -listnetworkserviceorder 2>> "$LOG_FILE" \
295     | grep -B 1 "Device: en0" | head -1 \
296     | sed -E 's/^([0-9\*]+\.) //' | true)
297 [ -z "$ENO_SERVICE" ] && ENO_SERVICE="Wi-Fi"
298
299 # Resolve the gateway's Tailscale IP.
300 GATEWAY_TS_IP=""
301 if [ -f "$PROJECT_ROOT/ADGUARD_IP.txt" ]; then
302     GATEWAY_TS_IP=$(cat "$PROJECT_ROOT/ADGUARD_IP.txt" 2>> "$LOG_FILE" \
303         | tr -d '\r' | awk 'NR==1{print $1; exit}')
304 fi
305 if [ -z "$GATEWAY_TS_IP" ] && command -v docker >> "$LOG_FILE" 2>&1 \
306     && docker compose ps --status running 2>/dev/null | grep -q 'tailscale'; then
307     GATEWAY_TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale \
308         tailscale ip -4 2>> "$LOG_FILE" \
309         | tr -d '\r' | awk 'NR==1{print $1; exit}')
310 fi
311
312 # Scenario classification state
313 SCENARIO=""
314 WARP_STATUS=""
315 SOCKS5_ENABLED=false
316 TS_ON_MESH=false
317 DEFAULT_VIA_UTUN=false
318 IPV6_LEAK=false
319
320 #
321 section "1/8 Host Tailscale mesh status"
322 #
323 if ! command -v tailscale >> "$LOG_FILE" 2>&1; then
324     fail "tailscale CLI not on PATH install via 'brew install tailscale' then re-run"
325     echo " (a Tailscale daemon is required for the exit-node path to work)"
326 else
327     set +e
328     TS_OUT=$(run_with_timeout 5 tailscale status 2>&1)

```

```

329 TS_EXIT=$?
330 set -uo pipefail
331 TS_HEAD=$(printf '%s\n' "$TS_OUT" | head -5 | tr -d '\r')
332 if [ "$TS_EXIT" -eq 137 ]; then
333     fail "tailscaled daemon is wedged/unresponsive (5s timeout)"
334     echo "Fix path: 'sudo brew services restart tailscale'"
335 elif printf '%s' "$TS_OUT" | grep -qE 'Logged out|not logged in|expired'; then
336     fail "tailscale is logged out / auth expired on this host"
337     echo "Fix path: 'sudo tailscale up' (browser auth once)"
338 elif printf '%s' "$TS_OUT" | grep -q '100\.'; then
339     ok "Host is on tailnet (100.x.x.x visible in status)"
340     TS_ON_MESH=true
341     if printf '%s' "$TS_OUT" | grep -q "$GATEWAY_TS_IP"; then
342         ok "Gateway $GATEWAY_TS_IP visible in host's peer list"
343     elif [ -n "$GATEWAY_TS_IP" ]; then
344         warn "Gateway $GATEWAY_TS_IP NOT in host's peer list possible auth/key mismatch"
345     fi
346     line "first 5 lines of 'tailscale status' "
347     line "$(printf '%s' "$TS_HEAD" | awk '{print "    "$0}')"
348 else
349     fail "tailscale output unrecognized:"
350     line "$(printf '%s' "$TS_OUT" | head -3 | awk '{print "    "$0}')"
351 fi
352 fi
353
354 #
355 section "2/8 Active SOCKS5 proxy on Wi-Fi"
356 #
357 SOCKS_OUT=$(networksetup -getsocksfirewallproxy "$ACTIVE_SERVICE" 2>> "$LOG_FILE" \
358     || networksetup -getsocksfirewallproxy Wi-Fi 2>> "$LOG_FILE" \
359     || echo "Could not query SOCKS5 proxy state")
360 if printf '%s' "$SOCKS_OUT" | grep -q "Enabled: Yes"; then
361     fail "SOCKS5 proxy IS enabled on $ACTIVE_SERVICE toggle.sh fell back to SOCKS5 mode last run"
362     SOCKS5_ENABLED=true
363     line "Full state:"
364     line "$(printf '%s' "$SOCKS_OUT" | awk '{print "    "$0}')"
365     line "Browse on macOS ignores system SOCKS5 by default traffic bypasses WARP."
366 else
367     ok "SOCKS5 proxy is OFF on $ACTIVE_SERVICE exit-node path should be active"
368 fi
369
370 #
371 section "3/8 Egress IP + WARP tunnel verification (definitive test)"
372 #
373 CDN_OUT=$(run_with_timeout 8 curl -s --max-time 6 \
374     https://www.cloudflare.com/cdn-cgi/trace 2>&1 || echo "(curl failed)")
375 CDN_WARP=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^warp=/{print $2; exit}')
376 CDN_IP=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^ip=/{print $2; exit}')
377 CDN_COLO=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^colo=/{print $2; exit}')
378 WARP_STATUS="$CDN_WARP"
379 if [ "$CDN_WARP" = "on" ]; then
380     ok "warp=on tunnel is hot, host traffic IS going through Cloudflare WARP"
381     line "Public IP: $CDN_IP"
382     line "Cloudflare: ${CDN_COLO:-unknown colo}"
383     line "whatismyip.com's 'local ISP' result is its ISP-database error, not yours."
384 elif [ "$CDN_WARP" = "off" ]; then
385     fail "warp=off host traffic is NOT going through WARP"
386     line "Public IP: $CDN_IP (NOT Cloudflare)"
387     warn "This is the actual leak."
388 else
389     warn "could not determine warp status (curl failed or returned unexpected shape)"
390     line "Raw output: $(printf '%s' "$CDN_OUT" | head -3 | tr '\n' '|')"
391 fi
392
393 #
394 section "4/8 IPv6 leak probe (bypasses IPv4 exit-node on campus networks)"
395 #
396 IPV6_OUT=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
397 if [ -n "$IPV6_OUT" ] && printf '%s' "$IPV6_OUT" | grep -qE '[0-9a-fA-F:]+$'; then
398     fail "host has live IPv6 egress $IPV6_OUT (A6 record bypasses WARP)"
399     IPV6_LEAK=true
400     line "Tailscale exit-node advertisement is IPv4-only, so IPv6 traffic skips it."
401     line "Quick-fix: 'sudo networksetup -setv6off $ACTIVE_SERVICE'"
402 else
403     ok "no IPv6 egress detected (good IPv6 won't bypass the tunnel)"
404 fi
405
406 #
407 section "5/8 Host egress route for real traffic (route -n get 1.1.1.1)"
408 #
409 # WHY NOT 'netstat -rn | grep default':

```

```

410 # On macOS, Tailscale does NOT replace the physical default route installed by
411 # DHCP. Instead it adds a second interface-scoped route bound to the utun
412 # interface that the kernel prefers for real traffic forwarding. The literal
413 # "default" entry (192.168.x.x via en0) is left untouched in the table as a
414 # BSD routing quirk it's not lying, it's just answering a different question.
415 # The correct question is: "where does a real destination actually resolve?"
416 # `route -n get 1.1.1.1` asks the kernel's forwarding logic directly.
417 ROUTE_GET=$(route -n get 1.1.1.1 2>/dev/null)
418 ROUTE_IFACE=$(echo "$ROUTE_GET" | awk '/interface:/{print $2}')
419 ROUTE_GATEWAY=$(echo "$ROUTE_GET" | awk '/gateway:/{print $2}')
420 if [ -z "$ROUTE_IFACE" ]; then
421     warn "route -n get 1.1.1.1 returned no interface routing table may be empty"
422 else
423     line " interface: $ROUTE_IFACE gateway: ${ROUTE_GATEWAY:-n/a}"
424     if echo "$ROUTE_IFACE" | grep -qE '^utun[0-9]+'; then
425         ok "1.1.1.1 resolves via $ROUTE_IFACE Tailscale interface-scoped route is winning"
426         DEFAULT_VIA_UTUN=true
427     elif echo "$ROUTE_IFACE" | grep -qE '^en[0-9]+'; then
428         fail "1.1.1.1 resolves via $ROUTE_IFACE (physical Wi-Fi) exit-node interface route not active"
429     else
430         warn "1.1.1.1 resolves via unexpected interface: $ROUTE_IFACE"
431     fi
432 fi
433
434 #
435 section "6/8 Last toggle.sh START-relevant lines in output.log (if any)"
436 #
437 if [ ! -f "$LOG_FILE" ]; then
438     note "output.log"
439     line " toggle.sh was never run from this directory. Run './toggle.sh' first,"
440     line " then re-run this diagnostic."
441 else
442     HITS=$(grep -E 'Exit node enabled|SKIP_EXIT_NODE|Pre-flight|Falling back to SOCKS|Starting|Successfully' \
443             "$LOG_FILE" 2>/dev/null | tail -8 || true)
444     if [ -z "$HITS" ]; then
445         note "matching lines in output.log"
446     else
447         line "$(printf '%s\n' "$HITS" | awk '{print "    "$0}')"
448     fi
449 fi
450
451 #
452 section "7/8 Gateway IP we should be pointing at"
453 #
454 if [ -n "$GATEWAY_TS_IP" ]; then
455     ok "gateway Tailscale IP: $GATEWAY_TS_IP"
456 else
457     warn "could not resolve gateway Tailscale IP"
458     line " (looked in ADGUARD_IP.txt and tried docker compose exec tailscale ip -4)"
459 fi
460
461 #
462 section "8/8 Tailscale System Extension (App Store conflict check)"
463 #
464 SE_PENDING_UNINSTALL=false
465 SE_ACTIVE=false
466 if command -v systemextensionsctl >/dev/null 2>&1; then
467     SE_OUT=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale')
468     if [ -n "$SE_OUT" ]; then
469         line " Tailscale System Extension(s) detected:"
470         line "$(printf '%s' "$SE_OUT" | awk '{print "    "$0}')"
471         if printf '%s' "$SE_OUT" | grep -q 'waiting to uninstall'; then
472             SE_PENDING_UNINSTALL=true
473             warn "System Extension is pending uninstall on next reboot."
474         else
475             SE_ACTIVE=true
476             warn "Active Tailscale System Extension detected. This will conflict with Homebrew Tailscale."
477         fi
478     else
479         ok "no Tailscale System Extension detected"
480     fi
481 else
482     note "systemextensionsctl not available (not on macOS or permission restricted)"
483 fi
484
485 #
486 section "CLASSIFICATION applies ONE of the four known scenarios"
487 #
488
489 if [ "$WARP_STATUS" = "on" ] && [ "$IPV6_LEAK" = "false" ]; then
490     SCENARIO="OK"

```

```

491 elif [ "$SE_PENDING_UNINSTALL" = "true" ]; then
492     SCENARIO="SE_PENDING"
493 elif [ "$SOCKS5_ENABLED" = "true" ]; then
494     SCENARIO="A"
495 elif [ "$IPV6_LEAK" = "true" ]; then
496     SCENARIO="B"
497 elif [ "$DEFAULT_VIA_UTUN" = "false" ] && [ "$TS_ON_MESH" = "true" ]; then
498     SCENARIO="C"
499 else
500     SCENARIO="UNKNOWN"
501 fi
502
503 case "$SCENARIO" in
504     SE_PENDING)
505         echo ""
506         echo -e " ${RED}${BOLD}VERDICT: Scenario SE_PENDING Tailscale System Extension Pending Uninstall${NC}"
507         echo " A prior App Store Tailscale System Extension is pending uninstall."
508         echo " Since System Integrity Protection (SIP) is enabled, macOS blocks manual"
509         echo " uninstallation of terminated system extensions until the next reboot."
510         echo " Brew-managed tailscaled cannot engage the Network Extension slot until this is resolved."
511         echo ""
512         echo "${BOLD}Recommended fix:${NC}"
513         echo " sudo shutdown -r now"
514         echo " # After reboot, run: ./toggle.sh"
515         ;;
516     A)
517         echo ""
518         echo -e " ${RED}${BOLD}VERDICT: Scenario A SOCKS5 fallback lane${NC}"
519         echo " toggle.sh's pre-flight checks failed last run. The script activated"
520         echo " SOCKS5 + local DNS proxy as a fallback. DNS gets hijacked but"
521         echo " browsers on macOS ignore the system SOCKS5 traffic egresses"
522         echo " direct through the local network."
523         echo ""
524         echo "${BOLD}Recommended fix:${NC}"
525         echo " echo 'GATEWAY_BYPASS_PING=true' >> $PROJECT_ROOT/.env"
526         echo " sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \"\"
527         if [ -n "$GATEWAY_TS_IP" ]; then
528             echo " --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
529         else
530             echo " --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true # fill in $GATEWAY_TS_IP"
531         fi
532         echo " sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
533         echo " ./toggle.sh"
534         ;;
535     B)
536         echo ""
537         echo -e " ${RED}${BOLD}VERDICT: Scenario B IPv6 leak over dual-stack campus/local IPv6${NC}"
538         echo " Tailscale exit-node advertises only IPv4. macOS sends AAAA queries"
539         echo " straight out en0, which is dual-stack on most campus APs IPv6"
540         echo " traffic bypasses the WARP tunnel entirely."
541         echo ""
542         echo "${BOLD}Recommended fix:${NC}"
543         echo " sudo networksetup -setv6off $ACTIVE_SERVICE"
544         echo " sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
545         echo " # Re-enable IPv6 later with: sudo networksetup -setv6automatic $ACTIVE_SERVICE"
546         ;;
547     C)
548         echo ""
549         echo -e " ${RED}${BOLD}VERDICT: Scenario C Tailscale route-freeze${NC}"
550         echo " Host's default route points at the Wi-Fi gateway instead of the"
551         echo " Tailscale utun* interface. The exit-node preference is set but the"
552         echo " routing-table assertion did not take effect (devref 10.26)."
553         echo ""
554         echo "${BOLD}Recommended fix:${NC}"
555         echo " sudo brew services restart tailscale"
556         echo " sudo route delete -net 0.0.0.0/1"
557         echo " sudo route delete -net 128.0.0.0/1"
558         echo " sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
559         if [ -n "$GATEWAY_TS_IP" ]; then
560             echo " sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \"\"
561             echo " --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
562         else
563             echo " sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true --exit-node=$TS_IP
564             echo " --exit-node-allow-lan-access=true"
565         fi
566         echo " ./toggle.sh"
567         ;;
568     OK)
569         echo ""
570         echo -e " ${GREEN}${BOLD}VERDICT: No host leak detected tunnel is working${NC}"
571         echo " The local ISP string from whatismyip.com is its IP-database"

```

```

571     echo "    misclassifying the Cloudflare egress IP. CDN-CGI's trace endpoint"
572     echo "    (which is what we use here) reports warp=on because that endpoint"
573     echo "    is hosted by Cloudflare itself and can see its own edge."
574     echo "    If you want a third-party confirmation:"
575     echo "        curl -s https://api.ipify.org; echo"
576     ;;
577 *)
578     echo ""
579     echo -e "    ${YELLOW}${BOLD}VERDICT: Could not classify automatically${NC}"
580     echo "    The diagnostic data did not match any known scenario cleanly."
581     echo "    Likely causes:"
582     echo "        - tailscaled is logged out / expired (see Section 1)"
583     echo "        - Docker is not running (gateway containers down)"
584     echo "        - ADGUARD_IP.txt missing AND docker compose exec failed"
585     echo "    Inspect the report at: $REPORT_FILE"
586     ;;
587 esac
588 echo ""
589
590 #
591 # Write the verdict block to the report file
592 #
593 SECOND_BLOCK=$(cat <<EOF
594
595 C L A S S I F I C A T I O N   R E S U L T
596
597 Scenario: $SCENARIO
598 WARP egress: $WARP_STATUS (cdn-cgi/trace)
599 SOCKS5 lane: $([ "$SOCKS5_ENABLED" = true ] && echo "ENABLED (browsers bypass)" || echo "off")
600 IPv6 leak: $([ "$IPV6_LEAK" = true ] && echo "YES (campus IPv6 egress)" || echo "no")
601 Default via: $([ "$DEFAULT_VIA_UTUN" = true ] && echo "utun* (Tailscale)" || echo "Wi-Fi gateway")
602 Host on mesh: $([ "$TS_ON_MESH" = true ] && echo "yes" || echo "no")
603 SE pending: $([ "$SE_PENDING_UNINSTALL" = true ] && echo "yes" || echo "no")
604 Gateway IP: ${GATEWAY_TS_IP:-unresolved}
605 EOF
606 )
607
608 printf '%s\n' "$SECOND_BLOCK" >&3
609
610 #
611 # --fix mode
612 #
613 if [ "$DO_FIX" = "true" ] && [ "$SCENARIO" != "OK" ] && [ "$SCENARIO" != "UNKNOWN" ]; then
614     echo ""
615     echo ""
616     echo " A P P L Y I N G   F I X (scenario $SCENARIO)"
617     echo ""
618     echo ""
619
620     case "$SCENARIO" in
621         SE_PENDING)
622             warn "System Extension uninstall is pending on next reboot."
623             line "Please run 'sudo shutdown -r now' to reboot the system."
624             ;;
625         A)
626             if [ -n "$GATEWAY_TS_IP" ]; then
627                 line "writing GATEWAY_BYPASS_PING=true to .env"
628                 ENV="$PROJECT_ROOT/.env"
629                 [ -f "$ENV" ] || touch "$ENV"
630                 if grep -q '^GATEWAY_BYPASS_PING=' "$ENV" 2>/dev/null; then
631                     sed -i 's/^GATEWAY_BYPASS_PING=.*GATEWAY_BYPASS_PING=true/' "$ENV"
632                 else
633                     printf '\nGATEWAY_BYPASS_PING=true\n' >> "$ENV"
634                 fi
635                 line "re-asserting tailscale exit-node (with --reset)"
636                 sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
637                     --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
638                     >> "$LOG_FILE" 2>&1 \
639                     && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
640                     || warn "tailscale up did not return success (check $LOG_FILE)"
641             else
642                 warn "no gateway Tailscale IP resolved skipping tailscale up"
643                 warn "fix manually: 'sudo tailscale up --reset ... --exit-node=<TS_IP>'"
644             fi
645             ;;
646         B)
647             line "disabling IPv6 on $ACTIVE_SERVICE while gateway is up"
648             sudo -n networksetup -setv6off "$ACTIVE_SERVICE" >> "$LOG_FILE" 2>&1 \
649                 && ok "IPv6 disabled on $ACTIVE_SERVICE (re-enable with 'setv6automatic')" \
650                 || warn "networksetup -setv6off failed (check $LOG_FILE)"
651             ;;

```



```

652 C)
653 line " restarting tailscaled (route-table fix)"
654 run_with_timeout 15 brew services restart tailscale >> "$LOG_FILE" 2>&1 \
655 || run_with_timeout 15 sudo -n brew services restart tailscale >> "$LOG_FILE" 2>&1
656 sleep 3
657 line " flushing stale host routes + DNS cache"
658 sudo -n route delete -net 0.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
659 sudo -n route delete -net 128.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
660 sudo -n dscacheutil -flushcache >> "$LOG_FILE" 2>&1 || true
661 sudo -n killall -HUP mDNSResponder >> "$LOG_FILE" 2>&1 || true
662 if [ -n "$GATEWAY_TS_IP" ]; then
663     line " re-asserting exit-node after restart"
664     sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
665         --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
666         >> "$LOG_FILE" 2>&1 \
667         && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
668         || warn "tailscale up did not return success (check $LOG_FILE)"
669 fi
670 ;;
671 esac
672
673 echo ""
674 echo "Re-verifying after fix..."
675 sleep 3
676 CDN_WARP_AFTER=$(run_with_timeout 8 curl -s --max-time 6 \
677     https://www.cloudflare.com/cdn-cgi/trace 2>&1 \
678     | awk -F=' ' /^warp={print $2; exit})
679 IPV6_AFTER=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
680 printf '\n[AFTER FIX]\n' >&3
681 printf ' warp status: %s\n' "$CDN_WARP_AFTER" >&3
682 printf ' IPv6 egress: %s\n' "${IPV6_AFTER:-none}" >&3
683 line " warp status: ${CDN_WARP_AFTER:-unknown}"
684 line " IPv6 egress: ${IPV6_AFTER:-none}"
685 if [ "$CDN_WARP_AFTER" = "on" ] && [ -z "$IPV6_AFTER" ]; then
686     echo ""
687     ok "Fix verified host is now routing through WARP. Remote clients unaffected."
688 else
689     echo ""
690     warn "Fix did not fully take effect on first try. Likely causes:"
691     line " tailscaled needed a longer settling window (re-run diagnostic in 30s)"
692     line " The classified scenario was wrong; paste this report for triage."
693     line " Tailscale needs a full auth refresh: 'sudo tailscale up'"
694 fi
695 fi
696
697 echo ""
698 echo ""
699 echo " Full report written to:"
700 echo " $REPORT_FILE"
701 echo ""
702 echo ""
703
704 #
705 # --watch mode: enter the continuous monitoring loop after baseline diagnostic
706 #
707 if [ "$DO_WATCH" = "true" ]; then
708     # Build baseline from the full diagnostic just completed
709     BASELINE_WARP="$WARP_STATUS"
710     BASELINE_DEFAULT=$(( [ "$DEFAULT_VIA_UTUN" = "true" ] && echo "utun" || echo "wifi/other" )
711
712     if [ "$SCENARIO" != "OK" ]; then
713         warn "Baseline diagnostic found issues (scenario: $SCENARIO)."
714         line " Watch loop will still run alerts fire on any STATE CHANGE from current baseline."
715         line " Fix the issue first? Re-run with: bash scripts/diagnose-host-leak.sh --fix"
716         echo ""
717     fi
718
719     run_watch_loop "$BASELINE_WARP" "$BASELINE_DEFAULT" "$IPV6_LEAK" "$WATCH_INTERVAL"
720 fi
721
722 exit 0

```

31 scripts/dns-proxy.py

```

1 #!/usr/bin/env python3
2 """

```

```

3 dns-proxy.py Local DNS proxy for nullexit gateway.
4
5 Listens on UDP port 53, receives DNS queries, forwards them over TCP
6 to AdGuard at 127.0.0.1:5354 (the Docker port mapping), and returns
7 the response via UDP. Handles the DNS-over-TCP wire format (2-byte
8 length prefix) correctly unlike socat which just forwards raw bytes.
9 """
10
11 import socket
12 import struct
13 import sys
14 import os
15 from concurrent.futures import ThreadPoolExecutor
16
17 LISTEN_ADDR = os.environ.get("LISTEN_ADDR", "127.0.0.1")
18 LISTEN_PORT = int(os.environ.get("LISTEN_PORT", 53))
19 TARGET_HOST = os.environ.get("TARGET_HOST", "127.0.0.1")
20 TARGET_PORT = int(os.environ.get("TARGET_PORT", 5354))
21
22
23 def handle_query(data: bytes, client_addr: tuple, sock: socket.socket) -> None:
24     """Forward a single DNS query over TCP and send the response back via UDP."""
25     try:
26         tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
27         tcp.settimeout(5)
28         tcp.connect((TARGET_HOST, TARGET_PORT))
29
30         # DNS over TCP: prepend 2-byte length (big-endian)
31         tcp.sendall(struct.pack("!H", len(data)) + data)
32
33         # Read the 2-byte response length
34         raw_len = tcp.recv(2)
35         if len(raw_len) < 2:
36             tcp.close()
37             return
38         resp_len = struct.unpack("!H", raw_len)[0]
39
40         # Read the full response
41         response = b""
42         while len(response) < resp_len:
43             chunk = tcp.recv(resp_len - len(response))
44             if not chunk:
45                 break
46             response += chunk
47
48         tcp.close()
49
50         # Send the response back over UDP (strip TCP length prefix)
51         if response:
52             sock.sendto(response, client_addr)
53     except Exception:
54         # Silently drop failed queries (malformed, timeout, etc.)
55         pass
56
57
58 def main() -> None:
59     # Create UDP socket
60     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
61     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
62
63     try:
64         sock.bind((LISTEN_ADDR, LISTEN_PORT))
65     except PermissionError:
66         print(f"dns-proxy: ERROR need root to bind port {LISTEN_PORT}", flush=True)
67         sys.exit(1)
68     except OSError as e:
69         print(f"dns-proxy: ERROR {e}", flush=True)
70         sys.exit(1)
71
72     print(f"dns-proxy: listening on UDP {LISTEN_ADDR}:{LISTEN_PORT} TCP {TARGET_HOST}:{TARGET_PORT}", flush=True)
73
74     # Use a thread pool to handle concurrent DNS queries efficiently without thread exhaustion
75     executor = ThreadPoolExecutor(max_workers=64)
76
77     while True:
78         try:
79             data, client_addr = sock.recvfrom(4096) # Support EDNS0 (up to 4096 bytes) to prevent truncation
80             executor.submit(handle_query, data, client_addr, sock)
81         except KeyboardInterrupt:
82             break

```

```

83         except Exception:
84             pass
85
86         executor.shutdown(wait=False)
87         sock.close()
88
89
90 if __name__ == "__main__":
91     main()

```

32 scripts/fix-docker-bridge-collision.sh

```

1  #!/bin/bash
2  # scripts/fix-docker-bridge-collision.sh  nullexit fix for devref 9.13
3  #
4  # Symptom: host default route points at Docker's bridge gateway (172.17.0.1)
5  # instead of the real Wi-Fi gateway. ALL host internet egress fails. Remote
6  # tailnet clients route through the gateway correctly (their tunnel is
7  # independent of the host's broken route table), so they look healthy while
8  # the host itself cannot reach anything.
9  #
10 # Root cause: Docker's default bridge claims 172.17.0.0/16 by default. When the
11 # host's physical Wi-Fi also lands in the same /16 extremely common on
12 # university networks which hand out 172.16.0.0/12 the kernel
13 # installs Docker's bridge as the default route, and every packet bound for
14 # the internet is silently absorbed by docker0 instead of egressing.
15 #
16 # This script:
17 # 1. Detects the actual collision (Docker's 172.17.0.0/16 vs the host's
18 #    Wi-Fi subnet, queried live via the ACTIVE interface not hardcoded en0)
19 # 2. Picks a non-colliding Docker bip using /8-family routing (works for
20 #    172.16.0.0/12 campus networks; naive prefix-match would pick 172.26/24
21 #    which still lives INSIDE a /12 of 172.16.0.0/12)
22 # 3. Backs up ~/.colima/default/colima.yaml with an ISO-8601 timestamp
23 # 4. Edits the file in place, idempotent (replace existing docker.bip /
24 #    append new docker.bip without disturbing other keys; skip YAML comments)
25 # 5. Restarts Colima so the VM rebuilds with the new bridge subnet
26 # 6. Rebinds Wi-Fi DHCP so the host re-learns its real Wi-Fi gateway
27 # 7. Verifies the new default route is no longer the Docker bridge
28 # 8. Re-runs toggle.sh to bring the gateway back up cleanly
29 # 9. Re-runs scripts/diagnose-host-leak.sh and prints the new verdict
30 #
31 # Usage:
32 #   bash scripts/fix-docker-bridge-collision.sh          # apply the fix
33 #   bash scripts/fix-docker-bridge-collision.sh --dry    # show what would change, then exit
34 #   bash scripts/fix-docker-bridge-collision.sh --skip-toggle # fix but don't restart gateway
35 #   bash scripts/fix-docker-bridge-collision.sh --help   # usage
36
37 set -uo pipefail
38 # NOTE: errexit (`set -e`) is intentionally NOT enabled so intermediate
39 # `command || warn` patterns can continue past non-fatal failures.
40 # Every critical failure (colima restart, sed, cp backup, toggle.sh)
41 # uses an explicit `|| die` so a half-applied state never escapes.
42
43 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
44 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
45 LOG_FILE="$PROJECT_ROOT/output.log"
46 source "$SCRIPT_DIR/common.sh"
47 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
48 COLIMA_YAML="$HOME/.colima/default/colima.yaml"
49 PLATFORM="$(uname -s)"
50
51 # Argument parsing
52 DRY_RUN=false
53 SKIP_TOGGLE=false
54 case "${1:-}" in
55     --dry|-n)      DRY_RUN=true ;;
56     --skip-toggle) SKIP_TOGGLE=true ;;
57     --help|-h)
58         sed -n '28,37p' "$0"
59         exit 0
60     ;;
61     *)
62         : # default: apply
63     ;;
64 *)

```

```

65     printf 'Unknown argument: %s\n' "$1" >&2
66     exit 2
67 ;;
68 esac
69
70
71
72 # Helper: print "[dry-run] $cmd" or actually run $cmd. Wraps the redirect
73 # INSIDE the function so outer `>> $FILE` statements never accidentally
74 # append dry-run noise to the real file. (Earlier draft didn't and the
75 # dry-run polluted $COLIMA_YAML.)
76 run() {
77     if [ "$DRY_RUN" = "true" ]; then
78         printf '    [dry-run] %s\n' "$*"
79         return 0
80     fi
81     printf '    $ %s\n' "$*"
82     "$@"
83 }
84
85 # Platform-aware sed in-place. macOS BSD sed needs an empty argument;
86 # GNU sed (Linux) does not. Earlier draft used `-i ''` everywhere and
87 # would have failed on Linux silently.
88 sed_inplace() {
89     local expr="$1"
90     local file="$2"
91     case "$PLATFORM" in
92         Darwin) run sed -i '' "$expr" "$file" ;;
93         *)      run sed -i "$expr" "$file" ;;
94     esac
95 }
96
97 # 0. Resolve the ACTIVE physical interface once (used everywhere)
98 # Replaces the earlier `en0` hardcoding. Same algorithm recover.sh uses:
99 # `route get default` if utun/tun, walk en0..en5 first with `status: active`
100 # + `inet` fall back to en0. This means the script also works on hosts
101 # whose primary NIC is en1/en2 (Thunderbolt Ethernet, USB-LAN, etc.).
102 resolve_active_iface() {
103     local iface
104     iface=$(route get default 2>> "$LOG_FILE" | awk '/interface:/{print $2; exit}')
105     if [ -z "$iface" ] || [ "${iface#utun}" != "$iface" ] || [ "${iface#tun}" != "$iface" ]; then
106         for i in en0 en1 en2 en3 en4 en5; do
107             if ifconfig "$i" 2>> "$LOG_FILE" | grep -q "status: active" \
108                 && ifconfig "$i" 2>> "$LOG_FILE" | grep -q "inet" ; then
109                 iface="$i"; break
110             fi
111         done
112     fi
113     [ -z "$iface" ] && iface="en0"
114     printf '%s\n' "$iface"
115 }
116 ACTIVE_IFACE=$(resolve_active_iface)
117
118 # 1. Detect collision
119 step "1/9 Detecting Docker-bridge vs Wi-Fi subnet collision"
120
121 # Host subnet on the ACTIVE interface (not hardcoded en0).
122 HOST_SUBNET=$(ifconfig "$ACTIVE_IFACE" 2>> "$LOG_FILE" \
123 | awk '/inet /{print $2; exit}')
124 HOST_GATEWAY=$(route get default 2>> "$LOG_FILE" \
125 | awk '/gateway:/{print $2; exit}')
126 DEFAULT_DEV=$(route get default 2>> "$LOG_FILE" \
127 | awk '/interface:/{print $2; exit}')
128
129 printf '    active iface: %s\n' "$ACTIVE_IFACE"
130 printf '    iface inet: %s\n' "${HOST_SUBNET:-none}"
131 printf '    default via: %s\n' "${HOST_GATEWAY:-none}"
132 printf '    default dev: %s\n' "${DEFAULT_DEV:-none}"
133
134 # Collision fires when the host's default-route gateway is 172.17.0.1 (Docker's
135 # bridge) OR when the host subnet is in 172.17.0.0/16. The full-bridge-collision
136 # case (gateway is literally the docker bridge IP) is the smoking gun.
137 COLLISION=false
138 if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#172.17.}" != "$HOST_GATEWAY" ]; then
139     COLLISION=true
140     fail "default gateway $HOST_GATEWAY collides with Docker's default bridge (172.17.0.0/16)"
141 elif [ -n "$HOST_SUBNET" ] && [ "${HOST_SUBNET#172.17.}" != "$HOST_SUBNET" ]; then
142     COLLISION=true
143     fail "host subnet $HOST_SUBNET overlaps Docker's 172.17.0.0/16 bridge"
144 else
145     ok "no collision detected between host Wi-Fi and Docker's default bridge"

```

```

146 fi
147
148 if [ "$COLLISION" = "false" ]; then
149     echo ""
150     echo "Nothing to fix. If you still see egress issues, run:"
151     echo "    bash scripts/diagnose-host-leak.sh"
152     exit 0
153 fi
154
155 # 2. Pick non-colliding bip via /8-family routing
156 step "2/9 Picking a non-colliding Docker bip for ~/.colima/default/colima.yaml"
157
158 # Earlier draft tried naive prefix-match: "if candidate's 3-octet prefix is
159 # not a literal prefix of host gateway, pick it". That breaks on /12 campus
160 # networks (172.16.0.0/12 covers 172.16-172.31): a candidate like
161 # 172.26.0.1/24 LIVES INSIDE the host's /12 even though the prefix test
162 # passes. Fix: classify the host's address family FIRST, then choose ALL
163 # candidates from a DIFFERENT RFC1918 family. /8 boundaries are coarse
164 # enough that no /24 picked this way can ever collide.
165 case "$HOST_GATEWAY" in
166     172.16.*|172.17.*|172.18.*|172.19.*|172.20.*|172.21.*|172.22.*|172.23.*|\
167     172.24.*|172.25.*|172.26.*|172.27.*|172.28.*|172.29.*|172.30.*|172.31.*)
168         HOST_FAMILY="172"
169         # Host is in 172.x jump past 172.31 entirely. Pick from 10.x / 192.168.x.
170         CANDIDATES="10.200.0.1/24 10.201.0.1/24 192.168.99.1/24"
171         ;;
172     10.*)
173         HOST_FAMILY="10"
174         # Host is in 10.x pick from 172.x (well outside any commonly-deployed
175         # 10.0.0.0/8 slice) / 192.168.x.
176         CANDIDATES="172.26.0.1/24 172.28.0.1/24 192.168.99.1/24"
177         ;;
178     192.168.*)
179         HOST_FAMILY="192"
180         CANDIDATES="172.26.0.1/24 10.200.0.1/24 10.201.0.1/24"
181         ;;
182     *)
183         HOST_FAMILY="unknown"
184         warn "host gateway $HOST_GATEWAY is not in any RFC1918 range (/8 family)"
185         warn "falling back to abstract candidate set; verify collision-free manually"
186         CANDIDATES="172.26.0.1/24 10.200.0.1/24 192.168.99.1/24"
187         ;;
188 esac
189
190 printf ' host address family: %s\n' "$HOST_FAMILY"
191 printf ' candidate pool:      %s\n' "$CANDIDATES"
192
193 # Sanity-check the chosen pool has at least one entry that doesn't literally
194 # share a /24 with the host gateway. Naive check (just /24 prefix) is fine
195 # here because we already picked across families.
196 CHOSEN=""
197 for cand in $CANDIDATES; do
198     PREFIX3=$(printf '%s' "$cand" | cut -d. -f1-3)
199     if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#$PREFIX3}" != "$HOST_GATEWAY" ]; then
200         warn "skip $cand (literal /24 collision with host gateway $HOST_GATEWAY)"
201         continue
202     fi
203     CHOSEN="$cand"
204     ok "chose bip=$CHOSEN (cross-family, no /24 literal collision)"
205     break
206 done
207
208 if [ -z "$CHOSEN" ]; then
209     CHOSEN="172.26.0.1/24"
210     warn "could not find a clean candidate; defaulting to $CHOSEN"
211     warn "verify with 'route get default' after restart"
212 fi
213
214 # 3. Backup colima.yaml
215 step "3/9 Backing up $COLIMA_YAML"
216 mkdir -p "$(dirname "$COLIMA_YAML")"
217 BACKUP="$HOME/.colima/default/colima.yaml.bak.$TIMESTAMP"
218 if [ -f "$COLIMA_YAML" ]; then
219     # `cp` is safe in dry-run because run() suppresses the inner command
220     # entirely no chance of accidentally writing a real backup.
221     run cp -p "$COLIMA_YAML" "$BACKUP" || die "backup failed"
222     if [ "$DRY_RUN" = "false" ]; then ok "backup: $BACKUP"; fi
223 else
224     warn "no existing $COLIMA_YAML (will create fresh)"
225 fi
226

```

```

227 # 4. Edit colima.yaml in place (idempotent)
228 step "4/9 Setting docker.bip=$CHOSEN in colima.yaml (in place)"
229
230 # Resolve the existing docker.bip (if any) without picking up commented-out
231 # YAML keys. awk-based extraction; the negated-comment check (`! /^[[:space:]]*#`)
232 # ensures `# bip: 1.2.3.4/24` is treated as a comment, not the active value.
233 EXISTING_BIP=""
234 if [ -f "$COLIMA_YAML" ]; then
235     EXISTING_BIP=$(awk '
236         /^[[:space:]]*#/{next}
237         /^[[:space:]]*bip:[[:space:]]*/ {print $2; exit}
238     ' "$COLIMA_YAML" 2>> "$LOG_FILE" || true)
239 fi
240
241 if [ -n "$EXISTING_BIP" ] && [ "$EXISTING_BIP" = "$CHOSEN" ]; then
242     ok "colima.yaml already has docker.bip=$CHOSEN no edit needed"
243 elif [ -n "$EXISTING_BIP" ]; then
244     printf ' (replacing existing docker.bip=%s with %s)\n' "$EXISTING_BIP" "$CHOSEN"
245     # Use sed inplace so Linux / macOS get the right syntax. Skip YAML comment
246     # lines so existing `# bip:` comments aren't rewired.
247     sed_inplace "s|^\([[:space:]]*bip:[[:space:]]*\).*\${1}$CHOSEN|" "$COLIMA_YAML" \
248     || die "sed replacement failed (colima.yaml malformed?)"
249     ok "colima.yaml updated"
250 else
251     # No existing `bip:` key. Branch:
252     # - file has `docker:` block append `bip:` under it
253     # - file has no `docker:` block append new docker: section
254     # - file does not exist create fresh
255     # Each branch is dry-run-aware so no branch can accidentally leak noise
256     # into $COLIMA_YAML during a preview.
257     HAS_DOCKER_BLOCK=false
258     if [ -f "$COLIMA_YAML" ] && grep -q '^docker:' "$COLIMA_YAML"; then
259         HAS_DOCKER_BLOCK=true
260     fi
261
262     if [ "$HAS_DOCKER_BLOCK" = "true" ]; then
263         if [ "$DRY_RUN" = "true" ]; then
264             printf ' [dry-run] would append to %s:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
265         else
266             printf '\n bip: %s\n' "$CHOSEN" >> "$COLIMA_YAML" \
267             || die "append failed"
268             ok "appended bip under existing docker: block"
269         fi
270     elif [ -f "$COLIMA_YAML" ]; then
271         if [ "$DRY_RUN" = "true" ]; then
272             printf ' [dry-run] would append to %s:\ndocker:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
273         else
274             printf '\ndocker:\n bip: %s\n' "$CHOSEN" >> "$COLIMA_YAML" \
275             || die "append failed"
276             ok "appended new docker: block at end of file"
277         fi
278     else
279         if [ "$DRY_RUN" = "true" ]; then
280             printf ' [dry-run] would create %s with:\ndocker:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
281         else
282             printf 'docker:\n bip: %s\n' "$CHOSEN" > "$COLIMA_YAML" \
283             || die "create failed"
284             ok "created fresh $COLIMA_YAML with docker.bip=$CHOSEN"
285         fi
286     fi
287 fi
288
289 # Always show the final colima.yaml docker section for visibility
290 printf '\n%b colima.yaml `docker:` section now reads:%b\n' "$BOLD" "$NC"
291 awk '
292     /^docker:/{p=1}
293     p && /^[^ ]/ && !/^docker:/{exit}
294     p
295     ' "$COLIMA_YAML" 2>/dev/null | sed 's/^ / /' || true
296
297 if [ "$DRY_RUN" = "true" ]; then
298     printf '\n %b(dry-run: exiting before colima restart / dhcp rebind / toggle.sh)%b\n' "$YELLOW" "$NC"
299     exit 0
300 fi
301
302 # 5. Restart Colima
303 step "5/9 Restarting Colima VM with new bip"
304 if ! command -v colima >> "$LOG_FILE" 2>&1; then
305     die "colima not on PATH install with 'brew install colima' first"
306 fi
307 # No timeout wrapper (would mask actual VM startup time). If colima hangs,

```

```

308 # the user kills with Ctrl+C and can manually `colima restart` from a fresh
309 # terminal the rest of this script is idempotent so re-running works.
310 colima restart >> "$LOG_FILE" 2>&1 || die "colima restart failed; check $LOG_FILE"
311 ok "colima restarted"
312
313 # 6. Rebind DHCP on the ACTIVE interface (not hardcoded en0)
314 step "6/9 Rebinding DHCP on $ACTIVE_IFACE so host re-learns its real gateway"
315
316 # Map ifname macOS network service name (Wi-Fi, USB 10/100 LAN, etc.).
317 # Same logic as the diagnostic + toggle.sh.
318 ACTIVE_SVC=""
319 if [ "$PLATFORM" = "Darwin" ]; then
320     ACTIVE_SVC=$(get_active_service)
321 fi
322
323 case "$PLATFORM" in
324     Darwin)
325         sudo -n networksetup -setdhcp "$ACTIVE_SVC" renew >> "$LOG_FILE" 2>&1 \
326             && ok "DHCP rebind issued on $ACTIVE_SVC" \
327             || warn "setdhcp renew failed; try 'sudo ipconfig set $ACTIVE_IFACE DHCP' manually"
328         ;;
329     Linux)
330         sudo -n dhclient -r "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 || true
331         sudo -n dhclient "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 \
332             && ok "dhclient rebind on $ACTIVE_IFACE" \
333             || warn "dhclient rebind failed"
334         ;;
335     *)
336         warn "unsupported platform $PLATFORM; skip DHCP rebind"
337         ;;
338 esac
339
340 # 7. Verify new default route
341 step "7/9 Verifying the new default route is NOT the Docker bridge"
342 sleep 2 # give DHCP + kernel a beat
343 NEW_DEFAULT=$(netstat -rn 2>> "$LOG_FILE" | awk '/^default/ {print $0; exit}')
344 printf ' %s\n' "${NEW_DEFAULT:-no default route detected}"
345 NEW_GATEWAY=$(printf '%s' "$NEW_DEFAULT" | awk '{print $2; exit}')
346
347 if [ -n "$NEW_GATEWAY" ] && [ "${NEW_GATEWAY#172.17.} " != "$NEW_GATEWAY" ]; then
348     fail "default route STILL points at 172.17.x.x DHCP rebind may need manual touch"
349     warn "try: sudo ipconfig set $ACTIVE_IFACE DHCP (or reconnect Wi-Fi in the menu bar)"
350 elif [ -n "$NEW_GATEWAY" ]; then
351     ok "default route is now via $NEW_GATEWAY (no longer Docker's bridge)"
352 else
353     warn "could not determine new gateway; verify with 'route get default'"
354 fi
355
356 # 8. Re-run toggle.sh to bring the gateway back up
357 if [ "$SKIP_TOGGLE" = "true" ]; then
358     step "8/9 Skipping toggle.sh (--skip-toggle)"
359     warn "gateway is currently down run ./toggle.sh manually when ready"
360 else
361     step "8/9 Re-running toggle.sh to bring the gateway back up"
362     if bash "$PROJECT_ROOT/toggle.sh" 2>&1 | tee -a "$LOG_FILE"; then
363         ok "toggle.sh completed"
364     else
365         warn "toggle.sh returned non-zero see $LOG_FILE for details"
366     fi
367 fi
368
369 # 9. Re-run the diagnostic + show verdict
370 step "9/9 Re-running host-leak diagnostic to confirm fix"
371 if bash "$SCRIPT_DIR/diagnose-host-leak.sh" 2>&1 | tee -a "$LOG_FILE"; then
372     LATEST_REPORT=$(ls -t "$PROJECT_ROOT"/host-leak-diagnostic-*.txt 2>/dev/null | head -1)
373     if [ -n "$LATEST_REPORT" ]; then
374         VERDICT=$(awk '/^ Scenario:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
375         WARP=$(awk -F': ' '/^ WARP egress:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
376         printf '\n Most recent verdict: Scenario=%s WARP=%s\n' \
377             "${VERDICT:-?}" "${WARP:-?}"
378         if [ "$VERDICT" = "OK" ]; then
379             ok "fix confirmed by diagnostic host traffic is going through WARP"
380         fi
381     fi
382 else
383     warn "diagnostic exited non-zero; verify manually"
384 fi
385
386 printf '\n%b%b\n' "$BOLD" "$NC"
387 printf '%bDone.%b Full transcript at %s\n' "$BOLD" "$NC" "$LOG_FILE"
388 printf '%b%b\n' "$BOLD" "$NC"

```


33 scripts/fix-ssh-delay.sh

```

1 #!/bin/bash
2 # scripts/fix-ssh-delay.sh Fix ~20s SSH connection delay caused by UseDNS
3 #
4 # macOS OpenSSH defaults UseDNS to "yes". When Tailscale SSH connects from
5 # a peer (phone/laptop), sshd performs a reverse-DNS (PTR) lookup on the
6 # client's 100.x.x.x Tailscale IP. Since DNS is hijacked to the nullexit
7 # gateway AdGuard Cloudflare WARP, and 100.64.0.0/10 is CGNAT private
8 # space, the PTR query times out (~15-20s) before SSH proceeds.
9 #
10 # This script uncomments "UseDNS no" in /etc/ssh/sshd_config and reloads
11 # sshd. Existing SSH sessions are NOT dropped.
12 #
13 # Usage:
14 #   sudo bash scripts/fix-ssh-delay.sh
15
16 set -euo pipefail
17
18 SSHD_CONFIG="/etc/ssh/sshd_config"
19
20 # Check we're root
21 if [ "$(id -u)" -ne 0 ]; then
22     echo "ERROR: This script must be run with sudo (it edits $SSHD_CONFIG)"
23     echo "Usage: sudo bash scripts/fix-ssh-delay.sh"
24     exit 1
25 fi
26
27 # Apply config (idempotent)
28 if grep -qE '^UseDNS no' "$SSHD_CONFIG" 2>/dev/null; then
29     echo "UseDNS is already set to 'no' in $SSHD_CONFIG"
30 elif grep -qE '^#UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
31     echo "Uncommenting 'UseDNS no' in $SSHD_CONFIG"
32     sed -i 's/^#UseDNS no/UseDNS no/' "$SSHD_CONFIG"
33 elif grep -qEi '^UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
34     echo "Changing existing UseDNS line to 'UseDNS no'"
35     sed -i 's/^UseDNS.*/UseDNS no/' "$SSHD_CONFIG"
36 else
37     echo "Appending 'UseDNS no' to $SSHD_CONFIG"
38     echo "" >> "$SSHD_CONFIG"
39     echo "UseDNS no" >> "$SSHD_CONFIG"
40 fi
41
42 # Verify
43 if ! grep -qE '^UseDNS no' "$SSHD_CONFIG"; then
44     echo "Failed to apply UseDNS no check $SSHD_CONFIG manually"
45     exit 1
46 fi
47
48 # Reload sshd (does NOT drop existing connections)
49 echo "Reloading sshd..."
50
51 SSHD_PID=$(pgrep -f /usr/sbin/sshd 2>/dev/null | head -1 || true)
52 if [ -n "$SSHD_PID" ]; then
53     kill -HUP "$SSHD_PID"
54     echo "Sent SIGHUP to sshd (PID $SSHD_PID) config reloaded"
55 else
56     echo "No running sshd found. macOS starts sshd on-demand per connection."
57     echo "The new 'UseDNS no' config will take effect on the next SSH connection."
58 fi
59
60 echo ""
61 echo "Done. SSH connections should now connect instantly."
62 echo "Existing sessions were not interrupted."

```

34 scripts/generate_tex.py

```

1 #!/usr/bin/env python3
2 import os
3

```

```

4 PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
5
6 IGNORE_DIRS = ['.git', '.github', 'adguard', '__pycache__', 'Recover Gateway.app', 'Toggle Gateway.app']
7 IGNORE_FILES = ['.env', 'nullexit_unified.tex', 'nullexit_unified.pdf', 'output.log', 'blocked.log', '.DS_Store',
8               '.lan_p2p_detected', '.signatures', 'ADGUARD_IP.txt', 'TUNNEL_FAILED_CLOSED.marker']
9 IGNORE_EXTS = ['.pdf', '.tex', '.png', '.jpg', '.jpeg', '.zip', '.tar', '.gz', '.log', '.aux', '.fls', '.fdb_latexmk',
10              '.out', '.toc', '.xdv', '.synctex(busy)']
11
12 # Exclude from code block inclusion, but keep in tree
13 SKIP_CONTENT_FILES = {'LICENSE'}
14
15 PRIORITY_FILES = [
16     'README.md',
17     'agent.md',
18     'devref.md',
19     'diagrams.md',
20     'toggle.sh',
21     'recover.sh',
22     'setup.sh',
23     'scripts/toggle-linux.sh',
24     'scripts/recover-linux.sh',
25     'scripts/setup-linux.sh',
26     'docker-compose.yml',
27     'scripts/common.sh',
28     'scripts/watcher.sh',
29     'scripts/crypto.sh'
30 ]
31
32 def get_language(filename):
33     ext = os.path.splitext(filename)[1].lower()
34     if ext == '.sh' or ext == '.applescript':
35         return 'bash'
36     elif ext == '.py':
37         return 'Python'
38     elif ext == '.go':
39         return 'Go'
40     elif ext in ['.yaml', '.yml']:
41         return 'bash'
42     elif ext == '.plist':
43         return 'XML'
44     elif ext == '.md':
45         return ''
46     return ''
47
48 def escape_tex(text):
49     return text.replace('_', '\\_')
50
51 def generate_tree(dir_path, prefix=""):
52     tree_str = ""
53     entries = sorted(os.listdir(dir_path))
54     entries = [e for e in entries if e not in IGNORE_DIRS and e not in IGNORE_FILES and os.path.splitext(e)[1].lower() not in IGNORE_EXTS]
55
56     for i, entry in enumerate(entries):
57         path = os.path.join(dir_path, entry)
58         is_last = (i == len(entries) - 1)
59         connector = "-- " if is_last else "|-- "
60         tree_str += f"{prefix}{connector}{entry}\n"
61
62         if os.path.isdir(path):
63             extension = " " if is_last else "| "
64             tree_str += generate_tree(path, prefix + extension)
65
66     return tree_str
67
68 def main():
69     tex_path = os.path.join(PROJECT_ROOT, 'nullexit_unified.tex')
70
71     preamble = r"""
72 \documentclass{article}
73 \usepackage[utf8]{inputenc}
74 \usepackage[margin=1in]{geometry}
75 \usepackage{listings}
76 \usepackage{xcolor}
77 \usepackage{hyperref}
78 \usepackage{tocloft}
79
80 \definecolor{codegreen}{rgb}{0,0.6,0}
81 \definecolor{codegray}{rgb}{0.5,0.5,0.5}
82 \definecolor{codepurple}{rgb}{0.58,0,0.82}
83 \definecolor{backcolour}{rgb}{0.97,0.97,0.96}

```

```

82 \lstdefinestyle{mystyle}{
83     backgroundcolor=\color{backcolour},
84     commentstyle=\color{codegreen},
85     keywordstyle=\color{blue}\bfseries,
86     numberstyle=\tiny\color{codegray},
87     stringstyle=\color{codepurple},
88     basicstyle=\ttfamily\scriptsize,
89     breakatwhitespace=false,
90     breaklines=true,
91     captionpos=b,
92     keepspaces=true,
93     numbers=left,
94     numbersep=5pt,
95     showspace=false,
96     showstringspaces=false,
97     showtabs=false,
98     tabsize=2,
99     frame=single,
100     rulecolor=\color{codegray},
101     extendedchars=true,
102     literate={-}1 {}{-}1 {}{{OK}}2 {}{-}1 {}{}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1
    }{}+}1 {}{v}1
103 }
104 \lstset{style=mystyle}
105
106 \title{Nullexit: Unified Project Document}
107 \author{Omar Alaaeldin}
108 \date{\today}
109
110 \begin{document}
111 \maketitle
112
113 \tableofcontents
114 \newpage
115
116 \section{Project Directory Tree}
117 \begin{verbatim}
118 nullexit/
119 """
120
121     tree = generate_tree(PROJECT_ROOT)
122     midamble = r"""end{verbatim}
123 \newpage
124 """
125
126     all_relpats = []
127     for root, dirs, files in os.walk(PROJECT_ROOT):
128         dirs[:] = sorted([d for d in dirs if d not in IGNORE_DIRS])
129         for file in sorted(files):
130             if file in IGNORE_FILES or file in SKIP_CONTENT_FILES or os.path.splitext(file)[1].lower() in
131             IGNORE_EXTS:
132                 continue
133
134             filepath = os.path.join(root, file)
135             relpath = os.path.relpath(filepath, PROJECT_ROOT)
136             all_relpats.append(relpath)
137
138     def sort_key(path):
139         if path in PRIORITY_FILES:
140             return (0, PRIORITY_FILES.index(path))
141         return (1, path)
142
143     all_relpats.sort(key=sort_key)
144
145     with open(tex_path, 'w', encoding='utf-8') as f:
146         f.write(preamble)
147         f.write(tree)
148         f.write(midamble)
149
150         for relpath in all_relpats:
151             lang = get_language(relpath)
152             lang_attr = f"[language={lang}]" if lang else ""
153
154             f.write(f"\section{{{escape_tex(relpath)}}}\n")
155             f.write(f"\begin{{{lstlisting}}}{lang_attr}\n")
156             try:
157                 with open(os.path.join(PROJECT_ROOT, relpath), 'r', encoding='utf-8', errors='ignore') as src:
158                     content = src.read()
159                     import re
160                     # Strip all non-ASCII characters (emojis, box drawings, em dashes, etc)
161                     content = re.sub(r'[\x00-\x7F]+', '', content)

```

```

161         f.write(content)
162         if not content.endswith('\n'):
163             f.write('\n')
164     except Exception as e:
165         f.write(f"Error reading file: {e}\n")
166     f.write("\\end{lstlisting}\n\n")
167
168     f.write("\\end{document}\n")
169
170     print(f"Generated {tex_path}")
171
172 if __name__ == '__main__':
173     main()

```

35 scripts/logger/go.mod

```

1 module nullexit/logger
2
3 go 1.22

```

36 scripts/logger/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "encoding/json"
6     "fmt"
7     "io"
8     "log"
9     "os"
10    "regexp"
11    "strings"
12    "time"
13 )
14
15 const (
16     queryLogPath = "/adguard_work/data/querylog.json"
17     procKmsgPath = "/proc/kmsg"
18     outputLogPath = "/app/blocked.log"
19 )
20
21 func logMessage(msg string) {
22     timestamp := time.Now().Format("2006-01-02 15:04:05")
23     formattedMsg := fmt.Sprintf("%s %s\n", timestamp, msg)
24     fmt.Print(formattedMsg)
25
26     f, err := os.OpenFile(outputLogPath, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
27     if err != nil {
28         log.Printf("Error writing to log file: %v\n", err)
29         return
30     }
31     defer f.Close()
32     f.WriteString(formattedMsg)
33 }
34
35 func tailFile(filepath string, out chan<- string) {
36     for {
37         if _, err := os.Stat(filepath); os.IsNotExist(err) {
38             time.Sleep(1 * time.Second)
39             continue
40         }
41         break
42     }
43
44     f, err := os.Open(filepath)
45     if err != nil {
46         log.Printf("Error opening file %s: %v\n", filepath, err)
47         return
48     }
49     defer func() {
50         if f != nil {

```

```

51     f.Close()
52 }
53 }()
54
55 // Seek to end
56 f.Seek(0, io.SeekEnd)
57 reader := bufio.NewReader(f)
58
59 for {
60     line, err := reader.ReadString('\n')
61     if err != nil {
62         if err == io.EOF {
63             stat, err := os.Stat(filepath)
64             if err == nil {
65                 currentOffset, _ := f.Seek(0, io.SeekCurrent)
66                 if stat.Size() < currentOffset {
67                     // File truncated or rotated
68                     f.Close()
69                     for {
70                         if _, err := os.Stat(filepath); os.IsNotExist(err) {
71                             time.Sleep(1 * time.Second)
72                             continue
73                         }
74                         break
75                     }
76                     f, _ = os.Open(filepath)
77                     reader = bufio.NewReader(f)
78                     continue
79                 }
80             }
81             time.Sleep(500 * time.Millisecond)
82             continue
83         }
84         log.Printf("Error reading file %s: %v\n", filepath, err)
85         time.Sleep(1 * time.Second)
86         continue
87     }
88     out <- line
89 }
90 }
91
92 type adguardLogEntry struct {
93     IP      string `json:"IP"`
94     QH      string `json:"QH"`
95     QT      string `json:"QT"`
96     Result struct {
97         IsFiltered bool `json:"IsFiltered"`
98         Reason      int  `json:"Reason"`
99         Rules       []struct {
100             Text string `json:"Text"`
101         } `json:"Rules"`
102     } `json:"Result"`
103 }
104
105 func monitorDNS() {
106     logMessage("[System] Starting DNS block logger...")
107     lines := make(chan string)
108     go tailFile(queryLogPath, lines)
109
110     reasonMap := map[int]string{
111         2: "CustomRule",
112         3: "BlockList",
113         4: "SafeBrowsing",
114         5: "ParentalControl",
115         6: "SafeSearch",
116         7: "BlockedService",
117     }
118
119     for line := range lines {
120         var data adguardLogEntry
121         if err := json.Unmarshal([]byte(line), &data); err != nil {
122             continue
123         }
124
125         res := data.Result
126         if res.IsFiltered || (res.Reason != 0 && res.Reason != 1) {
127             qh := data.QH
128             if qh == "" {
129                 qh = "unknown"
130             }
131             qt := data.QT

```

```

132     if qt == "" {
133         qt = "unknown"
134     }
135     clientIP := data.IP
136     if clientIP == "" {
137         clientIP = "unknown"
138     }
139
140     ruleText := "unknown rule"
141     if len(res.Rules) > 0 && res.Rules[0].Text != "" {
142         ruleText = res.Rules[0].Text
143     }
144
145     reasonStr, ok := reasonMap[res.Reason]
146     if !ok {
147         reasonStr = fmt.Sprintf("Reason-%d", res.Reason)
148     }
149
150     logMessage(fmt.Sprintf("[DNS] Blocked %s (Type: %s) for client %s | Reason: %s | Rule: %s", qh, qt,
151         clientIP, reasonStr, ruleText))
152     }
153 }
154
155 func monitorIPs() {
156     logMessage("[System] Starting IP block logger...")
157
158     prefixRe := regexp.MustCompile(`(IP_BLOCK_[A-Z_]+):`)
159     srcRe := regexp.MustCompile(`SRC=([0-9a-fA-F\.:]+)`)
160     dstRe := regexp.MustCompile(`DST=([0-9a-fA-F\.:]+)`)
161     protoRe := regexp.MustCompile(`PROTO=([A-Z0-9]+)`)
162     sptRe := regexp.MustCompile(`SPT=(\d+)`)
163     dptRe := regexp.MustCompile(`DPT=(\d+)`)
164     inRe := regexp.MustCompile(`IN=([a-zA-Z0-9\.\-_]+)`)
165     outRe := regexp.MustCompile(`OUT=([a-zA-Z0-9\.\-_]+)`)
166
167     f, err := os.Open(procKmsgPath)
168     if err != nil {
169         if os.IsPermission(err) {
170             logMessage("[System] ERROR: Insufficient permissions to read /proc/kmsg. Enable CAP_SYSLOG.")
171         } else {
172             logMessage(fmt.Sprintf("[System] ERROR in IP logger: %v", err))
173         }
174         return
175     }
176     defer f.Close()
177
178     reader := bufio.NewReader(f)
179     for {
180         line, err := reader.ReadString('\n')
181         if err != nil {
182             if err == io.EOF {
183                 time.Sleep(100 * time.Millisecond)
184                 continue
185             }
186             logMessage(fmt.Sprintf("[System] ERROR reading kmsg: %v", err))
187             time.Sleep(1 * time.Second)
188             continue
189         }
190
191         if strings.Contains(line, "IP_BLOCK") {
192             match := prefixRe.FindStringSubmatch(line)
193             if len(match) > 1 {
194                 prefix := match[1]
195
196                 src := "unknown"
197                 if m := srcRe.FindStringSubmatch(line); len(m) > 1 {
198                     src = m[1]
199                 }
200
201                 dst := "unknown"
202                 if m := dstRe.FindStringSubmatch(line); len(m) > 1 {
203                     dst = m[1]
204                 }
205
206                 proto := "unknown"
207                 if m := protoRe.FindStringSubmatch(line); len(m) > 1 {
208                     proto = m[1]
209                 }
210
211                 spt := ""

```

```

212     if m := sptRe.FindStringSubmatch(line); len(m) > 1 {
213         spt = m[1]
214     }
215
216     dpt := ""
217     if m := dptRe.FindStringSubmatch(line); len(m) > 1 {
218         dpt = m[1]
219     }
220
221     inIf := ""
222     if m := inRe.FindStringSubmatch(line); len(m) > 1 {
223         inIf = m[1]
224     }
225
226     outIf := ""
227     if m := outRe.FindStringSubmatch(line); len(m) > 1 {
228         outIf = m[1]
229     }
230
231     direction := "inbound"
232     if strings.HasSuffix(prefix, "DST") {
233         direction = "outbound"
234     }
235
236     listType := "MALICIOUS"
237     if !strings.Contains(prefix, "MALICIOUS") {
238         parts := strings.Split(prefix, "_")
239         if len(parts) >= 3 {
240             listType = "GEO_" + parts[2]
241         }
242     }
243
244     portStr := ""
245     if dpt != "" {
246         portStr = ":" + dpt
247     }
248
249     srcPortStr := ""
250     if spt != "" {
251         srcPortStr = ":" + spt
252     }
253
254     ifStr := ""
255     if inIf != "" && outIf != "" {
256         ifStr = fmt.Sprintf("IF: %s->%s", inIf, outIf)
257     } else if inIf != "" || outIf != "" {
258         ifStr = fmt.Sprintf("IF: %s%s", inIf, outIf)
259     }
260
261     logMessage(fmt.Sprintf("[IP] Blocked %s to %s%s (Proto: %s) from %s%s (%s) | List: %s", direction, dst,
        portStr, proto, src, srcPortStr, ifStr, listType))
262 }
263 }
264 }
265 }
266
267 func main() {
268     logMessage("[System] Nullexit Block Logger starting...")
269
270     go monitorDNS()
271     monitorIPs()
272 }

```

37 scripts/pf.conf

```

1 # scripts/pf.conf - Packet Filter configuration for nullexit killswitch
2 # Dynamically parameterized by pfctl macro override
3
4 # ext_if must be passed via command line, e.g. -D ext_if=en0
5 # If not passed, default to en0
6 ext_if = "en0"
7 ts_if = "utun+"
8
9 # Tables populated dynamically via pfctl
10 table <vpn_endpoints> persist
11 table <derp_relays> persist

```



```

12
13 # Core Rules
14 set skip on lo0 # Ensure local loopback is never blocked
15 scrub out all max-mss 1160 # TCP MSS Clamping: Prevents fragmentation on host TCP flows (like Tailscale
    control plane)
16 pass quick on $tst_if all # Allow all Tailscale traffic (essential to prevent SSH lockout)
17
18 # Default deny outbound WAN traffic
19 block out on $ext_if all
20
21 # Block inbound Screen Sharing (VNC) and SSH from local Wi-Fi, forcing them to use Tailscale
22 block in on $ext_if proto tcp from any to any port { 22, 5900 }
23
24 # Exceptions for tunnel handshakes and coordination
25 pass out quick on $ext_if proto udp to <vpn_endpoints> port 2408
26 pass out quick on $ext_if proto udp to <derp_relays>
27 pass out quick on $ext_if proto tcp to 192.200.0.0/24 port 443
28
29 # Allow local LAN traffic (AirDrop, local Tailscale P2P, etc)
30 pass out quick on $ext_if proto tcp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
31 pass out quick on $ext_if proto udp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
32 pass out quick on $ext_if proto icmp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
33
34 # Allow DHCP outbound (client port 68 to server port 67) so we can obtain/renew IP address
35 pass out quick on $ext_if proto udp from any port 68 to any port 67

```

38 scripts/reinstall-host-tailscale.sh

```

1 #!/usr/bin/env bash
2 #
3 # scripts/reinstall-host-tailscale.sh
4 #
5 # Complete uninstall + reinstall of the host-side Homebrew Tailscale.
6 #
7 # Use this when the brew LaunchAgent is wedged, Login Items entries are
8 # duplicated, "Tailscale is stopped" persists despite
9 # `brew services restart tailscale`, or you simply want a clean baseline
10 # before re-engaging the gateway as exit node.
11 #
12 # Idempotent. Safe to re-run. Use --dry to preview every step with zero
13 # changes. Use --yes to skip the per-step confirmation prompts.
14 #
15 # This script ONLY touches the HOST-side Tailscale (the one installed by
16 # Homebrew `brew install tailscale` and managed by launchd as
17 # homebrew.mxcl.tailscale). The nullexit gateway container's tailscaled
18 # (which lives inside the Colima VM at 100.100.21.8 and offers the exit
19 # node to this host) is unaffected.
20
21 set -u
22
23 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
24
25 DRY=0
26 YES=0
27
28 for arg in "$@"; do
29     case "$arg" in
30         --dry) DRY=1 ;;
31         --yes) YES=1 ;;
32         --help|-h) # NOTE: heredoc is UNQUOTED so $SCRIPT_DIR expands in the body. If you
33             # add new $-style placeholders here, they'll expand the same way.
34             cat <<USAGE
35 Usage: bash "$SCRIPT_DIR/reinstall-host-tailscale.sh" [--dry] [--yes]
36
37 --dry Print every step, make zero changes. Use to audit before
38 committing.
39 --yes Auto-confirm the prompts at every destructive step. Useful in
40 CI / fully-scripted recovery; do not use until you've read
41 the script.
42
43 After this script completes cleanly, you still must (MANUALLY):
44
45 1. Open System Settings Privacy & Security Local Network and
46 enable Tailscale. macOS will prompt automatically the first time
47 the fresh install of `tailscale` is invoked.
48

```

```

49 2. Re-engage the tailnet + exit node:
50     sudo tailscale up --ssh=true --accept-dns=false \
51         --exit-node="$(cat ADGUARD_IP.txt | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
52         --exit-node-allow-lan-access=true
53
54 3. Re-run ./toggle.sh from the project root.
55
56 4. Final verification:
57     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
58     expected: warp=on, ip=<Cloudflare>, colo=YYZ/YUL
59 USAGE
60     exit 0
61     ;;
62     *)
63     echo "Unknown flag: $arg" >&2
64     exit 2
65     ;;
66     esac
67 done
68
69 # ----- helpers -----
70
71 confirm() {
72     if [ "$YES" = 1 ] || [ "$DRY" = 1 ]; then
73         return 0
74     fi
75     printf "      %s [y/N] " "$1"
76     read -r ans
77     case "$ans" in
78         y|Y|yes|YES) return 0 ;;
79         *)
80             echo "      aborted by user"
81             exit 1
82             ;;
83     esac
84 }
85
86 header() {
87     printf '\n %s\n' "$1"
88 }
89
90 # with_timeout <seconds> <cmd...>
91 # Run a command in the background and kill it if it exceeds <seconds>.
92 # Returns the command's exit code if it finished in time, 124 (matching
93 # GNU `timeout` convention) if it timed out. macOS doesn't ship GNU
94 # coreutils `timeout`, so this is the bash-3.2-friendly replacement.
95 # stdout + stderr are discarded (we only care about exit code); redirect
96 # or pipe externally if you need the output.
97 with_timeout() {
98     local timeout_s="${1:-30}"
99     shift
100     "$@" >/dev/null 2>&1 &
101     local cmdpid=$!
102     local elapsed=0
103     while kill -0 "$cmdpid" 2>/dev/null; do
104         if [ "$elapsed" -ge "$timeout_s" ]; then
105             kill -9 "$cmdpid" 2>/dev/null
106             wait "$cmdpid" 2>/dev/null
107             return 124
108         fi
109         sleep 1
110         elapsed=$((elapsed+1))
111     done
112     wait "$cmdpid" 2>/dev/null
113     return $?
114 }
115
116 # ----- 0. Pre-flight -----
117 header "Pre-flight current host tailscale state"
118 echo "      brew: $(command -v brew >/dev/null 2>&1 && brew --version | head -1 || echo 'not on
119 PATH')"
119 echo "      tailscaled running: $(pgrep -lf tailscaled 2>/dev/null | wc -l | tr -d ' ') process(es)"
120 echo "      brew service tailscale: $(brew services list 2>/dev/null | awk '$1=="tailscale"{print $2, $3}' | tr
121 '\n' ' ' | sed 's/ $//')"
121 echo "      LaunchAgent plists (under ~/Library/LaunchAgents):"
122 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/~/ /' || echo "      (none)"
123
124 # Detect Tailscale's macOS Network Extension (residual from prior App Store
125 # Tailscale.app installs). Lives under /Library/Apple/SystemExtensions/ and
126 # is managed by `systemextensionsctl` NOT launchd. If loaded, brew
127 # tailscaled will NOT engage the Network Extension framework slot because

```

```

128 # the SE still has it claimed. Symptom: 'sudo tailscale status' returns
129 # "Tailscale is stopped" even when the daemon is alive and listening.
130 se_found=0
131 if command -v systemextensionsctl >/dev/null 2>&1; then
132     # Grab any line matching tailscale and network-extension
133     se_line=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale\..*network-extension' | head -
134         n 1)
135     if [ -n "$se_line" ]; then
136         se_found=1
137         # Extract the teamID (10-char uppercase/numeric word)
138         se_teamID=$(echo "$se_line" | tr -s ' ' '\n' | grep -E '^[A-Z0-9]{10}$' | head -n 1)
139         # Extract the bundleID (starts with io.tailscale or com.tailscale)
140         se_bundleID=$(echo "$se_line" | tr -s ' ' '\n' | grep -E '^(io|com)\.tailscale\.' | head -n 1 | sed 's
141             /(.*//)')
142         # If teamID is not found, fallback to '-'
143         if [ -z "$se_teamID" ]; then
144             se_teamID="-"
145         fi
146     fi
147 fi
148 if [ "$se_found" = 1 ] && [ -n "$se_bundleID" ]; then
149     echo "    Tailscale Network Extension System Extension is loaded (residual from App Store Tailscale.app
150         install)"
151     echo "    Team ID: $se_teamID, Bundle ID: $se_bundleID"
152     echo "    brew tailscaled cannot engage the NE slot while this SE has it claimed."
153     if [ "$DRY" = 1 ]; then
154         echo "    [dry] (would prompt) sudo systemextensionsctl uninstall $se_teamID $se_bundleID"
155     else
156         confirm "Run 'sudo systemextensionsctl uninstall $se_teamID $se_bundleID'? (frees the NE slot for brew
157             tailscale; non-fatal if it errors)"
158         sudo systemextensionsctl uninstall "$se_teamID" "$se_bundleID" 2>&1 \
159             || echo "    ! uninstall returned non-zero (may require reboot, non-fatal)"
160     fi
161 else
162     echo "    no Tailscale System Extension loaded"
163 fi
164 # Pattern used by the Step-1 bootout loops, in both dry + real branches.
165 # Widened from a bare /tailscale/ so the same loops also pick up any
166 # nullexit-labeled LaunchAgents (e.g., com.nullexit.wake-recovery, the
167 # caffeinate + post-wake recovery hook). The wake-recovery LaunchAgent
168 # is re-installed by ./toggle.sh / setup.sh on re-engage, so wiping it
169 # here is non-fatal and is part of restoring a clean baseline.
170 TAILSCALE_LABEL_RE='tailscale'
171 NULLEXIT_LABEL_RE='nullexit'
172 # ----- 1. brew services stop -----
173 header "Step 1 Stop the brew-managed service + unload LaunchAgents"
174 # Detect whether the brew-managed service is registered as $USER (gui/$UID
175 # domain) or as root (system domain). NOPASSWD sudoers can leave a stale
176 # root-owned registration after a prior 'sudo brew services ...' round;
177 # 'brew services stop' without sudo refuses to touch a root-owned plist
178 # (Error: Service 'tailscale' is started as 'root').
179 brew_status_line=$(brew services list 2>/dev/null | awk '$1=="tailscale"')
180 brew_status_user=$(echo "$brew_status_line" | awk '{print $3}')
181 if [ -n "$brew_status_line" ]; then
182     if [ "$DRY" = 1 ]; then
183         echo "    [dry] brew services stop tailscale (user=$brew_status_user)"
184     else
185         if [ "$brew_status_user" = "root" ]; then
186             confirm "Run 'sudo brew services stop tailscale' (service is registered as root)?"
187             sudo brew services stop tailscale 2>&1 || echo "    (sudo brew services stop returned non-zero non-fatal
188                 )"
189         else
190             echo "    brew services stop tailscale (user=$brew_status_user, no sudo needed)"
191             brew services stop tailscale 2>&1 || echo "    (brew reported it wasn't actually running)"
192         fi
193     fi
194     echo "    unload any tailscale-related LaunchAgents (prevents launchd from respawning after kill)"
195     if [ "$DRY" = 1 ]; then
196         echo "    [dry] for each 'tailscale|nullexit'-matched label in gui/$UID domain:"
197         while IFS= read -r label; do
198             [ -z "$label" ] && continue
199             echo "    [dry] launchctl bootout gui/$UID/$label"
200         done <<(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 &&
201             ($3 ~ ts || $3 ~ nx) {print $3}')
202         echo "    [dry] for each 'tailscale|nullexit'-matched label in system domain (root-owned brew plist):"
203         while IFS= read -r label; do
204             [ -z "$label" ] && continue

```

```

203     echo "                [dry] (would prompt) sudo launchctl bootout system/$label"
204 done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" '
NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
205 if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
206     echo "                [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macos"
207 fi
208 if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
209     echo "                [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macsys"
210 fi
211 else
212 # Bootout every gui-domain LaunchAgent whose label matches 'tailscale'
213 # OR 'nullexit'. The nullexit arm catches com.nullexit.wake-recovery,
214 # which holds caffeinate/watcher.sh alive across a daemon reset not
215 # desired while the script is establishing a fresh baseline. Awake-
216 # recovery is re-installed by ./toggle.sh / setup.sh on re-engage, so
217 # wiping here is non-fatal. The pattern stays narrower than a bare
218 # /tail/ or /null/ so unrelated Apple services are not unloaded.
219 while IFS= read -r label; do
220     [ -z "$label" ] && continue
221     echo "                launchctl bootout gui/$UID/$label"
222     launchctl bootout "gui/$UID/$label" 2>/dev/null || true
223 done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 &&
($3 ~ ts || $3 ~ nx) {print $3}')
224 # Bootout every system-domain LaunchDaemon/Agent whose label matches
225 # 'tailscale' OR 'nullexit'. This catches the root-owned brew plist left
226 # behind by prior 'sudo brew services ...' invocations that was the
227 # case that left tailscaled PID 47447 57734 respawning between sudo
228 # pkill runs. Defensive nullexit catch in case the user previously
229 # `sudo ln -s`'d the wake-recovery plist into /Library/LaunchDaemons.
230 while IFS= read -r label; do
231     [ -z "$label" ] && continue
232     confirm "Run 'sudo launchctl bootout system/$label'?"
233     sudo launchctl bootout "system/$label" 2>/dev/null || true
234 done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" '
NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
235 # Legacy Tailscale.app system-domain entries. Each one is checked and
236 # confirmed separately so we don't sudo-promp or sudo-bootout for nothing.
237 if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
238     confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macos'?"
239     sudo launchctl bootout system/com.tailscale.ipn.macos 2>/dev/null || true
240 fi
241 if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
242     confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macsys'?"
243     sudo launchctl bootout system/com.tailscale.ipn.macsys 2>/dev/null || true
244 fi
245 fi
246 echo "                waiting up to 10s for tailscaled to exit"
247 for i in 1 2 3 4 5 6 7 8 9 10; do
248     if [ "$DRY" = 1 ]; then
249         echo "                [dry] sleep 1; loop check would terminate here"
250         break
251     fi
252     sleep 1
253     if ! pgrep -lf tailscaled >/dev/null 2>&1; then
254         echo "                tailscaled exited in ${i}s"
255         break
256     fi
257     [ "$i" = 10 ] && echo "                tailscaled still alive after 10s Step 2 will deal with it"
258 done
259 else
260     echo "                brew reports no tailscale service loaded"
261 fi
262
263 # ----- 2. Kill any orphan tailscaled -----
264 header "Step 2 Kill any orphan tailscaled process(es) (may escalate to sudo)"
265 if ! pgrep -lf tailscaled >/dev/null 2>&1; then
266     echo "                no orphans"
267 else
268     echo "                Stray process(es):"
269     pgrep -lf tailscaled | sed 's/^/ /'
270     echo "                no-sudo: pkill tailscaled"
271     if [ "$DRY" = 1 ]; then
272         echo "                [dry] pkill tailscaled on failure: sudo pkill -9 tailscaled"
273     else
274         if pkill tailscaled 2>/dev/null; then
275             echo "                no-sudo pkill succeeded"
276         else
277             echo "                ! no-sudo pkill insufficient; cannot escalate yet"
278         fi
279         sleep 1
280     fi

```

```

281
282 # Recheck; if still alive, offer sudo escalation
283 if { [ "$DRY" = 0 ] && pgrep -lf tailscaled >/dev/null 2>&1; } || [ "$DRY" = 1 ]; then
284     if [ "$DRY" = 1 ]; then
285         echo "    [dry] sudo pkill -9 tailscaled (escalation would fire here)"
286     else
287         confirm "Run 'sudo pkill -9 tailscaled' to escalate?"
288         sudo pkill -9 tailscaled 2>&1
289         sleep 1
290     fi
291 fi
292
293 if [ "$DRY" = 0 ]; then
294     if pgrep -lf tailscaled >/dev/null 2>&1; then
295         echo "    Failed to kill stragglers bailing out"
296         pgrep -lf tailscaled | sed 's/^/' /'
297         exit 3
298     fi
299     echo "    all tailscaled processes gone"
300 fi
301 fi
302
303 # ----- 3. brew uninstall -----
304 header "Step 3 brew uninstall tailscale"
305 if ! brew list tailscale >/dev/null 2>&1; then
306     echo "    not currently installed via brew skip"
307 else
308     # Detect whether the keg directory is owner=$USER or owner=root. If the
309     # keg is root-owned (typically a residual of some prior 'sudo brew ...'
310     # cycle that did perms fixups), 'brew uninstall' without sudo refuses
311     # to remove the files and prints: 'Error: Could not remove tailscale
312     # keg! Do so manually: sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>'.
313     # Detect that and escalate to a sudoed uninstall, with a manual rm -rf
314     # fallback if `sudo brew uninstall` itself errors out (e.g., partial
315     # state where the keg dir is gone but brew still thinks it's installed).
316     # Detect keg state. Treat `(dir absent)` AND `(dir present + empty)` as the
317     # same `<unknown>` bucket so both escalate to sudo. The empty-parent case
318     # is what you get after a manual `sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>`
319     # from inside: brew's parent dir is left present-but-empty and brew still
320     # thinks the package is installed.
321     keg_dir="/opt/homebrew/Cellar/tailscale"
322     if [ -d "$keg_dir" ] && [ -n "$(ls -A "$keg_dir" 2>/dev/null)" ]; then
323         keg_owner="$(stat -f '%Su' "$keg_dir" 2>/dev/null || echo '<unknown>')"
324     else
325         keg_owner="<unknown>"
326     fi
327     if [ "$DRY" = 1 ]; then
328         echo "    [dry] brew uninstall tailscale (keg_owner=$keg_owner)"
329     else
330         # 'root' AND '<unknown>' both require sudo escalation. The '<unknown>'
331         # case is the partial-state path: keg dir was already removed (manually
332         # or by a prior failed uninstall) but brew's internal state still says
333         # installed. Without this branch the script would fall through to a
334         # non-sudo brew uninstall that fails again with the same 'Could not
335         # remove tailscale keg!' error.
336         if [ "$keg_owner" = "root" ] || [ "$keg_owner" = "<unknown>" ]; then
337             confirm "Run 'sudo brew uninstall tailscale' (keg=$keg_owner)?"
338             sudo brew uninstall tailscale 2>&1 || {
339                 echo "    ! sudo brew uninstall failed; offering manual rm -rf fallback"
340                 # Guard the glob so an empty/absent dir doesn't literal-expand the
341                 # '*' and confuse the user with a sudo error about a non-existent path.
342                 if [ -d /opt/homebrew/Cellar/tailscale ] && [ -n "$(ls -A /opt/homebrew/Cellar/tailscale 2>/dev/null)"
343 ]; then
344                     confirm "Run 'sudo rm -rf /opt/homebrew/Cellar/tailscale/*'?"
345                     sudo rm -rf /opt/homebrew/Cellar/tailscale/* 2>&1
346                 else
347                     echo "    keg dir already empty or absent nothing more to rm"
348                 fi
349             }
350             else
351                 confirm "Run 'brew uninstall tailscale' (removes LaunchAgent plist + linked symlinks)?"
352                 brew uninstall tailscale 2>&1
353             fi
354         fi
355 fi
356
357 # ----- 4. Wipe state directories -----
358 header "Step 4 Wipe stale state"
359 # User-owned
360 declare -a USER_PATHS
361 USER_PATHS=(

```

```

361 "$HOME/.local/share/tailscale"
362 "$HOME/.local/run/tailscaled.socket"
363 "$HOME/.local/run/tailscaled.state"
364 "$HOME/.local/state/tailscale"
365 "$HOME/Library/Application Support/Tailscale"
366 "$HOME/Library/Caches/com.tailscale.ipn.macos"
367 "$HOME/Library/Caches/Tailscale"
368 "$HOME/Library/Logs/Tailscale"
369 )
370 echo "    User-owned paths (no sudo):"
371 for p in "${USER_PATHS[@]"; do
372     if [ -e "$p" ]; then
373         if [ "$DRY" = 1 ]; then
374             echo "        [dry] rm -rf '$p'"
375         else
376             rm -rf "$p"
377             echo "        rm -rf '$p'"
378         fi
379     fi
380 done
381
382 # System
383 declare -a SUDO_PATHS
384 SUDO_PATHS=(
385     "/var/lib/tailscale"
386     "/var/cache/tailscale"
387 )
388 echo "    System paths under /var (will prompt for sudo if any are present):"
389 eligible_sudo=0
390 for p in "${SUDO_PATHS[@]"; do
391     if [ -e "$p" ]; then
392         eligible_sudo=1
393         if [ "$DRY" = 1 ]; then
394             echo "        [dry] sudo rm -rf '$p'"
395         else
396             if confirm "Run 'sudo rm -rf $p'?" ; then
397                 sudo rm -rf "$p" && echo "        rm -rf '$p'"
398             fi
399         fi
400     fi
401 done
402 [ "$eligible_sudo" = 0 ] && echo "        (none of /var/lib/tailscale, /var/cache/tailscale are present)"
403
404 # ----- 5. Wipe stale LaunchAgent plists -----
405 header "Step 5 Wipe stale LaunchAgent plist files"
406 declare -a PLIST_FILES
407 PLIST_FILES=(
408     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
409     "$HOME/Library/LaunchAgents/com.tailscale.ipn.macos.plist"
410     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist.bak"
411     "$HOME/Library/LaunchAgents/com.nullexit.wake-recovery.plist"
412 )
413 for f in "${PLIST_FILES[@]"; do
414     if [ -e "$f" ]; then
415         if [ "$DRY" = 1 ]; then
416             echo "        [dry] rm -f '$f'"
417         else
418             rm -f "$f"
419             echo "        rm -f '$f'"
420         fi
421     fi
422 done
423 echo "    Remaining tailscale|nullexit plists (should be empty):"
424 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/^/' | echo "        (none)"
425
426 # ----- 5.5. Drain Background-Items (Login Items) -----
427 header "Step 5.5 Drain Background-Items (Login Items)"
428 # macOS 13+ keeps a parallel "Background Items" database alongside
429 # classic LaunchAgents. Even when we wipe ~/Library/LaunchAgents/homebrew.mxcl.tailscale.plist
430 # in Step 5, the .btm file at
431 # ~/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm
432 # can still hold a Background-Items entry pointing at the (now-dead) plist
433 # path. Next login: macOS sees the entry, tries to bootstrap, finds the
434 # plist missing, and either silently regenerates it via brew's post-install
435 # hook (causing tailscaled respawn at next login) or refuses outright.
436 # `sfltool reset-login-items` is Apple's canonical API for wiping the
437 # entire .btm file. It DOES wipe all Login Items the user re-adds what
438 # they want via System Settings General Login Items. Acceptable here
439 # because the goal of this script is "fresh baseline for Tailscale".
440 BTM="$HOME/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm"
441 if [ -f "$BTM" ]; then

```

```

442 if [ "$DRY" = 1 ]; then
443     echo "    [dry] found $BTM; would prompt sfltool reset-login-items"
444 else
445     confirm "Run 'sfltool reset-login-items' (wipes ALL Login Items; rebuild via System Settings  General
446             Login Items)?"
447     if sfltool reset-login-items 2>&1; then
448         echo "        Login Items drained"
449     else
450         echo "        ! sfltool reset-login-items failed (non-fatal; do manually: System Settings  General  Login
451             Items)"
452     fi
453 fi
454 else
455     echo "        no backgrounditems.btm present"
456 fi
457 # ----- 6. brew install -----
458 header "Step 6 brew install tailscale (fresh)"
459 if brew list tailscale >/dev/null 2>&1; then
460     echo "        already installed skip"
461 else
462     confirm "Run 'brew install tailscale'?"
463     if [ "$DRY" = 1 ]; then
464         echo "        [dry] brew install tailscale"
465     else
466         brew install tailscale 2>&1
467     fi
468 fi
469 # ----- 6.5. Strip quarantine xattr from keg -----
470 header "Step 6.5 Strip quarantine xattr from tailscale install"
471 # brew's freshly poured bottles don't carry com.apple.quarantine (the
472 # bottles are content-addressed and Homebrew-verified), but the upstream
473 # installer (Tailscale.app from App Store) and any manual download do.
474 # Strip the xattr from the entire keg as a defensive safety net so the
475 # freshly-installed tailscaled binary and any helper binaries are not
476 # Gatekeeper-blocked on first launch. Idempotent no-op if no quarantine
477 # xattr is present.
478 quarantine_count="$(xattr -lr /opt/homebrew/Cellar/tailscale 2>/dev/null | grep -c '^[^']* com.apple.quarantine
479 ' || echo 0)"
480 if [ "${quarantine_count:-0}" -gt 0 ]; then
481     if [ "$DRY" = 1 ]; then
482         echo "        [dry] found $quarantine_count file(s) with com.apple.quarantine; would prompt xattr -dr"
483     else
484         confirm "Run 'xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale'? (strips quarantine xattr from
485             $quarantine_count file(s); non-fatal if it errors)"
486         if xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale 2>&1; then
487             echo "        quarantine xattr stripped"
488         else
489             echo "        ! xattr -dr failed (non-fatal; open /opt/homebrew/Cellar/tailscale in Finder, right-click
490                 tailscaled Open Anyway)"
491         fi
492     fi
493 else
494     echo "        no com.apple.quarantine xattr on tailscale install"
495 fi
496 # ----- 7. brew services start (NO sudo!) -----
497 header "Step 7 Start the service as $USER (NO sudo, with multi-tier self-healing)"
498 if ! command -v tailscale >/dev/null 2>&1; then
499     echo "        tailscale not on PATH after install brew install errored; aborting"
500     exit 4
501 fi
502 # Aliveness fan-out: ANY one of these is taken as proof of life. None of
503 # them alone is reliable pgrep misses tailscaled if it does setproctitle
504 # or fork-detach; launchctl's reported PID can be a shim that exited
505 # within 1s; tailscale-cli hangs while macOS Local-Network permission is
506 # unresolved. Combined, they cover each other's blind spots.
507 tailscaled_api_up() {
508     # Strategy A: tailscale CLI can talk to the local API. Canonical proof.
509     # Bound at 8s tailscale status itself returns quickly on success OR
510     # failure, but if the post-install state is wedged it can hang.
511     with_timeout 8 /opt/homebrew/bin/tailscale status >/dev/null 2>&1
512 }
513
514 tailscaled_proc_up() {
515     # Strategy B: any process has 'tailscaled' substring in argv.
516     pgrep -lf tailscaled >/dev/null 2>&1
517 }

```



```

518
519 tailscaled_lc_up() {
520     # Strategy C: launchctl-managed service has a non-dash, non-zero PID.
521     local lc_pid
522     lc_pid="$(launchctl list 2>/dev/null | awk '$3 == "homebrew.mxcl.tailscale" {print $1; exit}')"
523     if [ -n "$lc_pid" ] && [ "$lc_pid" != "-" ] && [ "$lc_pid" -gt 0 ] 2>/dev/null; then
524         return 0
525     fi
526     return 1
527 }
528
529 tailscaled_is_alive() {
530     tailscaled_api_up && return 0
531     tailscaled_proc_up && return 0
532     tailscaled_lc_up && return 0
533     return 1
534 }
535
536 confirm "Run 'brew services start tailscale'?"
537 if [ "$DRY" = 1 ]; then
538     echo " [dry] brew services start tailscale"
539     echo " [dry] (then poll up to 30s for tailscaled_is_alive)"
540     echo " [dry] (recovery tier 1: launchctl kickstart)"
541     echo " [dry] (recovery tier 2: launchctl bootout + bootstrap of explicit plist)"
542     echo " [dry] (recovery tier 3: brew services restart (stop+start cycle))"
543     echo " [dry] (recovery tier 4: direct /opt/homebrew/bin/tailscaled spawn (bypasses launchd))"
544 else
545     brew services start tailscale 2>&1
546     # Primary poll tailscaled_is_alive covers all 3 detection strategies.
547     echo " polling for tailscaled to come up (up to ~30s)"
548     tailscaled_up=0
549     for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
550         if tailscaled_is_alive; then
551             tailscaled_up=1
552             echo " tailscaled alive after ${i}*2s"
553             break
554         fi
555         sleep 2
556     done
557
558     # Recovery tier 1: kickstart the launchd-managed service. This is the
559     # cheapest and most-aligned-with-launchd fix when brew has loaded the
560     # LaunchAgent but tailscaled didn't fork. Kickstart tells launchd
561     # "re-spawn if not running" without forcing a teardown.
562     if [ "$tailscaled_up" = 0 ]; then
563         echo " ! Recovery tier 1: launchctl kickstart"
564         if launchctl kickstart -kp "gui/$UID/homebrew.mxcl.tailscale" 2>&1; then
565             for i in 1 2 3 4 5 6 7 8 9 10; do
566                 if tailscaled_is_alive; then
567                     tailscaled_up=1
568                     echo " tailscaled alive after kickstart (${i}*2s)"
569                     break
570                 fi
571                 sleep 2
572             done
573         else
574             echo " ! kickstart returned non-zero (LaunchAgent may not be loaded)"
575         fi
576     fi
577
578     # Recovery tier 2: explicit plist bootout + bootstrap. Forces a fresh
579     # launchd registration cycle for the brew-managed service, which fixes
580     # the wider class of "brew says started but daemon never spawned"
581     # wedge states that kickstart alone can't dig out of.
582     if [ "$tailscaled_up" = 0 ]; then
583         echo " ! Recovery tier 2: launchctl bootout + bootstrap of explicit plist"
584         launchctl bootout "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null || true
585         sleep 1
586         # Brew installs the canonical plist template at $HOMEBREW_PREFIX/Library/.
587         # On Apple Silicon that's /opt/homebrew/Library/LaunchAgents/. On Intel
588         # it's /usr/local/Library/LaunchAgents/. Detect which and pick the
589         # right one so this script works across both architectures.
590         if [ -f /opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
591             plist_path="/opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
592         elif [ -f /usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
593             plist_path="/usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
594         else
595             plist_path=""
596         fi
597         if [ -n "$plist_path" ]; then
598             if launchctl bootstrap "gui/$UID" "$plist_path" 2>&1; then

```

```

599     for i in 1 2 3 4 5 6 7 8 9 10; do
600         if tailscaled_is_alive; then
601             tailscaled_up=1
602             echo "          tailscaled alive after bootstrap (${i}*2s)"
603             break
604         fi
605         sleep 2
606     done
607     else
608         echo "          ! explicit bootstrap returned non-zero"
609     fi
610 else
611     echo "          ! could not locate brew plist template; skipping bootstrap tier"
612 fi
613 fi
614
615 # Recovery tier 3: brew services restart (forces a stop + start cycle,
616 # which unsticks LaunchAgents with stale 'started' state from prior
617 # half-cycles). This is the canonical brew-doc fix for 'services don't
618 # actually start even though brew says started'.
619 if [ "$tailscaled_up" = 0 ]; then
620     echo "          ! Recovery tier 3: brew services restart (forces stop+start cycle)"
621     if brew services restart tailscale 2>&1; then
622         for i in 1 2 3 4 5 6 7 8 9 10; do
623             if tailscaled_is_alive; then
624                 tailscaled_up=1
625                 echo "          tailscaled alive after brew restart (${i}*2s)"
626                 break
627             fi
628             sleep 2
629         done
630     else
631         echo "          ! brew services restart returned non-zero"
632     fi
633 fi
634
635 # Recovery tier 4: last-resort direct spawn of the daemon binary,
636 # bypassing launchd entirely. If tailscaled crashes here too, we'll
637 # capture its actual stderr in /tmp for the user to read (NOPASSWD
638 # sudoers often have local-network permission denied the captured
639 # stderr makes it obvious what to fix in System Settings).
640 if [ "$tailscaled_up" = 0 ]; then
641     echo "          ! Recovery tier 4: direct spawn (bypasses launchd; tail stderr to /tmp)"
642     /opt/homebrew/bin/tailscaled >/tmp/nullexit-tailscaled-spawn.log 2>&1 &
643     spawned_pid=$!
644     disown "$spawned_pid" 2>/dev/null || true
645     sleep 3
646     if tailscaled_is_alive; then
647         tailscaled_up=1
648         echo "          tailscaled alive via direct spawn (pid=$spawned_pid)"
649         echo "          (NOTE: not under launchd supervision a reboot, re-run of"
650         echo "          this script, or ./toggle.sh is required to recreate."
651         echo "          Last 5 lines of stderr captured to /tmp/nullexit-tailscaled-spawn.log:)"
652         tail -n 5 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/~/ /' || true
653     fi
654 fi
655
656 if [ "$tailscaled_up" = 0 ]; then
657     echo "          HARD FAIL: tailscaled refuses to stay alive across every recovery tier"
658     echo "          launchctl-managed state (if available):"
659     launchctl print "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null \
660         | grep -E 'state =|last exit code =|runs =|path =' \
661         | sed 's/~/ /' \
662         || echo "          (no launchctl state available for homebrew.mxcl.tailscale)"
663     echo "          Last 10 lines of /tmp/nullexit-tailscaled-spawn.log (if any):"
664     tail -n 10 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/~/ /' || echo "          (no log)"
665     echo "          Most likely remaining cause: macOS Local Network permission denied."
666     echo "          System Settings Privacy & Security Local Network toggle Tailscale ON."
667     exit 7
668 fi
669 fi
670 sleep 3
671
672 # ----- 8. Verify -----
673 header "Step 8 Verify fresh install"
674 echo "    brew service tailscale: $(brew services list 2>/dev/null | awk '1=="tailscale"{{print $2, $3}}' | tr
675     '\n' ' ' | sed 's/ $//')
676 echo "    tailscaled process: $(pgrep -lf tailscaled 2>/dev/null | head -1 || echo 'NONE (check brew logs)
677     ')"
678 ts_ver=""
679 if [ -x /opt/homebrew/bin/tailscale ]; then

```

```

678 ts_ver="$(/opt/homebrew/bin/tailscale version 2>/dev/null | head -1 | tr -d '\n')"
679 elif command -v tailscale >/dev/null 2>&1; then
680 ts_ver="$(tailscale version 2>/dev/null | head -1 | tr -d '\n')"
681 fi
682 echo "    tailscale version:      ${ts_ver:-binary missing}"
683 ts_status=""
684 if [ -x /opt/homebrew/bin/tailscale ]; then
685 ts_status="$(/opt/homebrew/bin/tailscale status 2>/dev/null | head -3)"
686 fi
687 if [ -n "$ts_status" ]; then
688 echo "    tailscale status:"
689 printf '%s\n' "$ts_status" | sed 's/^/    /'
690 else
691 echo "    tailscale status:      (binary missing or errored)"
692 fi
693
694 if { [ "$DRY" = 0 ] && /opt/homebrew/bin/tailscale status 2>/dev/null | grep -q '^Tailscale is stopped'; };
695 then
696 echo "    NOTICE: 'tailscale status' still reports stopped after a fresh install."
697 echo "    Two equally likely causes:"
698 echo "    (a) macOS hasn't yet shown the first-run 'Local Network' prompt for"
699 echo "    Tailscale open System Settings Privacy & Security Local"
700 echo "    Network. Toggle Tailscale ON."
701 echo "    (b) macOS denied Local Network permission at some prior point."
702 echo "    Either way: System Settings Privacy & Security Local Network."
703 echo "    If the entry isn't present, invoking any tailscale command will"
704 echo "    re-trigger the prompt automatically."
705 fi
706
707 # ----- 9. Done -----
708 header "Done"
709 if [ "$DRY" = 1 ]; then
710 echo "    (dry-mode finished; no follow-up commands were issued.)"
711 exit 0
712 fi
713
714 cat <<'DONE'
715
716 Manual follow-ups (all of these; the script stops short of running sudo
717 tailscale up so you can verify the brew install is actually live first):
718
719 1. System Settings Privacy & Security Local Network Tailscale ON
720 macOS will normally pop the prompt the first time you run `tailscale`.
721 If it didn't, trigger it via the menu bar check.
722
723 2. Re-engage the tailnet + exit node:
724 sudo tailscale up --ssh=true --accept-dns=false \
725 --exit-node="$(cat ADGUARD_IP.txt | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
726 --exit-node-allow-lan-access=true
727
728 3. Re-run: ./toggle.sh
729 (re-installs the com.nullexit.wake-recovery LaunchAgent wiped
730 in Step 5 to clear the slate and re-engages the gateway)
731
732 4. Final verification:
733 curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)=*'
734 expect: warp=on, ip=<Cloudflare IP>, colo=YYZ/YUL
735
736 Container-side (gateway / 100.100.21.8) Tailscale is unaffected by this
737 script. Other tailnet devices (S24, iPhone, Windows) are unaffected.
738
739 DONE

```

39 scripts/routing-fix.sh

```

1 #!/bin/sh
2 # routing-fix: Maintains routing for SOCKS5 proxy traffic through WARP (tun0)
3 # and FORWARD rules for Tailscale exit-node return traffic.
4 #
5 # Runs inside the warp container's network namespace. This script ensures:
6 # 1. Table 200 has default via tun0 (for SOCKS5 proxy traffic from 172.18.0.2)
7 # with the WARP endpoint exception via eth0 (prevents tunnel loop)
8 # 2. The CGNAT ip rule (100.64.0.0/10 table 52 tailscale routes) is maintained for Tailscale
9 # 3. FORWARD RELATED,ESTABLISHED rule allows return traffic for exit-node
10 # forwarded connections (S24 tailscale0 eth0 host internet)

```

```

11 #
12 # CRITICAL: Does NOT touch the main table's default route. Gluetun handles that
13 # internally via its own routing rules and WireGuard fwmark (0xca6c table 51820).
14 # Overriding the main table default would create a loop: encrypted WireGuard
15 # packets exiting tun0 would match default dev tun0 and re-enter the tunnel.
16
17 set -e
18
19 trap 'echo "routing-fix: Caught signal, exiting..."; exit 0' TERM INT QUIT
20
21 sleep 15 &
22 wait $! || true
23
24 echo "routing-fix: Setting up routing for SOCKS5 proxy through WARP (tun0)..."
25
26 # Dynamically detect Docker subnet from eth0
27 DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
28 DOCKER_GW=$(ip route show default | head -1 | awk '{print $3}')
29 WARP_ENDPOINT="${WARP_ENDPOINT}"
30 TAILSCALE_ALLOW_LAN_P2P="${TAILSCALE_ALLOW_LAN_P2P:-false}"
31 IP_BLOCKLIST_FILE="/userfilters/ip_blocklist.ipset"
32 LAST_IP_MTIME=""
33 echo "routing-fix: Docker network: ${DOCKER_NET}, gateway: ${DOCKER_GW}"
34
35 # Table 200: Used by ip rule 100 (from 172.18.0.2) for SOCKS5 proxy traffic
36 # WARP endpoint goes via eth0 to avoid tunnel loop, everything else via tun0
37 ip route add "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || \
38 ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
39 ip route replace default dev tun0 table 200 2>/dev/null || true
40
41 # Main table: Just ensure the Docker subnet route exists via eth0
42 # (gluetun owns the default route here do NOT touch it)
43 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
44
45 # FORWARD RELATED,ESTABLISHED: Allow return traffic for exit-node connections.
46 # The container has BOTH iptables (nftables backend) and iptables-legacy
47 # loaded. Gluetun's post-rules.txt uses the nftables backend, while Tailscale
48 # uses the legacy backend. Packets go through BOTH stacks, so we add the rule
49 # to both to ensure it survives regardless of which backend has policy DROP.
50 #
51 # Packet flow: S24 outgoing (tailscale0->eth0) ACCEPTed by first rule.
52 # Return traffic (eth0->eth0 via table 52 to host) needs RELATED,ESTABLISHED
53 # to match the conntrack entry created by the outgoing flow.
54 add_fwd_related_established() {
55     # nftables backend (used by gluetun's post-rules.txt)
56     if ! iptables -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
57         iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
58     fi
59     # legacy backend (used by Tailscale's ts-forward)
60     if command -v iptables-legacy >/dev/null 2>&1; then
61         if ! iptables-legacy -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
62             iptables-legacy -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
63         fi
64     fi
65 }
66
67 add_fwd_related_established
68
69 # Policy Routing for Tailscale P2P:
70 # Tailscale sends its encrypted mesh UDP packets from port 41642.
71 # If these go out tun0 (WARP), Cloudflare's strict NAT drops the returning
72 # hole-punch packets, causing Tailscale to fail P2P and fall back to DERP relays (high latency).
73 # We mark packets originating from port 41642 and force them out the main table (eth0).
74 #
75 # LAN P2P is controlled by two sources (in priority order):
76 # 1. /app/.lan_p2p_detected written by watcher.sh on macOS on every network change
77 #    using airport -I (802.1x detection) + AP isolation probe. Overrides .env.
78 # 2. TAILSCALE_ALLOW_LAN_P2P env var (from .env) static fallback.
79 # When disabled, we explicitly DROP local RFC1918-destined Tailscale UDP so the
80 # phone cleanly falls back to DERP rather than getting poisoned by macOS gvpnproxy SNAT.
81 add_tailscale_p2p_bypass() {
82     # Dynamic override from watcher.sh (network auto-detection) takes priority over .env
83     local allow_p2p="${TAILSCALE_ALLOW_LAN_P2P:-false}"
84     if [ -f "/app/.lan_p2p_detected" ]; then
85         allow_p2p=$(cat "/app/.lan_p2p_detected" 2>/dev/null | tr -d '[:space:]')
86     fi
87
88     # Always allow P2P directly to the Docker host gateway (the Mac running this container).
89     # The gateway container is always co-located with the host Mac on the same machine,
90     # so direct Tailscale P2P via the Docker bridge is always safe and avoids DERP overhead.
91     # This RETURN rule must be inserted BEFORE the general DROP rules so it takes priority.

```

```

92 local default_gw
93 default_gw=$(ip route show default | awk '{print $3}')
94 if [ -n "$default_gw" ]; then
95     if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null; then
96         iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null || true
97     fi
98 fi
99
100 # Also explicitly whitelist all physical IPs of the host Mac (e.g. 172.17.52.223).
101 # The Tailscale control plane registers the Mac using its physical IP, so the
102 # container will attempt P2P handshakes using that physical IP instead of the
103 # Docker bridge IP. If this physical IP falls within 172.16.0.0/12, it would be dropped.
104 if [ -f "/app/.host_ips" ]; then
105     while IFS= read -r host_ip; do
106         [ -z "$host_ip" ] && continue
107         if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null; then
108             iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null || true
109         fi
110     done < "/app/.host_ips"
111 fi
112
113 if [ "${allow_p2p}" != "true" ]; then
114     # Drop Tailscale UDP packets destined for local subnets to prevent macOS gvproxy SNAT
115     # endpoint poisoning. When disabled (campus/enterprise WPA2-Enterprise with AP isolation),
116     # dropping these forces the S24 to use DERP relays reliably.
117     for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
118         if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; then
119             iptables -t mangle -A OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null || true
120         fi
121     done
122 else
123     # Remove DROP rules if they exist (switching from restricted to allowed)
124     for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
125         while iptables -t mangle -D OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; do ;; done
126     done
127 fi
128
129 # Bypass STUN and P2P packets to public IPs (DERP relays and remote peers) out eth0
130 if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null; then
131     iptables -t mangle -A OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null || true
132 fi
133 if ! ip rule show | grep -q 'fwmark 0x8888'; then
134     ip rule add fwmark 0x8888 lookup 254 pref 98 2>/dev/null || true
135 fi
136 }
137
138 add_tailscale_p2p_bypass
139
140 create_log_drop_chain() {
141     local ipt="$1"
142     local chain="$2"
143     local prefix="$3"
144
145     if ! $ipt -N "$chain" 2>/dev/null; then
146         $ipt -F "$chain" 2>/dev/null || true
147     fi
148     $ipt -A "$chain" -j LOG --log-prefix "$prefix"
149     $ipt -A "$chain" -j DROP 2>/dev/null || true
150 }
151
152 add_country_block() {
153     # To add a country to the blocklist, simply define the 'BLOCKED_COUNTRIES' variable in .env with 2-letter ISO
154     # country codes.
155     # You can find the country code by looking at the filename on http://www.ipdeny.com/ipblocks/data/countries/
156     # For example, to block China (cn) and Russia (ru), set: BLOCKED_COUNTRIES="cn ru" in your .env file
157     for zone in $BLOCKED_COUNTRIES; do
158         set_name="block_${zone}"
159         zone_upper=$(echo "$zone" | tr 'a-z' 'A-Z')
160         chain_name="LOG_DROP_${zone_upper}"
161         log_prefix="IP_BLOCK_${zone_upper}: "
162
163         # Create the ipset if it doesn't exist
164         if ! ipset list "$set_name" >/dev/null 2>&1; then
165             ipset create "$set_name" hash:net 2>/dev/null || true
166             echo "routing-fix: Downloading ${zone_upper} IPs for blocklist..."
167             curl -sS "http://www.ipdeny.com/ipblocks/data/countries/${zone}.zone" | grep -v '^#' | while read -r
168                 subnet; do
169                     [ -n "$subnet" ] && ipset add "$set_name" "$subnet" 2>/dev/null || true
170                 done
171             fi
172         fi
173     done
174 }

```

```

171 # Apply LOG_DROP rule to both backends
172 for ipt in iptables iptables-legacy; do
173     command -v "$ipt" >/dev/null 2>&1 || continue
174
175     create_log_drop_chain "$ipt" "$chain_name" "$log_prefix"
176
177     # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
178     while $ipt -D FORWARD -m set --match-set "$set_name" dst -j DROP 2>/dev/null; do ;; done
179
180     if ! $ipt -C FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null; then
181         $ipt -I FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null || true
182     fi
183 done
184 done
185 }
186
187 add_ip_blocklist() {
188     # Only reload when the compiled file has actually changed (mtime check).
189     # This prevents a pointless ipset restore on every 30-second loop tick.
190     if [ ! -f "$IP_BLOCKLIST_FILE" ]; then
191         return 0
192     fi
193
194     CURRENT_MTIME=$(stat -c %Y "$IP_BLOCKLIST_FILE" 2>/dev/null || echo "0")
195     if [ "$CURRENT_MTIME" != "$LAST_IP_MTIME" ]; then
196         echo "routing-fix: IP blocklist changed, reloading..."
197         # ipset restore handles the atomic swap internally:
198         # create_new populate swap with live destroy_new
199         if ipset restore < "$IP_BLOCKLIST_FILE" 2>/dev/null; then
200             LAST_IP_MTIME="$CURRENT_MTIME"
201             echo "routing-fix: IP blocklist loaded ($(ipset list block_malicious | grep -c '[0-9]') entries)."
202         else
203             echo "routing-fix: Warning: ipset restore failed. Retrying next cycle."
204             return 1
205         fi
206     fi
207
208     # Apply FORWARD rules in BOTH iptables backends.
209     for ipt in iptables iptables-legacy; do
210         command -v "$ipt" >/dev/null 2>&1 || continue
211
212         create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_DST "IP_BLOCK_MALICIOUS_DST: "
213         create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_SRC "IP_BLOCK_MALICIOUS_SRC: "
214
215         # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
216         while $ipt -D FORWARD -m set --match-set block_malicious dst -j DROP 2>/dev/null; do ;; done
217         while $ipt -D FORWARD -m set --match-set block_malicious src -j DROP 2>/dev/null; do ;; done
218
219         if ! $ipt -C FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null; then
220             $ipt -I FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null || true
221         fi
222
223         if ! $ipt -C FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null; then
224             $ipt -I FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null || true
225         fi
226     done
227 }
228
229 # Start background block logger (DNS + IP)
230 if ! pgrep -f "nullexit-logger" >/dev/null; then
231     echo "routing-fix: Starting background block logger..."
232     nullexit-logger &
233 fi
234
235 add_country_block
236 add_ip_blocklist
237
238 echo 'routing-fix: Routes applied.'
239
240 UNHEALTHY_COUNT=0
241
242 # Re-assert loop (every 30 seconds)
243 while true; do
244     # Table 200 routes
245     ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
246
247     # Tunnel health check
248     if ! curl -m 3 -sS --interface tun0 https://cloudflare.com/cdn-cgi/trace >/dev/null 2>&1; then
249         UNHEALTHY_COUNT=$((UNHEALTHY_COUNT + 1))
250     else
251         UNHEALTHY_COUNT=0

```

```

252     if [ -f "/app/TUNNEL_FAILED_CLOSED.marker" ]; then
253         rm -f "/app/TUNNEL_FAILED_CLOSED.marker"
254     fi
255 fi
256
257 if [ "$UNHEALTHY_COUNT" -ge 3 ]; then
258     echo "routing-fix: Tunnel health check failed $UNHEALTHY_COUNT times. Failing closed."
259     ip route del default dev tun0 table 200 2>/dev/null || true
260     ip route replace blackhole default table 200 2>/dev/null || true
261     touch "/app/TUNNEL_FAILED_CLOSED.marker"
262 else
263     ip route replace default dev tun0 table 200 2>/dev/null || true
264 fi
265
266 # Main table: keep Docker subnet route
267 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
268
269 # CGNAT ip rule for Tailscale
270 if ! ip rule show | grep -q '100.64.0.0/10'; then
271     ip rule add to 100.64.0.0/10 lookup 52 pref 99 2>/dev/null || true
272     echo 'routing-fix: CGNAT rule missing, re-injected'
273 fi
274
275 # FORWARD RELATED, ESTABLISHED: Allow return traffic in both iptables
276 # backends. Gluetun may reset the nftables ruleset on VPN reconnect,
277 # and Tailscale may reset the legacy ruleset on auth refresh.
278 add_fwd_related_established
279
280 # Ensure P2P packets bypass WARP to maintain low latency
281 add_tailscale_p2p_bypass
282
283 # Enforce country blocklist
284 add_country_block
285
286 # Enforce IP blocklist (reloads automatically when file changes)
287 add_ip_blocklist
288
289 sleep 30 &
290 wait $! || true
291 done

```

40 scripts/rule-compiler/go.mod

```

1 module nullexit/rule-compiler
2
3 go 1.22

```

41 scripts/rule-compiler/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "bytes"
6     "crypto/md5"
7     "encoding/base64"
8     "encoding/hex"
9     "fmt"
10    "io"
11    "net/http"
12    "net/netip"
13    "os"
14    "os/exec"
15    "path/filepath"
16    "regexp"
17    "sort"
18    "strings"
19    "sync"
20    "time"
21 )
22
23 const (

```



```

24 BlackListPath = "black_list.txt"
25 WhiteListPath = "white_list.txt"
26 OutputPath = "adguard/work/userfilters/compiled_rules.txt"
27 IpOutputPath = "adguard/work/userfilters/ip_blocklist.ipset"
28 IpCacheDir = "adguard/work/userfilters/cache/ip"
29 CacheDir = "adguard/work/userfilters/cache"
30 )
31
32 var CoreLists = []string{
33     "https://raw.githubusercontent.com/anudeepND/blacklist/master/adservers.txt",
34     "https://raw.githubusercontent.com/anudeepND/blacklist/master/facebook.txt",
35     "https://raw.githubusercontent.com/crazy-max/WindowsSpyBlocker/master/data/hosts/spy.txt",
36     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.samsung.txt",
37     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.apple.txt",
38     "https://abp.oid.nl/basic/",
39     "https://adaway.org/hosts.txt",
40     "https://pgl.yoyo.org/adservers/serverlist.php?hostformat=adblockplus&showintro=1&mimetype=plaintext",
41     "https://raw.githubusercontent.com/nextdns/cname-cloaking-blocklist/master/domains",
42 }
43
44 var MediumAdditions = []string{
45     "https://raw.githubusercontent.com/lightswitch05/hosts/master/docs/lists/facebook-extended.txt",
46     "https://someonewhocares.org/hosts/zero/hosts",
47     "https://urlhaus.abuse.ch/downloads/hostfile/",
48 }
49
50 var HeavyAdditions = []string{
51     "https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts",
52     "https://raw.githubusercontent.com/Perflyst/PiHoleBlocklist/master/SmartTV-AGH.txt",
53     "https://raw.githubusercontent.com/DandelionSprout/adfilt/master/GameConsoleAdblockList.txt",
54 }
55
56 var IpBlockLists = []string{
57     "https://feodotracker.abuse.ch/downloads/ipblocklist.txt",
58     "https://www.spamhaus.org/drop/drop.txt",
59     "https://rules.emergingthreats.net/fwrules/emerging-Block-IPs.txt",
60     "https://cinsscore.com/list/ci-badguys.txt",
61 }
62
63 func getProfiles() map[string][]string {
64     profiles := make(map[string][]string)
65
66     light := append([]string(nil), CoreLists...)
67     light = append(light, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/light.txt")
68     profiles["light"] = light
69
70     medium := append([]string(nil), CoreLists...)
71     medium = append(medium, MediumAdditions...)
72     medium = append(medium, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/multi.txt")
73     profiles["medium"] = medium
74
75     heavy := append([]string(nil), CoreLists...)
76     heavy = append(heavy, MediumAdditions...)
77     heavy = append(heavy, HeavyAdditions...)
78     heavy = append(heavy, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt")
79     profiles["heavy"] = heavy
80
81     return profiles
82 }
83
84 func loadEnvProfile() string {
85     profile := "medium"
86
87     if bytesData, err := os.ReadFile(".env"); err == nil {
88         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
89         for scanner.Scan() {
90             line := strings.TrimSpace(scanner.Text())
91             if line != "" && !strings.HasPrefix(line, "#") {
92                 parts := strings.SplitN(line, "=", 2)
93                 if len(parts) == 2 {
94                     key := strings.TrimSpace(parts[0])
95                     val := strings.TrimSpace(parts[1])
96                     if key == "GATEWAY_RULE_PROFILE" {
97                         val = strings.Trim(val, "\"'")
98                         profile = strings.ToLower(val)
99                     }
100                 }
101             }
102         }
103     } else if !os.IsNotExist(err) {
104         fmt.Printf("Warning: Failed to read .env file for profile configuration (%v). Using default.\n", err)

```

```

105 }
106
107 if envVal := os.Getenv("GATEWAY_RULE_PROFILE"); envVal != "" {
108     profile = strings.ToLower(envVal)
109 }
110
111 profiles := getProfiles()
112 if _, exists := profiles[profile]; !exists {
113     fmt.Printf("Warning: Profile '%s' is invalid. Falling back to 'medium'.\n", profile)
114     profile = "medium"
115 }
116 return profile
117 }
118
119 func loadDomains(filepath string) map[string]bool {
120     domains := make(map[string]bool)
121     if bytesData, err := os.ReadFile(filepath); err == nil {
122         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
123         for scanner.Scan() {
124             line := strings.TrimSpace(scanner.Text())
125             if line != "" && !strings.HasPrefix(line, "#") {
126                 domains[strings.ToLower(line)] = true
127             }
128         }
129     }
130     return domains
131 }
132
133 func parseDomainsFromContent(content string) map[string]bool {
134     domains := make(map[string]bool)
135     scanner := bufio.NewScanner(strings.NewReader(content))
136     ipRegex := regexp.MustCompile(`^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$`)
137     agRegex := regexp.MustCompile(`^\|([a-z0-9.-]+)\|^$`)
138     domainRegex := regexp.MustCompile(`^[a-z0-9.-]+$`)
139
140     for scanner.Scan() {
141         line := strings.TrimSpace(scanner.Text())
142         if line == "" || strings.HasPrefix(line, "!") || strings.HasPrefix(line, "[") {
143             continue
144         }
145         parts := strings.SplitN(line, "#", 2)
146         line = strings.TrimSpace(parts[0])
147         if line == "" {
148             continue
149         }
150
151         fields := strings.Fields(line)
152         var rule string
153         if len(fields) >= 2 {
154             if ipRegex.MatchString(fields[0]) {
155                 rule = fields[1]
156             } else {
157                 rule = fields[0]
158             }
159         } else {
160             rule = fields[0]
161         }
162
163         rule = strings.TrimSpace(strings.ToLower(rule))
164
165         var domain string
166         if m := agRegex.FindStringSubmatch(rule); m != nil {
167             domain = m[1]
168         } else if domainRegex.MatchString(rule) {
169             domain = rule
170         } else {
171             domain = rule
172         }
173
174         if domain != "" && domain != "localhost" && domain != "0.0.0.0" && domain != "127.0.0.1" && domain != "
175             broadcasthost" {
176                 domains[domain] = true
177             }
178     }
179     return domains
180 }
181
182 // WARNING: Parsing AdGuardHome.yaml using regex instead of a proper YAML parser
183 // is brittle and assumes the exact format currently emitted by AdGuard.
184 // If an AdGuard version bump updates the configuration schema format,
185 // this parser will fail silently or loudly and should be refactored to use gopkg.in/yaml.v3.

```

```

185 func getAdguardNativeLists() []string {
186     yamlPath := "adguard/conf/AdGuardHome.yaml"
187     var enabledUrls []string
188     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
189         return enabledUrls
190     }
191
192     contentBytes, err := os.ReadFile(yamlPath)
193     if err != nil {
194         fmt.Printf("Warning: Failed to parse AdGuardHome.yaml for native lists (%v)\n", err)
195         return enabledUrls
196     }
197     content := string(contentBytes)
198
199     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"?(?^[a-zA-Z_]+:|\z)`
200     match := filtersRegex.FindStringSubmatch(content)
201     if len(match) > 1 {
202         filtersText := match[1]
203         blocks := strings.Split(filtersText, "\n - ")
204         urlRegex := regexp.MustCompile(`url:\s*(\S+)`)
205
206         for _, block := range blocks {
207             if strings.TrimSpace(block) == "" {
208                 continue
209             }
210             if strings.Contains(block, "enabled: true") {
211                 urlMatch := urlRegex.FindStringSubmatch(block)
212                 if len(urlMatch) > 1 {
213                     url := urlMatch[1]
214                     if !strings.Contains(url, "compiled_rules.txt") {
215                         enabledUrls = append(enabledUrls, url)
216                     }
217                 }
218             }
219         }
220     }
221     return enabledUrls
222 }
223
224 func getAdguardCompiledRulesCachePath() string {
225     yamlPath := "adguard/conf/AdGuardHome.yaml"
226     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
227         return ""
228     }
229
230     contentBytes, err := os.ReadFile(yamlPath)
231     if err != nil {
232         fmt.Printf("Warning: Failed to resolve compiled_rules.txt cache path (%v)\n", err)
233         return ""
234     }
235     content := string(contentBytes)
236
237     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"?(?^[a-zA-Z_]+:|\z)`
238     match := filtersRegex.FindStringSubmatch(content)
239     if len(match) > 1 {
240         blocks := strings.Split(match[1], "\n - ")
241         idRegex := regexp.MustCompile(`id:\s*(\d+)`)
242         for _, block := range blocks {
243             if !strings.Contains(block, "compiled_rules.txt") {
244                 continue
245             }
246             idMatch := idRegex.FindStringSubmatch(block)
247             if len(idMatch) > 1 {
248                 return fmt.Sprintf("adguard/work/data/filters/%s.txt", idMatch[1])
249             }
250         }
251     }
252     return ""
253 }
254
255 func handleFetchError(urlStr string, cacheFile string, err error) map[string]bool {
256     if _, statErr := os.Stat(cacheFile); statErr == nil {
257         fmt.Printf("-> Warning: Failed to fetch %s (%v). Falling back to expired local cache.\n", urlStr, err)
258         content, readErr := os.ReadFile(cacheFile)
259         if readErr == nil {
260             domains := parseDomainsFromContent(string(content))
261             fmt.Printf("-> Loaded from expired cache (%d domains).\n", len(domains))
262             return domains
263         }
264         fmt.Printf("-> Warning: Failed to read expired cache (%v). Using local lists only.\n", readErr)
265     } else {

```

```

266     fmt.Printf(" -> Warning: Failed to fetch %s (%v). Using local lists only for this source.\n", urlStr, err)
267 }
268 return make(map[string]bool)
269 }
270
271 func fetchRemoteDomains(urlStr string) map[string]bool {
272     os.MkdirAll(CacheDir, 0755)
273
274     hash := md5.Sum([]byte(urlStr))
275     urlHash := hex.EncodeToString(hash[:])
276     cacheFile := filepath.Join(CacheDir, fmt.Sprintf("%s.txt", urlHash))
277
278     if stat, err := os.Stat(cacheFile); err == nil {
279         fileAge := time.Since(stat.ModTime()).Seconds()
280         if fileAge < 86400 {
281             fmt.Printf("Loading remote blacklist from cache: %s\n", urlStr)
282             content, err := os.ReadFile(cacheFile)
283             if err == nil {
284                 domains := parseDomainsFromContent(string(content))
285                 fmt.Printf(" -> Loaded from cache (copy is %.1f hours old, %d domains).\n", fileAge/3600.0, len(domains))
286             }
287             return domains
288         }
289         fmt.Printf(" -> Warning: Failed to read cache file %s (%v). Re-fetching...\n", cacheFile, err)
290     }
291
292     fmt.Printf("Fetching remote blacklist from: %s ...\n", urlStr)
293
294     req, err := http.NewRequest("GET", urlStr, nil)
295     if err != nil {
296         return handleFetchError(urlStr, cacheFile, err)
297     }
298     req.Header.Set("User-Agent", "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)")
299
300     client := &http.Client{Timeout: 15 * time.Second}
301     resp, err := client.Do(req)
302     if err != nil {
303         return handleFetchError(urlStr, cacheFile, err)
304     }
305     defer resp.Body.Close()
306
307     if resp.StatusCode != 200 {
308         return handleFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
309     }
310
311     bodyBytes, err := io.ReadAll(resp.Body)
312     if err != nil {
313         return handleFetchError(urlStr, cacheFile, err)
314     }
315
316     content := string(bodyBytes)
317     domains := parseDomainsFromContent(content)
318
319     if len(domains) < 10 {
320         return handleFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only found %d domains. Possible 404 or bad URL", len(domains)))
321     }
322
323     err = os.WriteFile(cacheFile, bodyBytes, 0644)
324     if err != nil {
325         fmt.Printf(" -> Warning: Failed to save cache file (%v)\n", err)
326     }
327
328     fmt.Printf(" -> Successfully fetched and cached %d domains.\n", len(domains))
329     return domains
330 }
331
332 func optimizeSubdomains(domains map[string]bool, listName string) map[string]bool {
333     fmt.Printf("Optimizing %s by removing redundant subdomains...\n", listName)
334     rawCount := len(domains)
335     if rawCount == 0 {
336         return make(map[string]bool)
337     }
338
339     optimized := make(map[string]bool)
340     for domain := range domains {
341         if strings.ContainsAny(domain, "|~@$/") {
342             optimized[domain] = true
343             continue
344         }

```

```

345 parts := strings.Split(domain, ".")
346 hasParent := false
347
348
349 for i := 1; i < len(parts)-1; i++ {
350     parent := strings.Join(parts[i:], ".")
351     if domains[parent] {
352         hasParent = true
353         break
354     }
355 }
356
357 if !hasParent {
358     optimized[domain] = true
359 }
360 }
361
362 saved := rawCount - len(optimized)
363 reduction := float64(0)
364 if rawCount > 0 {
365     reduction = (float64(saved) / float64(rawCount)) * 100
366 }
367 fmt.Printf(" -> Reduced %s from %d to %d domains (-%d / %.1f%% reduction).\n", listName, rawCount, len(
    optimized), saved, reduction)
368 return optimized
369 }
370
371 func parseIpsFromContent(content string) map[string]bool {
372     ips := make(map[string]bool)
373     scanner := bufio.NewScanner(strings.NewReader(content))
374
375     for scanner.Scan() {
376         line := strings.TrimSpace(scanner.Text())
377         if line == "" || strings.HasPrefix(line, "#") || strings.HasPrefix(line, ";") {
378             continue
379         }
380
381         fields := strings.Fields(line)
382         if len(fields) > 0 {
383             token := strings.TrimRight(fields[0], ";,")
384             if _, err := netip.ParsePrefix(token); err == nil {
385                 ips[token] = true
386             } else if _, err := netip.ParseAddr(token); err == nil {
387                 ips[token] = true
388             }
389         }
390     }
391     return ips
392 }
393
394 func handleIpFetchError(urlStr string, cacheFile string, err error) map[string]bool {
395     if _, statErr := os.Stat(cacheFile); statErr == nil {
396         fmt.Printf(" -> Warning: fetch failed (%v). Falling back to stale cache.\n", err)
397         content, readErr := os.ReadFile(cacheFile)
398         if readErr == nil {
399             ips := parseIpsFromContent(string(content))
400             fmt.Printf(" -> %d IPs loaded from stale cache.\n", len(ips))
401             return ips
402         }
403         fmt.Printf(" -> Warning: stale cache also failed (%v).\n", readErr)
404     } else {
405         fmt.Printf(" -> Warning: %s failed (%v). No cache available. Skipping.\n", urlStr, err)
406     }
407     return make(map[string]bool)
408 }
409
410 func fetchRemoteIps(urlStr string) map[string]bool {
411     os.MkdirAll(IpCacheDir, 0755)
412
413     hash := md5.Sum([]byte(urlStr))
414     urlHash := hex.EncodeToString(hash[:])
415     cacheFile := filepath.Join(IpCacheDir, fmt.Sprintf("%s.txt", urlHash))
416
417     if stat, err := os.Stat(cacheFile); err == nil {
418         fileAge := time.Since(stat.ModTime()).Seconds()
419         if fileAge < 86400 {
420             fmt.Printf("Loading IP feed from cache: %s\n", urlStr)
421             content, err := os.ReadFile(cacheFile)
422             if err == nil {
423                 ips := parseIpsFromContent(string(content))
424                 fmt.Printf(" -> %d IPs (cache is %.1f%% old).\n", len(ips), fileAge/3600.0)

```

```

425     return ips
426 }
427 fmt.Printf(" -> Warning: cache read failed (%v). Re-fetching...\n", err)
428 }
429 }
430
431 fmt.Printf("Fetching IP feed: %s ...\n", urlStr)
432 req, err := http.NewRequest("GET", urlStr, nil)
433 if err != nil {
434     return handleIpFetchError(urlStr, cacheFile, err)
435 }
436 req.Header.Set("User-Agent", "Mozilla/5.0")
437
438 client := &http.Client{Timeout: 15 * time.Second}
439 resp, err := client.Do(req)
440 if err != nil {
441     return handleIpFetchError(urlStr, cacheFile, err)
442 }
443 defer resp.Body.Close()
444
445 if resp.StatusCode != 200 {
446     return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
447 }
448
449 bodyBytes, err := io.ReadAll(resp.Body)
450 if err != nil {
451     return handleIpFetchError(urlStr, cacheFile, err)
452 }
453
454 content := string(bodyBytes)
455 ips := parseIpsFromContent(content)
456 if len(ips) < 1 {
457     return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only %d IPs found. Possible
458     404", len(ips)))
459 }
460
461 err = os.WriteFile(cacheFile, bodyBytes, 0644)
462 if err != nil {
463     fmt.Printf(" -> Warning: cache write failed (%v)\n", err)
464 }
465
466 fmt.Printf(" -> %d IPs fetched and cached.\n", len(ips))
467 return ips
468 }
469
470 func compileIpBlocklist() int {
471     fmt.Println("\n IP Blocklist Compilation ")
472
473     allIps := make(map[string]bool)
474     var mu sync.Mutex
475     var wg sync.WaitGroup
476
477     maxThreads := 8
478     if len(IpBlockLists) < 8 {
479         maxThreads = len(IpBlockLists)
480     }
481     semaphore := make(chan struct{}, maxThreads)
482
483     for _, urlStr := range IpBlockLists {
484         wg.Add(1)
485         go func(u string) {
486             defer wg.Done()
487             semaphore <- struct{}{}
488             defer func() { <-semaphore }()
489
490             ips := fetchRemoteIps(u)
491             mu.Lock()
492             for ip := range ips {
493                 allIps[ip] = true
494             }
495             mu.Unlock()
496         }(urlStr)
497     }
498     wg.Wait()
499
500     fmt.Printf("Total unique entries before filtering: %d\n", len(allIps))
501
502     reservedStr := []string{
503         "10.0.0.0/8",
504         "172.16.0.0/12",
505         "192.168.0.0/16",

```

```

505     "127.0.0.0/8",
506     "169.254.0.0/16",
507     "100.64.0.0/10",
508     "0.0.0.0/8",
509 }
510 var reserved []netip.Prefix
511 for _, s := range reservedStr {
512     p, _ := netip.ParsePrefix(s)
513     reserved = append(reserved, p)
514 }
515
516 cleanIps := make(map[string]bool)
517 for entry := range allIps {
518     var prefix netip.Prefix
519     p, err := netip.ParsePrefix(entry)
520     if err != nil {
521         addr, err2 := netip.ParseAddr(entry)
522         if err2 != nil {
523             continue
524         }
525         prefix = netip.PrefixFrom(addr, addr.BitLen())
526     } else {
527         prefix = p
528     }
529
530     overlaps := false
531     for _, r := range reserved {
532         if r.Overlaps(prefix) {
533             overlaps = true
534             break
535         }
536     }
537     if !overlaps {
538         cleanIps[entry] = true
539     }
540 }
541
542 removed := len(allIps) - len(cleanIps)
543 if removed > 0 {
544     fmt.Printf(" -> Removed %d private/reserved entries.\n", removed)
545 }
546 fmt.Printf(" -> Final IP blocklist: %d entries.\n", len(cleanIps))
547
548 os.MkdirAll(filepath.Dir(IpOutputPath), 0755)
549
550 f, err := os.Create(IpOutputPath)
551 if err != nil {
552     fmt.Printf("Error writing IP output file: %v\n", err)
553     return len(cleanIps)
554 }
555 defer f.Close()
556
557 f.WriteString("# nullexit Compiled IP Blocklist\n")
558 f.WriteString("# Sources: Feodo Tracker, Spamhaus DROP/EDROP, Emerging Threats, CINS\n")
559 f.WriteString(fmt.Sprintf("# Entries: %d\n", len(cleanIps)))
560
561 f.WriteString("create block_malicious_new hash:net maxelem 200000 -exist\n")
562
563 var sortedIps []string
564 for ip := range cleanIps {
565     sortedIps = append(sortedIps, ip)
566 }
567 sort.Strings(sortedIps)
568
569 for _, ip := range sortedIps {
570     f.WriteString(fmt.Sprintf("add block_malicious_new %s -exist\n", ip))
571 }
572
573 f.WriteString("create block_malicious hash:net maxelem 200000 -exist\n")
574 f.WriteString("swap block_malicious block_malicious_new\n")
575 f.WriteString("destroy block_malicious_new\n")
576
577 fmt.Printf("IP blocklist written to %s\n", IpOutputPath)
578 return len(cleanIps)
579 }
580
581 func main() {
582     profile := loadEnvProfile()
583     fmt.Printf("Active memory profile: '%s'\n", strings.ToUpper(profile))
584
585     localBlackList := loadDomains(BlackListPath)

```



```

586 whiteList := loadDomains(WhiteListPath)
587
588 blackList := make(map[string]bool)
589 for k := range localBlackList {
590     blackList[k] = true
591 }
592
593 profiles := getProfiles()
594 urlsToFetch := profiles[profile]
595
596 fmt.Printf("\nStarting concurrent downloads for %d lists...\n", len(urlsToFetch))
597 startTime := time.Now()
598
599 var mu sync.Mutex
600 var wg sync.WaitGroup
601
602 maxThreads := 16
603 if len(urlsToFetch) < 16 {
604     maxThreads = len(urlsToFetch)
605 }
606 semaphore := make(chan struct{}, maxThreads)
607
608 for _, urlStr := range urlsToFetch {
609     wg.Add(1)
610     go func(u string) {
611         defer wg.Done()
612         semaphore <- struct{}{}
613         defer func() { <-semaphore }()
614
615         domains := fetchRemoteDomains(u)
616         mu.Lock()
617         for d := range domains {
618             blackList[d] = true
619         }
620         mu.Unlock()
621     }(urlStr)
622 }
623 wg.Wait()
624
625 fmt.Printf("Finished concurrent downloads in %.2f seconds using %d threads.\n", time.Since(startTime).Seconds(), maxThreads)
626
627 adguardNativeUrls := getAdguardNativeLists()
628 var adguardNativeDomains map[string]bool
629 if len(adguardNativeUrls) > 0 {
630     fmt.Printf("\nFetching %d AdGuard native list(s) for deduplication...\n", len(adguardNativeUrls))
631     adguardNativeDomains = make(map[string]bool)
632
633     maxNativeThreads := 4
634     if len(adguardNativeUrls) < 4 {
635         maxNativeThreads = len(adguardNativeUrls)
636     }
637     nativeSemaphore := make(chan struct{}, maxNativeThreads)
638
639     for _, urlStr := range adguardNativeUrls {
640         wg.Add(1)
641         go func(u string) {
642             defer wg.Done()
643             nativeSemaphore <- struct{}{}
644             defer func() { <-nativeSemaphore }()
645
646             domains := fetchRemoteDomains(u)
647             mu.Lock()
648             for d := range domains {
649                 adguardNativeDomains[d] = true
650             }
651             mu.Unlock()
652         }(urlStr)
653     }
654     wg.Wait()
655
656     if len(adguardNativeDomains) > 0 {
657         fmt.Println("Deduplicating compiled rules against AdGuard native lists...")
658         originalSize := len(blackList)
659         for d := range adguardNativeDomains {
660             delete(blackList, d)
661         }
662         fmt.Printf(" -> Removed %d redundant rules already covered by AdGuard.\n", originalSize-len(blackList))
663     }
664 }
665

```

```

666 var contradictions []string
667 for d := range whitelist {
668     if blacklist[d] {
669         contradictions = append(contradictions, d)
670     }
671 }
672
673 if len(contradictions) > 0 {
674     fmt.Printf("Detected %d contradictions. Whitelist taking priority.\n", len(contradictions))
675     sort.Strings(contradictions)
676     for _, domain := range contradictions {
677         if localBlackList[domain] {
678             fmt.Printf(" -> Removed local blacklist domain: %s\n", domain)
679         }
680         delete(blackList, domain)
681     }
682 }
683
684 blackList = optimizeSubdomains(blackList, "blacklist")
685 whitelist = optimizeSubdomains(whitelist, "whitelist")
686
687 os.MkdirAll(filepath.Dir(OutputPath), 0755)
688 outFile, err := os.Create(OutputPath)
689 if err != nil {
690     fmt.Printf("Error creating output file: %v\n", err)
691     return
692 }
693 defer outFile.Close()
694
695 outFile.WriteString("! Custom Compiled Rules (Auto-Generated)\n")
696 outFile.WriteString(fmt.Sprintf("! Memory Profile: %s\n", strings.ToUpper(profile)))
697 outFile.WriteString(fmt.Sprintf("! Total Block Rules: %d\n", len(blackList)))
698 outFile.WriteString(fmt.Sprintf("! Native AdGuard Rules: %d\n", len(adguardNativeDomains)))
699 outFile.WriteString(fmt.Sprintf("! Total Whitelist Rules: %d\n", len(whitelist)))
700
701 outFile.WriteString("! --- Blacklist Rules ---\n")
702
703 var sortedBlacklist []string
704 for d := range blackList {
705     sortedBlacklist = append(sortedBlacklist, d)
706 }
707 sort.Strings(sortedBlacklist)
708
709 for _, domain := range sortedBlacklist {
710     if strings.Contains(domain, "$") {
711         parts := strings.SplitN(domain, "$", 2)
712         dom, mod := parts[0], parts[1]
713         if strings.HasPrefix(dom, "|") {
714             if strings.HasSuffix(dom, "~") {
715                 outFile.WriteString(fmt.Sprintf("%s%s\n", dom, mod))
716             } else {
717                 outFile.WriteString(fmt.Sprintf("%s^%s\n", dom, mod))
718             }
719         } else {
720             outFile.WriteString(fmt.Sprintf("||%s^%s\n", dom, mod))
721         }
722     } else if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|") || strings.HasSuffix(domain, "~") {
723         outFile.WriteString(fmt.Sprintf("%s\n", domain))
724     } else {
725         outFile.WriteString(fmt.Sprintf("||%s^", domain))
726     }
727 }
728
729 outFile.WriteString("\n! --- Whitelist Rules ---\n")
730 var sortedWhitelist []string
731 for d := range whitelist {
732     sortedWhitelist = append(sortedWhitelist, d)
733 }
734 sort.Strings(sortedWhitelist)
735
736 for _, domain := range sortedWhitelist {
737     if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|") || strings.HasSuffix(domain, "~") || strings.HasPrefix(domain, "@@") {
738         rule := domain
739         if !strings.HasPrefix(domain, "@@") {
740             rule = "@@" + domain
741         }
742         outFile.WriteString(fmt.Sprintf("%s\n", rule))
743     } else {
744         outFile.WriteString(fmt.Sprintf("@@||%s^", domain))

```

```

745 }
746 }
747
748 fmt.Printf("\nSuccessfully compiled %d block rules and %d allow rules to %s\n", len(blackList), len(whiteList), OutputPath)
749
750 cachedFilterPath := getAdguardCompiledRulesCachePath()
751 if cachedFilterPath != "" {
752     input, err := os.ReadFile(OutputPath)
753     if err == nil {
754         err = os.WriteFile(cachedFilterPath, input, 0644)
755         if err == nil {
756             fmt.Printf("Updated AdGuard filter cache: %s\n", cachedFilterPath)
757         } else {
758             fmt.Printf("Warning: Could not update AdGuard filter cache %s (%v)\n", cachedFilterPath, err)
759         }
760     } else {
761         fmt.Printf("Warning: Could not read OutputPath to copy to cache (%v)\n", err)
762     }
763 } else {
764     fmt.Println("Warning: Could not determine compiled_rules.txt cache path from AdGuardHome.yaml; skipping cache update.")
765 }
766
767 adguardReachable := false
768
769 dockerComposeYamlExists := false
770 if _, err := os.Stat("docker-compose.yml"); err == nil {
771     dockerComposeYamlExists = true
772 }
773 dockerPath, err := exec.LookPath("docker")
774 if err == nil && dockerComposeYamlExists {
775     cmd := exec.Command(dockerPath, "compose", "ps", "--status", "running", "-q", "adguardhome")
776     out, err := cmd.Output()
777     if err == nil && strings.TrimSpace(string(out)) != "" {
778         fmt.Println("Force-recreating AdGuard Home container to drop persisted filter state...")
779         upCmd := exec.Command(dockerPath, "compose", "up", "-d", "--force-recreate", "--no-deps", "adguardhome")
780         err := upCmd.Run()
781         if err == nil {
782             fmt.Println("AdGuard Home force-recreated successfully.")
783             time.Sleep(8 * time.Second)
784             adguardReachable = true
785         } else {
786             fmt.Println("Failed to restart AdGuard Home. Is it running?")
787         }
788     }
789 }
790
791 if adguardReachable {
792     agPass := "nullexit"
793     if bytesData, err := os.ReadFile(".env"); err == nil {
794         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
795         for scanner.Scan() {
796             line := scanner.Text()
797             if strings.HasPrefix(line, "ADGUARD_PASSWORD=") {
798                 parts := strings.SplitN(line, "=", 2)
799                 if len(parts) == 2 {
800                     agPass = strings.Trim(strings.TrimSpace(parts[1]), "\"'")
801                 }
802                 break
803             }
804         }
805     }
806
807 creds := base64.StdEncoding.EncodeToString([]byte(fmt.Sprintf("admin:%s", agPass)))
808 req, err := http.NewRequest("POST", "http://127.0.0.1:3000/control/filtering/refresh", bytes.NewBuffer([]byte("{}")))
809 if err == nil {
810     req.Header.Set("Authorization", fmt.Sprintf("Basic %s", creds))
811     req.Header.Set("Content-Type", "application/json")
812     client := &http.Client{Timeout: 15 * time.Second}
813     resp, err := client.Do(req)
814     if err == nil {
815         fmt.Printf("Triggered AdGuard filter refresh via REST API (HTTP=%d).\n", resp.StatusCode)
816         resp.Body.Close()
817     } else {
818         fmt.Printf("Warning: AdGuard filter refresh API call failed (%v). New whitelist will not load until next refresh tick (~24h) or manual refresh.\n", err)
819     }
820 }
821 }

```

```

822
823 compileIpBlocklist()
824 }

```

42 scripts/setup-common.sh

```

1  #!/bin/bash
2  # 1. Docker
3  step "Checking Docker"
4
5  if ! command -v docker >> output.log 2>&1; then
6      echo ""
7      if [[ "$OSTYPE" == "darwin"* ]]; then
8          echo " Docker is not installed."
9          # Note: Homebrew aggressively rolls forward and discourages version pinning.
10         # This setup is confirmed stable on Colima v0.10.3 and Docker v29.6.1.
11         read -rp " Would you like to automatically install Colima & Docker via Homebrew? [y/N]: "
12         INSTALL_DOCKER
13         if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
14             step "Installing Colima and Docker..."
15             brew install colima docker docker-compose
16             step "Starting Colima VM..."
17             colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged
18         else
19             echo ""
20             echo " Please install Docker manually. Two options:"
21             echo " A) Colima (Recommended): brew install colima docker docker-compose && colima start --memory
0.6 --vm-type vz --network-address --network-mode bridged"
22             echo " B) Docker Desktop: https://www.docker.com/products/docker-desktop/"
23             die "Install Docker, then re-run this script."
24         fi
25     else
26         echo " Docker is not installed."
27         # Note: The Docker convenience script always fetches the latest stable release.
28         # This setup is confirmed stable on Docker Engine v29.6.1.
29         read -rp " Would you like to automatically install Docker? (Requires sudo) [y/N]: " INSTALL_DOCKER
30         if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
31             step "Installing Docker..."
32             curl -fsSL https://get.docker.com | sh
33             sudo usermod -aG docker "$USER"
34             echo -e "\n\033[0;32m[OK] Docker installed successfully!\033[0m"
35             echo -e "\033[0;33m[!] CRITICAL: You must log out of your SSH session and log back in for Docker
permissions to apply.\033[0m"
36             die "Please log out, log back in, and re-run setup.sh."
37         else
38             echo " Please install Docker manually:"
39             echo " curl -fsSL https://get.docker.com | sh"
40             echo " sudo usermod -aG docker \"$USER\" && newgrp docker"
41             die "Install Docker, then re-run this script."
42         fi
43     fi
44 fi
45 if ! docker info >> output.log 2>&1; then
46     echo ""
47     if [[ "$OSTYPE" == "darwin"* ]]; then
48         echo " Docker is installed but not running."
49         echo " If using Colima: colima start --memory 0.6 --vm-type vz --network-address --network-mode
bridged"
50         echo " If using Docker Desktop: open the app."
51     else
52         echo " Docker is installed but not running."
53         echo " Run: sudo systemctl start docker"
54     fi
55     die "Start Docker, then re-run this script."
56 fi
57
58 if ! docker compose version >> output.log 2>&1; then
59     die "Docker Compose v2 not found. Update Docker or install the compose plugin:\n https://docs.docker.com/
compose/install/"
60 fi
61
62 ok "Docker $(docker --version | grep -oE '[0-9.]+' | head -1) is running."
63
64 # 1.5 Python 3
65 step "Checking Python 3"

```

```

66
67 if ! command -v python3 >> output.log 2>&1; then
68     echo ""
69     if [[ "$OSTYPE" == "darwin"* ]]; then
70         echo " Python 3 is not installed."
71         echo " macOS includes it via Xcode Command Line Tools, or you can install it via Homebrew:"
72         echo "         brew install python3"
73         die "Install Python 3, then re-run this script."
74     else
75         echo " Python 3 is not installed."
76         echo " Please install it using your system's package manager. For example:"
77         echo "     Debian/Ubuntu:  sudo apt install python3"
78         echo "     Fedora:          sudo dnf install python3"
79         echo "     Arch:            sudo pacman -S python"
80         die "Install Python 3, then re-run this script."
81     fi
82 fi
83
84 # Note: This setup is confirmed stable on Python 3.9.6 (macOS default).
85 PYTHON_VER=$(python3 --version 2>&1 | head -n 1)
86 ok "$PYTHON_VER is installed."
87
88 # 2. Tailscale (host)
89 step "Checking Tailscale (host)"
90 # The host needs its own Tailscale installation separate from the Docker
91 # container so Toggle-Gateway can run 'tailscale down' before stopping
92 # containers and 'tailscale up' after starting them. Without this, the
93 # exit-node deadlock that setup.sh is designed to prevent can still occur.
94
95 # Resolve the tailscale binary: CLI (brew/package) or bundled macOS app
96 TAILSCALE_BIN=""
97 if command -v tailscale >> output.log 2>&1; then
98     TAILSCALE_BIN="tailscale"
99 elif [[ -x "/Applications/Tailscale.app/Contents/MacOS/Tailscale" ]]; then
100     TAILSCALE_BIN="/Applications/Tailscale.app/Contents/MacOS/Tailscale"
101 fi
102
103 RECOMMENDED_TS_VERSION="1.98.5"
104
105 if [[ -z "$TAILSCALE_BIN" ]]; then
106     warn "Tailscale not found on host installing..."
107
108     if [[ "$OSTYPE" == "darwin"* ]]; then
109         if command -v brew >> output.log 2>&1; then
110             step "Installing Tailscale CLI formula via Homebrew (standalone, no .app GUI)..."
111             # Note: We install Tailscale via Homebrew, targeting the stable version (recommended: 1.98.5)
112             brew install tailscale -q
113             TAILSCALE_BIN="tailscale"
114
115             step "Starting tailscaled as a per-user LaunchAgent (auto-starts at login)..."
116             if brew services start tailscale 2>&1 | grep -qE 'Successfully|started'; then
117                 ok "tailscaled LaunchAgent registered."
118             else
119                 warn "Could not auto-start tailscaled. Run 'brew services start tailscale' manually."
120                 warn "For a system-wide LaunchDaemon that runs before login, run instead:"
121                 warn "    sudo brew services start tailscale"
122             fi
123         else
124             echo ""
125             die "Homebrew not found. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):
126             https://tailscale.com/download/mac\n Then re-run this script."
127         fi
128     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
129         curl -fsSL https://tailscale.com/install.sh | sh
130         sudo systemctl enable --now tailscaled 2>> output.log || true
131         TAILSCALE_BIN="tailscale"
132     else
133         die "Unsupported OS. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):
134         https://tailscale.com/download\n Then re-run this script."
135     fi
136
137     TAILSCALE_VER=$(("${TAILSCALE_BIN}" version 2>> output.log | head -n 1)
138     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
139         warn "Tailscale installed, but version ($TAILSCALE_VER) differs from recommended version (
140         $RECOMMENDED_TS_VERSION)."
141     else
142         ok "Tailscale installed (version $TAILSCALE_VER)."
143     fi
144 else

```

```

144 TAILSCALE_VER=${${TAILSCALE_BIN} version 2>> output.log | head -n 1)
145 if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
146     warn "Tailscale is already installed ($TAILSCALE_VER), but recommended version is
147     $RECOMMENDED_TS_VERSION for host/container consistency."
148 else
149     ok "Tailscale is already installed at recommended version ($TAILSCALE_VER)."
150 fi
151
152 # Authenticate the host machine if it isn't already connected.
153 # This is an interactive step it opens a browser login URL.
154 # We do it now, before containers start, so Toggle-Gateway can
155 # safely run 'tailscale down/up' from day one.
156 if ! ${TAILSCALE_BIN} status >> output.log 2>&1; then
157     echo ""
158     echo " Your host machine needs to join your Tailscale network."
159     echo " A browser window or login URL will appear authenticate, then"
160     echo " return here. The script will continue automatically."
161     echo ""
162     if [[ "$OSTYPE" == "linux-gnu"* ]]; then
163         sudo ${TAILSCALE_BIN} up
164     else
165         ${TAILSCALE_BIN} up
166     fi
167     ok "Host machine connected to Tailscale."
168 else
169     ok "Host machine is already connected to Tailscale."
170 fi
171
172 # 3. wgcf
173 step "Checking wgcf (Cloudflare WARP key tool)"
174
175 if ! command -v wgcf >> output.log 2>&1; then
176     warn "wgcf not found installing..."
177
178     if [[ "$OSTYPE" == "darwin"* ]]; then
179         command -v brew >> output.log 2>&1 || die "Homebrew not found.\nInstall wgcf manually: https://github.
180         com/ViRb3/wgcf/releases"
181         brew install wgcf -q
182     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
183         ARCH=$(uname -m)
184         case $ARCH in
185             x86_64) WGCF_ARCH="amd64" ;;
186             aarch64) WGCF_ARCH="arm64" ;;
187             armv7l) WGCF_ARCH="armv7" ;;
188             *) die "Unsupported architecture: $ARCH.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/
189             releases" ;;
190         esac
191
192         echo " Fetching latest release..."
193         WGCF_VERSION=$(curl -sf https://api.github.com/repos/ViRb3/wgcf/releases/latest \
194             | grep "tag_name" | cut -d'"' -f4)
195         [[ -z "$WGCF_VERSION" ]] && die "Could not fetch wgcf version. Check your internet connection."
196
197         sudo curl -sL \
198             "https://github.com/ViRb3/wgcf/releases/download/${WGCF_VERSION}/wgcf_${WGCF_VERSION#v}_linux_${
199             WGCF_ARCH}" \
200             -o /usr/local/bin/wgcf
201         sudo chmod +x /usr/local/bin/wgcf
202     else
203         die "Unsupported OS. Install wgcf manually: https://github.com/ViRb3/wgcf/releases"
204     fi
205 fi
206
207 ok "wgcf is ready."
208
209 # 4. WARP profile
210 step "Generating Cloudflare WARP profile"
211
212 if [[ -f "wgcf-profile.conf" ]]; then
213     warn "wgcf-profile.conf already exists skipping key generation."
214 else
215     wgcf register --accept-tos 2>> output.log || die "wgcf register failed. Check your internet connection."
216     wgcf generate 2>> output.log || die "wgcf generate failed."
217     ok "WARP profile generated."
218 fi
219
220 # Parse keys (IPv4 address only gluetun doesn't need the IPv6 one)
221 PRIVATE_KEY=$(awk '/PrivateKey/{print $3}' wgcf-profile.conf)
222 PUBLIC_KEY=$(awk '/PublicKey/{print $3}' wgcf-profile.conf)

```

```

221 ADDRESSES=$(awk '/Address/{print $3}' wgcf-profile.conf | grep -v ':' | head -1)
222
223 [[ -z "$PRIVATE_KEY" ]] && die "Could not parse PrivateKey from wgcf-profile.conf"
224 [[ -z "$PUBLIC_KEY" ]] && die "Could not parse PublicKey from wgcf-profile.conf"
225 [[ -z "$ADDRESSES" ]] && die "Could not parse IPv4 Address from wgcf-profile.conf"
226
227 ok "WARP keys extracted."
228
229 # 5. Tailscale auth key
230 step "Tailscale authentication"
231
232 TS_AUTHKEY=""
233 if [[ -f ".env" ]]; then
234     EXISTING_TS_KEY=$(read_env_var TS_AUTHKEY)
235     if [[ -n "$EXISTING_TS_KEY" ]]; then
236         TS_AUTHKEY="$EXISTING_TS_KEY"
237         warn "Found existing TS_AUTHKEY in .env. Skipping prompt."
238     fi
239 fi
240
241 if [[ -z "$TS_AUTHKEY" ]]; then
242     echo ""
243     echo "Generate a key at: https://login.tailscale.com/admin/settings/keys"
244     echo "'Generate auth key' enable Reusable if you plan to re-run setup"
245     echo ""
246     read -rp "Paste your Tailscale auth key: " TS_AUTHKEY
247     echo ""
248 fi
249
250 [[ -z "$TS_AUTHKEY" ]] && die "Tailscale auth key is required."
251 [[ "$TS_AUTHKEY" != tskey-auth-* ]] && warn "Key doesn't look like a Tailscale auth key proceeding anyway."
252 ok "Auth key accepted."
253
254 # 6. AdGuard Home admin password
255 step "AdGuard Home admin account"
256
257 # Hardcoded default credentials to prevent lockout
258 # Username: admin
259 # Password: nullexit
260 if [[ -f ".env" ]]; then
261     EXISTING_AG_PASS=$(read_env_var ADGUARD_PASSWORD)
262     if [[ -n "$EXISTING_AG_PASS" ]]; then
263         ADGUARD_PASSWORD="$EXISTING_AG_PASS"
264         ADGUARD_PWD_ESC="$EXISTING_AG_PASS"
265         ok "Found existing ADGUARD_PASSWORD in .env."
266     else
267         ADGUARD_PASSWORD="nullexit"
268         ADGUARD_PWD_ESC="nullexit"
269         ok "Default password set to 'nullexit'."
270     fi
271 else
272     ADGUARD_PASSWORD="nullexit"
273     ADGUARD_PWD_ESC="nullexit"
274     ok "Default password set to 'nullexit'."
275 fi
276
277
278 # 7. Write .env
279 step "Writing .env"
280
281 if [[ -f ".env" ]]; then
282     read -rp ".env already exists. Overwrite? [y/N]: " OVERWRITE
283     if [[ "$OVERWRITE" != "y" && "$OVERWRITE" != "Y" ]]; then
284         warn "Keeping existing .env."
285     else
286         WRITE_ENV=1
287     fi
288 else
289     WRITE_ENV=1
290 fi
291
292 if [[ "${WRITE_ENV:-0}" == "1" ]]; then
293     # Preserve existing configuration flags if .env already exists
294     EXISTING_PROFILE="medium"
295     EXISTING_MSS="1120"
296     EXISTING_HIJACK="true"
297     EXISTING_EXIT="true"
298     EXISTING_THRESH="6"
299     EXISTING_KILL="false"
300     EXISTING_BLOCKED="kp il"
301     EXISTING_HOST_MTU="1200"

```



```

302
303     if [[ -f ".env" ]]; then
304         [[ -n "$(read_env_var GATEWAY_RULE_PROFILE)" ]] && EXISTING_PROFILE=$(read_env_var GATEWAY_RULE_PROFILE
305     )
306         [[ -n "$(read_env_var GATEWAY_MSS)" ]] && EXISTING_MSS=$(read_env_var GATEWAY_MSS)
307         [[ -n "$(read_env_var GATEWAY_HIJACK_HOST)" ]] && EXISTING_HIJACK=$(read_env_var GATEWAY_HIJACK_HOST)
308         [[ -n "$(read_env_var GATEWAY_USE_EXIT_NODE)" ]] && EXISTING_EXIT=$(read_env_var GATEWAY_USE_EXIT_NODE)
309         [[ -n "$(read_env_var WARP_FAIL_THRESHOLD)" ]] && EXISTING_THRESH=$(read_env_var WARP_FAIL_THRESHOLD)
310         [[ -n "$(read_env_var KILL_SWITCH)" ]] && EXISTING_KILL=$(read_env_var KILL_SWITCH)
311         [[ -n "$(read_env_var BLOCKED_COUNTRIES)" ]] && EXISTING_BLOCKED=$(read_env_var BLOCKED_COUNTRIES)
312         [[ -n "$(read_env_var HOST_MTU)" ]] && EXISTING_HOST_MTU=$(read_env_var HOST_MTU)
313     fi
314
315     cat > .env <<EOF
316 WIREGUARD_PRIVATE_KEY=${PRIVATE_KEY}
317 WIREGUARD_PUBLIC_KEY=${PUBLIC_KEY}
318 WIREGUARD_ADDRESSES=${ADDRESSES}
319 TS_AUTHKEY=${TS_AUTHKEY}
320 GATEWAY_RULE_PROFILE=${EXISTING_PROFILE}
321 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds and
322   have a healthy path.
323 GATEWAY_MSS=${EXISTING_MSS}
324 # Lower the host Tailscale MTU to 1200 to prevent fragmentation when passing through the 1280-byte WARP tunnel.
325 HOST_MTU=${EXISTING_HOST_MTU}
326 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
327   internet is NOT hijacked).
328 GATEWAY_HIJACK_HOST=${EXISTING_HIJACK}
329 GATEWAY_USE_EXIT_NODE=${EXISTING_EXIT}
330 WARP_FAIL_THRESHOLD=${EXISTING_THRESH}
331 KILL_SWITCH=${EXISTING_KILL}
332 ADGUARD_PASSWORD=${ADGUARD_PASSWORD}
333 BLOCKED_COUNTRIES="${EXISTING_BLOCKED}"
334 EOF
335     ok ".env written."
336 fi
337
338 # 8. Prepare directories
339 step "Preparing AdGuard directories"
340
341 mkdir -p adguard/conf adguard/work/userfilters
342
343 # Remove empty AdGuardHome.yaml that would cause a crash loop (same logic as Toggle-Gateway scripts)
344 if [[ -f "adguard/conf/AdGuardHome.yaml" && ! -s "adguard/conf/AdGuardHome.yaml" ]]; then
345     rm "adguard/conf/AdGuardHome.yaml"
346     warn "Removed empty AdGuardHome.yaml to prevent crash loop."
347 fi
348
349 ok "Directories ready."
350
351 # 9. Compile DNS filter rules
352 step "Compiling DNS filter rules (this may take a minute)"
353
354 if command -v python3 >> output.log 2>&1; then
355     python3 scripts/sync-rules.py
356     ok "Rules compiled."
357 else
358     warn "python3 not found skipping rule compilation."
359     warn "Run 'python3 scripts/sync-rules.py' manually after setup."
360 fi
361
362 # 10. Start containers
363 step "Starting containers"
364 # Note: tailscale depends on adguardhome being healthy (port 5335 up),
365 # so Docker Compose will hold it back automatically until the wizard is done.
366 docker compose up -d
367 ok "Containers starting."
368
369 # 11. Wait for AdGuard Home setup wizard
370 step "Waiting for AdGuard Home setup wizard"
371 echo " (up to 60 seconds...)"
372
373 MAX=60; ELAPSED=0
374 until docker exec routing-fix curl -sf http://127.0.0.1:3000 >> output.log 2>&1; do
375     sleep 2; ELAPSED=$((ELAPSED + 2))
376     [[ $ELAPSED -ge $MAX ]] && die "AdGuard Home didn't come up.\nDebug with: docker compose logs adguardhome"
377 done
378
379 ok "AdGuard Home is up."
380
381 # 12. Configure AdGuard Home via API
382 step "Configuring AdGuard Home"

```

```

380
381 # Run all API calls in a single docker exec sh -c so the session cookie file
382 # is shared across calls without leaving the container.
383 docker exec routing-fix sh -c "
384     set -e
385
386     # Step 1: Complete the setup wizard (sets admin creds + DNS port to 5335)
387     curl -sf -X POST http://127.0.0.1:3000/control/install/configure \
388     -H 'Content-Type: application/json' \
389     -d '{"web":{"ip":"0.0.0.0","port":3000},"dns":{"ip":"0.0.0.0","port":5335},"username":"","admin":"","password":"'${ADGUARD_PWD_ESC}'"}' \
390     >/dev/null || true
391
392     # Step 2: Wait for AdGuard to restart after wizard
393     sleep 4
394     WAITED=0
395     until curl -sf http://127.0.0.1:3000 >/dev/null 2>&1; do
396         sleep 2; WAITED=$((WAITED + 2))
397         [ \${WAITED} -ge 30 ] && { echo 'AdGuard did not restart in time'; exit 1; }
398     done
399
400     # Step 3: Login to get session cookie
401     curl -sf -X POST http://127.0.0.1:3000/control/login \
402     -H 'Content-Type: application/json' \
403     -d '{"name":"","admin":"","password":"'${ADGUARD_PWD_ESC}'"}' \
404     -c /tmp/agh.cookie >/dev/null
405
406     # Step 4: Point upstream DNS back through the WARP tunnel
407     curl -sf -X POST http://127.0.0.1:3000/control/dns_config \
408     -H 'Content-Type: application/json' \
409     -b /tmp/agh.cookie \
410     -d '{"upstream_dns":["127.0.0.1:53"],"bootstrap_dns":["1.1.1.1","8.8.8.8"],"upstream_mode":"","load_balance":""}' \
411     >/dev/null
412
413     # Step 5: Register the compiled rules file as a filter list
414     curl -sf -X POST http://127.0.0.1:3000/control/filtering/add_url \
415     -H 'Content-Type: application/json' \
416     -b /tmp/agh.cookie \
417     -d '{"name":"","url":"","opt/adguardhome/work/userfilters/compiled_rules.txt"}' \
418     >/dev/null || true
419
420     # Step 6: Force a filter refresh to load the rules immediately
421     curl -sf -X POST http://127.0.0.1:3000/control/filtering/refresh \
422     -H 'Content-Type: application/json' \
423     -b /tmp/agh.cookie \
424     -d '{"whitelist":false}' \
425     >/dev/null || true
426
427     rm -f /tmp/agh.cookie
428 "
429
430 ok "AdGuard Home configured."
431
432 # 13. Wait for Tailscale
433 step "Waiting for Tailscale to authenticate"
434 echo " (Tailscale only starts once AdGuard's healthcheck passes up to 90s)"
435
436 MAX=90; ELAPSED=0; TS_IP=""
437 until [[ -n "$TS_IP" ]]; do
438     TS_IP=$(docker exec tailscale tailscale ip -4 2>> output.log || true)
439     if [[ -z "$TS_IP" ]]; then
440         sleep 3; ELAPSED=$((ELAPSED + 3))
441         if [[ $ELAPSED -ge $MAX ]]; then
442             warn "Tailscale hasn't authenticated yet."
443             warn "Once it does, re-run the toggle script it will resolve the IP dynamically."
444             break
445         fi
446     fi
447 done
448
449 if [[ -n "$TS_IP" ]]; then
450     ok "Tailscale IP: ${TS_IP} (resolved dynamically by the toggle script on each startup)"
451 fi

```

43 scripts/unlock-files.sh

```
1 #!/bin/bash
2 # Resolve project root relative to this script's location, so the script
3 # works regardless of the caller's CWD.
4 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
6
7 echo "Unlocking .env..."
8 if [ -f "$PROJECT_ROOT/.env" ]; then
9     sudo cat "$PROJECT_ROOT/.env" > "$PROJECT_ROOT/.env.tmp" && mv "$PROJECT_ROOT/.env.tmp" "$PROJECT_ROOT/.env"
10     echo ".env unlocked"
11 fi
12
13 echo "Unlocking AdGuard cache files..."
14 for f in "$PROJECT_ROOT"/adguard/work/userfilters/compiled_rules.txt \
15         "$PROJECT_ROOT"/adguard/work/userfilters/ip_blocklist.ipset \
16         "$PROJECT_ROOT"/adguard/work/userfilters/cache/*.txt \
17         "$PROJECT_ROOT"/adguard/work/userfilters/cache/ip/*.txt \
18         "$PROJECT_ROOT"/adguard/work/data/filters/*.txt; do
19     if [ -f "$f" ]; then
20         cat "$f" > "$f.tmp" && mv "$f.tmp" "$f"
21         echo "Unlocked $f"
22     fi
23 done
24
25 echo ""
26 echo "All locked files have been successfully bypassed via atomic rename!"
```

44 white_list.txt

```
1 0.client-channel.google.com
2 1drv.com
3 2.android.pool.ntp.org
4 akamaihd.net
5 akamaitechnologies.com
6 akamaized.net
7 amazonaws.com
8 android.clients.google.com
9 api-adservices.apple.com
10 api.ipify.org
11 api.rlje.net
12 app-api.ted.com
13 appleid.apple.com
14 apps.skype.com
15 appsbackup-pa.clients6.google.com
16 appsbackup-pa.googleapis.com
17 appspot-preview.l.google.com
18 apt.sonarr.tv
19 aspnetscdn.com
20 attestation.xboxlive.com
21 ax.phobos.apple.com.edgesuite.net
22 books-analytics-events.apple.com
23 brightcove.net
24 c.s-microsoft.com
25 cdn.cloudflare.net
26 cdn.embedly.com
27 cdn.optimizely.com
28 cdn.vidible.tv
29 cdn2.optimizely.com
30 cdn3.optimizely.com
31 cdnjs.cloudflare.com
32 cert.mgt.xboxlive.com
33 clientconfig.passport.net
34 clients1.google.com
35 clients2.google.com
36 clients3.google.com
37 clients4.google.com
38 clients5.google.com
39 clients6.google.com
40 connectivitycheck.android.com
41 connectivitycheck.gstatic.com
42 continuum.dds.microsoft.com
43 cpms.spop10.ams.plex.bz
44 cpms35.spop10.ams.plex.bz
```

45 cse.google.com
46 ctldl.windowsupdate.com
47 cws.conviva.com
48 d2c8v52ll5s99u.cloudfront.net
49 d2gatte9c95jao.cloudfront.net
50 dashboard.plex.tv
51 dataplicity.com
52 def-vef.xboxlive.com
53 delivery.vidible.tv
54 dev.virtualearth.net
55 device.auth.xboxlive.com
56 display.ugc.bazaarvoice.com
57 displaycatalog.mp.microsoft.com
58 dl.delivery.mp.microsoft.com
59 dl.dropbox.com
60 dl.dropboxusercontent.com
61 dns.msftncsi.com
62 download.sonarr.tv
63 drift.com
64 driftt.com
65 dynupdate.no-ip.com
66 ecn.dev.virtualearth.net
67 edge.api.brightcove.com
68 eds.xboxlive.com
69 fonts.gstatic.com
70 forums.sonarr.tv
71 g.live.com
72 geo-prod.do.dsp.mp.microsoft.com
73 geo3.ggpht.com
74 gfwsl.geforce.com
75 giphy.com
76 github.com
77 github.io
78 googleapis.com
79 gravatar.com
80.gstatic.com
81 help.ui.xboxlive.com
82 hls.ted.com
83 i.ytimg.com
84 il.ytimg.com
85 iadsdk.apple.com
86 imagesak.secureserver.net
87 img.vidible.tv
88 imgix.net
89 imgs.xkcd.com
90 instantmessaging-pa.googleapis.com
91 intercom.io
92 jquery.com
93 jsdelivr.net
94 keystone.mwbsys.com
95 lastfm-img2.akamaized.net
96 licensing.xboxlive.com
97 live.com
98 livepassdl.conviva.com
99 login.live.com
100 login.microsoftonline.com
101 manifest.googlevideo.com
102 meta-db-worker02.pop.ric.plex.bz
103 meta.plex.bz
104 meta.plex.tv
105 microsoftonline.com
106 msftncsi.com
107 my.plexapp.com
108 nexusrules.officeapps.live.com
109 nine.plugins.plexapp.com
110 no-ip.com
111 node.plexapp.com
112 notes-analytics-events.apple.com
113 notify.xboxlive.com
114 npr-news.streaming.adswizz.com
115 ns1.dropbox.com
116 ns2.dropbox.com
117 o1.email.plex.tv
118 o2.sg0.plex.tv
119 ocsp.apple.com
120 office.com
121 office.net
122 office365.com
123 officeclient.microsoft.com
124 om.cbsi.com
125 onedrive.live.com

```

126 outlook.live.com
127 outlook.office365.com
128 pings.conviva.com
129 placeholder.it
130 placeholder.imgix.net
131 players.brightcove.net
132 pricelist.skype.com
133 products.office.com
134 proxy.plex.bz
135 proxy.plex.tv
136 proxy02.pop.ord.plex.bz
137 pubsub.plex.bz
138 pubsub.plex.tv
139 raw.githubusercontent.com
140 redirector.googlevideo.com
141 res.cloudinary.com
142 s.gateway.messenger.live.com
143 s.marketwatch.com
144 s.youtube.com
145 s.ytimg.com
146 s1.wp.com
147 s2.youtube.com
148 s3.amazonaws.com
149 sa.symcb.com
150 secure.avangate.com
151 secure.brightcove.com
152 secure.surveymonkey.com
153 services.sonarr.tv
154 skyhook.sonarr.tv
155 spclient.wg.spotify.com
156 ssl.p.jwpcdn.com
157 staging.plex.tv
158 status.plex.tv
159 t.co
160 t0.ssl.ak.dynamic.tiles.virtualearth.net
161 t0.ssl.ak.tiles.virtualearth.net
162 tawk.to
163 tedcdn.com
164 themoviedb.com
165 thetvdb.com
166 tinyurl.com
167 title.auth.xboxlive.com
168 title.mgt.xboxlive.com
169 traffic.libsyn.com
170 tvdb2.plex.tv
171 tvthemes.plexapp.com
172 twimg.com
173 ui.skype.com
174 video-stats.l.google.com
175 videos.vidible.tv
176 vidtech.cbsinteractive.com
177 weather-analytics-events.apple.com
178 widget-cdn.rpxnow.com
179 win10.ipv6.microsoft.com
180 wp.com
181 ws.audioscrobbler.com
182 www.dataplicity.com
183 www.googleapis.com
184 www.msftconnecttest.com
185 www.msftncsi.com
186 www.no-ip.com
187 www.youtube-nocookie.com
188 xbox.ipv6.microsoft.com
189 xboxexperiencesprod.experimentation.xboxlive.com
190 xflight.xboxlive.com
191 xkms.xboxlive.com
192 xsts.auth.xboxlive.com
193youtu.be
194 youtube-nocookie.com
195 yt3.ggpht.com
196 zee.cws.conviva.com
197
198 # YouTube family base domains so every subdomain (video stream servers on
199 # *.googlevideo.com, image hosts on *.ytimg.com, profile thumbnails on
200 # *.ggpht.com, etc.) is automatically allowlisted via the AdGuard '||domain^'
201 # wildcard. The specific s.youtube.com / manifest.googlevideo.com entries
202 # above are kept for clarity but will be optimized away by sync-rules.py
203 # because their parents are now in this list.
204 youtube.com
205 googlevideo.com
206 ytimg.com

```

```
207 ggph.t.com
208 gvt1.com
209 gvt2.com
210 gvt3.com
211 gvt1.net
212 gvt2.net
213 gvt3.net
214
215 edge-mqtt.facebook.com
216 gateway.instagram.com
217 i.instagram.com
218
219 facebook.com
220 fb.com
221 fbcdn.net
222 fbsbx.com
223 messenger.com
224 whatsapp.com
225 web.whatsapp.com
226 instagram.com
227 cdninstagram.com
```